Search You Can See Into

A textbook on the RAG Workbench: a grammar for search pipelines, the engine that runs it, and the agent that edits it.

Oct 16, 2026 · @Manuel Odendahl

About this book

This book explains how the RAG Workbench turns a folder of markdown into ranked, cited answers, and why each layer is built the way it is. It follows one question through eleven project reports: *Why did ChatGPT show no app actions after OAuth worked?*

The system has four layers, taken in order. A **grammar** describes a search pipeline as data. A **compiler** turns that description into a typed graph, the IR. An **engine** builds the graph's indexes and runs its query nodes. A **generative UI** and an **agent** sit on top, and both read and change the same graph through the same edit operations.

Three readers

- **Computer scientists** get information retrieval from first principles (BM25, rank fusion, ranking metrics), a small compiler from specs to a typed DAG, content-addressed incremental computation, and closures that survive serialization.
- **Programmers** get the API, how to write presets and components, how to read a trace when a query misses, and how to change a running pipeline without breaking it.
- **Designers** get the decisions a search experience is made of: what typing does versus Enter, what a result shows, how cost and citations surface, and the grammar that lays out the page.

Reading paths

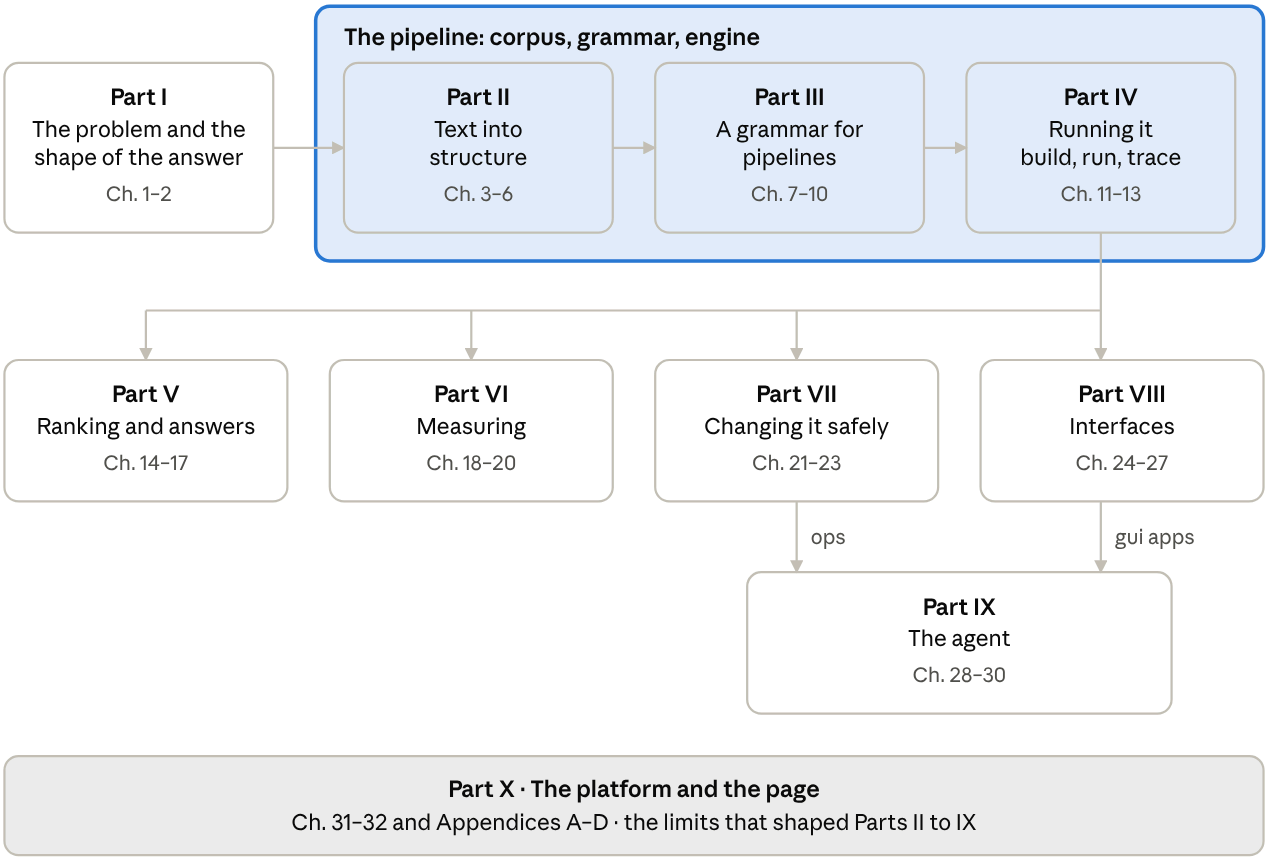
Reader	Read closely	Then	Skim
Computer scientist	Parts II, III, IV, VI	Parts V, VII	Parts VIII, X
Programmer	Parts I, III, IV, VII	Parts V, VI, IX	Ch. 5 (BM25 math), Part X
Designer	Parts I, VIII	Ch. 15, 18, 23, 29	Parts III, IV, X

Conventions

- Every code sample is the real `rag` and `gui` API as it runs in the workbench. Pseudocode is labelled as pseudocode.
- Every chapter ends with a **Lab** that names a preset in the editor's picker; loading it reproduces the chapter's examples.
- Trace boxes show real output from the running page, with each line explained.
- Steps that spend the reader's Claude usage are marked, the same way the workbench marks them.

Parts II to IV build the pipeline that every later part works on

Map of the book · each arrow points to a part that builds on the one it leaves



map of the book · ten parts, the pipeline core outlined

Read Parts II to IV in order. After them, Parts V to VIII can be read in any order; Part IX assumes VII and VIII, and Part X describes the limits all of them were built within.

Part I · The problem and the shape of the answer

Search quality is found by iteration, so the pipeline has to be something you can see, measure and change while it runs. Part I sets up the problem and the page built around it.

- **1. Retrieval is ranking under a budget.** The eleven reports total 535 KB and a prompt holds 256 KB, so something must choose what Claude reads, and the choice is a ranking. *Lab: Minimal search.*
  - 1.1 The running example: eleven reports and one question
  - 1.2 Why not send everything: the prompt budget in bytes
  - 1.3 Three jobs: retrieve, pack, answer
  - 1.4 Three failures you can name: the section was missed, misranked, or misread
- **2. A workbench, not a script.** A pipeline that is data can be drawn, diffed, scored and edited live; a pipeline that is code can only be rerun. *Lab: Reports · hybrid + rerank + answer.*
  - 2.1 The edit, measure, keep loop
  - 2.2 What must be visible: the graph, the trace, the scores
  - 2.3 A tour of the page: editor, sheet, inspector, agent
  - 2.4 Presets as worked examples, and how to read one

Part II · Text into structure

What can be found depends on how text was cut and counted before anyone asked a question. Part II builds the corpus side: documents, chunks and indexes.

- **3. Parsing markdown into documents.** Frontmatter is metadata, headings form a path, and fences, tables and callouts are kept whole, because each is used differently later.
  - 3.1 Frontmatter as fields, not text
  - 3.2 The heading stack and section paths
  - 3.3 Blocks: paragraph, list, fence, table, callout
  - 3.4 Links between documents: wiki links and related notes
  - 3.5 What the parser gets wrong, and why that is acceptable here
- **4. Chunking decides what can be found.** The chunk is the unit that gets ranked, cited and scored, so its size and boundaries set the ceiling on every later stage. *Labs: Chunk sizes, Table rows as chunks, Code search, Outline search, Which report?*
  - 4.1 Sections as the default unit

    - 4.2 Token budgets, atomic blocks and the short tail
    - 4.3 Overlap and heading prefixes
    - 4.4 Several chunk streams over one corpus: rows, code, cards
    - 4.5 Stable chunk ids, and what depends on them
- **5. BM25 from first principles.** A lexical score is three ideas multiplied together: rare terms matter more, repeated terms matter less each time, and long sections should not win by length. *Lab: Field weights.*
    - 5.1 Tokens, stop words and a small stemmer
    - 5.2 Term frequency and saturation (k1)
    - 5.3 Length normalization (b)
    - 5.4 Inverse document frequency
    - 5.5 Fields and weights: title, heading path, callout, text
    - 5.6 Scoring "rebalance law" by hand, then checking it against the engine
- **6. Metadata as an index.** Frontmatter terms answer different questions than body text: they filter, facet and boost rather than rank. *Labs: Faceted search, Query expansion.*
    - 6.1 A terms index over type, tags and aliases
    - 6.2 Facet counts from a result set
    - 6.3 Synonyms in the index versus rewriting the query

Part III · A grammar for pipelines

A pipeline written as a chain of builder calls is a value, not a running program: it can be compiled, checked, drawn, varied and stored. Part III defines that grammar and the IR it compiles to.

- **7. Specs, not programs.** `rag.corpus` and `rag.plan` return immutable specs, so a variant is a new value and the original never changes under you. *Lab: Minimal search.*
    - 7.1 Corpus specs and plan specs
    - 7.2 Immutability, and why variants cost nothing
    - 7.3 The chain infers the wiring; the graph form states it
    - 7.4 Fragments and the shared `kit`
- **8. Components and typed ports.** Every step is a registered component with a phase and typed inputs and outputs, so wiring mistakes are caught before anything runs. *Labs: Your own components, Code search.*

    - 8.1 The registry: type, phase, ports, trigger
    - 8.2 Two typed streams: files → docs → chunks → index, and queries → hits → context → answer
    - 8.3 Method names generated from the registry
    - 8.4 `rag.define`: components that chain like built-ins
- **9. Lambdas that survive serialization.** A lambda is stored as source text and run without its closure, so a pipeline can be saved, reloaded and edited by an agent. *Labs: Rerank with a lambda, Table rows as chunks.*
    - 9.1 Functions as source text
    - 9.2 Closed lambdas: the guard and the free-identifier error
    - 9.3 Getting values in: signals, params as the last argument, `rag.bake`
    - 9.4 Four phases: build callbacks, build slots, bakes, live slots
    - 9.5 Which signals a lambda reads: static scan and runtime tracking
- **10. Compiling to an IR.** Compilation checks ports, ids and cycles, orders the nodes, derives the outputs and gives every build node a content digest. *Lab: Graph form.*
    - 10.1 The IR: nodes, edges, signals, outputs
    - 10.2 What the checker rejects, with the real error messages
    - 10.3 Topological order and derived outputs
    - 10.4 Digests: naming a node by what it would compute
    - 10.5 Reading the DAG tab

Part IV · Running it

The engine runs an IR in two phases: a build that turns files into indexes once, and query runs that happen on every keystroke. Both avoid repeated work by recording what each node's output depended on.

- **11. Builds that remember.** A build node is named by a digest of its type, params, input digests and file contents, so an edit rebuilds only the nodes downstream of it. *Labs: Chunk sizes, Doc cards.*
    - 11.1 The build phase and the live phase
    - 11.2 The digest chain: type, params, inputs, file contents
    - 11.3 The build memo and the dirty set
    - 11.4 Map components and the per-item cache, in memory and in the database
    - 11.5 Builds that cost Claude calls: budgets, deferral and `pending()`

- **12. Queries at runtime.** A session reruns only the live nodes whose params, inputs or signal reads changed, and each node's trigger decides whether it runs while the user types or waits for Enter. *Labs: Answer first, Ranking playground.*
  - 12.1 Signals: the named inputs of a query run
  - 12.2 Triggers: `change` nodes run on every keystroke, `submit` nodes wait for Enter
  - 12.3 The session memo: input identity plus recorded signal reads
  - 12.4 Four node statuses: ran, cached, skipped, error
  - 12.5 Cancellation, and what a keystroke costs
- **13. Traces and explanations.** A run returns one trace entry per node and a score breakdown per hit, so a miss can be attributed to the stage that caused it. *Lab: Compare stages.*
  - 13.1 The trace: status, counts in and out, time, calls
  - 13.2 `why` and `parts`: where a hit's score came from
  - 13.3 `rag.explain`: following one chunk through every node
  - 13.4 Missed, misranked, misread: diagnosing chapter 1's three failures from a trace

Part V · Ranking and answers

The live half of the pipeline turns a question into ranked hits, packs the best of them into a context, and asks Claude for a cited answer. Part V takes those steps in order, then gives Claude a larger role in each.

- **14. Retrieval and fusion.** Scores from different indexes are on different scales, so they are combined by rank, not by adding scores. *Labs: Fusion, Reports · hybrid + rerank + answer.*
  - 14.1 The retrieve node: one index, several queries
  - 14.2 Why two BM25 scores cannot be added
  - 14.3 Reciprocal rank fusion: 1/(k + rank), weights, and what k = 60 does
  - 14.4 Max fusion, and when it is the better rule
  - 14.5 Filters and limits: `filter.where`, filter lambdas and `s.k`
- **15. Context and answers.** The answer can cite only what was packed, so packing is a ranking decision under a byte budget. *Labs: Context expansion, Two answers.*
  - 15.1 Expanding hits: neighbouring sections and linked reports, with score decay
  - 15.2 Packing under a budget: rank order, per-document caps, the 40,000-character default

    - 15.3 Evidence labels and the answer prompt
    - 15.4 Citations as data, and an answer that says what is missing
    - 15.5 Streaming the answer and showing its sources
- **16. Rewriting and reranking.** A ranking can change without touching an index: before retrieval by changing the queries, and after it by reordering the hits. *Labs: Query expansion, Rerank with a lambda, Your own components, Rewrite + rerank, HyDE.*
    - 16.1 Rewrite lambdas: acronyms and synonyms
    - 16.2 `rewrite.llm`: keyword queries in the corpus's own vocabulary
    - 16.3 HyDE: searching with a drafted answer
    - 16.4 Rerank lambdas: heuristics over fields the index never scored
    - 16.5 `rerank.llm`: passages, scores and stated reasons
    - 16.6 Why steps that call Claude wait for Enter
- **17. Claude as the retriever.** An agentic node gives Claude search and read tools and lets it choose the queries, trading predictability and calls for recall. *Labs: Agentic answer, Doc cards.*
    - 17.1 Four tools: search, read, outline, documents
    - 17.2 A read budget, and evidence labels assigned on read
    - 17.3 Rounds in the trace: what was searched and what was read
    - 17.4 Routing by build-time enrichment: summaries and questions per document
    - 17.5 Fixed pipeline or agent: comparing both on the same goldens

Part VI · Measuring

A change to a pipeline is an improvement only if it moves a number on questions chosen before the change was made. Part VI defines those questions, the numbers, and how pipelines are compared with them.

- **18. Golden questions.** A golden case is a question plus patterns for the document and section that should answer it, so relevance is checked by a rule rather than by a stored chunk id. *Labs: Reports · hybrid + rerank + answer, Synthetic evals.*
    - 18.1 A case: question, expected document, expected section, expected answer text
    - 18.2 Patterns instead of ids: why cases survive rechunking
    - 18.3 Writing cases from the questions users actually ask
    - 18.4 Which run a case exercises: `change` by default, `submit` when the answer is scored

    - 18.5 Synthetic cases: questions Claude writes for sampled sections
- **19. Ranking metrics.** hit@k, MRR, nDCG and recall each reward a different shape of result list, and choosing one is choosing how far down the list users are assumed to read. *Lab: Field weights.*
    - 19.1 hit@k: whether a relevant section was on the first screen
    - 19.2 MRR: how far down the first relevant section was
    - 19.3 nDCG@k: credit discounted by position and normalised by the best possible list
    - 19.4 recall@k: how many of the relevant sections were found
    - 19.5 All four by hand for one case
    - 19.6 Twelve cases, and how large a difference has to be before it means anything
- **20. Variants, sweeps and judges.** Pipelines are compared on the same cases with the same metrics, parameter grids are scored automatically, and Claude judges what string patterns cannot. *Labs: Chunk sizes, Parameter sweep, Judged evals.*
    - 20.1 Variants as patched plans; `compare` and its deltas
    - 20.2 A sweep is a grid of `set` ops, and each configuration rebuilds only what it changed
    - 20.3 Reading a sweep: the best configuration, ties, and overfitting to twelve questions
    - 20.4 `answerHas`: the cheapest check on an answer
    - 20.5 Judges: groundedness and length as lambdas that call Claude
    - 20.6 What an eval run costs, and when to spend it

Part VII · Changing it safely

A pipeline that a person and an agent both edit while it serves queries needs edits that are small, checked and reversible. Part VII makes every change an operation on the IR and gives the workspace that applies them a history.

- **21. Edits as operations.** Ten operations cover every change to an IR, and each op list is applied by recompiling, so an edit that would break the graph is rejected before it takes effect. *Lab: the `ops` list at the end of every preset.*
    - 21.1 The ten ops: set, unset, insert, remove, replace, wire, signal, trigger, output, rename
    - 21.2 Paths into params, and setting a lambda from source text
    - 21.3 `plan.patch(ops)`: the same ops at authoring time

    - 21.4 Dry runs: which nodes go live, which rebuild, and how many Claude calls the rebuild would cost
    - 21.5 Why operations and not a text diff of the script
- **22. Workspaces and revisions.** A workspace holds the current IR and a revision number, and an op list planned against an older revision is refused rather than silently merged. *Lab: Reports · hybrid + rerank + answer, with the agent.*
    - 22.1 A workspace: IR, engine, revision, listeners
    - 22.2 `expect`: optimistic concurrency between the agent and the editor
    - 22.3 The receipt: live changes, nodes to rebuild, removals
    - 22.4 Read-only workspaces, and errors written for the agent that will read them
- **23. History: undo, branches and snapshots.** Every applied op list is stored with the IR it replaced, so undo restores a previous pipeline exactly and a branch lets a risky change be scored before it lands. *Lab: the History tab.*
    - 23.1 Undo and redo restore stored IRs; revisions only count up
    - 23.2 Branches, and why a merge replays ops instead of merging graphs
    - 23.3 Snapshots: what the agent saves before each step
    - 23.4 Keeping changes in the script, and exporting a standalone page

Part VIII · Interfaces

The page a search runs in is described the same way as the pipeline: a tree built once from a grammar, with props that are values or closed lambdas. Part VIII builds that grammar, the runtime that keeps it current, and the decisions a search page is made of.

- **24. A grammar for pages.** An app is state, derived values and a view tree built once; each prop is a value, a closed lambda `(s, ctx) => …`, or a bake evaluated at build time. *Labs: Corpus dashboard, Result map.*
    - 24.1 `gui.app`: title, `use(plan)`, state, derive, watch, view
    - 24.2 Layout: row, col, grid, card, tabs, when
    - 24.3 Content: text, note, stat, table, list, chart, code
    - 24.4 Controls bound to state keys: slider, number, select, toggle, input, button
    - 24.5 Props as lambdas, and why the tree is built only once
    - 24.6 `canvas`: drawing with `(s, ctx, g)`

- **25. A reactive runtime.** The runtime records which state keys each component read while rendering, and when a key changes it re-renders only those components. *Lab: Ranking playground.*
  - 25.1 Read tracking through a proxy
  - 25.2 From a state change to a redraw: set, derive, refresh
  - 25.3 Plan signals are state keys: the bridge between the page and the session
  - 25.4 Scheduling query runs by trigger
  - 25.5 Errors that stay inside the component that raised them
- **26. Search components.** The search components are ordinary components that read the plan's signals and `s.out[nodeId]`, so any node's output can be shown anywhere on the page. *Labs: Compare stages, Answer first, Document browser.*
  - 26.1 `searchBox`: a query signal, a submit action, and a box that keeps focus while results redraw
  - 26.2 `results` and `hits`: showing any hits node, not only the last one
  - 26.3 `facets`: counts from the result set, written back as a filter signal
  - 26.4 `answer` and `chunk`: streamed text, citation links, the section viewer
  - 26.5 `trace`, `dag` and `status`: the pipeline as part of the page
- **27. Designing search experiences.** A search page is a set of decisions about time, trust and cost: what each keystroke does, what a result shows, and when the reader is told that an action spends Claude usage. *Labs: Faceted search, Answer first, Document browser, Result map.*
  - 27.1 Type versus Enter: instant ranking, deliberate answers
  - 27.2 What a result shows: title, heading path, snippet, score, reason
  - 27.3 Answer first or results first
  - 27.4 Citations the reader can check in one click
  - 27.5 Empty, loading, partial and failed states
  - 27.6 Marking the actions that cost Claude calls
  - 27.7 Phone width: one column, no horizontal scroll

Part IX · The agent

The agent is Claude with one tool that runs JavaScript in the page, so it works through the same `rag`, `gui` and workspace objects as a programmer and is held to the same checks. Part IX explains that design and the controls around it.

- **28. One tool: eval.** Instead of one tool per action, the agent gets a single tool that runs code against the page's API, so its abilities grow with the API and its mistakes arrive as ordinary errors. *Lab: the suggestion chips in the agent panel.*
  - 28.1 Why one eval tool rather than many narrow tools
  - 28.2 The scope: `rag`, `gui`, `wb`, `ws`, `ui`, `chat`, `ref`, `log`, `console`
  - 28.3 Results as short YAML, truncated near 9,000 characters
  - 28.4 Errors with line numbers, and how the agent recovers from them
  - 28.5 The system prompt: an API reference plus the current state of the page
- **29. Run modes and reversibility.** Auto, Ask and Read-only decide whether a step runs, waits for approval or is refused when it mutates, and every step that runs is snapshotted so the person can revert it. *Lab: the mode switch in the agent panel.*
  - 29.1 Auto: run, snapshot, report
  - 29.2 Ask: approve one step, or the rest of the turn
  - 29.3 Read-only: mutating calls throw, and branches stay open for experiments
  - 29.4 Reverting to before any step
  - 29.5 Editing the live pipeline with ops versus editing the script
- **30. Answers the agent can draw.** The agent puts tables, charts, hit lists, eval reports, DAGs and whole gui apps into the conversation, and links into the workbench from its prose. *Lab: ask the agent for a search page.*
  - 30.1 `chat` outputs: md, table, chart, hits, evals, dag, ui, download
  - 30.2 `ref` links: pointing at a query, a chunk, a node or a tab
  - 30.3 `ui`: running a query, opening a tab or a section for the person
  - 30.4 Conversations saved per user, and what is clipped when they are stored
  - 30.5 A worked session: diagnose a miss, branch, fix, score, merge

Part X · The platform and the page

The workbench runs as a hosted page whose only services are a Claude sampler, a small per-user database and file downloads. Part X describes those limits, which shaped Parts II to IX, and the page built inside them.

- **31. The artifact runtime.** Every design choice in this book fits inside four services and a 256 KiB prompt, and this chapter says which choice each limit forced.
  - 31.1 `claude.use`: sample, db, user, downloads
  - 31.2 Sampling: the prompt cap, tools, `.json`, and `limits()`

    - 31.3 Why there are no embeddings, and what lexical retrieval plus Claude covers instead
    - 31.4 Persistence: one document per user, with transcripts, files and cache collections
    - 31.5 Uploading reports: what a new file does to digests, caches and evals
    - 31.6 The same library in node: testing without a page
- **32. The page itself.** The Lattice design system gives the workbench one accent colour and two line styles, and both carry meaning: dashed is build time and dotted is runtime.
    - 32.1 Ink on paper, with one signal blue
    - 32.2 Dashed for build, dotted for runtime, in the DAG, the editor and the agent panel
    - 32.3 Layout: editor, search sheet, inspector, agent
    - 32.4 The preset picker: stashed edits and restoring a session
    - 32.5 Checking the page: node tests and a headless run of every preset

Appendices

- **A. API reference.** Every `rag` and `gui` builder method and component type, with its phase, ports, default trigger and options.
- **B. Preset catalogue.** The 29 presets by group, each with the chapters it illustrates.
- **C. Formats.** The IR, the ten ops, trace entries and eval reports, as JSON with field-by-field notes.
- **D. Glossary.** Chunk, digest, signal, trigger, bake, golden case, evidence label and the other terms as this book defines them.

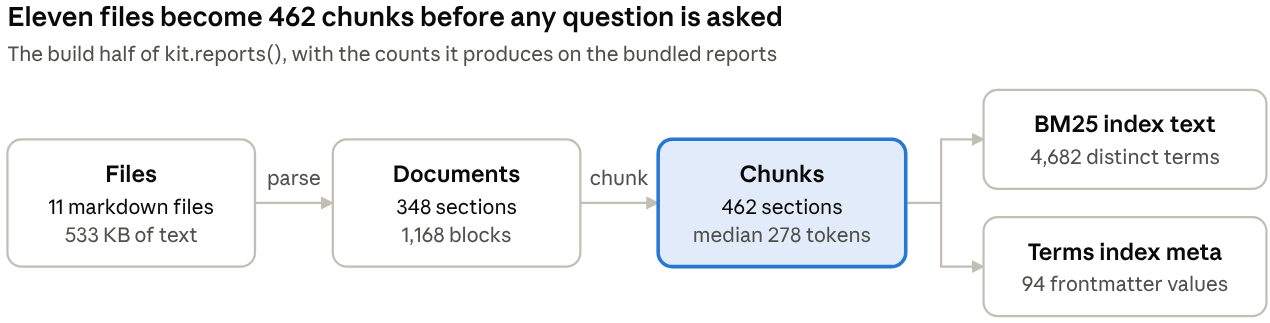
Part II · Text into structure

What can be found depends on how text was cut and counted before anyone asked a question. This part builds the corpus side of the pipeline: documents, chunks and indexes.

3. Parsing markdown into documents

This chapter explains how a markdown file becomes a *document*: a JavaScript object with an id, a title, a metadata record, an ordered list of sections, and a list of links to other documents. Every later stage reads these fields and never the raw file, so any distinction the parser does not make is a distinction no later stage can use. By the end of the chapter you should be able to predict, for any line in a report, which field of which section it ends up in.

The corpus side of every pipeline in this book has the same four stages. Files are parsed into documents, documents are cut into chunks, and chunks are indexed. Part II takes them in that order, one chapter each, and the picture below gives the counts each stage produces on the eleven bundled reports.



the corpus side of the pipeline · files, documents, chunks, indexes

In the grammar, parsing is one call in a corpus chain:

```
const corpus = rag.corpus('reports')
  .from(rag.files('*.md'))      // source.files: every uploaded file whose
name matches
  .parse(rag.markdown())        // parse.markdown: one document per file
  .chunk(rag.sections({ depth: 3, maxTokens: 450 }))
  .index('text', rag.bm25());
```

The parser is also exported on its own, which is how the outputs in this chapter were produced:

```
const doc = rag.util.parseMarkdown({ name: file.name, text: file.text });
```

For the CoinVault report, the document looks like this, with long arrays abridged:

```
{
  id:    'project-report-coinvault-mcp-from-local-conformance-to-chatgpt-2',
  title: 'PROJECT REPORT - CoinVault MCP - From Local Conformance to ChatGPT
2026-07-28',
  h1:    'CoinVault MCP: From Local Conformance to ChatGPT 2026-07-28',
  meta:  { type: 'article', status: 'active', tags: [ 'article', 'project-
report', 'coinvault', … ],
           aliases: [ 'CoinVault ChatGPT MCP acceptance', … ], tickets: [ …
], … },
  sections: [ /* 33 sections */ ],
  links: [ 'PROJECT REPORT - CoinVault Remote MCP - Official Go SDK and
Employee OAuth Authorization' ]
}
```

Two details are worth noticing before the sections that follow. The `id` is a slug of the *file name*, cut to 64 characters, which is why it ends in `-2`: the cut falls in the middle of `2026`. Renaming the title in the frontmatter leaves the id alone; renaming the file changes it, and with it every chunk id derived from it (section 4.5). The `title` comes from the frontmatter `title` field when there is one, then from the first `#` heading, then from the file name. Here the frontmatter wins, so the friendlier H1 is kept separately as `h1`.

3.1 Frontmatter as fields, not text

A report begins with a block of YAML between two `---` lines. The CoinVault report's begins like this:

```
---
title: "PROJECT REPORT - CoinVault MCP - From Local Conformance to ChatGPT
2026-07-28"
aliases:
  - CoinVault ChatGPT MCP acceptance
  - CoinVault hosted OAuth debugging
  - CoinVault MCP 2026-07-28
status: active
type: article
```

```
created: 2026-09-03
repo: /home/manuel/workspaces/2026-08-28/coinvault-oidc-mcp/coinvault
tickets:
  - CV-MCP-ACCEPT-001
  - CV-OAUTH-OBS-001
tags:
  - article
  - project-report
  - coinvault
  - mcp
  - oauth
  …
---
```

The parser reads this block into `doc.meta` and removes it from the body. It does not leave it in the text, and the reason is a matter of counting. If `type: article` and the eleven tags stayed in the body, the words *project*, *report* and *article* would appear in the first chunk of nearly every report, and a lexical score would treat them as evidence. Worse, the evidence would be attached only to the first chunk of each document, because only that chunk contains the frontmatter. As fields, the same information can do what it is good at: filter a result list, count facets and match a question against a closed vocabulary (chapter 6).

The parser understands a small subset of YAML, chosen to cover what notes tools write:

Form in the file	Parsed value
`status: active`	the string `'active'`
`title: "PROJECT REPORT - …"`	the string without its quotes
`tags: [a, b]`	the array `['a', 'b']`
a key with no value, followed by `- item` lines	an array of the items
`publish: false`, `count: 12`	the boolean `false`, the number `12`
an indented `key: value` under another key	nothing: the line is dropped (section 3.5)

Every chunk carries a copy of its document's metadata, so that a filter can test a chunk without looking up its document. The copy is not complete. `pickMeta` drops the keys `title`, `created`, `updated`, `source_ticket_path`, `source_design_doc`,

`related_vault_notes` and `derived-from`, and any string longer than 120 characters, because those values identify a file rather than describe its content and would only add noise to facet lists. The first chunk of the order-grain TTC report carries this:

```
{ "aliases": ["TTC Customer Clustering", "thetreecenter.com segmentation"],
  "tags": ["project", "r", "shiny", "clustering", "marketing"],
  "status": "active", "type": "project",
  "repo": "/home/manuel/code/gec/2026-09-03--marketing-analysis", "publish":
false }
```

3.2 The heading stack and section paths

A section is the run of blocks between one heading and the next. What makes it useful is its *path*: the titles of every heading it sits under, from the outermost in. The parser computes paths with a stack of open headings. When it reads a heading of level *L*, it pops every open heading whose level is *L* or deeper, pushes the new one, and records the stack's titles as the new section's path.

```
Pseudocode: the heading stack

stack = []
for each heading (level L, title T) in the file, in order:
    if this is the first level-1 heading: remember T as h1; continue
    while stack is not empty and top(stack).level >= L: pop(stack)
    push(stack, {level: L, title: T})
    start a new section with path = titles of stack
```

Text before the first heading becomes a section with level 0 and an empty path, which the chunker later names `intro`. The first H1 is not a section at all: it is the document's heading, and treating it as a section would put the document title at the front of every path. Heading titles are passed through `plain()`, which strips backticks, bold markers and link syntax, so a heading written with inline code, such as `## 4.1 The `typed-nil` deployment failure` gets the path element `4.1 The typed-nil deployment failure`.

These are the first ten sections of the CoinVault report, with the kinds of blocks each one holds:

```
0  []
paragraph,paragraph,paragraph,callout
2  ["1. The acceptance problem"]
paragraph,list,paragraph
3  ["1. The acceptance problem","1.1 Exact deployed identity"]
paragraph,table,paragraph
2  ["2. Architecture at acceptance"]
paragraph,fence,paragraph
3  ["2. Architecture at acceptance","2.1 Repository responsibilities"]
table,paragraph
2  ["3. Local proof before hosted debugging"]
paragraph,paragraph,fence,paragraph,…
2  ["4. Deploying a stable public MCP service"]
paragraph,paragraph,fence,paragraph
3  ["4. Deploying a stable public MCP service","4.1 The typed-nil deployment
failure"]  paragraph,fence,…
2  ["5. Making safe failures diagnosable"]
paragraph,fence,paragraph,paragraph,fence,paragraph
2  ["6. OAuth discovery and registration incompatibilities"]   (no blocks)
```

The last line shows a rule that is easy to miss. Section 6 has a heading and no text of its own: its content lives entirely in 6.1 and 6.2. The parser keeps it anyway, because a section with a heading is still a node in the outline and its title is still part of its children's paths. Only a level-0 section with no text is dropped. Across the corpus the parser produces 348 sections: 11 intros, 173 at level 2 and 164 at level 3. No report uses level 4 or deeper.

The path matters later in three ways. It is a field the BM25 index weights three times as heavily as body text, because a heading is the author's own summary of what follows (chapter 5). It is the second half of every result's label, `title › path`, which is how a reader decides whether to open a hit. And it is what golden cases match against when they check whether the right section was found (chapter 18), which is why evals survive changes to chunking.

3.3 Blocks: paragraph, list, fence, table, callout

Inside a section, the parser divides lines into blocks. It recognises a block by its first line and then consumes lines until the block ends:

Kind	Starts with	Ends at	In the corpus	Used later for
fence	``` or ~~~, optional language	the same marker	195	atomic during chunking; code chunks; `hasCode`
table	a line starting with `|`	the first line not starting with `|`	51	atomic during chunking; one chunk per row
callout	`> [!variant]`	the last `>` line	10	its own BM25 field, `callout`
quote	`>` without `[!…]`	the last `>` line	1	ordinary text
list	`-`, `*`, `+` or `1.`	a line that is neither an item nor indented	106	split by item, never mid-item
paragraph	anything else	a blank line or another block	805	split by sentence

Fences are checked first, so a `#` line inside a code block is never mistaken for a heading. Tables keep both their raw text and a parsed form, a header row and data rows, with each cell passed through `plain()`. The last table in the CoinVault report parses to:

```
{ kind: 'table',
  header: [ 'Visible symptom', 'Actual cause', 'Corrective change' ],
  rows: [ [ 'Browser ECS never healthy', 'Typed nil verifier activated RAG
branch', 'Return a genuinely nil verifier interface' ],
          [ 'DCR returned 503', 'Legacy SQLite lacked last_used_at, then
payload', 'Add migration support; reset explicitly disposable first-dev
state' ],
          … ] }
```

Keeping the parsed form is what lets the *Table rows as chunks* preset turn every row into a chunk that reads `Visible symptom: DCR returned 503` and so on, without parsing the table a second time.

Fences keep their language tag. In this corpus that is `ts` 80 times, `text` 43, `js` 27, `mermaid` 19, and a handful of `go`, `json`, `r`, `bash` and others. The language becomes a facet in the *Code search* preset, and `mermaid` fences are dropped by the default chunker,

because a diagram's source is mostly arrows and node ids that match nothing a person would type.

Callouts are Obsidian's `> [!summary]` blocks. All ten in this corpus are summaries at the top of a report, and each is a list of bolded findings. The CoinVault one begins: *"Acceptance was staged by boundary. Loopback transport, deterministic OIDC, real domain tools, …"*. The chunker copies callout text into a separate `callout` field of the chunk, so an index can weight an author's summary above ordinary text (section 5.5).

3.4 Links between documents: wiki links and related notes

Reports refer to each other with wiki links, `[[Title of another note]]`. The parser collects every link target in the file, including those in the frontmatter's `related_vault_notes`, strips any `#heading` or `|label` part, and stores the distinct targets in `doc.links`. It does not resolve them: a link is only a string at this point.

Resolution happens once all documents exist, when the engine assembles a build's artifacts. A target matches a document if, after lowercasing, removing a `PROJECT REPORT -` or `PROJ -` prefix and collapsing punctuation, it equals that document's title, file name or one of its aliases. Targets that match nothing are dropped. The Datalab report links to six notes; two of them are in this corpus, so it resolves to two neighbours:

```
datalab         → workbench-core, workbench-stabilization
identity        → workbench-core, workbench-stabilization
workbench-core  → workbench-stabilization
stabilization   → workbench-core, datalab
(the other seven reports resolve to no links)
```

The CoinVault report's one link names a different CoinVault report that was never uploaded, so it resolves to nothing. That is the normal case for a set of files exported from a larger vault, and the code treats it as normal. The resolved graph is what `expand.links` follows when it adds sections from linked reports to an answer's context (chapter 15).

3.5 What the parser gets wrong, and why that is acceptable here

The parser is about 90 lines of line-by-line pattern matching, not a CommonMark implementation. This test file shows its main gaps:

```
---
title: Edge cases
owner:
  name: Ada
  team: search
tags: [a, b]
---
# Edge cases

Setext heading
--------------

- item one
lazy continuation line

> [!note] Heads up
> second line
```

It parses to a document whose `meta` is `{ title: 'Edge cases', tags: ['a', 'b'] }` and whose only section holds these blocks:

```
paragraph  "Setext heading\n--------------"
list       "- item one"
paragraph  "lazy continuation line"
callout    note: "Heads up\nsecond line"
```

Three things went wrong. The nested `owner` mapping vanished without an error, because indented `key: value` lines are not part of the supported subset. The underlined *setext* heading became a paragraph, so the section structure is flatter than the author intended. And the unindented line after the list item, which CommonMark treats as a continuation of the item, became a separate paragraph. The callout, by contrast, came through correctly.

These gaps are acceptable for this corpus because none of them occurs in it. A scan of the eleven reports finds no setext headings, no nested frontmatter keys and exactly one H1 per file; the reports come from one notes tool that writes one dialect. The decision rests on a principle that applies to every stage of Part II: a parser error matters only when it changes what can be retrieved. A dropped `owner.team` would matter if someone built a facet on it, and a setext heading would matter because its section would vanish from every path. If you upload files from another source, open the Corpus tab and compare a few documents' sections with the files before trusting any score.

The key points to take from this chapter:

- A document is the parser's whole output, and later stages read only its fields: `id`, `title`, `meta`, `sections` and `links`.
- Frontmatter becomes fields so that it can filter and facet without inflating word counts in the first chunk of each report.
- A section's path is computed with a stack of open headings, and the first H1 is the document's heading rather than a section.
- Fences, tables and callouts are recognised as kinds because each is used differently later: kept whole, split into rows, or weighted as a summary.
- Links are stored as strings at parse time and resolved against titles, file names and aliases only when all documents exist.
- The parser handles one dialect of markdown, and its gaps are acceptable only because the corpus was checked for them.

The next chapter cuts these sections into chunks, the unit that every later stage ranks, cites and scores.

Lab: Minimal search. Load the preset and open the Corpus tab in the inspector. Find the CoinVault report and compare its section list with the ten lines in section 3.2. Then search for `typed nil` and check that the result's label shows the path `4. Deploying a stable public MCP service › 4.1 The typed-nil deployment failure`.

4. Chunking decides what can be found

This chapter explains how documents are cut into *chunks*, and why that cut sets an upper bound on everything that follows. A chunk is the unit the index scores, the unit a result list shows, the unit the answer cites and the unit an eval checks. No later stage can return half a chunk or join two, so a fact split across two chunks has to be found twice, and a fact buried in a large chunk has to compete with everything else in it. By the end of the chapter you should be able to read a chunker's parameters and predict roughly how many chunks it produces, how large they are and what their ids will be.

A chunk is a plain object. The default chunker produces chunks with these fields:

Field	Example	Used by
`id`	`project-report-coinvault-…-chatgpt-2#4-1-the-typed-nil-deployment-failure`	fusion, neighbours, the UI, citations
`docId`, `title`	the parent document's id and title	labels, filters, link expansion
`headingPath`	`['4. Deploying a stable public MCP service', '4.1 The typed-nil deployment failure']`	BM25 field, labels, evals
`text`	the section's blocks joined back into markdown	BM25 field, answers
`callout`	text of any `[!summary]` callouts	BM25 field
`meta`	the document's metadata, minus file-only keys	filters, facets, the terms index
`tokens`	`240`	packing budgets, statistics
`part`, `parts`	`1`, `1`	splitting, ids
`hasCode`, `hasTable`, `kind`	`true`, `false`, `'section'`	filters, rerank heuristics

4.1 Sections as the default unit

The default chunker, `rag.sections()`, starts from the sections the parser found. The reason is that a section is a unit the author chose. A report's author put a heading where the topic changed, so a section usually answers one question, and its heading path is a ready-made label for it. A fixed window of 450 tokens would cut wherever the count ran out, often in the middle of the explanation a reader needed.

The `depth` parameter decides which headings start a new chunk. A section at level `depth` or shallower starts a new group; a deeper section is appended to the group before it, with its heading kept as a line of text. With the default `depth: 3`, every H2 and H3 in these reports starts a chunk. With `depth: 2`, each H3 folds into its H2. For the CoinVault report that turns section 4 and its subsection 4.1 into one group of 467 tokens, which is then split into two parts:

```
depth 3:  …-chatgpt-2#4-deploying-a-stable-public-mcp-service          218
tokens
          …-chatgpt-2#4-1-the-typed-nil-deployment-failure             240
tokens
depth 2:  …-chatgpt-2#4-deploying-a-stable-public-mcp-service          366
tokens  part 1/2
          …-chatgpt-2#4-deploying-a-stable-public-mcp-service-2        101
tokens  part 2/2
```

The folded chunk still contains the words *typed-nil*, so a lexical search still finds it. What it has lost is the heading path `… › 4.1 The typed-nil deployment failure`, and that loss is larger than it looks. The heading path is a weighted field in the index (chapter 5), it is the label a reader scans, and it is what the golden cases test. On the twelve goldens, `depth: 2` drops hit@5 from 0.917 to 0.667. All four cases that now miss expect a section that is an H3, such as *What does the rebalance law say?*, which expects `5.9 The rebalance law`: the right text is retrieved, but no chunk is labelled with that section any more. A chunker can therefore make a pipeline look worse to its evals without making it worse at retrieval, and the cure is to read misses, not only scores (chapter 13).

4.2 Token budgets, atomic blocks and the short tail

A section can be longer than anyone wants to read or rank as one unit, so the chunker splits groups that exceed `maxTokens`. Token counts here are an estimate, `Math.ceil(characters / 4)`, not the output of a model's tokenizer. The estimate is cheap and stable, and the budget only needs to be roughly right: 450 estimated tokens is about 1,800 characters, or a long screen of prose.

The split works on blocks, not characters:

```
Pseudocode: splitting one group

units = []
for each block in the group:
    if tokens(block) <= max:                          keep it whole
    else if block is a fence or table
         and tokens(block) <= 3 * max:                keep it whole (atomic)
    else if block is a fence, table or list:          split it into runs of
lines
    else:                                             split it into runs of
sentences
parts = pack units greedily, starting a new part when the next unit would
pass max
if there are 2+ parts and the last has fewer than minTokens (30):
    merge the last part into the one before it
```

Each rule protects something. Code and tables are allowed to exceed the budget up to three times, because half a code sample or a table without its header row is wrong rather than merely short. Lists split between items and paragraphs between sentences, so no part ends mid-sentence. The short-tail rule prevents a final fragment of a dozen words from becoming a chunk of its own, where it would rank badly and cite worse.

The budget has a large effect on the number and size of chunks. These are the real counts for the eleven reports, with `depth: 3`:

`maxTokens`	Chunks	Median tokens	90th percentile	Largest	Over budget
200	873	144	197	568	66
450	462	278	423	743	8
900	342	322	699	901	1

The total is about 125,000 estimated tokens in every row; only the cut points move. Doubling the budget from 450 to 900 removes only a quarter of the chunks, because most sections are already shorter than 450 tokens: the budget only affects the sections that exceed it. The *over budget* column counts chunks kept whole by the atomic rule. The largest at 450 is a 743-token table in the Workbench Core report's section *2. The problem*.

At 200 tokens, the typed-nil section from section 4.1 above becomes two chunks of 139 and 101 tokens, the second with the id suffix `-2`. A question about the restart loop now has two candidate chunks, each holding part of the explanation. Whether that is better

depends on the question. On the goldens, run with the *Chunk sizes* preset's neighbour expansion, 200 tokens moves one case from rank 6 to rank 2 and two others from rank 1 to 3, while 900 tokens gives the best MRR of the three:

Variant	hit@5	MRR	nDCG@10
450 (base)	0.917	0.931	0.867
200-tok	1.000	0.903	0.746
900-tok	1.000	0.958	0.903
h2-only	0.667	0.563	0.509
overlap-40	1.000	1.000	0.871

Twelve questions are too few to choose a budget from these numbers alone; one case moving five places changes MRR by about 0.07. Chapter 19 makes that argument precise. What the table does show is that chunk size is a parameter to measure, not a constant to look up.

4.3 Overlap and heading prefixes

Two options change what goes *into* a chunk rather than where it is cut. `overlap: n` prefixes each part after the first with the last *n* estimated tokens of the part before it, about 4*n* characters. Its purpose is to keep a sentence that introduces an idea next to the sentence that completes it when a split falls between them. The cost is visible in the text. This is the start of the second part of a TTC section with `overlap: 40`:

```
es: when flags are produced by a join that can fail, classify by the failure
mode you can
detect, and never let an empty flag mean "none of the above" silently.
Plant categories come from keyword matching on genus names (maple, spruce, …
```

The overlap is measured in characters, so it begins in the middle of a word. Overlap also duplicates text: the same sentences now appear in two chunks, and a query matching them scores both. With `overlap: 40` the corpus grows from 124,977 to 130,491 estimated tokens and 24 chunks exceed the budget instead of 8. On the goldens it happens to produce the only perfect MRR in the table above, which is a reason to keep it as a variant and test it on more questions, not a reason to make it the default.

`prefixHeading: true` writes the heading path as the first line of the chunk's text:

```
4. Deploying a stable public MCP service › 4.1 The typed-nil deployment
failure
The first shared-image deployment left MCP healthy but put the browser
service into a restart loop. …
```

This matters for consumers that only see `text`: an answer prompt, a reranker or a rewrite step that reads the chunk as plain prose. The BM25 index does not need it, because it already indexes `headingPath` as a separate, weighted field, and prefixing would count those heading words a second time at text weight. The default leaves it off; the Chunk sizes preset suggests it as a variant to measure.

4.4 Several chunk streams over one corpus: rows, code, cards

A corpus is not limited to one chunker. Each `.chunk()` call adds a chunk stream, each `.index()` call can name the stream it indexes with `{ from }`, and a plan can retrieve from several indexes and fuse the results (chapter 14). This is how one set of documents serves questions of different shapes. The *Table rows as chunks* preset adds a stream with one chunk per table row:

```
const corpus = kit.reports()
  .chunk('rows', (doc, ctx, p) => doc.sections.flatMap(sec => sec.blocks
    .filter(b => b.kind === 'table' && b.rows.length >= p.minRows)
    .flatMap(t => t.rows.map(row => ({
      kind: 'row',
      headingPath: [...sec.path, row[0]],
      text: t.header.map((h, j) => `${h}: ${row[j] ?? ''}`).join('\n'),
    })))), { minRows: 2 })
  .index('rows', rag.bm25({ fields: { headingPath: 2, text: 1 } }), { from:
'rows' });
```

The lambda receives a parsed document and returns plain objects; the chunker fills in `docId`, `title`, `meta`, `tokens` and the id. For the question *which HTTP status does ErrConflict map to*, the top hit is now a chunk that contains exactly the answer:

```
{ id: 'project-report-agentforum-web-a-protobuf-contract-an-http-adapte#row-
errconflict',
  kind: 'row',
  headingPath: [ 'The HTTP server', 'Adapter discipline', 'ErrConflict' ],
  text: 'Service sentinel: ErrConflict\nHTTP: 409\nError.code: conflict' }
```

In the section stream the same fact is one row in a 300-token section about adapter discipline, competing with every other row. The other chunking presets apply the same idea to other shapes:

Preset	Stream	Chunks	Built from
Table rows as chunks	`rows`	350	one per row of tables with 2+ rows
Code search	`code`	145	one per fence of 3+ lines, except mermaid, with `meta.lang`
Outline search	`outline`	337	one per heading: its path and its first sentence
Which report?	`docs`	11	one card per document: title, tags, summary callout, H2 titles

Each stream answers a different kind of question. Rows answer lookups of a single value. Code chunks answer questions phrased in identifiers. Outline chunks answer *where is X discussed?* and are small enough to scan as a list. Document cards answer *which report covers X?*, where returning a section would be too specific. Building them costs little: the four streams together took under 100 ms to build in node.

4.5 Stable chunk ids, and what depends on them

A chunk id is built from the document id and the slug of the last heading in the chunk's path: `docId#slug`. A section before the first heading uses `intro`. A chunk from a stream whose kind is not `section` puts the kind in front, as in `#row-errconflict` or `#heading-10-3-the-migration-note`. When two chunks would get the same id, which happens to the parts of a split section and to sections with identical headings, the second gets `-2`, the third `-3`, in reading order. The CoinVault intro is split into two parts, `#intro` and `#intro-2`.

The point of this scheme is that an id is predictable from the document and its outline, and does not change when unrelated text changes. Editing a paragraph in section 7 leaves every id in the document as it was. Changing the chunker's parameters is different: at 200 tokens the typed-nil section gains a `-2` chunk that does not exist at 450, and at `depth: 2` the id `#4-1-the-typed-nil-deployment-failure` disappears altogether.

Several parts of the workbench rely on ids, and it helps to know which:

- Fusion merges hits from different indexes by id. Two indexes built over the same stream share ids, so a section found by both is counted once with both ranks (chapter 14).
- Neighbour expansion uses each document's reading order of ids to add the section after a hit (chapter 15).
- The UI's selected chunk, citation links in answers, `ref.chunk(id)` links written by the agent and `rag.explain(result, id)` all name a chunk by id, so they break when a parameter change renames it.
- Golden cases deliberately do not use ids. They match the expected document and section with patterns against titles and heading paths, so a case written at 450 tokens still works at 200. As section 4.1 showed, they do depend on heading paths.
- The engine's per-item build cache is keyed by a hash of the whole input item and the step's parameters. A chunk whose id or text changed is a different item, so a per-chunk enrichment step computes it again (chapter 11).

The key points to take from this chapter:

- The chunk is the unit of ranking, display, citation and evaluation, so the chunker sets the ceiling for every later stage.
- The default chunker groups by heading up to `depth` and splits by an estimated token budget, keeping code and tables whole up to three times the budget.
- Most sections in this corpus are shorter than 450 tokens, so the budget changes the chunk count less than its size suggests.
- Folding headings into their parents removes them from the heading path, which hurts the label, the index and the evals at once.
- Overlap and heading prefixes add text to chunks; overlap duplicates evidence and heading prefixes duplicate heading words already indexed as a field.
- Several chunk streams over one corpus let one pipeline answer lookups, code questions, outline questions and document questions.
- Chunk ids are predictable from the outline and stable under text edits, but not under changes to chunking parameters.

The next chapter turns chunks into scores: what a BM25 index counts, and why its formula has the shape it does.

Lab: Chunk sizes, Table rows as chunks, Code search, Outline search, Which report? Load *Chunk sizes*, open the Evals tab, tick the four variants and run them; you should see the table in section 4.2. Then open the Misses list for `h2-only` and confirm that the

rebalance-law case retrieves the right text under the wrong label. In *Table rows as chunks*, search `which HTTP status does ErrConflict map to` and compare the row chunk with the section chunk that contains it. In *Outline search*, search `migration` and notice that every result is one line long.

5. BM25 from first principles

This chapter derives the score the workbench uses to rank chunks against a query, and then checks one score by hand against the engine. BM25 is a formula from the 1990s that still sets the baseline in lexical search, and it is worth knowing in detail because every number the Trace tab shows for a lexical retriever is one of its terms. The formula combines three ideas: a term that appears in few chunks is better evidence than one that appears in many; each further occurrence of a term in a chunk adds less than the one before; and a long chunk should not win merely because it has room for more words. By the end of the chapter you should be able to look at a hit's `why` breakdown and say which of those three ideas produced each number.

The score of a chunk *d* for a query *q* is a sum over the query's terms:

```
score(d, q) = Σ over terms t in q of   idf(t) · tf(t,d) · (k1 + 1) / (
tf(t,d) + k1 · (1 − b + b · len(d) / avglen) )

idf(t) = ln( 1 + (N − df(t) + 0.5) / (df(t) + 0.5) )
```

Here *N* is the number of chunks in the index, *df(t)* is the number of chunks containing *t*, *tf(t,d)* is how often *t* occurs in *d*, *len(d)* is the chunk's length in terms, and *avglen* is the mean length. The two constants are `k1 = 1.2` and `b = 0.75`, the engine's defaults and the values most systems use. The rest of the chapter takes the pieces in order: what counts as a term, then the tf part, the length part, the idf part, and finally how fields change *tf* and *len*.

5.1 Tokens, stop words and a small stemmer

Before anything can be counted, text must be cut into terms. The engine's `tokenize` function does four things, and it does exactly the same four things to chunks at build time and to queries at search time. That symmetry is the whole point: a query term can only match an indexed term that went through the same transformation.

1. It finds runs of letters, digits and underscores. Everything else, including `/`, `-`, `.` and backticks, separates terms.

2. It splits each run at underscores and at lowercase-to-uppercase boundaries, and also keeps the joined form. `parseMarkdown` yields `parse`, `markdown` and `parsemarkdown`; `next_cursor` yields `next`, `cursor` and `nextcursor`. A query can then match an identifier by its parts or by its whole name.
3. It lowercases each term, drops single letters (but not single digits) and drops 98 stop words such as *the*, *what*, *does* and *why*.
4. It stems what is left with a short list of suffix rules.

These are real outputs:

```
tokenize('What does the rebalance law say?')   → [ 'rebalanc', 'law', 'say' ]
tokenize('parseMarkdown next_cursor HTTP2')    → [ 'pars', 'markdown',
'parsemarkdown', 'next', 'cursor', 'nextcursor', 'http2' ]
tokenize('ChatGPT')                            → [ 'chat', 'gpt', 'chatgpt' ]
tokenize('ErrConflict 409')                    → [ 'err', 'conflict',
'errconflict', '409' ]
```

The stemmer is not Porter's algorithm. It removes plural endings (*-ies* becomes *-y*, *-es* after *x*, *ch*, *sh* and *z*, a final *-s* in most other cases), then *-ing*, *-ed* and *-ly* on long enough words, then a doubled final consonant and a final *-e*. The aim is to make *rebalance*, *rebalanced* and *rebalancing* the same term, and it does:

```
rebalance → rebalanc    rebalancing → rebalanc    rebalanced → rebalanc
policies → policy       classes → class           running → run
quickly → quick
laws → laws             tags → tags               tokens → token
```

The last line shows the stemmer's main limitation. The plural rule only applies to words longer than four letters, so *laws* and *tags* keep their *s*, and a query for *laws* does not match a chunk that only says *law*. The length limit exists to protect short words such as *bus*, *gas* and *this* from being cut; the price is a few short plurals that fail to merge. A larger stemmer would fix this case and introduce others, and the place to measure the trade is the evals, not intuition.

5.2 Term frequency and saturation (k1)

The simplest score would count how often each query term appears in the chunk. The problem is that the tenth occurrence of a word is much weaker evidence than the first. The first tells you the chunk is about the topic; the tenth mostly tells you the chunk is long

or repetitive. BM25 handles this by passing the count through a function that rises quickly and then levels off:

```
tf · (k1 + 1) / (tf + k1)          with k1 = 1.2 and no length correction
```

tf	1	2	3	5	10	20
contribution	1.000	1.375	1.571	1.774	1.964	2.075

The contribution can never exceed *k1 + 1*, which is 2.2. *k1* controls how quickly the curve flattens: a small *k1* makes one occurrence nearly as good as many, and a large *k1* makes the score closer to a raw count. One practical consequence is that a chunk which mentions every query term once usually beats a chunk which mentions one query term many times, because the second chunk's extra occurrences are worth little and its missing terms are worth nothing.

5.3 Length normalization (b)

A long chunk has more words, so it contains any given term more often by chance. Without a correction, ranking would favour the longest sections in the corpus. BM25 corrects by making *k1* effectively larger for chunks longer than average, which flattens their curve:

```
K(d) = k1 · (1 − b + b · len(d) / avglen)
contribution = tf · (k1 + 1) / (tf + K(d))
```

With *b* = 0.75 and a term that occurs five times, the contribution depends on length like this:

len / avglen	0.5	1.0	2.0
K(d)	0.75	1.20	2.10
contribution for tf = 5	1.913	1.774	1.549

The parameter *b* sets how strongly length counts. At *b* = 0 length is ignored; at *b* = 1 a chunk twice the average length needs roughly twice the occurrences for the same contribution. Chunking (chapter 4) and *b* interact: a corpus of uniformly sized chunks

barely needs length normalization, while a corpus mixing one-line outline chunks with 700-token tables needs it a great deal. The *Field weights* preset includes a `b-0.3` variant for this reason.

5.4 Inverse document frequency

The last idea is that matching a rare term is better evidence than matching a common one. In this corpus *workbench* appears in 269 of the 462 chunks, so a chunk containing it has told you very little. *law* appears in 7, so a chunk containing it is one of very few candidates. The idf factor turns that intuition into a weight:

```
idf(t) = ln( 1 + (N − df + 0.5) / (df + 0.5) )        with N = 462
```

Term	df	idf
law	7	4.123
oauth	22	3.024
rebalanc	24	2.939
agent	139	1.200
workbench	269	0.541
a term in half the chunks	231	0.693
a term in 400 chunks	400	0.145

The `1 +` inside the logarithm is a variant used by Lucene and most modern systems. The original formula has no such term and goes negative for a term that appears in more than half the chunks, which would make matching a common word *lower* a chunk's score. With the `1 +`, a common term contributes a small positive amount and never a penalty. The table also explains why stop words are removed: *the* would have df close to 462 and an idf close to zero, so removing it costs almost nothing in ranking and saves a large posting list.

5.5 Fields and weights: title, heading path, callout, text

So far a chunk has been a single bag of words. The workbench's chunks have structure: a document title, a heading path, an optional summary callout and the body text. A word in a heading is stronger evidence than the same word in a paragraph, because the author used

the heading to say what the section is about. The index handles this with field weights, in the style of BM25F: before scoring, each field's term counts are multiplied by the field's weight, and the chunk's length is the weighted sum of its field lengths.

```
rag.bm25({ fields: { title: 2, headingPath: 3, callout: 2, text: 1 }, k1:
1.2, b: 0.75 })
```

These are the weights `kit.reports()` uses. The defaults, without `callout`, are `{ title: 2, headingPath: 3, text: 1 }`. A field can be any path into the chunk, including `meta.lang`, which the *Code search* preset weights at 2 so that a query containing *ts* or *go* prefers code in that language.

Weights change rankings in ways that are easy to predict for one query and hard to predict across many. For *opaque server revision token*, the weighted index ranks first the Identity section titled *5. Server revision is an opaque concurrency token*, because four query terms appear in its heading at weight 3. With `fields: { text: 1 }`, that section falls out of the top three, and *2. The three-domain identity model* leads. Across the twelve goldens, the same change has mixed effects:

Variant of the *Field weights* preset	hit@5	MRR	nDCG@10
title 2, headingPath 3, callout 2, text 1	1.000	0.933	0.847
text 1 only	1.000	1.000	0.809
headingPath 6	1.000	0.938	0.842
b 0.3	1.000	0.938	0.829

Text-only puts the first relevant section at rank 1 for all twelve questions, while the weighted index ranks one of them fifth; but the weighted index places more of the relevant sections near the top, which is what nDCG rewards. Neither is simply better. Chapter 19 explains how to read a disagreement between metrics.

The title weight deserves one more remark. Every chunk of a document carries the same title, so a query word that appears in a title raises *every* section of that report. That is useful for a question that names a project and harmful for a question that happens to share a word with an unrelated report's title.

5.6 Scoring “rebalance law” by hand, then checking it against the engine

The query *rebalance law* comes from the golden case *What does the rebalance law say?*, after tokenization has removed *what*, *does* and *the* and kept *say*. Using the two-word form keeps the arithmetic short. The index is the one `kit.reports()` builds: 462 chunks, 4,682 distinct terms, mean weighted length 168.004.

The engine's top hit is the Workbench Core section *5. The core's modules › 5.9 The rebalance law*, with a score of 12.6758 and this breakdown:

```
{ id: 'project-report-pbui-workbench-core-a-headless-engine-a-pure-plan#5-9-
the-rebalance-law',
  score: 12.6758,
  why: { law: 7.08, rebalanc: 5.59 } }
```

To reproduce it, first count the chunk's terms per field. The title has 11 terms, the heading path 7, the callout none and the text 163. Its weighted length is therefore:

```
len = 2·11 + 3·7 + 2·0 + 1·163 = 206
```

Then count each query term per field and weight the counts:

Term	in title	in heading path	in text	weighted tf	df
law	0	1	2	3·1 + 1·2 = 5	7
rebalanc	0	1	6	3·1 + 1·6 = 9	24

The length factor is shared by both terms:

```
K = 1.2 · (1 − 0.75 + 0.75 · 206 / 168.004) = 1.2 · 1.16962 = 1.40354
```

Now each term's contribution:

```
law:       idf = ln(1 + (462 − 7 + 0.5) / (7 + 0.5))   = ln(61.733) = 4.1228
           tf part = 5 · 2.2 / (5 + 1.40354)          = 11 / 6.40354 =
1.71780
           contribution = 4.1228 · 1.71780            = 7.0822

rebalanc:  idf = ln(1 + (462 − 24 + 0.5) / (24 + 0.5)) = ln(18.898) = 2.9391
           tf part = 9 · 2.2 / (9 + 1.40354)          = 19.8 / 10.40354 =
1.90320
           contribution = 2.9391 · 1.90320            = 5.5937

score = 7.0822 + 5.5937 = 12.6759
```

The engine reports 12.6758; the difference in the fourth decimal comes from rounding the intermediate values above. Three observations follow from the arithmetic, one for each idea in the formula. *law* contributes more than *rebalanc* even though it occurs fewer times, because it is rarer: idf 4.12 against 2.94. *rebalanc* occurs nearly twice as often as *law* but its tf part is only 11 percent higher, which is saturation at work. And the chunk is 23 percent longer than average, which raises *K* from 1.2 to 1.40 and costs both terms a little. The heading path supplies 3 of the 5 weighted occurrences of *law*; without field weights the same chunk still ranks first, but with a score of 10.32 instead of 12.68 and a narrower margin over the second hit.

The second hit, *1. Project status* (part 2) of the same report, scores 7.39: it mentions both terms in passing, in a longer chunk with no heading match. That gap of more than five points between first and second is what a confident lexical match looks like. When the gap is small, the trace is telling you that BM25 could not tell the candidates apart, and the later stages of Part V exist for that case.

The key points to take from this chapter:

- The same tokenizer runs at build time and at query time, so a query term matches only what went through the same splitting, lowercasing, stop-word removal and stemming.
- The tf part saturates at k1 + 1, so repeated occurrences add less and less, and covering every query term matters more than repeating one.
- Length normalization raises the saturation constant for long chunks, with b setting how strongly length counts.
- idf weights rare terms above common ones, and the Lucene-style `1 +` keeps every matching term's contribution positive.
- Field weights multiply term counts and lengths per field, which lets headings and summaries count more than body text.

- A hit's `why` object lists each term's contribution, so any lexical score in the workbench can be decomposed and checked.

The next chapter turns to the part of each document that BM25 does not score: the frontmatter fields, which filter and facet rather than rank.

Lab: Field weights. Load the preset, search `rebalance law` and open the Trace tab: the `why` values on the first hit should be the 7.08 and 5.59 computed above. Then open Evals, tick `text-only`, `headings-x6` and `b-0.3`, and run them to reproduce the table in section 5.5. As an exercise, add a variant that sets `k1` to 2.0 and predict, before running it, whether the rebalance-law section's margin over the second hit grows or shrinks.

6. Metadata as an index

This chapter is about the frontmatter fields that chapter 3 kept out of the text. They describe a document rather than say anything in it: its type, its status, its tags, the other names people use for it. Those values form a small, closed vocabulary, and a closed vocabulary answers different questions than prose does. It can filter a result list exactly, it can be counted to show what a result list contains, and it can recognise a project by name. It ranks badly, because every section of a document carries the same values. By the end of the chapter you should know which of those jobs to give metadata in a pipeline, and what goes wrong when it is given the wrong one.

6.1 A terms index over type, tags and aliases

`kit.reports()` builds a second index next to the BM25 one:

```
.index('meta', rag.terms({ keys: ['type', 'tags', 'status', 'aliases'] }))
```

A terms index is an inverted map from each metadata value, lowercased, to the chunks that carry it, kept separately per key. Over the eleven reports it holds 94 distinct values. Because chunks inherit their document's metadata, the counts are counts of *chunks*:

Key	Values	Largest
`type`	4	project-report 254, project 99, spec 77, article 32
`status`	3	complete 267, active 118, proposed 77
`tags`	47	project-report 286, pbui 254, workbench 254, typescript 254, architecture 254, …

Key	Values	Largest
`aliases`	44	*ttc tier b* 17, *coinvault mcp 2026-07-28* 32, *workbench-core* 83, …

Searching it does not tokenize or stem. The query is lowercased and its punctuation turned into spaces, and a value matches when its words occur in the query as a contiguous phrase. Each match adds a weight to every chunk that carries the value, and the weight falls slowly as the value becomes more common:

```
weight(value, key) = 1 / ln(2 + n / 10)        n = number of chunks with that value under that key
```

For *what happened in TTC Tier B*, the alias *ttc tier b* matches, it is carried by 17 chunks, and each of them scores 1 / ln(3.7) = 0.7643. That is the index doing its job: a name the team uses for a project, which may appear nowhere in the report's text, routes the question to the right document. All 17 chunks tie, because all 17 carry the same alias; the terms index identifies a document and leaves the choice of section to BM25 and fusion.

The same mechanism produces a failure that is worth studying. For *Which project deals with OAuth?*, two values match. *oauth* is a tag of the CoinVault report, carried by 32 chunks, worth 1 / ln(5.2) = 0.6065. But *project* is also a value: it is the `type` of the three TTC reports and the two Agentforum reports, and one of their tags, carried by 99 chunks under each key, and the question contains the word *project*. Each of those 99 chunks collects 0.4038 twice, for 0.8076, and outranks every CoinVault chunk. In the *Which report?* preset, which fuses this index with a BM25 index over document cards, the TTC order-grain report comes first and CoinVault second. (The five matching reports tie in the meta index; the order-grain report's id sorts first, and fusion with the card index leaves it on top.)

The question used *project* as an ordinary word, and the index read it as a value. BM25 is protected from this by idf, which gives a word found everywhere almost no weight; the terms index has only the gentle `n / 10` decay. There are three remedies, each with a cost. The preset already uses the first: it gives the meta retriever half the weight of the text retriever in fusion (`weights: { docs: 1, meta: 0.5 }`), which limits the damage but does not remove it. The second is to index only keys whose values rarely occur as ordinary words, such as `aliases` and specific tags, and to leave `type` and `status` to filters. The third is to filter out values that are also common query words. The general lesson is that a closed vocabulary is precise when a user *chooses* a value and noisy when values are *recognised* in free text.

6.2 Facet counts from a result set

The better use of metadata is to let the user choose. A *facet* is a list of values for one key, each with the number of results that carry it; clicking a value filters the results to it. The *Faceted search* preset wires this up:

```
const plan = rag.plan('facets').over(kit.reports())
  .signals({ q: 'sync', k: 10, filters: {}, codeOnly: false })
  .retrieve('text', { k: 80 })
  .filter.where()
  .filter((s, ctx, hit) => !s.codeOnly || hit.text.includes('```'), { id:
'code_only' })
  .limit(30);

// in the app
c.facets('filters', { fields: ['type', 'tags', 'status'], from: 'text', max:
10 })
```

The `facets` component counts values over the hits of the node named in `from`, here the BM25 retriever, so the counts describe the result set *before* any filter. For the query *sync*, the 80 retrieved chunks give these counts:

```
type     project-report 79 · spec 1
tags     architecture 79 · pbui 79 · project-report 79 · typescript 79 ·
workbench 79 · testing 61
         sync 60 · idempotency 42 · identity 42 · optimistic-concurrency 42
status   complete 79 · proposed 1
```

The counts say something a ranked list cannot say at a glance: 79 of the 80 hits come from the PBUI reports, and the one exception is the agent runtime spec. A reader who wanted the spec's view of synchronisation now knows the result set holds exactly one candidate for it.

Clicking a value writes it into the `filters` signal, and `filter.where()` keeps the hits whose metadata matches. The rule is: within one key, a hit must carry *any* of the selected values; across keys, it must satisfy *every* key. Values compare case-insensitively, and the special key `doc` filters by document id. The trace for `filters: { tags: ['pbui'] }` shows each node's input and output counts:

```
text       ran   1 → 80
filter     ran  80 → 79
limit      ran  79 → 30
```

Adding `type: ['article']` to the same filter leaves 0 hits, because no PBUI chunk in this result set is an article.

Two decisions in this design are worth stating, because they are choices a designer could make differently. First, counts are taken before filtering, so selecting a tag does not make the other tags' counts collapse to the intersection. The counts keep describing the result set the user started from, which keeps the other values clickable, at the price that a number on a chip is not always the number of results you will get. A selected value with no hits stays visible with a count of 0, so the user can see why the list is empty and undo it. Second, the counts are counts of chunks, not documents: *pbui 79* means 79 sections, from several reports. A facet over documents would need to count distinct `docId`s, and would say *4* instead.

6.3 Synonyms in the index versus rewriting the query

A lexical index finds only the words that were written. A reader who asks about *LCA* will not match a report that always writes *latent class analysis*, and a reader who asks about *stateless* mode will not match a section that talks about *server/discover*. The workbench offers two ways to close that gap without calling a model, and the *Query expansion* preset uses both.

The first is a synonyms table attached to the corpus. It is a build node, so its table can be computed from the documents with a bake:

```
// the preset: a literal table
const corpus = kit.reports()
  .synonyms({ 'oauth': ['authorization', 'token'], 'cursor': ['sequence',
'next_cursor'],
              'stateless': ['server/discover', 'sessionless'] });

// an example: a table baked from every document's aliases at build time
.synonyms(rag.bake(ctx => {
  const table = {};
  for (const d of ctx.docs) for (const a of [].concat(d.meta.aliases ?? []))
    (table[String(a).toLowerCase()] ??= []).push(d.title);
  return table;
}))
```

The table is built once, but it is not written into the index. It is applied inside BM25 when a query is scored: if the query contains a key as a phrase, the tokens of that key's synonyms are added as extra query terms with weight 0.5, unless they are already in the query. The

engine's own term list for one query shows the effect:

```
without the table:  chat 1 · gpt 1 · chatgpt 1 · need 1 · stateless 1 · mode
1
with the table:     chat 1 · gpt 1 · chatgpt 1 · need 1 · stateless 1 · mode
1 · server 0.5 · discover 0.5 · sessionless 0.5
```

For *why did ChatGPT need stateless mode*, the top hit stays the same, *12. Stateful versus stateless Streamable HTTP*, but its score rises from 21.27 to 25.13 and the third hit changes from *15. Failure chronology* to *18. Current status*. The half weight is a hedge: a synonym is evidence that the chunk is about the same thing, but weaker evidence than the word the user typed.

The second way is a rewrite step in the plan, which produces additional queries. The preset's rewrite lambda receives a table of acronyms as a parameter and adds one expanded query for each acronym it finds:

```
.rewrite((s, ctx, queries, p) => {
  const q = String(s.q).toLowerCase();
  const extra = Object.entries(p.acronyms)
    .filter(([a]) => new RegExp('\\b' + a + '\\b').test(q))
    .map(([a, full]) => q.replace(new RegExp('\\b' + a + '\\b', 'g'), full));
  return [...queries, ...extra];
}, { acronyms: { lca: 'latent class analysis', dcr: 'dynamic client
registration', … } })
.retrieve('text', { k: 30, perQuery: true })
```

For *what is the knee rule for LCA*, the rewrite node outputs two queries, the original and *what is the knee rule for latent class analysis*. The retriever scores each separately and keeps each chunk's best score. The top hit, *Latent class analysis: the interpretable alternative*, goes from 9.08 to 20.08, and the second hit becomes *What model selection actually established*, the section a golden case expects for the related question about latent classes.

The two mechanisms differ in ways that decide where each belongs:

	Synonyms table	Rewrite step
Where it lives	the corpus, as a build node	the plan, as a live node
Can be computed from documents	yes, with `rag.bake`	only from params and signals

	Synonyms table	Rewrite step
Effect on scoring	adds terms at weight 0.5 to the same query	adds whole queries, each scored at full weight
Visible in the trace	in a hit's `why` terms	as the rewrite node's output list
Applies to	every retriever over that BM25 index	the retrievers downstream of the step

On the twelve goldens, removing either one changes almost nothing: hit@5 stays at 1.0 and MRR at 0.933, and only nDCG@10 moves, from 0.857 to 0.847 without the synonyms. That is not evidence that expansion is useless. None of the twelve questions uses an acronym or the word *stateless*, so the goldens do not exercise what these steps do. A team that adds an expansion should add the questions that motivated it to the goldens at the same time, or the evals will report that the change did nothing (chapter 18).

The key points to take from this chapter:

- Frontmatter values form a closed vocabulary, which is precise for filtering and counting and noisy for ranking.
- A terms index matches values as phrases in the question and identifies documents, leaving the choice of section to BM25 and fusion.
- Recognising values in free text can misfire when a value is also an ordinary word, as *project* did for the OAuth question.
- Facet counts are computed over the result set before filtering, and they count chunks, not documents.
- Selected values within a key combine with *or*, and different keys combine with *and*.
- A synonyms table is built once and applied inside BM25 at half weight; a rewrite step adds whole queries in the plan.
- An expansion step needs golden cases that use the vocabulary it expands, or the evals cannot see its effect.

This completes the corpus side of the pipeline: documents, chunks, a lexical index and a metadata index. Part III steps back from what each stage computes and asks how a pipeline is written down, so that it can be compiled, compared and changed while it runs.

Lab: Faceted search, Query expansion. In *Faceted search*, search `sync`, read the facet counts and confirm that one hit is not from the PBUI reports; then select the tag `pbui` and watch the Trace tab's counts for the filter node. Select `type: article` as well and

look at how the empty state is shown. In *Query expansion*, search `what is the knee rule for LCA`, open the Trace tab to see the rewrite node's two queries, then run the evals with the `no-rewrite` and `no-synonyms` variants. Finally, add a golden case that uses an acronym and run the comparison again.

Part III · A grammar for pipelines

A pipeline written as a chain of builder calls is a value, not a running program: it can be compiled, checked, drawn, varied and stored. This part defines that grammar and the IR it compiles to.

7. Specs, not programs

This chapter explains what a pipeline *is* in the workbench. The short answer is that it is a value: a list of node records that says which components run, in what order, with which parameters. Nothing in that list has run yet. By the end of the chapter you should be able to read a pipeline script and say exactly what data it produces before it touches a single file, and you should understand why the workbench insists on that separation.

The alternative is the obvious one. A search script could open the files, split them, build an index and answer queries, all as ordinary function calls. That works once. It does not let you draw the pipeline, compare two versions of it, change one parameter while it is serving queries, or hand it to an agent that has to edit it without rerunning everything. Each of those needs the pipeline as data first and as computation second. Parts IV and VII depend on this, so it is the first thing Part III establishes.

7.1 Corpus specs and plan specs

A pipeline has two halves, split by when they run. The **corpus** half runs once per set of files: it reads markdown, cuts it into chunks and builds indexes. The **plan** half runs once per query: it takes the signals the page sends (the query text `q`, the result count `k`, any filters) and turns them into ranked hits, a packed context and an answer. The library has one builder for each half. Here is the whole of the *Minimal search* preset's pipeline:

```
const corpus = rag.corpus('reports')
  .from(rag.files('*.md'))
  .parse(rag.markdown())
  .chunk(rag.sections({ maxTokens: 450 }))
  .index('text', rag.bm25());

const plan = rag.plan('minimal').over(corpus)
  .signals({ q: 'rebalance law', k: 10 })
  .retrieve('text', { k: 20 });
```

None of these calls reads a file. Each one appends a node record to a list and returns a new spec holding the longer list. The arguments are data too: `rag.sections({ maxTokens: 450 })` returns a plain object that names a component type and its parameters.

```
rag.sections({ maxTokens: 450 })
// → { "$c": "chunk.sections", "p": { "maxTokens": 450 } }
```

Asking the corpus spec for its JSON shows the node list it has accumulated:

```
{ "kind": "corpus", "id": "reports", "nodes": [
  { "phase": "build", "id": "files",   "t": "source.files",   "in": [],
"p": { "glob": "*.md" } },
  { "phase": "build", "id": "parse",   "t": "parse.markdown", "in":
["files"],  "p": {} },
  { "phase": "build", "id": "chunks",  "t": "chunk.sections", "in":
["parse"],  "p": { "maxTokens": 450 } },
  { "phase": "build", "id": "ix-text", "t": "index.bm25",     "in":
["chunks"], "p": { "name": "text" } } ] }
```

Every node has the same five fields, and they are worth fixing in memory because the rest of the book manipulates them:

Field	Meaning	Example
`id`	The node's name, unique in the pipeline. Generated if you do not give one.	`chunks`, `ix-text`
`t`	The component type, a key into the registry (chapter 8).	`chunk.sections`
`in`	The ids of the nodes whose outputs this node consumes, in port order.	`["parse"]`
`p`	The parameters, as JSON. Functions become stored lambdas (chapter 9).	`{ "maxTokens": 450 }`
`phase`	`build` for corpus nodes, `live` for plan nodes.	`build`

The plan spec looks the same, with live nodes and a signals object:

```
{ "kind": "plan", "id": "minimal", "signals": { "q": "rebalance law", "k": 10
},
  "nodes": [ { "id": "text", "t": "retrieve", "phase": "live", "in": ["q"],
"p": { "index": "text", "k": 20 } } ] }
```

Two details in that record show the builder doing work for you. The retrieve node was named `text` after the index it reads, because the `retrieve` component names its nodes that way. And its input is `q`, the query signal, even though the script never mentioned `q` in the `.retrieve` call. Declaring a `q` signal made it the plan's current source of queries, and `.retrieve` takes queries. Section 7.3 makes these inference rules explicit.

The plan holds a reference to its corpus (`.over(corpus)`), not a copy of the corpus nodes. Compiling the plan (chapter 10) concatenates the two node lists into one graph.

7.2 Immutability, and why variants cost nothing

Every builder method returns a new spec and leaves the receiver alone. The spec object itself is frozen, so an accidental assignment to it fails rather than silently changing a pipeline someone else is holding:

```
Object.isFrozen(plan)    // → true
Object.isFrozen(corpus)  // → true

const small = corpus.chunk(rag.sections({ maxTokens: 200 }));
small.toJSON().nodes.map(n => n.id)   // → ["files", "parse", "chunks", "ix-
text", "chunks_2"]
corpus.toJSON().nodes.map(n => n.id)  // → ["files", "parse", "chunks", "ix-
text"]
```

Internally each method copies the node array and the small cursor object (section 7.3) and shares every node record with the original. Extending a spec therefore costs one array copy, whatever the size of the pipeline. Freezing is shallow, so the guarantee rests on the library never mutating a spec's state after construction, which it does not.

Immutability matters most for **variants**. A variant is a second pipeline that differs from the first in a few places, built so the two can be scored against each other. Because the original cannot change, a variant can be described as "that plan, plus these edits":

```
const k5 = plan.variant('k5', [{ op: 'set', node: 'text', path: 'k', value: 5
}]);
```

The variant does not copy and modify the retrieve node. It records the edit as an operation, the same `set` operation the workspace uses in Part VII, and applies it when the plan is compiled:

```
{ "kind": "plan", "id": "k5", "signals": { "q": "rebalance law", "k": 10 },
  "nodes": [ { "id": "text", "t": "retrieve", "phase": "live", "in": ["q"],
"p": { "index": "text", "k": 20 } } ],
  "ops": [ { "op": "set", "node": "text", "path": "k", "value": 5 } ] }
```

After compilation the two graphs differ in exactly one parameter, which `rag.util.diffIr` reports:

```
[ { "change": "param", "node": "text", "path": "k", "from": 20, "to": 5 } ]
```

This is why the *Rerank with a lambda* preset can declare two comparison pipelines in two lines, one that removes the rerank node and one that triples its heading weight. Neither line can disturb the main plan, and the eval sheet can run all three side by side. `variant` also accepts a function instead of an op list, `plan.variant('id', p => p.limit(5))`, for edits that are easier to write as builder calls.

There is a second benefit that only shows up later. Chapter 11 names every build node by a digest of what it would compute. Two variants that share their corpus share those digests, so building the second variant reuses every index the first one built. Immutable specs make that sharing safe: a node record that cannot change cannot invalidate a cached result behind the engine's back.

7.3 The chain infers the wiring; the graph form states it

A chain of builder calls never names its edges. The builder infers each node's `in` list from a **cursor**: a small map from an output type to the id of the node that most recently produced it. The corpus cursor tracks `files`, `docs` and `chunks`. When `.parse` is called, its input is whatever `files` points at; when `.chunk` is called, its input is the current `docs`, and afterwards `chunks` points at the new node.

The plan cursor tracks `queries`, a list of current `hits` streams, `context` and `answer`. The hits entry is a list because retrieval fans out: each `.retrieve` call starts a new hits stream and adds it to the list. Any step that takes a single hits stream checks that list, and if it holds more than one stream it refuses to guess which one you meant:

```
rag.plan('bad').over(kit.reports()).signals({ q: '' })
  .retrieve('text').retrieve('meta').limit(5);
// RagCompileError: limit: 2 hit streams (text, meta) need a fuse step first,
e.g. .fuse.rrf()
```

The fix is to say how the streams combine. `.fuse.rrf()` takes a variadic hits input, consumes every open stream and leaves one stream behind (chapter 14 explains the fusion rule itself).

Named chunk streams follow a related rule. In the *Table rows as chunks* preset, `.chunk('rows', lambda)` adds a second chunker that emits one chunk per table row. Because the corpus already has a main chunk stream, the named one does not move the `chunks` cursor; it is reachable only by name. That is why the index built over it says where its input comes from:

```
kit.reports()
  .chunk('rows', (doc, ctx, p) => /* one chunk per table row */, { minRows: 2
})
  .index('rows', rag.bm25({ fields: { headingPath: 2, text: 1 } }), { from:
'rows' });
```

Inference keeps simple pipelines short, but a reader has to know the cursor rules to see the edges. When a pipeline branches and merges, it is clearer to state every edge. `rag.graph` does that. Each node gets an explicit id and an explicit `from`:

```
const g = rag.graph('minimal-g', g => {
  g.over(corpus);
  const q = g.signal('q', 'rebalance law');
  g.signal('k', 10);
  const t = g.node('text', rag.retrieve('text', { k: 20 }), { from: q });
  g.output('hits', t);
});
```

The graph form is not a second representation. It calls the same internal method the chain calls, with the inference switched off, and it compiles to the same IR. Comparing the compiled node lists of the chain version and the graph version above gives `true`. The only difference is in the outputs: the graph pinned `hits` explicitly with `g.output`, so its IR lists `pinned: ["hits"]`, while the chain lets the compiler derive the outputs (section 10.3).

	Chain form	Graph form
Edges	Inferred from the cursor	Stated with `from`
Node ids	Generated unless given (`{ id }`)	Always given
Outputs	Derived by the compiler	Pinned with `g.output`, else derived
Best for	Linear pipelines, presets, quick edits	Branches, merges, several answers from one context
Compiles to	The same IR	The same IR

The *Graph form* preset writes a fused two-retriever plan this way. Its fuse node lists both retrievers in `from`, and its fuse weights are keyed by those node ids (`{ lex: 1, tags: 0.3 }`), which is easier to check when the ids are visible in the source.

7.4 Fragments and the shared `kit`

Because a spec is a value and every builder method is a function from spec to spec, a reusable piece of pipeline is just such a function. `rag.fragment` gives it a name, and `.use` applies it with parameters:

```
const hybrid = rag.fragment('hybrid', (p, o) =>
  p.retrieve('text', { k: o.k ?? 40 }).retrieve('meta', { k: 20 }).fuse.rrf({
k: 60 }));

const hp = rag.plan('h').over(kit.reports()).signals({ q: '', k: 8 })
  .use(hybrid, { k: 30 })
  .limit(s => s.k);

rag.compile(hp).summary()
// → files:source.files → q:signal → k:signal → parse:parse.markdown →
chunks:chunk.sections
//   → ix-text:index.bm25 → ix-meta:index.terms → text:retrieve →
meta:retrieve → fuse:fuse.rrf → limit:limit
```

A fragment has no special status after it is applied. The compiled graph contains ordinary nodes, and nothing records that three of them came from `hybrid`. That is deliberate: the engine, the evals and the agent all work on the graph, and a graph with no hidden

structure is one they can all read.

The workbench's presets share a few such pieces through `kit`, which is ordinary `rag` and `gui` code loaded before every script:

- `kit.reports()` returns the standard corpus spec: markdown sections at depth 3 and at most 450 tokens, a BM25 index named `text` with field weights `title: 2, headingPath: 3, callout: 2, text: 1`, and a terms index named `meta` over `type`, `tags`, `status` and `aliases`.
- `kit.goldens()` returns the twelve golden questions for the bundled reports (chapter 18).
- `kit.searchApp(plan, options)` returns the standard search page layout (chapter 26).

Presets extend `kit.reports()` rather than copying it. The *Code search* preset adds a code chunker and an index over it; the *Table rows* preset adds a row chunker and an index over that. Because `kit.reports()` returns a fresh immutable spec, neither extension can leak into another preset.

Key points

- A pipeline is a list of node records with an id, a component type, input ids, JSON parameters and a phase. Builder calls only extend that list.
- Specs are immutable. A variant is the original plus recorded operations, so the original can never change under you, and two variants can share every build result they have in common.
- The chain form infers edges from a cursor keyed by output type. It refuses to guess when several hit streams are open, and named chunk streams must be referenced by name.
- The graph form states every edge and compiles to the same IR. Use it when the pipeline branches.
- A fragment is a function from spec to spec. After it is applied, it leaves only ordinary nodes behind.

The node records in this chapter carried types like `chunk.sections` and `retrieve`, and the builder knew that `retrieve` takes queries and gives hits. Chapter 8 explains where that knowledge lives.

Lab: Minimal search. Load the preset and open the IR tab to see the node list described in 7.1. Change `maxTokens` to 200 and rebuild; the Build passes list shows `files` and `parse` reused. Then add `const k5 = plan.variant('k5', [{ op: 'set', node: 'text', path: 'k', value: 5 }]);` and return `variants: [k5]` to score both plans on the goldens. Finally, load *Graph form* and compare its IR tab with *Reports · hybrid + rerank + answer* to see that both forms produce the same kind of graph.

8. Components and typed ports

Chapter 7 showed node records whose `t` field named a component type: `chunk.sections`, `retrieve`, `fuse.rrf`. This chapter explains what stands behind those names. Every type is an entry in a registry that declares the component's phase, the type of data it takes and gives, and how it runs. The builder methods are generated from that registry, the compiler checks wiring against it, and `rag.define` adds to it. Understanding the registry is what lets you predict which pipelines compile and extend the grammar without touching the library.

The design goal is to catch a whole class of mistakes before anything runs. Building indexes over 535 KB of markdown takes a few hundred milliseconds, and a query plan that ends in a Claude call takes seconds and spends usage. A pipeline that feeds a packed context into a reranker, or a query signal into an answer node, should fail at compile time with a message that names the bad edge, not at the end of a build or a paid call.

8.1 The registry: type, phase, ports, trigger

The registry is a `Map` from a type name to a definition object. Two helpers populate it, `build(type, def)` and `live(type, def)`, which differ only in the phase they set. Here is the complete definition of `limit`, one of the simplest live components:

```
live('limit', {
  in: ['hits'], out: 'hits',
  args: a => (typeof a === 'number' || isLambda(a) || typeof a === 'function'
    ? { k: typeof a === 'number' ? a : lam(a) } : { ...(a || {}) }),
  describe: p => `k ${typeof p.k === 'number' ? p.k : 's.k'}`,
  run: (s, ctx, hits, p) => hits.slice(0, Number(p.k ?? s.k ?? 8)),
});
```

The fields fall into three groups. Some describe the component to the compiler, some tell the builder how to turn call arguments into parameters, and the rest are the implementation.

Field	Group	What it does
`phase`	compiler	`build` (runs once per corpus) or `live` (runs per query).
`in`, `out`	compiler	Port types. `in` is a list, one entry per input; `out` is one type.
`trigger`	compiler	`submit` for components that wait for Enter. Defaults to `change`.
`uses`	compiler	Extra dependencies that are not data inputs, such as the index a retriever reads.
`reads`	compiler	Signals the component reads inside its own code (section 9.5).
`lambdas`	compiler	Parameter names that hold functions the component calls itself, rather than values to resolve first.
`args`	builder	Turns the call's arguments into a params object: `limit(10)` becomes `{ k: 10 }`.
`id`	builder	Chooses the default node id: `retrieve('text')` makes a node named `text`.
`describe`	display	A one-line summary for the DAG and the agent: `k s.k`.
`run`	live	`(s, ctx, input, p) => output`, called once per query.
`map` / `all`	build	`map(item, ctx, p)` per input item, cached per item; or `all(inputs, ctx, p)` over the whole input.

`rag.components()` lists the registry. The bundled library defines 27 types:

Type	Phase	In	Out	Trigger
`source.files`, `source.inline`	build	none	files	
`parse.markdown`	build	files	docs	

Type	Phase	In	Out	Trigger
`chunk.sections`, `chunk`	build	docs	chunks	
`enrich`, `enrich.llm`	build	chunks	chunks	
`index.bm25`, `index.terms`	build	chunks	index	
`synonyms`, `bake`	build	docs	value	
`retrieve`	live	queries	hits	change
`rewrite` / `rewrite.llm`	live	queries	queries	change / submit
`fuse.rrf`, `fuse.max`	live	hits*	hits	change
`filter`, `filter.where`, `limit`, `expand.neighbors`, `expand.links`, `map`	live	hits	hits	change
`rerank` / `rerank.llm`	live	hits	hits	change / submit
`pack`	live	hits	context	change
`answer`	live	context	answer	submit
`agentic`	live	queries	answer	submit

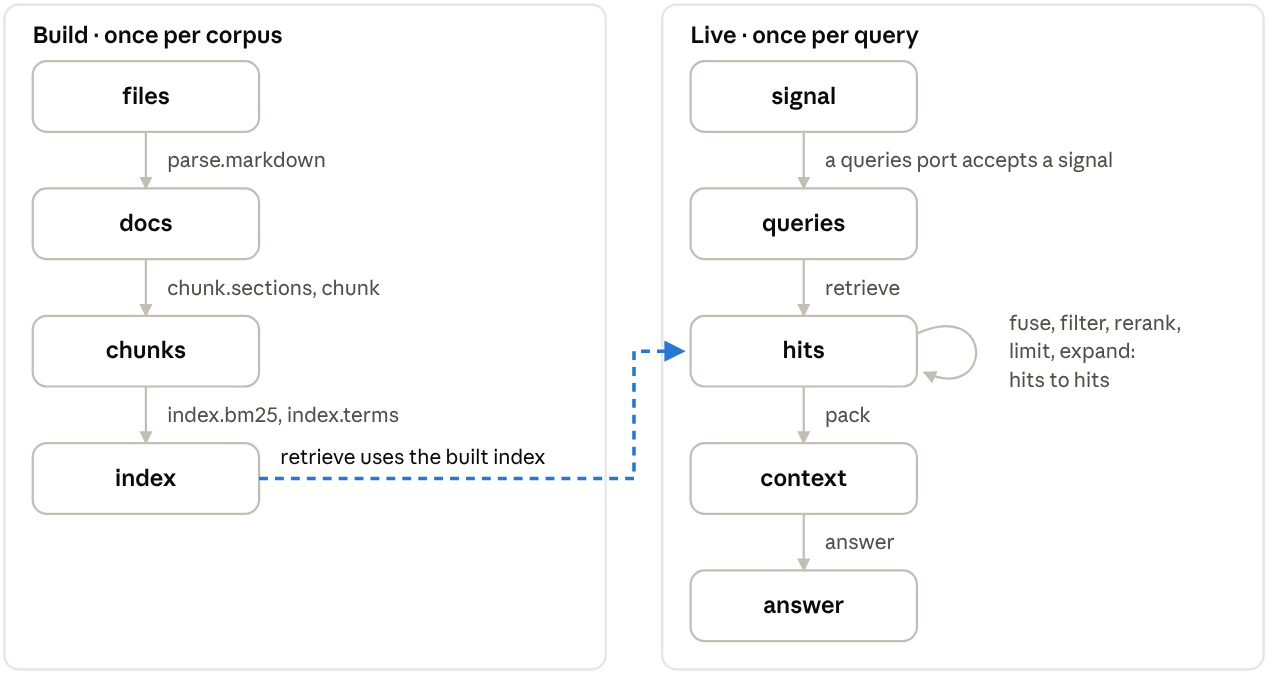
The pattern in the trigger column is not an accident. Every live component that calls Claude declares `trigger: 'submit'`, so typing never spends usage. Chapter 12 builds the scheduling on top of that one field.

8.2 Two typed streams

The port types form two chains, one per phase.

Two typed streams; only an index crosses from build to live

Boxes are port types, arrow labels are the components that convert one type to the next



port types of the two streams · build left, live right

The build stream converts `files` (name and text) to `docs` (frontmatter, heading tree, typed blocks), `docs` to `chunks` (the units that get ranked), and `chunks` to an `index`. A side branch from `docs` produces a `value`, used for tables such as query synonyms. The live stream starts from a **signal**, a named input the page sets, and converts `queries` (a list of strings) to `hits` (ranked chunk ids with scores), `hits` to a `context` (packed text plus evidence labels) and `context` to an `answer`. Most live components sit on the `hits` row and map hits to hits: they fuse, filter, rerank, cut or expand the list without changing its type.

The compiler checks each edge with one function. A port accepts a source if their types match, with one exception: a `queries` port also accepts a signal, which is how the `q` string reaches the first retriever. Fusion components declare their input as `hits*`, a variadic port that takes any number of hits streams.

```
const accepts = (port, type) =>
  port.replace('*', '') === type || (port.startsWith('queries') && type ===
'signal');
```

The two streams meet in exactly one place, and it is not a data port. A retriever needs a built index, but an index is not something a query run passes along; it is an artifact the build left behind. The `retrieve` definition therefore declares it as a dependency through `uses`, not `in`:

```
live('retrieve', {
  in: ['queries'], out: 'hits', source: true, lambdas: ['fn'],
  id: p => (p.index ? String(p.index) : 'retrieve'),
  uses: p => (p.index ? ['ix-' + p.index] : []),
  // …
});
```

The compiler turns that into an edge of a third kind. Every edge in the IR carries one of three labels, and the DAG tab draws them differently:

Edge kind	Comes from	Example	Drawn as
`data`	An `in` entry	`chunks → ix-text`	solid (dashed when the source is a build node)
`artifact`	A `uses` entry	`ix-text → text`	dashed
`signal`	A signal in `in`, or a signal a lambda reads	`q → text`, `k → limit`	blue

The direction rule follows from the phases. A build node may not read a live node, because the build runs before any query exists; the compiler reports `a build node cannot read live node '…'` if one tries. Live nodes read build results only through `uses`.

When wiring is wrong, the compiler names the node, the input and both types. Feeding the query signal straight into a packer, or a packed context into a limit:

```
1 problem:
  pack: input 'q' gives signal but this port takes hits

2 problems:
  b: input 'p' gives context but this port takes hits
  x: input 'a' gives hits but this port takes context
```

A wrong number of inputs and a retriever pointed at an index that does not exist produce messages of the same shape. The second lists the indexes that do exist, which is usually all the reader needs to fix it:

```
1 problem:
  b (pack) takes hits but is wired to [a, q]

1 problem:
  a: uses 'ix-nope', which does not exist (indexes: text, meta)
```

8.3 Method names generated from the registry

No builder method is written by hand for a component. After the registry is filled, two functions derive the API from it.

`refreshFactories` groups types by the part before the dot and creates one factory per family on `rag`. A family with a bare member and variants becomes a function with properties: `rag.rerank` is a function that makes a `rerank` component, and `rag.rerank.llm` makes a `rerank.llm` component. A family with only variants becomes a plain object: `rag.fuse` has the keys `rrf` and `max`. A few short aliases are added for the build components presets use most: `rag.files`, `rag.markdown`, `rag.sections`, `rag.bm25` and `rag.terms`. Each factory runs the type's `args` normalizer and returns the `{ $c, p }` value seen in chapter 7.

`installFamilies` does the same for the plan builder. For every live family it defines a getter on `PlanSpec.prototype`. Reading `plan.fuse` calls the getter, which builds an object whose `rrf` and `max` methods add a node to *that* plan. Listing the prototype's own properties shows the result: the hand-written methods (`as`, `over`, `signals`, `patch`, `variant`, `use`, `then`, `toJSON`) followed by the generated ones (`retrieve`, `rewrite`, `fuse`, `filter`, `rerank`, `limit`, `expand`, `pack`, `answer`, `agentic`, `map`).

Getters on the prototype, rather than methods attached to each plan, are what make later definitions work. A plan created before `rag.define('diversify', …)` runs still gains a `diversify` method afterwards, because every plan looks up its methods on the same prototype:

```
const p = rag.plan('x');
rag.define('diversify', { phase: 'live', in: 'hits', out: 'hits', run: (s,
ctx, hits) => hits });
typeof p.diversify   // → "function"
```

The corpus builder is different on purpose. Its methods (`from`, `parse`, `chunk`, `enrich`, `index`, `synonyms`, `then`) are fixed, and each takes a component value as an argument: `.chunk(rag.sections({ … }))`, `.index('text', rag.bm25())`. Build stages are few and their order is fixed, so the method names describe the stage and the argument picks the implementation. Live steps are many and freely ordered, so the method name is the component.

8.4 `rag.define`: components that chain like built-ins

`rag.define(type, def)` validates a definition, fills in defaults, adds it to the registry and reruns both generators. The defaults are the common case: a live component takes and gives `hits`, a build component takes `docs` and gives `chunks`. The *Your own components* preset defines a step that keeps at most `perDoc` sections from each document:

```
rag.define('diversify', {
  phase: 'live', in: 'hits', out: 'hits', params: { perDoc: 2 },
  describe: p => `≤ ${p.perDoc} per doc`,
  run: (s, ctx, hits, p) => {
    const seen = new Map();
    return hits.filter(h => {
      const d = (ctx.chunk(h.id) || {}).docId;
      const n = seen.get(d) || 0;
      seen.set(d, n + 1);
      return n < p.perDoc;
    });
  },
});

const plan = rag.plan('custom').over(kit.reports())
  .signals({ q: 'testing and verification', k: 10 })
  .retrieve('text', { k: 60 })
  .diversify({ perDoc: 1 })
  .limit(s => s.k);
```

The new node is indistinguishable from a built-in one in the IR: `{ id: "diversify", t: "diversify", phase: "live", in: ["text"], p: { perDoc: 1 }, out: "hits" }`. Its effect is visible in the results. Without it, the top five for *testing and verification* include two sections of *PBUI Workbench Core*, both headed "7. Verification". With `perDoc: 1`, the fourth and fifth places go to *PBUI Datalab* ("12.1 Gates") and *Agentforum Web* ("Testing and validation").

A build component works the same way. The *Code search* preset defines `chunk.codeBlocks`, whose `map` receives a parsed document and returns one chunk per fenced code block with at least `minLines` lines. The returned objects need only `headingPath`, `text` and any extra fields; `define` wraps them in full chunk records and assigns stable ids, exactly as the built-in chunkers do. Because the type has a dot, it becomes a variant of the existing `chunk` family and is called as `rag.chunk.codeBlocks({ minLines: 3 })`.

`define` checks its input, and its errors say what shape it expected:

```
component type 'Boost-Recent' should look like 'family' or 'family.variant'
define needs phase: 'build' or 'live'
a live component needs run(s, ctx, input, p)
```

Two properties of defined components need care. First, a component's `run` is ordinary JavaScript in the script, not a stored lambda, so it may close over anything. The cost is that the IR records only the type name and params. A saved IR that mentions `diversify` can run only in a page where the `define` call has run, which is why presets put their definitions at the top of the script. Second, the compiler cannot scan `run` for signal reads the way it scans lambdas (section 9.5). A component that reads a signal inside `run` should say so with `reads`, as the preset's `boost.tags` does with `reads: () => ['boostTag']`, so the DAG shows the signal edge and the engine knows to rerun the node when that signal changes.

Key points

- Every component type is a registry entry declaring its phase, typed ports, trigger and implementation. The compiler, the builders and the DAG all read the same entry.
- Data flows through two typed chains: files, docs, chunks, index at build time, and signal, queries, hits, context, answer per query. Most live steps map hits to hits.
- The only connection from build to live is a `uses` dependency, which becomes an artifact edge. A build node may never read a live node.
- Wiring errors are reported at compile time with the node, the input and both types named.
- Builder methods are generated from the registry. Live families become prototype getters, so components defined later appear on every plan.

- `rag.define` adds a component that chains like a built-in. Its code lives in the script, so a saved IR needs the definition to run again, and signals read inside `run` must be declared with `reads`.

The components in this chapter took their behaviour from code in the registry. Many presets instead pass a function as a parameter: a rerank lambda, a chunker, `s => s.k`. Those functions end up inside the IR, which is JSON. Chapter 9 explains how.

Labs: Your own components, Code search. In *Your own components*, select the `diversify` node in the DAG tab and change `perDoc` from 2 to 1; only the live nodes rerun. Then try wiring a mistake on purpose with an op in the script, such as `{ op: 'wire', node: 'limit', in: ['q'] }`, and read the compiler's message. In *Code search*, search for `protojson UnmarshalOptions` and use the `lang` facet; then change `minLines` to 10 and rebuild to see how many code chunks remain.

9. Lambdas that survive serialization

The registry covers the steps the library anticipated. Search is full of steps it did not: a chunker that turns table rows into chunks, a reranker that rewards heading overlap, a result count that follows a slider. The grammar lets you write those steps as functions and pass them as parameters. This chapter explains what happens to a function once it is inside a pipeline, why the workbench stores its source text instead of the function object, and what that costs you as a programmer. The cost is one rule, and most of the chapter is about living with it: **a lambda in a pipeline cannot see the variables around it.**

9.1 Functions as source text

A pipeline has to be JSON. Chapter 7 gave the reasons: the workspace saves it to the database, the agent reads it as text, ops edit it, the compiler digests it, and `diffIr` compares two versions of it. A JavaScript function object fits none of these. It cannot be serialized, two functions with the same text are never equal, and a function received from somewhere else cannot be inspected.

So the builder converts every function it finds in a component's parameters into a record holding its source text. The conversion is `lam`:

```
rag.util.lam((s, ctx, hit) => hit.tokens > s.minTok)
// → { "$fn": "(s, ctx, hit) => hit.tokens > s.minTok" }
```

The text comes from `Function.prototype.toString`, which returns the function's source as written. Some functions have no usable source. A built-in such as `Math.max` prints as `function max() { [native code] }`, and a bound function prints the same way, so `lam` refuses both:

```
native or bound functions cannot be stored as lambdas; write an arrow
function
```

The stored record is what the IR holds. A filter step written as `.filter((s, ctx, hit, p) => hit.tokens > p.minTok, { minTok: 60 })` compiles to a node whose parameters are:

```
{ "fn": { "$fn": "(s, ctx, hit, p) => hit.tokens > p.minTok" }, "minTok": 60
}
```

When the node runs, `fnOf` compiles the text back into a function and caches it by label and source, so each distinct lambda is compiled once per page. Because the source is data, it can also be produced by something other than a JavaScript function: an op can replace a lambda with a string, `{ op: 'replace', node: 'filter', fn: '(s, ctx, hit) => hit.tokens > 40' }`, which is how the agent edits lambdas.

9.2 Closed lambdas: the guard and the free-identifier error

Source text loses the function's closure. Consider a script that defines a threshold and uses it in a filter:

```
const minTok = 60;
const plan = rag.plan('f').over(kit.reports())
  .signals({ q: 'rebalance law', k: 10 })
  .retrieve('text', { k: 40 })
  .filter((s, ctx, hit) => hit.tokens > minTok);
```

The stored text is `(s, ctx, hit) => hit.tokens > minTok`. When that text is compiled again later, perhaps in a reloaded page or a workspace the agent built, `minTok` does not exist. Ordinary `new Function` compilation would resolve the name against the page's global scope, find nothing, and throw a `ReferenceError` that mentions neither the pipeline nor the node. Worse, if the page happened to have a global with that name, the lambda would silently read it.

The library prevents both outcomes by compiling every lambda inside a scope it controls. In pseudocode, the compiled wrapper is:

```
// pseudocode of compileSource(src, label)
guard = Proxy where
  has(name)  = name is not in ALLOWED          // claim every name except
safe globals
  get(name)  = throw "<label> references free identifier `name` …"
return new Function('__g', 'with (__g) { return (' + src + ') }')(guard)
```

The `with` statement makes every identifier lookup in the lambda body ask the proxy first. The proxy's `has` trap claims every name except a short allow-list of standard globals (`Math`, `JSON`, `Array`, `Object`, `String`, `Number`, `Date`, `RegExp`, `Map`, `Set`, `Promise`, `Intl`, `parseInt`, `console` and a few others). The lambda's own parameters and local variables are resolved before the `with` scope is consulted, so they work normally. Any other name reaches the proxy's `get` trap, which throws an error naming the node and the identifier. Running the plan above gives:

```
filter lambda 'filter' references free identifier `minTok`. Lambdas are
stored as source
and run without their closure: use their arguments, a signal, or rag.bake.
```

The error is raised when the lambda first runs, not when the plan compiles. The compiler does check every lambda's syntax, so a truncated edit is rejected at once (`filter.fn: Unexpected token ')'`), but finding free identifiers statically would need a JavaScript parser and scope analysis, and the guard is exact without one. In practice the gap is small. A live lambda that references a free name fails on the first query: the run's `errors` list carries the message, the trace marks the node `status: "error"`, and the node's input passes through so the rest of the page still renders. A build lambda with the same mistake stops the build, and the build report carries the same message prefixed with the node id, for example `rows: chunk lambda 'rows' references free identifier minRows`.

The rule is the same for every lambda in the system, including the generative UI in Part VIII. It is what makes a pipeline a self-contained value: everything a lambda depends on is either in its arguments or in the IR beside it.

9.3 Getting values in: signals, params as the last argument, `rag.bake`

A closed lambda still needs data from outside its body. The grammar provides three channels, each for a different kind of value.

Params, as the last argument. A component that calls a lambda passes the node's own parameters as the lambda's final argument. Anything you put in the params object beside the function is therefore visible to it and is stored next to it in the IR. The free-identifier example above becomes:

```
.filter((s, ctx, hit, p) => hit.tokens > p.minTok, { minTok: 60 })
```

This is the right channel for constants that belong to the step: weights, thresholds, small lookup tables. Because they live in `p`, they can be edited in the DAG tab's node panel or with a `set` op (`{ op: 'set', node: 'filter', path: 'minTok', value: 100 }`) without touching the lambda's text. The *Rerank with a lambda* preset uses four of them: `{ heading: 1.5, code: 0, minTokens: 60, shortPenalty: 0.6 }`.

Signals. Every live lambda receives the run's signals as its first argument, `s`. A value that the page or the user changes at run time belongs there: the query, `k`, a filter object, a slider position. Declare it with `.signals({ minTok: 60 })` and read it as `s.minTok`. Chapter 12 shows that the engine reruns only the nodes whose signal reads changed, so moving a slider bound to `minTok` reruns the filter and what follows it, not the retriever.

Bakes. Some values depend on the corpus and should be computed once after it is built, not hand-written and not recomputed per query. `rag.bake(fn)` marks a parameter as such a value. Its source is stored as `{ "$bake": "…" }`, and after the build the engine calls it once with `{ docs, chunks, tokens, slug }` and substitutes the result. A rerank step that halves the score of hits from the two largest documents can compute that list from the chunks:

```
.rerank((s, ctx, hits, p) => hits
    .map(h => ({ ...h, score: h.score * (p.common.includes(h.docId) ? 0.5 :
1) }))
    .sort((x, y) => y.score - x.score),
  { common: rag.bake(ctx => {
      const n = {};
      for (const c of ctx.chunks) n[c.docId] = (n[c.docId] || 0) + 1;
      return Object.entries(n).sort((a, b) => b[1] - a[1]).slice(0, 2).map(e
=> e[0]);
    }) })
```

The IR stores the bake's source. After a build, `engine.baked(node)` shows the substituted value the lambda will see:

```
{ "common": [ "project-report-pbui-workbench-core-a-headless-engine-a-pure-plan",
              "agent-runtime-spec-portable-eval-agent" ] }
```

The same mechanism builds query-expansion tables. A synonyms node given `rag.bake(ctx => …)` can read every document's `aliases` frontmatter and map each alias to the document title, so a query for *ttc tier b* expands to the full report name. The baked table for the bundled reports starts with `lca`, `ttc customer clustering`, `thetreecenter.com segmentation`, `coinvault chatgpt mcp acceptance`.

Channel	Set where	Changes when	Use for
Params (`p`, last argument)	Next to the lambda in the IR	An op or a DAG edit changes it	Constants of the step: weights, thresholds
Signals (`s`, first argument)	`.signals({ … })`, then the page	Every run, from the UI	Values the user controls: query, k, filters, sliders
Bakes (`rag.bake`)	Computed after the build	The corpus changes	Values derived from the documents: alias tables, document sizes

9.4 Four phases: build callbacks, build slots, bakes, live slots

A function in a pipeline script can run at four different times. Knowing which one applies tells you what arguments it gets, whether it is stored, and how often it runs.

Form	Runs	Receives	Stored in the IR
Build callback, e.g. `rag.graph(id, g => …)`, `plan.variant(id, p => …)`, `gui` view builders	Once, while the spec is being built	A builder	No
Build slot, e.g. `.chunk('rows', (doc, ctx, p) => …)`, `.enrich(name, (chunk, ctx, p) => …)`	At build time, once per input item, cached by digest	The item, a build context, params	Yes, as `$fn`

Form	Runs	Receives	Stored in the IR
Bake, `rag.bake(ctx => …)`	Once after the build	`{ docs, chunks, tokens, slug }`	Yes, as `$bake`
Live slot, e.g. `.filter((s, ctx, hit, p) => …)`, `.limit(s => s.k)`	Per query	Signals, a live context, the input, params	Yes, as `$fn`

A build callback is ordinary JavaScript. It runs inside your script, may close over anything, and leaves only its result behind; that is why `rag.graph`'s callback can use local constants freely. The other three forms are stored and run closed.

A build slot runs once per item of its input. The *Table rows as chunks* preset's chunker receives each parsed document and returns one chunk per table row:

```
.chunk('rows', (doc, ctx, p) => doc.sections.flatMap(sec => sec.blocks
  .filter(b => b.kind === 'table' && b.rows.length >= p.minRows)
  .flatMap(t => t.rows.map(row => ({
    kind: 'row',
    headingPath: [...sec.path, row[0]],
    text: t.header.map((h, j) => `${h}: ${row[j] ?? ''}`).join('\n'),
  }))))), { minRows: 2 })
```

Over the eleven reports it ran 11 times and produced 350 row chunks. One of them is the row that answers *which HTTP status does ErrConflict map to*: heading path `The HTTP server › Adapter discipline › ErrConflict`, text `Service sentinel: ErrConflict / HTTP: 409 / Error.code: conflict`. Raising `minRows` to 5 changes the node's digest, reruns the slot and leaves 293 chunks.

Live slots come in two kinds, and the difference is the `lambdas` field of the registry entry from chapter 8. A parameter listed there, such as `fn` on `filter` or `rerank`, is handed to the component as a function, which calls it with `(s, ctx, input, p)`. Any other parameter that holds a lambda is a **value slot**: the engine evaluates it with `(s, ctx)` just before the node runs and passes the result as an ordinary value. `limit(s => s.k)` is a value slot. Its stored form is `{ k: { "$fn": "s => s.k" } }`, and the `limit` component sees `p.k` as a number. Any parameter of any live component can be made dynamic this way, without the component's author having planned for it.

9.5 Which signals a lambda reads: static scan and runtime tracking

The engine needs to know which signals each live node depends on, for two reasons. The DAG draws a signal edge from `k` to `limit`, and Part VII's dry runs report which nodes an edit will rerun. Both must work without running anything. Separately, the session that executes queries must never reuse a node's cached output after a signal it read has changed. That must be exact.

The library uses two mechanisms, one for each need. At compile time, `signalReads` scans each live lambda's source with a regular expression. It finds the name of the first parameter and collects every `name.prop`, `name?.prop` and `name['prop']` that follows; if the first parameter is a destructuring pattern, it collects the destructured names. The results become the node's `reads` list and its signal edges:

```
signalReads('(s, ctx, hit) => hit.tokens > s.minTok')          // →
["minTok"]
signalReads('({ q, k }) => q.length + k')                       // → ["q",
"k"]
signalReads("(sig) => sig['boostTag'] + sig?.k")                // →
["boostTag", "k"]
signalReads('(s) => { const key = "k"; return s[key]; }')       // → []
```

The last line shows the limit of a textual scan: a computed property name is invisible to it. That is acceptable for drawing, but not for caching. So at run time the session passes each node a proxy over the signals object that records every property read, with the value it had. Before reusing a node's previous output, the session checks that every recorded read still has the same value. A plan whose limit reads a signal through a computed key demonstrates the difference:

```
rag.plan('d').over(corpus).signals({ q: 'rebalance law', k: 10, field: 'k' })
  .retrieve('text', { k: 20 })
  .limit(s => { const key = s.field; return s[key]; });
```

The static scan finds only `["field"]`. Running the plan twice, with `k` changed from 10 to 3 between runs, shows the session getting it right anyway:

```
run 1:  text ran 20    limit ran 10
run 2:  text cached 20  limit ran 3
recorded reads for limit: { field: "k", k: 3 }
```

The same tracking is what makes ordinary plans cheap to rerun. In the filter plan from section 9.3, with `minTok` and `k` as signals, changing only `k` reuses the retriever and the filter and reruns only `limit`; changing `minTok` reruns the filter and `limit` and still reuses the retriever. Chapter 12 builds the session memo on this.

Key points

- Functions in pipeline parameters are stored as source text (`$fn`, `$bake`), so the IR stays JSON that can be saved, diffed, digested and edited by ops.
- Stored lambdas run closed. A guard proxy turns any free identifier into an error that names the node and the identifier. The compiler checks syntax; free identifiers are reported on first run.
- Values reach a closed lambda through three channels: params as the last argument for step constants, signals as the first argument for values the user changes, and `rag.bake` for values derived from the corpus.
- Functions run in four phases. Build callbacks run once and are not stored; build slots run per item at build time; bakes run once after the build; live slots run per query.
- A parameter listed in a component's `lambdas` is called by the component. Any other lambda parameter is a value slot evaluated with `(s, ctx)` before the node runs.
- A static scan of the source draws signal edges; runtime tracking of actual reads keeps the cache correct even when the scan misses a read.

With components, ports and lambdas in place, a script can describe any pipeline the workbench runs. Chapter 10 follows what happens when that description is compiled: which checks run, in what order the nodes end up, and how each node gets a name derived from what it computes.

Labs: Rerank with a lambda, Table rows as chunks. In *Rerank with a lambda*, select the `rerank` node in the DAG tab: its lambda source and its four params are shown separately. Change `heading` from 1.5 to 3 and compare with the `heading-x3` variant on the Evals tab. Then add `const boost = 2;` above the plan and use `boost` inside the lambda; search once and read the free-identifier error in the trace. In *Table rows as chunks*, search for *which HTTP status does ErrConflict map to*, then change `minRows` to 5 and watch the Build passes list rebuild only the row chunker and its index.

10. Compiling to an IR

A spec is convenient to write but awkward to execute. Its corpus and plan halves live in separate objects, its signals are a side table, some of its edits are still pending as ops, and nothing has checked that its wiring is legal. `rag.compile` turns a spec into an **IR** (intermediate representation): one flat, checked, ordered graph that every later stage reads. The engine builds and runs the IR, the workspace edits it, the evals score it, the DAG tab draws it and the agent reads it. This chapter describes that graph, the checks that guard it, the order its nodes are stored in, and the digests that name its build nodes by what they compute.

10.1 The IR: nodes, edges, signals, outputs

Here is the compiled IR of the *Minimal search* plan with one more step, `.limit(s => s.k)`, so that it contains a signal read. Node records are shortened to one line each; nothing has been removed.

```
{ "kind": "rag", "version": 1, "id": "minimal", "corpus": "reports",
  "signals": { "q": "rebalance law", "k": 10 },
  "nodes": [
    { "id": "files",   "t": "source.files",   "phase": "build",  "in": [],
"p": { "glob": "*.md" },       "out": "files",  "uses": [], "reads": [] },
    { "id": "q",       "t": "signal",         "phase": "signal", "in": [],
"p": {},                       "out": "signal" },
    { "id": "k",       "t": "signal",         "phase": "signal", "in": [],
"p": {},                       "out": "signal" },
    { "id": "parse",   "t": "parse.markdown", "phase": "build",  "in":
["files"],  "p": {},                      "out": "docs",   "uses": [],
"reads": [] },
    { "id": "chunks",  "t": "chunk.sections", "phase": "build",  "in":
["parse"],  "p": { "maxTokens": 450 },    "out": "chunks", "uses": [],
"reads": [] },
    { "id": "ix-text", "t": "index.bm25",     "phase": "build",  "in":
["chunks"], "p": { "name": "text" },     "out": "index",  "uses": [],
"reads": [] },
    { "id": "text",    "t": "retrieve",       "phase": "live",   "in": ["q"],
"p": { "index": "text", "k": 20 }, "out": "hits", "uses": ["ix-text"],
"reads": [] },
    { "id": "limit",   "t": "limit",          "phase": "live",   "in":
["text"],   "p": { "k": { "$fn": "s => s.k" } }, "out": "hits", "uses": [],
"reads": ["k"] } ],
  "edges": [ ["files","parse","data"], ["parse","chunks","data"],
["chunks","ix-text","data"],
            ["q","text","signal"], ["ix-text","text","artifact"],
["text","limit","data"], ["k","limit","signal"] ],
  "outputs": { "hits": "limit" },
  "pinned": [],
  "digest": "02ef3933d734f5" }
```

Compared with the spec, four things have changed.

- **The two halves are one list.** The corpus nodes and the plan nodes share one `nodes` array, and the plan's reference to its corpus survives only as the `corpus` field.
- **Signals are nodes.** Each signal becomes a node of type `signal` with phase `signal`, so an edge can start from it and the DAG can draw it. The `signals` object keeps the default values.

- **Each node carries derived fields.** `out` is the node's output type from the registry, `uses` lists its artifact dependencies, and `reads` lists the signals its lambdas read (section 9.5). These are computed, not written by hand, and they are recomputed on every compile.
- **Edges are explicit.** Each edge is a triple `[from, to, kind]`, where the kind is `data`, `signal` or `artifact` as defined in section 8.2. Data and signal edges come from `in`; artifact edges come from `uses`; the `k → limit` signal edge comes from the lambda's reads.

The IR also carries methods, attached as non-enumerable properties so they do not appear in its JSON: `ir.node(id)`, `ir.downstream(id)`, `ir.upstream(id)`, `ir.summary()`, `ir.mermaid()` and `ir.describe(id)`. For this graph, `ir.downstream('chunks')` is `["ix-text", "text", "limit"]`, which is exactly the set of nodes a change to the chunker can affect.

10.2 What the checker rejects, with the real error messages

Compilation happens in `finalize`, which walks the nodes once, collects every problem it finds, and throws a single error listing all of them. It does not stop at the first problem. An agent editing a pipeline, or a person reading the error panel, can fix everything in one pass instead of discovering problems one compile at a time.

Check	Example message
Node ids are well formed	`node id 'ix.text' must start with a letter and use letters, digits, _ or - (no dots)`
Node ids are unique	`duplicate node id 'a'`
The component type exists	`x: unknown component 'rerank.semantic'`
The number of inputs matches the ports	`b (pack) takes hits but is wired to [a, q]`
Every input exists	`limit: input 'rerank' does not exist`
Each input's type fits its port	`limit: input 'text' gives hits but this port takes context`
A build node reads no live node	`c: a build node cannot read live node 'x'`
Every `uses` target exists	`a: uses 'ix-nope', which does not exist (indexes: text, meta)`

Check	Example message
Every lambda parses	`filter.fn: filter.fn: Unexpected token ')'`
The graph is acyclic	`the graph has a cycle through: text, limit`
Every output points at a node	`output hits points at missing node 'x'`

The error message starts with a count. Rewiring the retriever of the minimal plan to read from `limit`, which reads from the retriever, breaks two rules at once, and both are reported:

```
after ops: 2 problems:
  text: input 'limit' gives hits but this port takes queries
  the graph has a cycle through: text, limit
```

The prefix `after ops:` says the problem appeared after a list of ops was applied, which is how the workspace reports edits that would break the graph. Ops can also fail before compilation, when the op itself makes no sense; those messages name the op's position and kind: `op 1 (insert): node 'text' already exists`, `op 1 (set): set needs a path into the node params, e.g. "k" or "fields.title"`, `op 1 (move): unknown op 'move'. Ops: set unset insert remove replace wire signal trigger output rename`.

Two limits of the checker are worth knowing. It checks lambda syntax but not free identifiers, which surface on first run (section 9.2). And a duplicate id currently produces a second, empty line, `the graph has a cycle through:` with no node names, because the ordering step cannot place two nodes with the same id. Fixing the duplicate removes both lines.

10.3 Topological order and derived outputs

The nodes array in the IR is in **topological order**: every node appears after all the nodes it has an edge from. The engine relies on this. A build or a query run is a single pass over the array, and when a node's turn comes, its inputs are already computed.

The ordering is Kahn's algorithm. Count each node's incoming edges, start with the nodes that have none, and repeatedly emit a node and decrement the counts of its successors, emitting each successor when its count reaches zero. Ties are broken by the order nodes were declared. That is why the minimal plan's summary reads:

```
files:source.files → q:signal → k:signal → parse:parse.markdown →
chunks:chunk.sections
  → ix-text:index.bm25 → text:retrieve → limit:limit
```

The signals appear second and third because they have no inputs and were declared before `parse` became ready. If the algorithm ends with nodes it never emitted, those nodes lie on a cycle, and the checker lists them. The same pass therefore detects cycles at no extra cost.

After ordering, the compiler decides the plan's **outputs**: which node's result the page shows as *the* hits, *the* context and *the* answer. You can pin an output explicitly, with `g.output` in the graph form or an `output` op. Otherwise the compiler derives it:

- `answer` is the last node in order whose output type is `answer`.
- `context` is the last node whose output type is `context`.
- `hits` is the last rerank node if there is one; otherwise the input of the context node; otherwise the last node producing hits.

The hits rule needs explaining. The list a user should see is the best-ordered list before the pipeline starts cutting and expanding it for the answer. A rerank step is the most deliberate ordering in a plan, so its output wins. Without one, what went into the packer is what the answer was built from. A full hybrid plan shows the rule in action:

```
… text:retrieve → meta:retrieve → fuse:fuse.rrf → rerank:rerank.llm! →
limit:limit → pack:pack → answer:answer!

outputs: { "answer": "answer", "context": "pack", "hits": "rerank" }
after { op: 'remove', node: 'rerank' }: { "answer": "answer", "context":
"pack", "hits": "limit" }
```

The `!` after a node in the summary marks a `submit` trigger: that node waits for Enter (chapter 12). Pinned outputs are listed in the IR's `pinned` array and survive edits; derived ones are recomputed every time the graph changes.

10.4 Digests: naming a node by what it would compute

The last thing compilation produces is a set of **digests**. A digest is a short hash that identifies a piece of computation by its inputs. The engine computes one per build node, and the compiler computes one for the whole IR. The build-node digests are what chapter 11's incremental builds are keyed on, so it is worth seeing exactly what goes into them.

In pseudocode, the digest of a build node is:

```
// pseudocode of engine.digests(ir), in topological order
digest(n) = hash([ n.t, n.p, [digest(i) for i in n.in], fileContents(n) ])
fileContents(n) = hashes of every file's name and text, if n is source.files;
otherwise ''
hash(v) = cyrb53(stable(v))   // stable = JSON with object keys sorted
```

Three properties follow from that definition, and each has a purpose.

- **A digest covers everything upstream.** Each node's digest includes its inputs' digests, so a change to a parameter changes the digest of that node and of every build node downstream of it, and of no other.
- **File contents count.** The `source.files` node hashes each file's name and text. Uploading a new report changes the first digest and therefore every build digest after it.
- **The node id does not count.** The digest says what the node computes, not what it is called. Renaming a node leaves its digest, and its cached result, intact.

Changing the chunker's `maxTokens` from 450 to 200 shows the first property:

```
node      before          after
files     967fb737993c00  967fb737993c00  same
parse     d8c4fd64af9808  d8c4fd64af9808  same
chunks    7f4be2a1d17826  7b7903cada3901  changed
ix-text   b7e8f0afe8c72b  670946ba2ee997  changed
```

The build reports that follow are what the digests buy. The first build computed everything: 11 files, 11 parsed documents, 462 chunks and an index, in about 220 ms, most of it chunking and indexing. The second build reused `files` and `parse` and recomputed only `chunks` (now 873 chunks) and `ix-text`. Renaming `chunks` to `sections` instead produces the same four digests as the original, and the build after the rename reuses all four nodes.

Live nodes have no build digest. Changing the retriever's `k` from 20 to 40 leaves every build digest unchanged, so no rebuild happens; the change affects only query runs, which chapter 12 caches differently.

The whole-IR digest, the `digest` field in the JSON above, is a hash of every node's id, type, inputs, params and trigger, plus the signals and outputs. Unlike the build digests, it changes on a rename and on any live edit: the three versions above (original, `maxTokens:`

`200`, `k: 40`) have IR digests `02ef3933d734f5`, `92d021d6f7f3e2` and `164e199c974419`. Eval reports record it as `irDigest`, so a score can always be traced to the exact graph that produced it.

Digest	Covers	Changes on	Used for
Build node	Type, params, input digests, file contents	Upstream build edits, file changes	Reusing built chunks and indexes (chapter 11)
Whole IR	Ids, types, wiring, params, triggers, signals, outputs	Any edit, including renames and live params	Tagging eval reports and history

The hash function is cyrb53, a fast non-cryptographic 53-bit hash, printed as 14 hex digits. It is applied to a *stable* serialization in which object keys are sorted, so `{ k: 20, index: 'text' }` and `{ index: 'text', k: 20 }` hash the same. A non-cryptographic hash is adequate because digests here only need to distinguish a user's own pipeline versions from each other, not resist an adversary.

10.5 Reading the DAG tab

The DAG tab in the inspector draws the current IR. Everything on it is one of the fields described in this chapter.

The layout is by layer. Each node is placed one column to the right of its furthest predecessor, so every edge points rightwards, and live nodes are pulled as far right as their consumers allow. The columns are headed *build · per corpus* and *live · per query*. Within a column, build nodes come first, then signals, then live nodes.

On the tab	Comes from
Node name and small type label	`id` and `t`
Solid edge	A `data` edge between live nodes
Dashed grey edge	A `data` edge out of a build node, or an `artifact` edge
Blue edge	A `signal` edge
Dotted green edge	A data edge that carried at least one item in the last run

On the tab	Comes from
`llm` tag	The node's trigger is `submit` (it waits for Enter and may spend Claude calls)
`λ` tag	A parameter holds a `$fn` or `$bake`
Number on a build node	Items it produced in the last build
`in→out` on a live node	Counts from the last run's trace; `skip` if the node was skipped

Selecting a node opens its detail panel: its params, its lambda source, its last build or run statistics. Editing a param there does not change the script; it applies a `set` op to the workspace, which recompiles the IR, recomputes the digests and rebuilds only the dirty nodes. That loop, edit an op, recompile, rebuild the difference, is the subject of Parts IV and VII. The same graph is available as text with `ir.mermaid()`, which draws build nodes as parallelograms, signals as circles and live nodes as rectangles, with `-->` for data, `-.->` for artifact and `-.-` for signal edges.

Key points

- The IR is one flat graph: all corpus and plan nodes in one list, signals as nodes, and explicit edges labelled `data`, `signal` or `artifact`. Derived fields (`out`, `uses`, `reads`) are recomputed on every compile.
- The checker collects every problem and throws one error that lists them all, each naming the node, the input and the types involved. Free identifiers in lambdas are the one class of error it leaves to run time.
- Nodes are stored in topological order, so a build or a run is a single pass. The ordering pass also detects cycles.
- Outputs are pinned or derived. The derived hits output prefers a rerank node, then the packer's input, then the last hits node.
- A build node's digest hashes its type, params, input digests and, for the source, file contents, but not its id. It changes exactly when the node's result would change.
- The whole-IR digest changes on any edit and ties eval reports to the graph that produced them.

Part III has described a pipeline as a value and shown how it becomes a checked graph with names derived from content. Part IV runs that graph: chapter 11 shows the engine using build digests to rebuild only what changed, and chapter 12 shows the session

caching live nodes by input identity and recorded signal reads.

Lab: Graph form. Load *Graph form* and open the IR tab. Find the five signal edges: `q` into `lex`, `tags` and `answer`, `filters` into `filter`, and `k` into `top`, the last one coming from the lambda `s => s.k`. Add a node to the graph, `g.node('n2', rag.retrieve('text', { k: 10 }), { from: q })`, without listing it in the fuse node's `from`. The plan still compiles, because a dangling retriever breaks no rule, but `n2` now runs on every query and nothing reads its hits. List it in `from` and check that `fuse` has three inputs. Finally, in the DAG tab, select `chunks`, change `maxTokens`, and watch the build nodes downstream of it turn dirty while `files` and `parse` stay reused.

Part IV · Running it

The engine runs an IR in two phases: a build that turns files into indexes once, and query runs that happen on every keystroke. Both avoid repeated work by recording what each node's output depended on.

11. Builds that remember

This chapter explains how the engine decides what to compute when a pipeline changes. The answer rests on one idea: every build node is named by a digest of what it would compute, and the engine keeps every output it has computed under that name. Once that is in place, an edit to the chunker rebuilds the chunks and the indexes after them, and nothing else; switching back to the previous setting costs nothing; and a report you upload recomputes the parse of that one file. By the end of the chapter you should be able to predict, before pressing Build, which nodes a change will recompute and which it will reuse.

All output in this chapter comes from running the real `rag.js` in node over the eleven bundled reports. Timings are from one machine and will differ on yours; counts and digests will not.

11.1 The build phase and the live phase

Every node in a compiled IR has a phase. Build nodes turn files into documents, documents into chunks, and chunks into indexes. Live nodes take a query and turn it into hits, a context and an answer. The engine runs the two phases at different times and for different reasons. A build runs when the corpus or a build parameter changes. A live run happens on every keystroke.

Here is the IR for the smallest useful pipeline, the corpus from `kit.reports()` with one retriever on top:

```
const corpus =
rag.corpus('reports').from(rag.files('*.md')).parse(rag.markdown())
  .chunk(rag.sections({ depth: 3, maxTokens: 450 }))
  .index('text', rag.bm25({ fields: { title: 2, headingPath: 3, callout: 2,
text: 1 } }))
  .index('meta', rag.terms({ keys: ['type', 'tags', 'status', 'aliases'] }));
const plan = rag.plan('chunks').over(corpus).signals({ q: 'rebalance law', k:
8 }).retrieve('text', { k: 30 });
rag.compile(plan).summary();
```

```
files:source.files → q:signal → k:signal → parse:parse.markdown →
chunks:chunk.sections
  → ix-text:index.bm25 → ix-meta:index.terms → text:retrieve
```

Five of these nodes are build nodes (`files` , `parse` , `chunks` , `ix-text` , `ix-meta`), two are signals, and one, `text` , is live. The split exists because the two halves have very different costs. On the first build, parsing eleven files took 36.5 ms, sectioning them into 462 chunks took 75.4 ms, and building the BM25 index took 101.5 ms. A query against the finished index took about one millisecond. If each keystroke rebuilt the index, typing would be two hundred times slower than it needs to be.

	Build phase	Live phase
Inputs	files, build params	signals (`q` , `k` , filters, sliders) and build outputs
Outputs	`files` , `docs` , `chunks` , `index` , `value`	`queries` , `hits` , `context` , `answer`
Runs when	the corpus or a build param changes	the query or another signal changes
Remembered by	digest, across edits (this chapter)	per node, across runs (chapter 12)
Typical cost	100–250 ms, or Claude calls	0.1–30 ms, or Claude calls on Enter

Live nodes never call the build. They read its results through the run context: `ctx.index('text')` , `ctx.chunk(id)` , `ctx.neighbors(id)` . The engine assembles those from the stored build outputs with `engine.artifacts(ir)` , which returns `null` if any

build node of the IR has not been computed yet. A session that finds no artifacts builds first; that is why the very first query after loading a preset is slower than the second.

11.2 The digest chain: type, params, inputs, file contents

To reuse work, the engine needs a name for each output that changes exactly when the output would change. The output itself is no use for this, because computing it is the cost being avoided. The engine instead names a node by everything its output depends on. This is the complete function:

```
digests(ir) {
  const out = new Map();
  for (const n of ir.nodes) {
    if (n.phase !== 'build') continue;
    let extra = '';
    if (n.t === 'source.files') extra = this.files().map(f =>
this.fileDigest(f)).join(',');
    out.set(n.id, hash([n.t, n.p, n.in.map(i => out.get(i) || i), extra]));
  }
  return out;
}
```

A node's digest is the hash of four things: its component type, its params, the digests of its inputs, and, for the source node only, a hash of each file's name and text. The nodes are visited in topological order (chapter 10), so an input's digest is always known before the node that reads it. `hash` is a 53-bit string hash over a canonical serialization, `stable`, which sorts object keys, so `{ depth: 3, maxTokens: 450 }` and `{ maxTokens: 450, depth: 3 }` produce the same digest. Lambdas take part through their source text.

The digest therefore names a node by what it would compute rather than by what it did compute. You can check this by recomputing one by hand. For the `chunks` node of the pipeline above:

```
const d = engine.digests(ir), n = ir.node('chunks');
// n = { t: 'chunk.sections', p: { depth: 3, maxTokens: 450 }, in: ['parse']
}
rag.util.hash([n.t, n.p, n.in.map(i => d.get(i)), '']);   // 'a9202e4e3c054c'
d.get('chunks');                                          // 'a9202e4e3c054c'
```

Because each digest includes its inputs' digests, a change anywhere propagates down the edges and never up them. A new `maxTokens` changes the digest of `chunks`, which changes the input list of both indexes, which changes theirs. The digest of `parse` does not mention `chunks` at all, so it stays the same.

The most common misunderstanding is to read the digest as a hash of the node's output. It is not, and the difference matters in one place, which section 11.5 comes back to: two different outputs can exist under one digest if the engine ever stores something other than the full result of the computation the digest names.

11.3 The build memo and the dirty set

The engine keeps a map from digest to output, `engine.memo`. Building a node means: compute its digest; if the memo has it, reuse the stored value; otherwise compute the value and store it. `engine.dirty(ir)` lists the build nodes whose digests are not in the memo, which is exactly the set a build would compute.

Here is a sequence of four builds on one engine, with the per-node report each returns (`rep.nodes`). The first builds everything. The second builds the same IR again. The third applies one op, `{ op: 'set', node: 'chunks', path: 'maxTokens', value: 200 }`. The fourth builds the original IR once more.

```
dirty before [ 'files', 'parse', 'chunks', 'ix-text', 'ix-meta' ]

# first build ok true ms 218.8
   files    source.files    967fb737993c00 computed count 11
   parse    parse.markdown  d8c4fd64af9808 computed count 11
   chunks   chunk.sections  ebe4a56ec01071 computed count 462
   ix-text  index.bm25      ed7fac9a111070 computed count 462
   ix-meta  index.terms     5fb5ef38ae6f1b computed count 462

# same IR again ok true ms 0.3            (every node: reused)

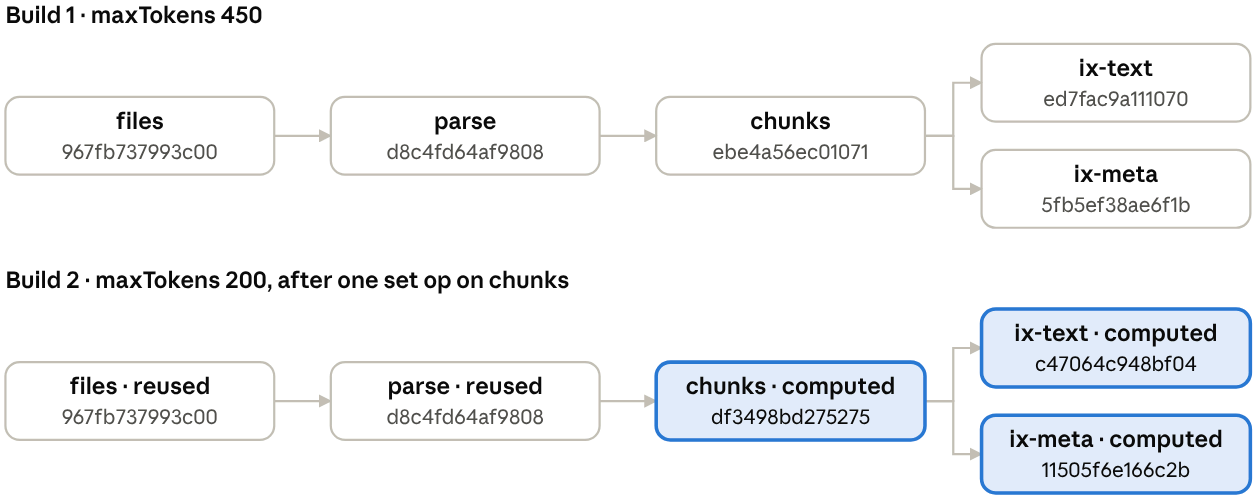
dirty after set maxTokens [ 'chunks', 'ix-text', 'ix-meta' ]
# maxTokens 200 ok true ms 119.5
   files    source.files    967fb737993c00 reused   count 11
   parse    parse.markdown  d8c4fd64af9808 reused   count 11
   chunks   chunk.sections  df3498bd275275 computed count 873
```

```
   ix-text  index.bm25      c47064c948bf04 computed count 873
   ix-meta  index.terms     11505f6e166c2b computed count 873

# back to 450 ok true ms 0.2              (every node: reused)
```

Changing maxTokens renames only the nodes downstream of it

Build-node digests from two real builds of the same corpus



A digest hashes the node's type, params and input digests, so a change propagates down the edges and never up them.

digest chain · two builds, one changed param

Three properties of this design show up in the output.

- An edit costs what is downstream of it, not what is in the pipeline. Halving the chunk size reused the parse and spent 119.5 ms on chunks and indexes, against 218.8 ms for the full build.
- Undo is free. Going back to 450 tokens found the original digests still in the memo and took 0.2 ms. This is what makes the variant comparisons of chapter 20 cheap: four chunking variants cost four chunk builds once, and every later comparison between them is a lookup.
- The dirty set is a prediction you can read before spending anything. The workbench shows it after every edit, and the agent's dry runs (chapter 21) report it before an op list is applied.

The memo is never evicted during a session. Each entry holds a full output, so a long session of chunking experiments holds every chunk list and index it ever built. For eleven reports this is a few megabytes; for a corpus a hundred times larger it would need an eviction policy, which the engine does not have.

11.4 Map components and the per-item cache, in memory and in the database

The memo works at the granularity of a node, which is too coarse for one common edit: changing a single file. Every file feeds the source node, so any edit gives `files` a new digest, and with it every node downstream. Without something finer, adding one paragraph to one report would reparse all eleven.

The finer mechanism belongs to map components. A component registered with `map` instead of `all` processes its input one item at a time: `parse.markdown` maps over files, `chunk.sections` and `chunk` map over documents, `enrich` and `enrich.llm` map over chunks. Index builders use `all`, because an index needs every chunk at once. For a map component the engine keys each item separately:

```
const fnKey = hash([n.t, p]);                 // component type and params
const key = fnKey + ':' + hash(item);       // ...and this one item's content
if (this.items.has(key)) { results[i] = this.items.get(key); st.cached++;
continue; }
```

The item key does not mention the node id or its inputs' digests, only the transformation and the item. Here is the build after appending an `## Addendum` section to one of the eleven reports:

```
dirty after file edit [ 'files', 'parse', 'chunks', 'ix-text', 'ix-meta' ]
# one file edited ok true ms 113.7
   files    source.files    4d202e7d0e1ae0 computed count 11  computed 0
cached 0
   parse    parse.markdown  c5b18768ffabfb computed count 11  computed 1
cached 10
   chunks   chunk.sections  a9202e4e3c054c computed count 463 computed 1
cached 10
   ix-text  index.bm25      95a5b278407bc9 computed count 463
   ix-meta  index.terms     e2554e8611b322 computed count 463
```

Every node is dirty, as the digest chain requires, but the parse computed one document and took ten from the item cache, and the chunker did the same. The chunk count went from 462 to 463: the new section. The two indexes were rebuilt from scratch, because `all` components have no items to reuse, and they account for most of the 113.7 ms.

The item cache lives in memory on the engine. For asynchronous components, the ones that may call Claude, the engine also consults `env.cache`, an object with `get(key)` and `set(key, value)`. In the workbench that object writes to the `cache` collection of the

user's database document, so an enrichment computed yesterday is found again today. The effect is visible with two engines that share one cache, the second standing in for a reloaded page:

```
# A: build({calls:true})                    enrich_llm  computed 11  cached
0   calls 11
# B: fresh engine, same db cache            enrich_llm  computed 0   cached
11  calls 0
```

Engine B had an empty memo and an empty item cache, so it reparsed and rechunked locally, which takes milliseconds. The eleven Claude calls it would have needed were all answered from the database.

11.5 Builds that cost Claude calls: budgets, deferral and `pending()`

Some build nodes call Claude. The engine recognizes them with one predicate:

```
function costsCalls(n) {
  const def = REG.get(n.t) || {};
  if (def.phase !== 'build') return false;
  return !!def.llm || Object.values(n.p || {}).some(v => isLambda(v) &&
/\bsample\b/.test(v.$fn));
}
```

A node costs calls if its component is marked `llm: true`, as `enrich.llm` is, or if one of its lambdas mentions `sample`. The second test reads source text, so it can be fooled by a lambda that mentions the word without calling it; it errs toward caution.

A build that spends the user's Claude usage should not start because someone changed a chunk size. The workbench therefore configures the engine with `buildCalls: 'explicit'`. In that mode an ordinary build still runs every node, but a costly map component does not call Claude for items it has not seen; it passes each such item through unchanged and counts it as deferred. The Doc cards preset shows this. Its corpus adds a card per document and enriches the cards with a summary, questions and keywords:

```
.chunk('cards', (doc, ctx) => [{ kind: 'card', headingPath: ['Document
card'],
  text: [doc.title, ...doc.sections.filter(s => s.level === 2).map(s =>
s.title)].join('\n') }])
.enrich('llm', rag.enrich.llm({ tier: 'quick' }), { from: 'cards' })
.index('cards', rag.bm25({ fields: { title: 2, text: 1, 'llm.summary': 2,
'llm.questions': 3, 'llm.keywords': 3 } }), { from: 'enrich_llm' })
```

The build reports below come from a node script with a mock sampler standing in for Claude; the call counts are real, the enrichment text is not.

```
# A: build, calls not allowed ok true calls 0
   cards       computed count 11  computed 11
   enrich_llm  computed count 11  computed 0  deferred 11
   ix-cards    computed count 11
pending [ { node: 'enrich_llm', items: 11 } ]   dirty [ 'enrich_llm' ]
artifacts? true
```

The deferred output is stored in a separate map, `engine.partial`, not in the memo. That is why `enrich_llm` is still dirty, and why `engine.pending(ir)` can list it with the number of items still waiting. It is also why the pipeline works at once: `artifacts()` accepts partial outputs, so the cards index exists and routes queries lexically on titles and headings. The workbench shows the pending count in its status line, and the Build button calls `build(ir, { calls: true })`:

```
# A: build({calls:true}) ok true calls 11
   enrich_llm  computed count 11  computed 11  cached 0
progress 1/11 2/11 3/11 4/11 5/11 6/11 7/11 8/11 9/11 10/11 11/11
pending []
```

The `onProgress` callback receives one event per item, which the page turns into “Building enrich_llm 7/11”. A component can also be given a per-build budget, `budget: { calls: 4 }`. Items past the budget are skipped the same way deferred items are, so each press of Build advances the enrichment by at most four calls and the cache keeps what was already paid for:

```
# B: new prompt, budget 4     enrich_llm  computed 4  cached 0  skipped 7
pending: 7 items
# B: again with budget 4      enrich_llm  computed 4  cached 4  skipped 3
```

A build also accepts an `AbortSignal`. When it fires, the map worker throws before its next item, the report comes back with `ok: false` and `errors: ['enrich_llm: build stopped']`, and the items finished before the abort stay in the item cache and the database.

There is one gap in this bookkeeping, and it is instructive because it follows directly from section 11.2. The deferred build stored the cards index in the memo under its digest, and that digest is computed from the digest of `enrich_llm`, which is the same whether the enrichment is partial or complete. When Build later completes the enrichment, the index's digest has not changed, so the engine reuses the index built over the unenriched cards. In the report above, `ix-cards` shows as reused after the full build. A query for a word that appears only in the enrichment demonstrates it:

```
after explicit, then calls: 0 hits
direct full build:          5 hits
```

The rule the design needs is that a digest must name one output. A partial output is a different output, so anything computed from it needs a different name, for example the input digest with a marker appended while the input is partial. `engine.artifacts()` already applies this rule to its own cache key, appending `~` for partial nodes; the build loop does not yet apply it to downstream digests. Until it does, rebuilding after a completed enrichment requires a change to the index node, or a fresh engine.

The key points of the chapter:

- Every build node is named by a digest of its type, its params, its inputs' digests and, at the source, the files' contents. The name is fixed before any work is done.
- The engine stores each computed output under its digest and never evicts it during a session, so returning to an earlier setting is free and the dirty set predicts the cost of an edit.
- Map components cache each item under its transformation and content, so one edited file reparses one file, and enrichments survive reloads through the database cache.
- Builds that call Claude are deferred until the user asks for them, can be capped by a call budget, and report what is still pending.

A build turns files into artifacts once. The next chapter turns to the other half of the engine, the query runs that read those artifacts many times a second, and to the same idea applied at a finer grain: remembering what each live node's output depended on.

Lab: Chunk sizes, Doc cards. Load *Chunk sizes*, open the Corpus tab and change `maxTokens` in the editor from 450 to 200, then back. Watch which passes the build report marks as reused. Then load *Doc cards*: before pressing Build, the Cards column reads “press Build to enrich” and the status line reads “enrich_llm: 11 items wait for Claude · press Build to spend those calls”. Press Build once and note the call count the status line reports.

12. Queries at runtime

This chapter explains what happens between a keystroke in the search box and a redrawn result list. The build gave us artifacts; a query run reads them through the live nodes of the plan. Two mechanisms keep that cheap enough to run on every keystroke. Each node has a trigger that says whether it runs while the user types or waits for Enter, and a session remembers, per node, exactly which inputs and signal values its last output depended on. Together they mean that most keystrokes rerun two or three nodes in under a millisecond, and that Claude is called only when the user asks for it.

The traces in this chapter come from node runs of the real engine. The plan in sections 12.2 to 12.5 contains three nodes that call Claude; in these runs a mock sampler stood in for Claude, returning fixed queries, a fixed ranking and a one-sentence answer after a 20–40 ms delay. The statuses, counts and call counts are what the engine produced; the Claude text and the timings of the Claude nodes are the mock's.

12.1 Signals: the named inputs of a query run

A signal is a named value that live nodes read: the query `q`, the result count `k`, the facet selection `filters`, or any slider a page defines. A plan declares its signals with defaults:

```
const plan = rag.plan('reports').over(corpus)
  .signals({ q: 'Why did ChatGPT show no app actions after OAuth worked?', k:
8, filters: {} })
  .rewrite.llm({ n: 3 })
  .retrieve('text', { k: 40 })
  .retrieve('meta', { k: 20 })
  .fuse.rrf({ weights: { text: 1, meta: 0.3 } })
  .filter.where()
  .rerank.llm({ top: 20, keep: s => s.k })
  .pack({ budget: 30000 })
  .answer();
```

Each signal becomes a node of phase `signal` in the IR, so a signal can be an input like any other node. A run takes a partial set of values and merges it over the defaults, `{ ...ir.signals, ...signals }`; anything not passed keeps its default. The search box writes `q`, the facet list writes `filters`, and a slider writes the key it is bound to (chapter 25).

A node can depend on a signal in two ways, and the IR records both as edges. The first is as a data input: the first retriever's input port has type `queries`, and when the node before it is a signal the session coerces the string into a one-element query list. The second is as a read: a lambda param such as `keep: s => s.k`, or a component that declares `reads`, as `filter.where` does for `filters`. The Ranking playground's plan shows both kinds:

```
["q","text","signal"]  ["q","failures_2","signal"]  ["rrfK","fuse","signal"]
["q","rerank","signal"]  ["failureBoost","rerank","signal"]
["headingBoost","rerank","signal"]
["k","limit","signal"]
```

These edges are what the DAG tab draws as dotted lines from the signal chips to the nodes that read them. They come from a static scan of each lambda's source (chapter 9.5). The session does not rely on them to decide what to rerun; it records reads as they happen, which section 12.3 explains.

12.2 Triggers: `change` nodes run on every keystroke, `submit` nodes wait for Enter

Every run has a trigger, either `change` or `submit`, and every live node has a trigger too. The page runs with `change` while the user types, after a 140 ms pause, and with `submit` when they press Enter or click the submit button. A node runs on a `change` run only if its own trigger is `change`; on a `submit` run every node runs. A node's trigger is its component's default unless the plan overrides it:

Node	Component	Trigger	Why
`rewrite`	`rewrite.llm`	submit	one Claude call per query
`text`, `meta`	`retrieve`	change	an index lookup, about a millisecond
`fuse`	`fuse.rrf`	change	arithmetic over two lists
`filter`	`filter.where`	change	a metadata test per hit

Node	Component	Trigger	Why
`rerank`	`rerank.llm`	submit	one Claude call over twenty passages
`pack`	`pack`	change	string assembly under a byte budget
`answer`	`answer`	submit	one streamed Claude call

The rule is mechanical: components that call Claude default to `submit`, everything else to `change`. It is also a design decision, discussed from the reader's side in chapter 27.1. Typing should feel instant and cost nothing; an answer should be something the user asked for.

A node that does not run still has to produce something, because the nodes after it need an input. Each component says what it produces when skipped. `rewrite.llm` and `rerank.llm` declare `skip: 'pass'`, so a skipped rewrite forwards the original query and a skipped rerank forwards the fused hits unchanged. `answer` declares `skip: 'null'`, so a skipped answer is no answer. Here is the first run of the plan above with the `change` trigger:

```
# change, first run 26.5 ms, calls 0
  rewrite    skipped    1→1        0ms calls 0
  text       ran        1→40     1.8ms calls 0
  meta       ran        1→20    23.1ms calls 0
  fuse       ran        2→40     0.3ms calls 0
  filter     ran       40→40     0.1ms calls 0
  rerank     skipped   40→40       0ms calls 0
  pack       ran       40→14     0.3ms calls 0
  answer     skipped   14→0        0ms calls 0
```

Each line gives the node, its status, the size of its input and output, its time and its Claude calls. The two Claude nodes in the middle of the pipeline are skipped and pass their input through; the lexical pipeline still ranks 40 fused sections and packs 14 of them into the 30 KB budget. Pressing Enter on the same query runs everything:

```
# submit, same q 86.2 ms, calls 3
  rewrite    ran        1→4     21.1ms calls 1
  text       ran        4→40     1.4ms calls 0
  meta       ran        4→20     0.7ms calls 0
  fuse       ran        2→40     0.1ms calls 0
  filter     ran       40→40       0ms calls 0
  rerank     ran       40→3     21.4ms calls 1
  pack       ran        3→3        0ms calls 0
  answer     ran        3→3     40.9ms calls 1
```

The rewrite turned one query into four (the original and three rewrites), so both retrievers reran with new input. The rerank kept the three passages the mock ranked. The answer cited `[E1]`. Three calls, one per Claude node.

12.3 The session memo: input identity plus recorded signal reads

A trigger decides whether a node may run. The session memo decides whether it needs to. A session (`engine.session(ir)`, or the one each page and workspace holds) keeps one memo entry per live node, recording the output and everything that output depended on. Before running a node, the session checks the entry:

```
const nd = hash([n.t, n.p, n.trigger]);
const wants = trigger === 'submit' || triggerOf(n) === 'change';
const m = this.memo.get(n.id);
const sameInputs = m && m.inputs.length === inputs.length &&
m.inputs.every((v, i) => v === inputs[i]);
const sameReads = m && Object.entries(m.reads).every(([k, v]) => stable(s[k])
=== v);
if (m && m.digest === nd && sameInputs && sameReads && (m.full || !wants)) {
/* reuse: status 'cached' */ }
```

Four conditions must all hold. The node's definition (type, params, trigger) must be unchanged. Each input must be the identical object the last run received. Every signal the node read last time must have the same value now. And the stored output must be a full one, unless this run would skip the node anyway.

The second condition compares by identity, `===`, not by content. That is sound because the session reuses outputs: if a node was cached, its output is the very object it returned before, so the next node sees an identical input and can be cached too. Identity also makes the comparison constant time; comparing forty hits by content on every keystroke would cost more than some nodes take to run. The one place identity would fail is the

query itself, because coercing the string `q` into a list makes a new array each run. The session keeps a small cache, `session.queries(v)`, that returns the same array for the same query text, and that is enough to let the retrievers be cached when only another signal changed.

The third condition is where the reads come in. While a node runs, the session hands it the signals through a proxy that writes down every key the node reads and the value it saw:

```
const reads = {};
const sp = new Proxy(s, { get: (t, k) => { if (typeof k === 'string')
reads[k] = stable(t[k]); return t[k]; } });
```

The proxy is passed both to the component's `run` and to every lambda param, so `keep: s => s.k` records `k` even though the lambda belongs to the plan rather than the component. After the runs above, the recorded reads were:

```
rewrite: [q]   text: []   meta: []   fuse: []   filter: [filters]   rerank:
[k, q]   pack: []   answer: [q]
```

The retrievers read no signals at all. They depend on the query only through their input. This matters for the next three runs. Pressing Enter again with nothing changed reruns nothing:

```
# submit again, nothing changed 0.1 ms, calls 0     (all eight nodes: cached)
```

Selecting a facet changes `filters`. Only `filter` read it, so the run reuses everything before it:

```
# change: filters set 0.6 ms, calls 0
  rewrite    cached     1→4        0ms
  text       cached     4→40       0ms
  meta       cached     4→20       0ms
  fuse       cached     2→40       0ms
  filter     ran       40→4      0.3ms
  rerank     skipped    4→4        0ms
  pack       ran        4→4        0ms
  answer     skipped    4→0        0ms
```

Two details deserve a look. The rewrite is `cached`, not `skipped`: its last output was a full one from the Enter press, its input and reads are unchanged, so the four Claude-written queries are reused on a `change` run without a new call. The page keeps the benefit of the rewrite for as long as the question stays the same. The rerank, on the other hand, is `skipped`: its input changed (four filtered hits instead of forty), so its stored ranking no longer applies and a `change` run may not call Claude to replace it.

Now the user edits the question. The new `q` is a new input for the rewrite, which may not run on `change`, so it passes the raw query through and every node after it runs on the lexical path:

```
# change: typing 1 ms, calls 0
  rewrite    skipped    1→1        0ms
  text       ran        1→40     0.6ms
  ...
  answer     skipped   14→0        0ms
```

The Ranking playground makes the same mechanism visible with sliders. Its plan reads `s.rrfK` in the fusion, `s.headingBoost` and `s.failureBoost` in a rerank lambda, and `s.k` in the limit. Moving one slider at a time:

```
# rrfK 60→20                # failureBoost 1→3          # k 8→4
  text       cached            text       cached          text       cached
  failures_2 cached            failures_2 cached          failures_2 cached
  fuse       ran               fuse       cached          fuse       cached
  rerank     ran               rerank     ran             rerank     cached
  limit      ran               limit      ran             limit      ran
```

Each slider reruns the node that reads it and the nodes after it, and nothing before it. The session never consulted the IR's signal edges to work this out; the recorded reads were enough.

12.4 Four node statuses: ran, cached, skipped, error

Every trace entry carries one of four statuses. They are worth learning exactly, because the trace view, the DAG's highlighting and the agent's diagnoses all use them.

Status	Meaning	Output used downstream
`ran`	the node executed in this run	what it returned

Status	Meaning	Output used downstream
`cached`	the memo matched; the node did not execute	the stored output, the identical object
`skipped`	a `submit` node on a `change` run, with no usable stored output	its input (`skip: 'pass'`) or `null` (`skip: 'null'`)
`error`	the node threw	a fallback: its input for hits and queries, `null` for answers

An error does not stop the run. If a rerank lambda throws, the session records `rerank: <message>` in `result.errors`, forwards the unreranked hits, and runs the rest of the plan, so the page shows results with an error marker instead of a blank. The one exception is cancellation, which section 12.5 covers.

A `skipped` entry is also stored in the memo, marked as not full. A later `change` run with the same input may reuse it as `skipped` again; a `submit` run never will, because a skipped node owes the user a real output once they press Enter.

12.5 Cancellation, and what a keystroke costs

A user who types faster than a run completes produces runs that nobody will see. The page handles this in three places. It waits 140 ms after the last keystroke before starting a `change` run. It holds one `AbortController` per search runtime and aborts the previous run when it starts a new one. And it numbers runs, discarding any result whose number is no longer the latest. Here is the relevant part of the gui runtime:

```
schedule(trigger) {
  clearTimeout(this.debounce);
  if (trigger === 'submit') return this.runNow('submit');
  this.debounce = setTimeout(() => this.runNow('change'), 140);
}
async runNow(trigger = 'change') {
  const seq = ++this.runSeq;
  if (this.ctl) this.ctl.abort();
  const ctl = this.ctl = new AbortController();
  const r = await this.session.run(this.signals(), { trigger, signal:
ctl.signal, onText: … });
  if (seq !== this.runSeq) return r;   // a newer run started; drop this
result
  …
}
```

The session passes the signal to the sampler, so an aborted run also stops its Claude call. When a node fails with the code `cancelled`, the session records the error and stops instead of continuing to the next node. Here the abort fired 70 ms into a `submit` run, while the answer was streaming:

```
# submit, aborted at 70ms 71.1 ms, calls 3 [ 'answer: cancelled' ]
  rewrite    ran        1→4     19.4ms calls 1
  ...
  rerank     ran       40→3       21ms calls 1
  pack       ran        3→3        0ms calls 0
  answer     error      3→0        0ms calls 0 ERR cancelled
```

The run counted three calls because the answer's call had started. The rewrite and rerank results were stored in the memo before the abort, so if the user presses Enter again on the same question, those two nodes come back `cached` and only the answer calls Claude.

Putting the numbers together gives the cost model of the page. The first `change` run after a build took 26.5 ms, most of it spent warming the terms index. Later `change` runs took between 0.1 and 1 ms and made no calls. A `submit` run on this plan makes three calls, and a repeated `submit` with nothing changed makes none. A designer can promise the user that typing is free and Enter costs a fixed, small number of calls, and the engine keeps that promise by construction.

The key points of the chapter:

- Signals are the named inputs of a query run; a node depends on one either as a data input or by reading it, and the session records reads as they happen.
- A run's trigger and each node's trigger decide whether a node may run; nodes that call Claude default to `submit`, so typing never spends calls.
- The session reuses a node's output when its definition, its inputs by identity and the values of the signals it read are all unchanged, so a slider reruns only the nodes from its first reader onward.
- Every trace entry is `ran`, `cached`, `skipped` or `error`; errors fall back and continue, and cancellation stops the run.

A run's trace says which nodes ran. It also records, for every node that produces hits, the top of its list with scores and their parts, which is enough to follow one section through the whole pipeline. That is the subject of the next chapter.

Lab: Answer first, Ranking playground. In *Answer first*, type a question and open the Trace tab before pressing Enter: `rerank` and `answer` show as skipped. Press Enter and watch them run. Then pick a facet in the Sources tab and confirm that the trace shows the retrievers as cached. In *Ranking playground*, move one slider at a time and read off which nodes rerun; then predict the result for `rrfK` before you move it.

13. Traces and explanations

This chapter is about the question every search builder asks a hundred times: why did this result come first, and why is the one I wanted not there? The engine answers it from data it already keeps. Every run returns a trace with one entry per live node, and every node that produces hits records the top of its list with the score of each hit and the parts that score was made of. `rag.explain` reads those lists and follows one section through every node. With it, chapter 1's three failures (missed, misranked, misread) stop being guesses and become something you can read off a path.

All runs in this chapter are lexical and call no model, so every number is the engine's own.

13.1 The trace: status, counts in and out, time, calls

`session.run` returns a result object. Its enumerable fields are the data of the run:

Field	Contents
`hits`	the plan's hits output, the list the page shows

Field	Contents
`context`, `answer`	the packed context and the answer, when the plan has them
`outputs`	every node's output by node id; the page reads these as `s.out[nodeId]`
`trace`	one entry per live node, in execution order
`errors`	`nodeId: message` for every node that failed
`ms`, `calls`	wall time of the run and Claude calls it made

It also has three non-enumerable methods: `top(n, from)` lists the first `n` hits of the output or of any node, `chunk(id)` returns a section, and `explain(id)` is section 13.3's subject.

A trace entry records what happened at one node: `node`, the component type `t`, `status` (chapter 12.4), `in` and `out` (the sizes of its input and output), `ms`, `calls`, and, when present, `notes` and `error`. A node whose output type is `hits` also gets `top`, its first fifty hits with `id`, `score` and whichever of `why`, `parts` and `reason` the node attached. `result.toJSON()` prints the trace in one line per node, which is the form the agent reads:

```
text ran 1.3ms 1→40
failures_2 ran 0.1ms 1→6
fuse ran 0.3ms 2→46
rerank ran 0.7ms 46→46
limit ran 0.2ms 46→8
```

These five lines are the first run of the Ranking playground for "Why was the orders tab empty?". They already say a good deal. The failure-table index found 6 rows against the section index's 40; fusion merged them into 46 distinct ids, so no section appeared in both lists; the rerank reordered without dropping; the limit cut to `k`. The counts are often the fastest diagnosis there is. A retriever with `1→0` found nothing; a filter with `40→0` removed everything; a fusion whose output size equals the sum of its inputs found no overlap between its retrievers.

Timing is the second column to read. A node that takes most of a run's time is the one to look at when typing feels slow, and a Claude node's time dominates every `submit` run it is part of.

13.2 `why` and `parts`: where a hit's score came from

Scores mean different things at different nodes, and each kind of node explains its own. A BM25 retriever attaches `why`, the contribution of each query term to the score, keeping the five largest. Fusion attaches `parts`, the rank the hit had in each list it was fused from. A rerank attaches `reason`, a short string from the lambda or from Claude. Here is the same section as the first hit of `text` and of `fuse`, from the playground run with `rrfK` set to 20:

```
text  {"id":"…#failure-modes-encountered","score":11.0288,"why":
{"tab":4.74,"empty":4.36,"order":1.93},"q":"Why was the orders tab empty?"}
fuse  {"id":"…#failure-modes-encountered","score":0.04762,"parts":{"text":1}}
```

The BM25 score, 11.03, is the sum of three term contributions (chapter 5 derives each one). “tab” contributes most, more than “empty”, and “order” least, because “order” occurs in 137 of the 462 sections, against 15 for “tab”, so its inverse document frequency is low. The query's other words, “why” and “was”, are stop words and contribute nothing. The `q` field says which of the node's queries produced the best score, which matters once a rewrite has turned one question into four.

The fused score is 0.04762, which is 1/(20 + 1): reciprocal rank fusion with `k = 20` gives a hit at rank 1 in one list exactly that much (chapter 14.3). The `parts` field shows the hit came from `text` at rank 1 and did not appear in `failures_2` at all. A hit found by both retrievers would show both ranks and a score near twice as large.

Two cautions apply when reading these numbers. Scores are comparable only within one node: 11.03 and 0.048 measure different things, and a rerank lambda that multiplies the fused score produces a third scale. And `why` shows the five largest terms, not all of them, so for long queries the listed contributions can sum to less than the score.

13.3 `rag.explain`: following one chunk through every node

`result.explain(id)` walks the trace in order and, for every entry with a `top` list, reports the rank the section had there, or `null` if it was absent from a list that had input. The implementation is short enough to read whole:

```
function explain(result, id) {
  const out = { chunk: id, path: [] };
  for (const t of result.trace) {
    if (!t.top) continue;
    const i = t.top.findIndex(h => h.id === id);
    if (i < 0) { if (t.in) out.path.push({ node: t.node, rank: null });
continue; }
    const h = t.top[i];
    out.path.push({ node: t.node, rank: i + 1, score: h.score, ...(h.why ? {
terms: h.why } : {}),
      ...(h.parts ? { parts: h.parts } : {}), ...(h.reason ? { reason:
h.reason } : {}) });
  }
  const c = result.chunk(id);
  if (c) out.where = `${c.title} › ${c.headingPath.join(' › ')}`;
  return out;
}
```

The golden case “How does the eval tool report line numbers in errors?” (chapter 18) expects a section under “5. The eval tool” of the agent runtime spec. On a plan with a section retriever, a metadata retriever, fusion and `limit(8)`, the first relevant section comes fifth, which gives the case an MRR of 0.2. Explaining the section the user wanted, “5.2 Execution”, and the one that beat it:

```
explain(…#5-2-execution-2)                                  explain(…#21-6-canvas-
2)   (rank 1)
  text   rank 5  score 11.0819                                 text   rank 1
score 12.6489
         terms eval 3.33  line 2.9  tool 2.6  number 2.25          terms eval
2.62  number 2.36  tool 2.12
  meta   rank null                                                      line
1.98  error 1.91
  fuse   rank 5  score 0.01538  parts { text: 5 }            meta   rank null
  limit  rank 5  score 0.01538  parts { text: 5 }            fuse   rank 1
score 0.01639  parts { text: 1 }
  where: Agent runtime spec … › 5. The eval tool › 5.2 Execution
```

The path tells the whole story. Nothing after the section retriever changed the order: the metadata index found nothing (`rank null` at `meta` for both), fusion with a single contributing list preserves rank, and the limit kept the first eight. The ranking was decided by BM25 alone, and BM25 preferred “21.6 Canvas” because that section happens to

contain all six query terms (eval, tool, report, line, number, error; the why field lists only the largest five), while “5.2 Execution” contains four, each scoring higher than in the Canvas section but not enough to make up for the two it lacks. The remedy is therefore a ranking change, not a retrieval change: a heavier heading-path weight (chapter 5.5) or a rerank that rewards heading overlap (chapter 16.4) would both lift a section titled “The eval tool” over one titled “Canvas”.

One property of the current implementation limits where `explain` can be used. Only entries for nodes that ran carry a `top` list; a `cached` entry records counts and status but not the hits. On a run where most nodes were cached, such as a facet click after a search, `explain` reports only the nodes that ran. To follow a section through every node, explain a run in which they all ran: the first run of a fresh session, or a run after a change to the query.

13.4 Missed, misranked, misread: diagnosing chapter 1's three failures from a trace

Chapter 1 named three ways a search can fail the reader. With `explain` each one has a recognizable signature in the path of the section that should have been found.

Failure	Signature in the path	Where to fix it
Missed	`rank: null` at every retriever	the corpus side or the query: chunking, fields, synonyms, rewriting
Misranked	a rank at the retriever, then `null` at a later node, or a rank above `k` at the output	ranking: weights, fusion, rerank, `k`
Misread	rank within `k` at the output and the section in the packed evidence, yet the answer is wrong	the answer step: packing order, budget, prompt

A missed section is one no retriever found. The question “Why did the pod keep dying?” is about the incident in section 4.1 of the CoinVault report, “The typed-nil deployment failure”, in which the browser service went into a restart loop. The path of that section is empty all the way down:

```
Why did the pod keep dying? | query terms: pod keep dying | text:null
meta:null  fuse:null  limit:null
```

The query terms column explains why. After stop words are removed and stems taken, the query is `pod keep dying`, and the section says “restart loop”, “startup failed” and “browser service”. Not one query term occurs in it. No ranking change can repair this, because nothing downstream ever sees the section. The fixes are on the vocabulary side: index-time synonyms (chapter 6.3), a query rewrite that produces “crash loop” or “restart” (chapter 16), or enrichment that adds such keywords to the section at build time (chapter 17.4).

A misranked section is one a retriever found and a later stage lost. A rephrasing of the same question shows it:

```
why did the browser container keep crashing | browser container keep crash |
text:18  meta:null  fuse:18  limit:null
```

The word “browser” matched, so the section was retrieved at rank 18, and fusion kept that rank. The limit cut the list to eight, and the section was gone. The section retriever has `k: 40`, so it was well inside the candidate set. This is the failure a rerank exists for: a step that looks at more than term overlap and can move rank 18 to rank 3. It is also the case where raising `k` hides the problem from the eval while leaving it in front of the reader, who still has to scroll past seventeen sections.

The eval-tool question in section 13.3 is a milder misranking: the section survived to the output at rank 5, so it counts as a hit at 5 and costs only MRR.

A misread is the one failure the retrieval trace cannot show, because retrieval succeeded. Its signature is a section that reaches the output within `k`, appears in `answer.evidence` with a label such as `E3`, and yet the answer is wrong or says the evidence is missing. Three checks narrow it down. If `E3` is absent from `answer.cited`, the model had the section and did not use it; look at where the section sits in the packed context and how much text surrounds it. If the context’s `bytes` is close to the pack budget, other evidence the answer needed may have been left out, because the pack node stops at the first section that does not fit. If the section is cited and the answer still misstates it, the problem is the prompt or the model tier. Showing a real misread needs a real model answer, so this book leaves it to the Lab and to the answer checks of chapter 20.4.

The key points of the chapter:

- A run returns a trace with one entry per live node, and its status, counts, time and calls are usually enough to locate a problem before looking at any hit.
- Each kind of node explains its own scores: BM25 hits carry per-term contributions, fused hits carry their source ranks, and reranked hits carry a reason.

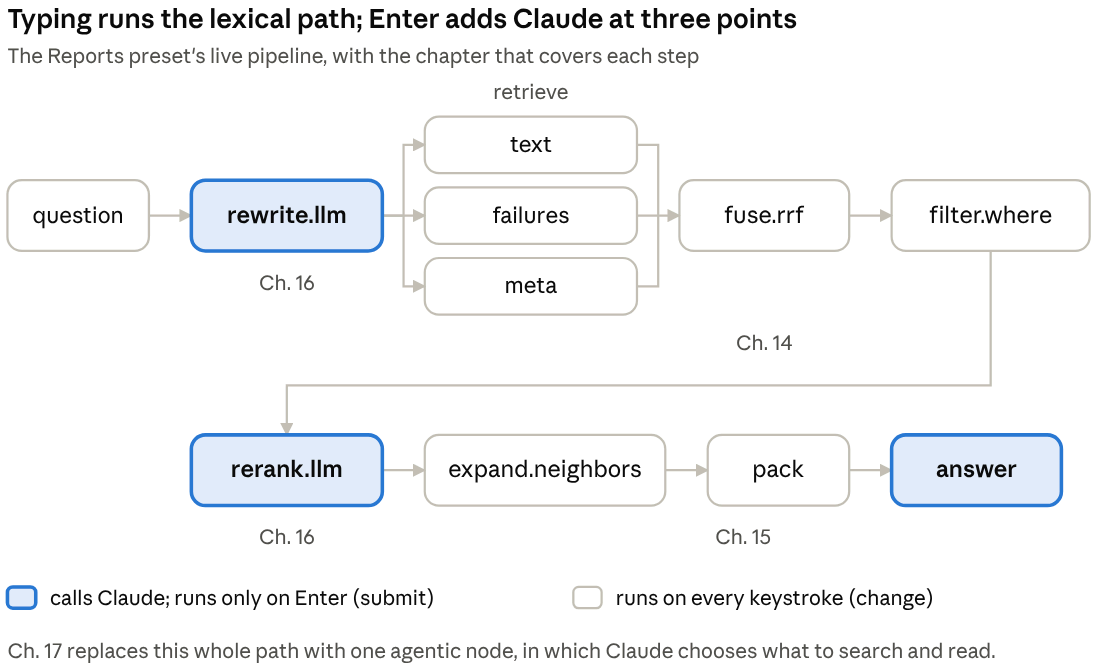
- `result.explain(id)` follows one section through every node that ran and reports its rank and score at each, so the node where a section was lost can be read off its path.
- Missed, misranked and misread failures have distinct signatures in that path, and each points to a different part of the pipeline to change.

Part IV has now covered the whole engine: builds that are reused by digest, query runs that are reused by input and signal reads, and traces that explain what both did. Part V uses this machinery to build better rankings, starting with the question the fused scores above raised: how to combine lists whose scores are not on the same scale.

Lab: Compare stages. Load *Compare stages* and type “cursor semantics tension”. The three columns show the `text`, `fuse` and `limit` outputs side by side. Pick a section that moves between columns, open the Trace tab and the agent panel, and ask the agent to run `explain` on it; the path it returns should match the columns you see. Then try the two typed-nil questions from section 13.4 and confirm their paths.

Part V · Ranking and answers

The live half of the pipeline turns a question into ranked hits, packs the best of them into a context, and asks Claude for a cited answer. This part takes those steps in order, then gives Claude a larger role in each.



the live pipeline of the Reports preset · three steps call Claude

The chapters follow this pipeline. Chapter 14 covers the lexical core in the top row, chapter 15 the path from hits to an answer, and chapter 16 the two places where Claude can rewrite the queries or reorder the hits.

14. Retrieval and fusion

This chapter explains how a pipeline with several indexes produces one ranked list. A `retrieve` node asks one index for hits, and a `fuse` node combines several such lists into one. The central claim is that scores from different indexes are on different scales, so they should be combined by rank rather than by adding scores. By the end you should be able to compute a fused score by hand, predict what the fusion constant does, and choose between the two fusion rules the library ships.

The examples use the hybrid corpus from the *Reports* preset: a `text` BM25 index over sections, a `failures` BM25 index over the rows of failure tables, and a `meta` terms index over frontmatter. The question is the book's running example, *Why did ChatGPT show no app actions after OAuth worked?* All output is from `rag.js` running in node over the eleven reports, with `change` runs, so no Claude calls are involved.

14.1 The retrieve node: one index, several queries

A `retrieve` node has one input port of type `queries` and one output of type `hits`. When it is wired to the `q` signal, the session turns the question string into a one-element list (chapter 12). When a rewrite step sits in front of it, the list can hold several phrasings. The node's whole job is in a dozen lines of `rag.js`:

```
const qs = p.perQuery === false ? [queries.join(' ')] : queries;
const best = new Map();
for (const q of qs) for (const h of ix.search(q, p.k ?? 20, { synonyms:
ctx.synonyms })) {
  const prev = best.get(h.id);
  if (!prev || h.score > prev.score) best.set(h.id, { ...h, q });
}
return [...best.values()].sort((a, b) => b.score - a.score)
  .slice(0, p.k ?? 20).map(h => ({ ...h, via: ctx.node.id }));
```

Each query is searched on its own, and a chunk found by several queries keeps its best score. Every hit records the query that found it in `q` and the node that produced it in `via`. The alternative, `perQuery: false`, joins the queries into one string, so a chunk that matches several phrasings accumulates term contributions from all of them. Here is the difference for *why did DCR fail* with a rewrite lambda that adds two phrasings:

```
perQuery (default)
1. 13.596   q="last_used_at migration"            CoinVault › 5.1
Observability channels
2. 12.7352  q="dynamic client registration 503"  CoinVault › 1. The
acceptance problem
3. 11.022   q="last_used_at migration"            CoinVault › 15. Failure
chronology and diagnostic lesson

perQuery: false
1. 23.7946  q="why did DCR fail dynamic client registration 503 last_used_at
migration"  CoinVault › 15. Failure chronology
2. 22.0751  (same joined query)
CoinVault › 5.1 Observability channels
3. 16.862   (same joined query)
CoinVault › 6.2 …
```

The joined form rewards the failure-chronology section, which mentions the 503, the registration and the migration together; the per-query form lets each phrasing put forward its own best section. Neither is always right. Per-query is the default because it keeps a bad rewrite from changing the scores of the original question.

A retrieve node's id defaults to the index name, so `retrieve('text')` becomes node `text`. If that id is taken, the compiler appends a suffix. In the Reports preset a chunk node is already called `failures`, so the second retriever becomes `failures_2`. Section 14.3 shows why this matters.

For the running question the three retrievers return these lists:

Rank	`text` (BM25 over sections)	`failures_2` (BM25 over failure rows)	`meta` (terms)
1	26.476 · 11. Why ChatGPT still showed no actions	16.541 · row: ChatGPT connected but showed no actions	1.213 · 1.1 Exact deployed identity
2	19.340 · 6.1 RFC 9728 path-specific metadata	14.626 · row: ChatGPT said OAuth unsupported	1.213 · 1. The acceptance problem
3	19.121 · 15. Failure chronology and diagnostic lesson	6.045 · row: Authorization returned invalid_scope	1.213 · 10. The hidden tool-scope propagation bug

Rank	`text` (BM25 over sections)	`failures_2` (BM25 over failure rows)	`meta` (terms)
4	18.522 · 9. Consent behavior in a real browser	2.271 · row: Inspector consent returned invalid_grant	1.213 · 11. Why ChatGPT still showed no actions
hits	40	6	20

All three lists point at the CoinVault report, which is correct. They disagree about everything else, including the scale of their numbers.

14.2 Why two BM25 scores cannot be added

A BM25 score (chapter 5) is a sum over query terms of an IDF weight times a saturated, length-normalized term frequency. Every part of that formula is computed inside one index: the IDF depends on how many chunks of *that* collection contain the term, the length normalization depends on *that* collection's average length, and the field weights are *that* index's configuration. Two indexes over different chunk streams therefore produce numbers that cannot be compared. The `text` index's best score above is 26.5; the failure-row index's best is 16.5; the terms index gives every chunk of the matching document the same 1.213.

The effect holds even for the same chunk and the same query. In the *Fusion* preset, the section *Metadata: two representations, one* of the Agentforum report scores 24.28 in the `text` index and 3.93 in a `headings` index that sees only heading paths. If scores were added, the `text` index would decide every ranking and the other indexes would contribute rounding noise.

Normalizing each list, for example to the range 0 to 1, looks like a fix but depends on the shape of each list. The `meta` list is flat: twenty chunks tie at 1.213, because the terms index scores a match on a frontmatter value with `1 / log(2 + n/10)` and gives that score to every chunk of the document. Its order within the tie is alphabetical by chunk id, which carries no information. Any normalization of that list turns an arbitrary tie-break into apparent preference.

Ranks avoid the problem. Rank 1 means "the best this retriever could find," whatever the retriever's scale, and a rank-based rule only needs to decide how much a rank-1 vote is worth compared with a rank-10 vote.

14.3 Reciprocal rank fusion: 1/(k + rank), weights, and what k = 60 does

Reciprocal rank fusion (Cormack, Clarke and Büttcher, 2009) scores a chunk by summing, over the lists it appears in, the reciprocal of a constant plus its rank:

score(c) = Σ_lists w_list / (k + rank_list(c))

The library's `fuse.rrf` is a direct transcription. Its input port is `hits*`, which accepts any number of hit lists; each list's weight is looked up by the id of the node that produced it:

```
lists.forEach((list, li) => {
  const src = ctx.node.in[li], wt = w[src] ?? 1;
  arr(list).forEach((h, r) => {
    const a = acc.get(h.id) || { id: h.id, score: 0, parts: {} };
    a.score += wt / (k + r + 1);
    a.parts[src] = r + 1;
    acc.set(h.id, a);
  });
});
```

Each fused hit carries `parts`, the rank it had in each input list, which is what the Trace tab shows when you select a fused row. For the running question with `k: 60` and weights `{ text: 1, failures: 0.8, meta: 0.3 }` the top three are:

Fused rank	Section	parts	By hand	Engine
1	11. Why ChatGPT still showed no actions	text 1, meta 4	1/61 + 0.3/64 = 0.02108	0.02108
2	15. Failure chronology and diagnostic lesson	text 3, meta 7	1/63 + 0.3/67 = 0.02035	0.02035
3	1. The acceptance problem	text 6, meta 2	1/66 + 0.3/62 = 0.01999	0.01999

Two things in this table deserve attention. First, the failure row *ChatGPT connected but showed no actions*, rank 1 in the failure index and arguably the most direct answer in the corpus, is not in the fused top ten. It appears in one list only, so its score is 1/61 = 0.01639, while the tenth fused hit has 0.01733. Sections appear in both `text` and `meta`, because the terms index votes for every chunk of a matching document; failure rows are a separate chunk stream that `meta` never sees. RRF rewards agreement among retrievers,

and a retriever over its own chunk stream cannot receive agreement. The remedies are a larger weight for that retriever, a smaller `k` (below), or a terms index over the failure rows as well.

Second, the weight `failures: 0.8` in the preset never applies. Weights are keyed by input node id, the retriever's id is `failures_2`, and a missing key falls back to weight 1. The compiler does not check weight keys against input ids, so the mistake is silent. You can see it by printing the node:

```
reports fuse in [ 'text', 'failures_2', 'meta' ] {"k":60,"weights":
{"text":1,"failures":0.8,"meta":0.3}}
```

The constant `k` sets how quickly a vote loses value with rank. With small `k`, rank 1 is worth far more than rank 2; with large `k`, all ranks are worth nearly the same and the score approaches a weighted count of the lists a chunk appears in. Here are the top three for four values:

k	1st	2nd	3rd
1	0.56 · 11. Why ChatGPT still showed no actions	0.5 · failure row *ChatGPT connected but showed no actions*	0.348 · 6.1 RFC 9728 path-specific metadata
10	0.112 · 11. Why ChatGPT …	0.0946 · 15. Failure chronology	0.0933 · 6.1 RFC 9728 …
60	0.02108 · 11. Why ChatGPT …	0.02035 · 15. Failure chronology	0.01999 · 1. The acceptance problem
600	0.00216 · 11. Why ChatGPT …	0.00215 · 15. Failure chronology	0.00215 · 1. The acceptance problem

At `k = 1` the failure row reaches second place on the strength of a single first-place vote. At `k = 600` the second through fourth scores differ in the fifth decimal, so the order is decided almost entirely by how many lists a chunk is in and their weights. The value 60 comes from the original paper, where it worked well across many TREC runs; it is a reasonable default, not a law.

The *Fusion* preset scores four variants on the twelve golden questions (chapter 18). These are real results:

Variant	hit@5	MRR	nDCG@10
fusion (rrf, k 60, weights text 1, headings 0.7, meta 0.3)	1	0.938	0.850
rrf-k10	1	0.944	0.862
fuse-max	1	0.933	0.847
text-weight-only	1	0.933	0.847
equal-weights	1	0.896	0.865

The differences come from two or three questions. *How does protojson serialize int64 values?* finds its section at rank 1 under max fusion and rank 4 under the default RRF, while *How does the eval tool report line numbers in errors?* goes the other way, from rank 5 to rank 1. With twelve cases, one question moving changes MRR by several hundredths; chapter 19 discusses how large a difference must be before it means anything.

`fuse.rrf` also takes `limit` (default 60), the length of the fused list, and sets `via` on each hit to the fuse node's id.

14.4 Max fusion, and when it is the better rule

`fuse.max` keeps, for each chunk, the hit with the highest raw score from any list:

```
for (const list of lists) for (const h of arr(list))
  if (!acc.has(h.id) || acc.get(h.id).score < h.score) acc.set(h.id, h);
return [...acc.values()].sort((a, b) => b.score - a.score);
```

This compares raw scores across lists, which section 14.2 argued against, and the running question shows the consequence: all of the top six fused hits come from `text`, whose scores run highest. On the goldens, `fuse-max` produces exactly the same numbers as `text-weight-only`; with these three indexes, max fusion is text-only retrieval.

Max fusion is the right rule when the lists share a scale. That happens when several lists come from the same index, for example several retrieve nodes over `text` with different query rewrites, or when one index is primary and the others exist to add chunks the primary missed, never to reorder it. In the first case the retrieve node's `perQuery` mode already does max fusion internally.

14.5 Filters and limits: `filter.where`, filter lambdas and `s.k`

After fusion, a pipeline usually narrows the list. `filter.where` reads a signal (default `filters`) holding an object of frontmatter keys to accepted values, and keeps hits whose chunk metadata contains one of the accepted values for every key. Values within a key are alternatives; keys are all required; matching ignores case; the key `doc` matches the document id. An empty object passes every hit through, which is why the facet panel's initial state is `{}`.

A filter lambda has the shape `(s, ctx, hit, p) => boolean`, where `hit` is a view with the chunk's `title`, `headingPath`, `text`, `meta` and `tokens` already attached. The *Faceted search* preset chains both:

```
rag.plan('facets').over(kit.reports())
  .signals({ q: 'sync', k: 10, filters: {}, codeOnly: false })
  .retrieve('text', { k: 80 })
  .filter.where()
  .filter((s, ctx, hit) => !s.codeOnly || hit.text.includes('```'), { id:
'code_only' })
  .limit(30);
```

These are the trace counts for five signal settings:

Signals	text	filter	code_only	limit
`{}`	1→80	80→80	80→80	80→30
`filters: { tags: ['go'] }`	1→80	80→18	18→18	18→18
`filters: { tags: ['go', 'r'] }`	1→80	80→18	18→18	18→18
`filters: { type: 'spec' }`	1→80	80→1	1→1	1→1
`tags: ['go']`, `codeOnly: true`	1→80	80→18	18→6	6→6

The `type: 'spec'` row shows the cost of filtering after retrieval: only one of the 80 retrieved sections came from the one spec document, so the user sees one result even though the spec has dozens of sections. A filter can only remove; it cannot ask the index for more. When filters are expected, either retrieve more (`k: 80` instead of 20) or filter inside the retriever with a custom lambda that calls `ix.searchIn(docId, q, k)`.

`limit` cuts the list to `k`. Its parameter can be a number or a lambda, and `limit(s => s.k)` reads the `k` signal, so moving a result-count slider reruns only the limit node (chapter 12).

The key points:

- A retrieve node searches each query separately by default and keeps each chunk's best score, so one bad phrasing cannot lower the scores of the others.
- BM25 scores from different indexes are on different scales, because IDF, length normalization and field weights are all computed per index; even one chunk can score 24 in one index and 4 in another.
- Reciprocal rank fusion combines lists by rank, rewards chunks that several retrievers agree on, and lets `k` trade the value of a first place against the value of agreement.
- Fusion weights are keyed by input node id. A key that names no input is ignored without an error, as the Reports preset's `failures` weight shows.
- Max fusion is correct only when the lists share a scale; across different indexes it reduces to the highest-scoring index.
- Filters can only remove hits, so a pipeline that expects narrow filters must retrieve more than it shows.

The fused, filtered list is still a list of ids and scores. The next chapter turns its best entries into the text Claude reads and the answer the user sees.

Lab: Fusion; Reports · hybrid + rerank + answer. Load *Fusion*, open the Evals tab, tick all variants and run them; then find the protojson question in the per-case table and select the `fuse` node in the DAG tab to see its `parts`. Set `k` to 1 and then to 600 and watch the order of the top five change. In *Reports*, select the fuse node and compare its `weights` keys with the ids of its inputs; rename the weight key to `failures_2`, rerun the running question, and see where the failure row lands.

15. Context and answers

This chapter covers the three steps between a ranked list and an answer: expanding the best hits with nearby material, packing them into a context under a size budget, and asking Claude to answer from that context with citations. The point to take away is that the answer can cite only what was packed. Packing is therefore the last ranking decision in the pipeline, and it is made by simple, inspectable rules rather than by the model.

The examples use the *Context expansion* preset and its question, *How does batch-preserving sync relate to idempotent operation ids?* That question is useful because its answer spans two reports: the Workbench Identity report defines operation ids, and the Workbench Core report describes the sync loop. The prompt text below is the real prompt the engine builds. Claude's replies come from a mock sampler that returns a fixed string, and they are marked as mock wherever they appear.

The plan is:

```
rag.plan('context').over(kit.reports())
  .signals({ q: 'How does batch-preserving sync relate to idempotent
operation ids?', k: 6 })
  .retrieve('text', { k: 30 })
  .limit(s => s.k)
  .expand.neighbors({ before: 0, after: 1, top: 4 })
  .expand.links({ decay: 0.4, max: 3, top: 3 })
  .pack({ budget: 40000, order: 'doc' })
  .answer();
```

Its trace on a `submit` run:

```
text       ran  1→30
limit      ran 30→6
neighbors  ran  6→8
links      ran  8→10
pack       ran 10→8
answer     ran  8→8   calls 1
```

The rest of the chapter explains each of those counts.

15.1 Expanding hits: neighbouring sections and linked reports, with score decay

A section is a unit of retrieval, not a unit of meaning. The section that matches a question best often sets up an idea that the next section completes. `expand.neighbors` adds, for each of the first `top` hits, the sections just before and after it in reading order. Reading order comes from the first chunk stream of the corpus, so a neighbour of a section is always another section of the same document.

```
hits.forEach((h, i) => {
  out.push(h);
  if (i >= (p.top ?? 5)) return;
  for (const n of ctx.neighbors(h.id, { before: p.before ?? 0, after: p.after
?? 1 })) {
    if (have.has(n.id) || extra >= (p.maxChars ?? 6000)) continue;
    have.add(n.id);
    extra += n.text.length;
    out.push({ id: n.id, score: +(h.score * (p.decay ?? 0.5)).toFixed(5),
via: ctx.node.id, of: h.id });
  }
});
```

Three details matter. A neighbour is inserted directly after the hit that brought it, so list order keeps the two together. Its score is the parent's score times `decay` (0.5 by default), which keeps it below its parent if a later step sorts. And it records its parent in `of`, which the results list shows as a chip. In the example, the top four hits each asked for one following section, but only two were new:

```
1. 18.1984  via text                                  6.1 Batch identity
2.  9.0992  via neighbors  of 6.1                     6.2 Concrete send
identity
3. 17.116   via text                                  18. Tests as identity
laws
4.  8.558   via neighbors  of 18.                     19. Consumer and
packaging verification
5. 17.0725  via text                                  6. Two levels of
operation identity
6. 17.0014  via text                                  1. The problem was
semantic, not syntactic
…
```

The neighbour of section 6 is 6.1, already present; the neighbour of section 1 is section 2, which was already at rank 6. The `maxChars` cap (6,000 characters by default) bounds how much expansion can add, whatever `top` says.

All eight hits so far come from one report. `expand.links` reaches across reports. At build time the engine resolves each document's `[[wiki links]]` and its `related_vault_notes` frontmatter to document ids by matching titles, file names and aliases. In this corpus four reports have links:

Report	Linked reports
PBUI Workbench Identity	2
PBUI Datalab on Workbench Core	2
PBUI Workbench Stabilization	2
PBUI Workbench Core	1
the other seven	0

For each of the first `top` hits, `expand.links` visits the linked documents not yet represented, searches each one for the question with `searchIn`, and adds the best section, scored at the parent's score times `decay`. With `decay: 0.4` and `max: 3` it added two sections:

```
 9. 7.27936  via links  of 6.1   PBUI Workbench Core › 5. The core's modules
› 5.10 Batch-preserving sync, and persistence
10. 7.27936  via links  of 6.1   PBUI Workbench Stabilization ›
(introduction)
```

The first of these is exactly the missing half of the question: the section that describes the sync loop. It did not rank in the top six by itself because the question's words about ids and idempotency dominate. Following an author's link found it, which is a retrieval signal that BM25 cannot see.

15.2 Packing under a budget: rank order, per-document caps, the 40 KB default

`pack` turns hits into one string and an evidence table. Its rules are short enough to state in full:

```
for (const h of hits) {
  const c = ctx.chunk(h.id);
  const n = perDoc.get(c.docId) || 0;
  if (n >= (p.maxPerDoc ?? 6)) continue;            // cap per document
  const piece = c.text.length;
  if (bytes + piece > budget && evidence.length) break; // stop at the first
overflow
  perDoc.set(c.docId, n + 1);
  bytes += piece;
  evidence.push({ label: 'E' + (evidence.length + 1), id: c.id, docId:
c.docId, title: c.title, headingPath: c.headingPath });
}
```

Packing walks the list in order; it does not re-rank. It skips a chunk whose document already supplied `maxPerDoc` pieces (six by default), and it stops at the first chunk that would overflow the budget, except that the first chunk always enters, so a context is never empty when there are hits. The budget counts characters of chunk text (JavaScript string length), not bytes; the default of 40,000 is about 40 KB for English text. It leaves most of Claude's prompt allowance (chapter 31) for the instructions and the question, and it keeps the cost of each answer predictable.

In the example, ten hits became eight pieces. The Identity report contributed eight of the ten hits, so the cap of six dropped its last two (*13. Invalid aggregate isolation* and *2. The three-domain identity model*), and the two linked sections entered after them. The result was 8,361 characters from three reports. Changing the budget and cap gives:

budget	maxPerDoc	pieces	characters	reports
40,000	6	8	8,361	3
8,000	6	7	7,791	2
40,000	2	4	3,724	3
1,000	6	1	899	1

The per-document cap is a diversity rule. Without it, a question that one report answers well would fill the whole context with that report, and a second report holding the other half of the answer would never be read. The 8,000-character row shows how the budget and the cap interact: the six Identity pieces use 6,435 characters, the cap skips the remaining Identity hits, the Workbench Core section (1,356 characters) still fits, and the

Stabilization introduction is the first piece that would overflow. Had the cap been higher, two more Identity sections would have entered first and the Core section, the sync half of the answer, would not have fitted.

`order: 'doc'` changes the order of the text, not the selection. Pieces are grouped by document and sorted by their position in the document, so Claude reads each report's sections in the order their author wrote them. Labels are assigned before reordering, in rank order, so in this example the context begins with `[E7]`, the Workbench Core section.

15.3 Evidence labels and the answer prompt

Each piece in the context starts with its label, title and heading path. This is the real beginning of the context for the example:

```
[E7] PBUI Workbench Core: A Headless Engine, a Pure Planner, and the Hard
Cutover of the React Shell › 5. The core's modules › 5.10 Batch-preserving
sync, and persistence
`sync/index.ts` is the union of the agentlogic and turboproof loops that the
earlier report described, rewritten so the outbox holds whole committed
batches:
…

---

[E1] PBUI Workbench Identity: Local Revisions, Opaque Server Tokens, and
Idempotent Sync Operations › 6. Two levels of operation identity › 6.1 Batch
identity
…
```

The `answer` node wraps the context in a fixed frame. This is the prompt it sent, with the evidence elided:

```
Answer the question using only the evidence below, drawn from a set of
engineering project reports. Cite evidence inline as [E1], [E2]. If the
evidence does not answer the question, say what is missing. Be specific and
concise: under 200 words unless the question needs more. Use plain markdown.

<evidence>
[E7] PBUI Workbench Core …
…
</evidence>

Question: How does batch-preserving sync relate to idempotent operation ids?
```

The whole prompt was 9,980 characters, sent at the `default` model tier. Each sentence of the instruction has a job. *Using only the evidence below* tells the model not to answer from its own knowledge, which for private project reports would be invention. *Cite evidence inline as [E1]* fixes a syntax that code can parse afterwards. *Say what is missing* gives the model a correct thing to do when retrieval failed, instead of an incentive to guess.

The `instructions` parameter replaces that first paragraph; the evidence block and the question are always appended. The *Two answers* preset uses this to branch one context into two answers:

```
const ctxt = g.node('pack', rag.pack({ budget: 30000 }), { from: top });
g.node('brief', rag.answer({ tier: 'quick', instructions: 'Answer in at most
two sentences, using only the evidence below. Cite as [E1].' }), { from: ctxt
});
g.node('detailed', rag.answer({ instructions: 'Answer thoroughly from the
evidence below. End with a short bulleted list of the key facts. Cite every
fact as [E1], [E2].' }), { from: ctxt });
```

Its trace shows one pack feeding two answer nodes, one call each:

```
text:ran 1→30 | top:ran 30→8 | pack:ran 8→8 | brief:ran 8→8 calls 1 |
detailed:ran 8→8 calls 1
```

15.4 Citations as data, and an answer that says what is missing

The answer node returns a value, not just text:

```
{ text: "MOCK answer citing [E1] and [E2].",        ← mock sampler output
  evidence: [ { label: "E1", id: "…#6-1-batch-identity", docId: "…", title:
"…", headingPath: […] }, … 8 entries ],
  cited: [ "E1", "E2" ],
  truncated: false }
```

`cited` is computed by matching `\bE\d+\b` in the text. `evidence` maps every label to a chunk id. Together they let the interface turn `[E2]` into a link to section 6.2, show which packed pieces were cited and which were not, and let evals score an answer-only plan by the sections it used (chapter 18). They also bound what a citation can prove: the engine checks that a label exists, not that the sentence beside it is supported by that section. Checking support is the job of a judge, which chapter 20 adds.

When retrieval returns nothing, the answer node does not call Claude at all. A question with no matching terms produces this, with zero calls in the trace:

```
{ text: "No evidence was retrieved for this question.", evidence: [], cited:
[] }
```

That message is written by the engine, not the model. When there is evidence but it does not answer the question, the instruction asks the model to say what is missing; that is a request, and only an eval with `answerHas` or a judge can tell whether it was honoured.

15.5 Streaming the answer and showing its sources

`answer` has trigger `submit` and `skip: 'null'`. On a `change` run (typing) it is skipped and its output is `null`; the trace shows `answer:skipped` and the answer panel keeps the last answer or stays empty. On a `submit` run it calls Claude and streams text as it arrives through the run's `onText(nodeId, text)` callback, which the `answer` component uses to show the response as it is written. The node id in the callback is what lets a page with two answer nodes route each stream to its own panel. With the mock sampler, the stream is a single 33-character delivery; with Claude it arrives in many pieces.

For a designer, these mechanics translate into a few decisions:

- The answer area should say that it waits for Enter, because typing alone will never fill it.
- Each `[E#]` in the text should link to the section it names, and the list of sources should mark which pieces were cited, so the reader can check a claim in one click.
- A source list should show the title and heading path of each piece, which is the same information the model saw in each piece's header.

- `truncated: true` means the model stopped at its output limit; the interface should say so rather than present a cut-off answer as complete.

The key points:

- Expansion adds sections the ranking missed, from the same document by reading order or from linked documents by the author's own links, and marks each added hit with the hit that brought it.
- Packing walks the list in order, caps each document's share, and stops at the first piece that would overflow the budget, so the context is a predictable function of the list.
- The budget is counted in characters of chunk text, 40,000 by default.
- Evidence labels are assigned in rank order and survive reordering, so a citation always names the same section regardless of where it appears in the prompt.
- An answer is a value with text, evidence and parsed citations; the engine verifies that cited labels exist, not that claims are supported.
- With no evidence, the engine answers for itself and spends no call.

Everything in this chapter is deterministic except the final call. The next chapter gives Claude two earlier places to act, before retrieval and between retrieval and packing.

Lab: Context expansion; Two answers. In *Context expansion*, press Enter and look at the result chips: the `neighbors` and `links` rows name the hit that brought them. Set `maxPerDoc` on the pack node to 2 and check which reports enter the context; then set `budget` to 8000 with maxPerDoc back at 6 and see which piece is the first to drop. In *Two answers*, edit the `brief` node's `instructions` in the DAG tab and press Enter; the trace shows one pack and two answer calls.

16. Rewriting and reranking

A ranking can be changed without touching an index. Before retrieval, a pipeline can change the queries: add expansions of an acronym, keyword phrasings of a question, or a drafted answer. After retrieval, it can change the order: boost by features the index never scored, or let Claude read the passages and reorder them. This chapter covers both places, first with lambdas that cost nothing and then with Claude. It ends with the rule that keeps Claude's steps off the keystroke path.

As in the previous chapter, prompts are the real prompts the engine builds, and every Claude reply shown is from a mock sampler. The mocks were written to exercise the code paths, including malformed replies; they say nothing about how Claude would actually

rank these passages.

16.1 Rewrite lambdas: acronyms and synonyms

A `rewrite` node maps a list of queries to a list of queries. Its lambda has the shape `(s, ctx, queries, p) => string[]`; the engine deduplicates the result and drops empty strings. The *Query expansion* preset uses one to expand acronyms from a table passed as a parameter:

```
.rewrite((s, ctx, queries, p) => {
  const q = String(s.q).toLowerCase();
  const extra = Object.entries(p.acronyms)
    .filter(([a]) => new RegExp('\\b' + a + '\\b').test(q))
    .map(([a, full]) => q.replace(new RegExp('\\b' + a + '\\b', 'g'), full));
  return [...queries, ...extra];
}, { acronyms: { lca: 'latent class analysis', dcr: 'dynamic client
registration', /* … */ } })
.retrieve('text', { k: 30, perQuery: true })
```

The table is a parameter, not a closure variable, because lambdas are stored as source and run without their closures (chapter 9). For *what is the knee rule for LCA* the node produces two queries, and the second changes what is found:

```
queries ["what is the knee rule for LCA", "what is the knee rule for latent
class analysis"]

with rewrite
without rewrite
1. 20.078   Tier B › The mixture models… › Latent class analysis       1.
9.0812  Tier B › The mixture models… › Latent class analysis
2. 16.5276  Broken Exports › What model selection actually established   2.
7.7937  Tier B › What was learned
3. 16.2573  Tier B › The mixture models… › Latent class analysis…      3.
7.3555  Broken Exports › What model selection actually established
```

The top hit records `q: "what is the knee rule for latent class analysis"` and its term contributions `{ latent: 5.77, class: 5.25, knee: 4.81, analysis: 4.24 }`. The word *LCA* alone matched little, because the reports mostly spell the term out.

The same corpus also has an index-time synonym table (chapter 6). The two mechanisms differ in where they act and how much weight they give:

```
index.terms('why did the oauth cursor break', synonyms)
→ oauth 1, cursor 1, break 1, authorization 0.5, token 0.5, sequenc 0.5, next 0.5, nextcursor 0.5
```

	Synonym table	Rewrite lambda
Acts on	the terms of one query, inside the index	the list of queries, before retrieval
Weight of added words	0.5	1, in a query of their own
Effect with `perQuery`	none; it is still one query	each phrasing is searched and ranked on its own
Visible in the trace	as term contributions in `why`	as the `rewrite` node's output list
Good for	near-synonyms that should help but not dominate	expansions that are the question in other words

16.2 `rewrite.llm`: keyword queries in the corpus's own vocabulary

Users ask in their own words; reports are written in their authors' words. `rewrite.llm` asks Claude to bridge the gap. This is the exact prompt for the *Rewrite + rerank* preset's question:

```
Rewrite the question as 3 short keyword queries (2-6 words each) for a BM25 index over engineering project reports. Use the vocabulary such reports would use, including likely technical terms. Reply with only JSON: {"queries": [string]}.

Question: Why did ChatGPT say OAuth was unsupported?
```

It is sent at the `quick` tier through `sample.json`, which parses the reply. The node keeps the original question first and appends at most `n` new queries. With the mock reply, its output was:

```
["Why did ChatGPT say OAuth was unsupported?",      ← always kept
 "stateless discovery tools/list",                  ← mock reply
 "chatgpt connector no actions",                    ← mock reply
 "mcp session initialize"]                          ← mock reply
```

Keeping the original matters for the same reason `perQuery` is the default: if the rewrite is poor, the original question still gets its own ranking. The prompt asks for keywords rather than sentences because BM25 ignores grammar; *2-6 words* keeps each query focused on one idea.

16.3 HyDE: searching with a drafted answer

HyDE (hypothetical document embeddings, Gao et al., 2022) was proposed for vector search: draft a plausible answer, then search with the draft instead of the question, because an answer resembles other answers more than a question does. The idea transfers to BM25 because a drafted answer uses the vocabulary of the documents. The library has no HyDE component; the *HyDE* preset is a rewrite lambda that calls Claude:

```
.rewrite(async (s, ctx, queries, p) => {
  if (!s.hyde) return queries;
  const r = await ctx.sample(`Write one plausible paragraph (about ${p.words}
words) from an engineering project report that answers: ${s.q}\nNo preamble,
no headings.`, { modelTier: 'quick' });
  ctx.note(r.text.slice(0, 160));
  return [...queries, r.text];
}, { trigger: 'submit', words: 80 })
```

Three features of the grammar meet here. The lambda reads a signal, `hyde`, so a toggle on the page turns the step off without editing code. It sets its own trigger to `submit`, because it calls Claude. And it records the draft with `ctx.note`, which the page shows in a card beside the results.

With a mock paragraph about latent classes and BIC, the results were:

```
with the draft (mock)                                    hyde off
1. 52.1927  Broken Exports › What model selection…        1. 20.2441  Broken
Exports › What model selection…
2. 39.3875  Tier B › … › Latent class analysis…          2. 19.0413  Tier B ›
The mixture models…
3. 32.672   Tier B › (introduction)                       3. 17.2575  Tier B
› (introduction)
```

All of the top six hits were found by the draft, not by the question. That is a property of the retriever, not of HyDE: `perQuery` keeps each chunk's best raw score, and an 80-word draft matches far more terms than a 15-word question, so its scores are larger. This is the scale problem of section 14.2 inside a single retriever. If the original question should keep a voice, give the draft its own retrieve node and fuse the two lists by rank.

16.4 Rerank lambdas: heuristics over fields the index never scored

BM25 scores terms. It does not know that a heading which repeats the question's words is probably the answer, that a section with a code block is what a programmer wants, or that a 40-token section is too short to answer anything. A `rerank` lambda can. It receives the hits as views, with `title`, `headingPath`, `text`, `meta` and `tokens` attached, and returns hits in its preferred order. This is the *Rerank with a lambda* preset:

```
.rerank((s, ctx, hits, p) => {
  const words = String(s.q).toLowerCase().match(/[a-z0-9]{4,}/g) || [];
  const overlap = t => words.filter(w => t.toLowerCase().includes(w)).length
/ Math.max(1, words.length);
  return hits.map(h => {
    const o = overlap(h.headingPath.join(' '));
    let score = h.score * (1 + p.heading * o);
    if (h.text.includes('```')) score *= 1 + p.code;
    if (h.tokens < p.minTokens) score *= p.shortPenalty;
    return { id: h.id, score, reason: `heading overlap ${o.toFixed(2)}` };
  }).sort((a, b) => b.score - a.score);
}, { heading: 1.5, code: 0, minTokens: 60, shortPenalty: 0.6 })
```

For *Why did ChatGPT show no app actions?* the long words are `chatgpt`, `show` and `actions`. The section *11. Why ChatGPT still showed no actions* contains all three in its heading (`show` matches inside *showed*), so its overlap is 1.00 and its score goes from 20.90 to 52.26. Every other top section has overlap 0 and keeps its score. Raising `heading` to 3 widens the gap to 83.61 and lets a PBUI section with overlap 0.33 into the top five. That is the usual shape of a boost: one parameter that helps the case it was designed for and starts to pull in unrelated material when pushed.

Whatever the lambda returns is normalized: strings become hits with score `1/(rank)`, and objects keep `id`, `score`, `via` and `reason`. `reason` is shown in the results list and the trace, so a heuristic can explain itself. Because the node's trigger is `change`, the lambda runs on every keystroke at zero calls; the trace for a keystroke is `text 1→40 | rerank 40→40 | limit 40→10`.

When a heuristic is worth reusing, `rag.define` turns it into a component (chapter 8). The *Your own components* preset defines `diversify` (at most `perDoc` sections per document) and `boost.tags` (multiply the score of chunks tagged with the `boostTag` signal). Its trace for *testing and verification* with `boostTag: 'go'` is `text 1→60 | tags 60→60 | diversify 60→16 | limit 16→10`. The two components pull in opposite directions, and the counts show it: without either, the top ten come from five reports; the tag boost alone concentrates them on three reports, seven of the ten from Workbench Stabilization; adding diversify spreads them back over six.

16.5 `rerank.llm`: passages, scores and stated reasons

A lexical score cannot tell whether a passage answers a question or merely mentions its words. Claude can, at the price of a call. `rerank.llm` sends the first `top` hits (20 by default) as numbered passages, each clipped to 700 characters:

```
Rank the passages by how directly they answer the question. Reply with only
JSON: {"ranking": [{"p": "p3", "score": 0.0-1.0, "reason": "under 12
words"}]}, best first, including only passages that help answer it.

Question: Why did ChatGPT say OAuth was unsupported?

Passages:
[p1] PROJECT REPORT - CoinVault MCP - From Local Conformance to ChatGPT 2026-
07-28 › 15. Failure chronology and diagnostic lesson
| Visible symptom | Actual cause | Corrective change | … | ChatGPT said OAuth
unsupported | Missing RFC 9728 path metadata | …
…
```

For twenty passages the prompt was 15,905 characters. The reply is parsed defensively. The mock returned five entries, two of them faulty, and the engine handled each:

Mock entry	Engine's treatment
`p3`, 0.94, "explains the stateless discovery path"	kept, rank 1
`p1`, 0.81, "names the no-actions symptom"	kept, rank 2
`p3` again, 0.5	dropped: already ranked
`p99`, 0.4	dropped: no such passage
`p7`, 0.42, "background on tool scopes"	kept, rank 3

The output keeps only passages Claude ranked, cut to `keep`, which defaults to the `k` signal. Two consequences follow. The reranker can shrink the result list: `keep` was 8 but the reply named 3 passages, so the user saw 3 results. And a passage Claude omits is gone even if BM25 liked it, which is the intent of *including only passages that help* but also the risk. If the reply contains no usable entry, the node falls back to the lexical order, so a malformed reply degrades the result instead of emptying it. Each kept hit carries Claude's `reason`, which the results list shows beside the title.

16.6 Why steps that call Claude wait for Enter

A user typing a twelve-word question produces a dozen `change` runs. If each run called Claude twice, one question would cost two dozen calls and each keystroke would wait on a model. The library therefore gives every component that calls Claude the trigger `submit`, and gives each a skip rule that says what flows downstream when it does not run:

Node	Trigger	When skipped
`rewrite.llm`	submit	passes its input queries through
`rerank.llm`	submit	passes its input hits through
`answer`	submit	outputs `null`
`agentic`	submit	outputs `null`
HyDE rewrite lambda	submit (set in the preset)	passes its input queries through

Node	Trigger	When skipped
`rewrite`, `rerank`, `filter` lambdas	change	run on every keystroke

The *Rewrite + rerank* preset shows both kinds of run:

```
submit  rewrite:ran 1→4 calls 1 | text:ran 4→40 | filter:ran 40→40 |
rerank:ran 40→3 calls 1
change  rewrite:skipped 1→1     | text:ran 1→40 | filter:ran 40→40 |
rerank:skipped 40→40
```

While the user types, the page shows lexical results for the question as typed. On Enter, the same plan adds the rewrites and the reranking. The design keeps the page responsive and makes the cost of a search a decision the user takes, one press at a time. Chapter 12 describes the session rules behind it, and chapter 27 how a page should show which actions spend calls.

The key points:

- A rewrite step changes the queries and leaves the index alone; with `perQuery`, each phrasing is ranked on its own and the original question always keeps its own ranking.
- Index-time synonyms add terms at half weight inside one query; rewrites add whole queries at full weight. They answer different needs.
- HyDE is a rewrite that searches with a drafted answer; because long queries score higher, the draft dominates a per-query retriever unless it is fused by rank.
- A rerank lambda can score anything the hit view exposes, explain itself with `reason`, and costs nothing per keystroke.
- `rerank.llm` reads clipped passages, ignores malformed entries, keeps only what Claude ranked, and falls back to the lexical order when the reply is unusable.
- Every step that calls Claude runs on Enter and passes its input through when skipped, so typing never waits on a model.

Rewriting and reranking keep Claude at the edges of a fixed pipeline. The next chapter puts Claude in charge of the retrieval itself.

Lab: Query expansion; Rerank with a lambda; Your own components; Rewrite + rerank; HyDE. In *Query expansion*, try *what is the knee rule for LCA* and *why did DCR fail*, then compare the `no-rewrite` and `no-synonyms` variants in the Evals tab. In *Rerank with a lambda*, set `heading` to 3 and watch which section enters the top five. In *Your own*

components, pick a tag in the Boost tag menu. In *Rewrite + rerank*, type a question, then press Enter and open the Trace tab: the rewrite and rerank rows change from skipped to ran, each with one call. In *HyDE*, compare the results with the toggle on and off.

17. Claude as the retriever

Every pipeline so far has had a fixed shape: the same nodes run in the same order for every question, and Claude, where it appears, fills one slot. This chapter covers the two ways the workbench gives Claude a larger role. The `agentic` node hands Claude a set of tools over the indexes and lets it choose what to search and what to read. Build-time enrichment uses Claude once per document, ahead of any question, to write summaries and likely questions that a lexical index can then match. The chapter ends by comparing an agent with a fixed pipeline on the same golden questions.

The tool definitions, prompts, budgets and error messages below are real. The agent runs were driven by a scripted policy in place of Claude: a mock sampler that calls the tools in a fixed sequence. Those runs show what the node records and enforces. They do not show what Claude would choose to do.

17.1 Four tools: search, read, outline, documents

The *Agentic answer* preset is a one-node plan:

```
rag.plan('agentic').over(corpus)
  .signals({ q: 'Across all projects, which bugs came from something silently
being empty or nil?', k: 8 })
  .agentic({ maxReads: 8, indexes: ['text', 'rows'], tier: 'default' });
```

The node's input is the query list and its output is an answer, the same type `answer` produces, so the answer panel, the citation links and the evals work unchanged. Its question is chosen to defeat a fixed pipeline: the answer is scattered across failure tables in several reports, under headings that rarely use the words *empty* or *nil*.

The node gives Claude four tools. These are their descriptions as Claude receives them:

Tool	Description
`search`	Search the report corpus with a short keyword query. Returns up to 8 sections as {id, doc, heading, snippet}. Indexes: text, rows. Search several phrasings.
`read`	Read one section by id. Returns its full text and the evidence label to cite, like [E2].
`outline`	List a document's sections as {id, heading}. Pass a section id or a document id.
`documents`	List all documents as {id, title, type, tags}.

The split between `search` and `read` is the important design choice. Search results are cheap: an id, the document title, the heading path and a 220-character snippet. Reading costs more context: the full section, clipped at 6,000 characters. Claude can therefore scan many candidates and pay for the few it needs, which is the same retrieve-then-pack structure as a fixed pipeline, with the model deciding both steps. A real search result looks like this:

```
{ "id": "project-report-coinvault-mcp-…#row-browser-ecs-never-healthy",
  "doc": "PROJECT REPORT - CoinVault MCP - From Local Conformance to ChatGPT
2026-07-28",
  "heading": "15. Failure chronology and diagnostic lesson › Browser ECS
never healthy",
  "snippet": "Visible symptom: Browser ECS never healthy Actual cause: Typed
nil verifier activated RAG branch Corrective change: Return a genuinely nil
verifier interface" }
```

`outline` and `documents` let Claude move through the corpus by structure rather than by words: list the reports, then a report's sections, then read one. The prompt states the method and the citation format:

```
You answer questions about a corpus of engineering project reports using the
tools. Search with several phrasings, read the sections you rely on, then
answer. Cite read sections inline as [E1], [E2] using the labels read()
returns. Stop searching as soon as you can answer. Final answer: plain
markdown, under 250 words.

Question: Across all projects, which bugs came from something silently being
empty or nil?
```

17.2 A read budget, and evidence labels assigned on read

An agent with unlimited reads can fill its context with marginal sections, take many turns, and cost more than the answer is worth. The node enforces `maxReads` (8 by default) inside the `read` tool. When the budget is spent, a read of a new section returns an error that tells Claude what to do next:

```
read budget of 8 sections used; answer now from what you read
```

Re-reading a section already read does not count, and returns the same label. A read of an id that does not exist returns:

```
no section no-such-id; use an id returned by search
```

Both messages are written for the model, not for a person. A tool error is part of the conversation the model sees, so the error should say how to recover.

Evidence labels are assigned when a section is first read, in the order of reading: the first section read is `[E1]`. The node collects the read sections into the same `evidence` table that `pack` builds (chapter 15), so a citation from the agent links to a section exactly as a citation from `answer` does. The difference is who chose the evidence: in a fixed pipeline the ranking chose it; here Claude did, one read at a time.

17.3 Rounds in the trace: what was searched and what was read

An agentic run is one node in the trace, so the trace alone would hide everything the agent did. The node therefore records each tool call in two places. `answer.rounds` lists every call with its arguments; the trace entry's `notes` list the searches and reads in a compact form. This is the scripted run (the sequence of calls is the script's, not Claude's):

```
agentic  ran  in 1  out 8  calls 1
  search("silently empty nil") → 8
  search("typed nil interface") → 8
  search("empty result no error") → 8
  read(…coinvault…#row-browser-ecs-never-healthy) → [E1]
  read(…ttc…order-grain…#row-datatable-polling-an-empty-response) → [E2]
  read(…ttc…order-grain…#row-empty-orders-tab-on-load) → [E3]
  read(…coinvault…#4-1-the-typed-nil-deployment-failure) → [E4]
  …
  read(…stabilization…#row-known-optional-binding-left-empty) → [E8]

rounds: documents · search × 3 · read × 7 · outline · read
```

Two details of the record are worth knowing. The notes cover `search` and `read` only; `outline` and `documents` appear in `rounds` but not in the trace notes. And the trace's `calls` column counts invocations of the sampler, so a whole agentic run counts as 1 however many model turns the tool loop takes inside that invocation. The column is therefore a lower bound on usage for this node; the number of rounds is a better measure of how hard the agent worked.

On a `change` run the node is skipped and outputs `null`, like `answer`: the page does nothing while the user types.

17.4 Routing by build-time enrichment: summaries and questions per document

The agent spends Claude's effort per question. Enrichment spends it per document, once, at build time. The *Doc cards* preset makes one card per report from its title and second-level headings, asks Claude to enrich each card, and indexes the result:

```
kit.reports()
  .chunk('cards', (doc, ctx) => [{
    kind: 'card', headingPath: ['Document card'],
    text: [doc.title, ...doc.sections.filter(s => s.level === 2).map(s =>
s.title)].join('\n'),
  }])
  .enrich('llm', rag.enrich.llm({ tier: 'quick' }), { from: 'cards' })
  .index('cards', rag.bm25({ fields: { title: 2, text: 1, 'llm.summary': 2,
'llm.questions': 3, 'llm.keywords': 3 } }), { from: 'enrich_llm' });
```

This is the prompt each card is sent with, and the start of a real card:

```
For this section of an engineering report, reply with only JSON: {"summary":
string (one sentence), "questions": [3-5 questions this section answers],
"keywords": [5-8 search keywords, including synonyms not in the text]}.

Document: TTC Customer Clustering — Order-Grain Segmentation and the Shiny
Explorer
Section: Document card

TTC Customer Clustering — Order-Grain Segmentation and the Shiny Explorer
Why this project exists
Project shape
The data model
…
```

The questions and keywords are the point. A user asks *which report explains why a criterion keeps adding clusters*; the report says *BIC kept improving as components were added*. A keyword list written with synonyms not in the text, weighted 3 in the card index, can carry *criterion* and *clusters* to the right card.

The plan routes with a retrieve lambda: find the best three cards, then search inside those three documents, four sections each, and fuse with plain retrieval:

```
.retrieve((s, ctx, queries, p) => {
  const q = queries.join(' ');
  const docs = ctx.index('cards').search(q, p.docs).map(h =>
ctx.chunk(h.id).docId);
  return docs.flatMap(d => ctx.index('text').searchIn(d, q, p.perDoc));
}, { id: 'routed', docs: 3, perDoc: 4 })
.fuse.rrf({ weights: { text: 1, routed: 1.2 } })
```

The enrichment costs eleven quick calls, one per report. The workbench defers it to an explicit Build (chapter 11), so before that press the cards hold only titles and headings. That state is instructive, because it shows routing without enrichment:

```
build   enrich_llm: computed, deferred 11, calls 0      pending: [{ node:
"enrich_llm", items: 11 }]

cards for "Which report explains why a criterion keeps adding clusters?"
2.8841  CoinVault card        why { report: 2.88 }
2.2654  Tier B card           why { cluster: 2.27 }
2.1362  Broken Exports card   why { cluster: 2.14 }

routed (in output order)
1. 4.1578   CoinVault › 19. Engineering rules worth preserving
2. 2.5095   CoinVault › (introduction)
…
5. 12.5382  Tier B › What was learned
6. 12.2974  Tier B › The mixture models, and a lesson about BIC

fused
1. 0.03459  { text: 2, routed: 5 }   Tier B › What was learned
…
8. 0.03078  { text: 30, routed: 1 }  CoinVault › 19. Engineering rules worth
preserving
```

Two things went wrong, and both are instructive. The bare cards routed first to CoinVault because its title begins *PROJECT REPORT* and the question contains *report*; a card with headings only has too few words to outvote one accidental match. And the lambda returns its hits grouped by document without sorting, so RRF reads the CoinVault sections as ranks 1 to 4 and the Tier B sections, scoring three times higher, as ranks 5 to 8. The CoinVault section, rank 30 of 40 in plain retrieval, entered the fused top eight on the strength of that position. A routing lambda should sort its output by score, or interleave documents, before fusion reads its ranks. Enrichment would address the first problem, because a summary and keywords give the card index real content to match; the second is a bug in the lambda that enrichment cannot fix.

There is also a known engine bug on this path. After a deferred build, pressing Build runs the eleven enrichment calls, but the downstream `ix-cards` index is reused rather than rebuilt: its digest depends on the enrichment node's parameters, which did not change, not on the enriched output. In a node test, a search for a keyword that only the mock enrichment produced found 0 cards after Build, and 3 cards after a fresh build on a new engine. Until the engine is fixed, reloading the page after Build (which rebuilds from the

per-item cache without new calls) makes the enriched cards searchable; in the same test, a fresh engine sharing the cache rebuilt the index with 11 cached items, 0 calls, and found all 3 cards.

17.5 Fixed pipeline or agent: comparing both on the same goldens

Evals score a plan by the ranked list it returns (chapter 18). An agentic plan returns an answer, not a list, so evals fall back to the answer's evidence in reading order. That makes an agent and a fixed pipeline comparable with the same cases and metrics. These are real numbers for a lexical pipeline (`retrieve('text', { k: 40 }).limit(s => s.k)`) and for the agentic plan driven by a simple scripted policy that searches the question and reads the top five results:

Plan	hit@5	MRR	nDCG@10	sampler calls
fixed lexical	1	0.933	0.847	0
agentic, scripted policy (search once, read 5)	1	0.933	0.653	12

The scripted policy finds the same first relevant section as the lexical pipeline, because it is the lexical pipeline with extra steps. Its nDCG is lower because five reads make a shorter list than ten results, and nDCG@10 gives credit for relevant sections further down. The comparison shows the method, not a verdict on agents: an agent driven by Claude will search different phrasings, read different sections, and score differently from run to run. Those differences are the point of running the comparison.

The broader trade-off is between two kinds of system:

	Fixed pipeline	Agentic node
What decides the evidence	a ranking you can read in the trace	Claude, one search and read at a time
Same question, same result	yes	not guaranteed
Cost per question	0 to 3 calls, known in advance	1 invocation with an unknown number of turns
Latency	milliseconds until the answer call	several model turns

	Fixed pipeline	Agentic node
Questions spread across many reports	limited by fusion and packing	can follow leads across reports
How to debug a miss	the trace shows which stage dropped it	`rounds` shows what was searched and read

A reasonable practice is to build the fixed pipeline first, measure it, and use an agent for the questions the goldens show it cannot answer, keeping both as variants scored on the same cases.

The key points:

- The agentic node gives Claude search, read, outline and documents tools over the indexes; cheap searches and expensive reads mirror the retrieve-then-pack structure of a fixed pipeline.
- A read budget is enforced inside the tool, and tool errors are written to tell the model how to recover.
- Evidence labels are assigned in reading order, so an agent's citations link to sections exactly as a fixed pipeline's do.
- `rounds` and the trace notes record what the agent did; the trace's call count is one per sampler invocation, not per model turn.
- Build-time enrichment spends Claude once per document so that a lexical index can match questions the text does not phrase; a routing lambda must still sort its output before fusion reads its ranks.
- After a deferred enrichment build, the downstream index is currently reused instead of rebuilt; reloading the page rebuilds it from the cache.
- Evals compare agents and fixed pipelines on the same cases by scoring the agent's evidence in reading order.

This chapter ends the tour of the live pipeline. Part VI turns to the question every chapter so far has raised: when a change moves a result, how do you know it was an improvement?

Lab: Agentic answer; Doc cards. In *Agentic answer*, press Enter, then read the *Tool calls* card: each line is a search or a read, and the labels in the answer match the reads. Lower `maxReads` to 3 in the DAG tab and ask again. In *Doc cards*, ask the default question before pressing Build and look at the `routed` row in the Trace tab; then press Build (eleven quick calls), reload the page, and ask again. Open the *Cards* tab to read the summaries Claude wrote.

Part VI · Measuring

A change to a pipeline is an improvement only if it moves a number on questions chosen before the change was made. This part defines those questions, the numbers, and how pipelines are compared with them.

18. Golden questions

This chapter defines the unit of measurement for everything that follows: the golden case. A golden case is a question together with a rule that decides, for any chunk the pipeline returns, whether that chunk answers the question. By the end of the chapter you should be able to read a case, predict which chunks it will accept, write new cases that measure what users actually need, and recognise when a case has stopped measuring the pipeline and started measuring itself.

The reason for starting here is simple. Parts II to V describe dozens of choices: chunk size, field weights, fusion constants, rerankers, packing budgets. Every one of them changes the ranking, and most changes help some questions and hurt others. Without a fixed set of questions chosen before the change, "better" means "better on the query I happened to try last", and the pipeline drifts toward whatever the author tested by hand. Golden cases replace that habit with a number that is computed the same way every time.

18.1 A case: question, expected document, expected section, expected answer text

An eval set is built with the same immutable-builder pattern as corpora and plans. `rag.evals(id)` returns an empty set; `.case(question, expect)` returns a new set with one more case; `.metrics([...])` names the numbers to compute. The workbench's twelve hand-written cases live in `kit.js` as rows of question, document pattern, section pattern and, optionally, answer pattern:

```
const CASES = [
  ['Why did ChatGPT show no actions after OAuth worked?', 'CoinVault', /no
actions|stateless/i],
  ['What rule picked eight latent classes?', /Tier B|Broken Exports/, /latent
class|model selection/i, /knee|ten percent|10%/i],
  ['How are arrays of objects flattened into metadata terms?', 'SQLite-Backed
Forum', /metadata/i],
  // … nine more
];

let ev = rag.evals('goldens');
for (const [q, doc, section, answerHas] of CASES)
  ev = ev.case(q, { doc, section, ...(answerHas ? { answerHas } : {}) });
ev = ev.metrics(['hit@5', 'mrr', 'ndcg@10']);
```

Serialised with `ev.toJSON()`, the first two cases read:

```
{ "q": "Why did ChatGPT show no actions after OAuth worked?",
  "expect": { "doc": "CoinVault", "section": "/no actions|stateless/i" } }
{ "q": "What rule picked eight latent classes?",
  "expect": { "doc": "/Tier B|Broken Exports/",
              "section": "/latent class|model selection/i",
              "answerHas": "/knee|ten percent|10%/i" } }
```

The `expect` object is not a list of correct chunk ids. It is a predicate, and the engine applies it to chunks with a function called `relevant`. Its rules are short enough to state completely:

- **The document pattern** is tested against the chunk's document title, the document's file name, its document id and every alias in its frontmatter. A match on any of them counts.
- **The section pattern** is tested against the chunk's heading path joined with `›`, for example `5. The eval tool › 5.2 Execution`. The flag `any: true` skips this test, so the case accepts any chunk from the right document.
- **A string pattern** matches as a case-insensitive substring. A regular expression is applied as written. An array of patterns matches if any member does.
- **A chunk is relevant** when both the document test and the section test pass.

Two numbers fall out of this predicate for every case. The **rank** is the 1-based position of the first relevant hit in the pipeline's result list, or `null` when no relevant hit appears. **R** is the number of chunks in the whole corpus that the predicate accepts, whether or not the

pipeline returned them. Rank drives hit@k and MRR; R is the denominator of recall and fixes the ideal ranking that nDCG compares against. Chapter 19 defines those metrics; here it is enough to know that every case report carries both numbers.

The optional `answerHas` pattern is different in kind. It is tested against the text of the generated answer, not against chunks, and it is only scored when the run produced an answer. Section 18.4 explains when that happens and chapter 20 uses it.

18.2 Patterns instead of ids: why cases survive rechunking

A chunk id in this corpus looks like `project-report-coinvault-mcp-from-local-conformance-to-chatgpt-2#11-why-chatgpt-still-showed-no-actions`. It is built from the document id and the slug of the section heading, and a section split by the token budget gets a numeric suffix per piece. Change `maxTokens` from 450 to 200 and many sections split into two or three pieces with new ids. A case that said "the answer is chunk X" would be wrong after every chunking experiment, which is exactly the experiment Part II told you to run.

A pattern on headings survives that change. When the Execution section splits in two, both pieces carry the heading path `5. The eval tool › 5.2 Execution`, and both satisfy `/eval tool|execution/i`. The case keeps working, and R grows to count the extra pieces.

The protection is real but it has a limit, and the limit shows up in a real run. The Chunk sizes preset compares four chunking variants over the twelve goldens. Here are R and rank per case for the 450-token default and for the `h2-only` variant, which chunks at heading depth 2 instead of 3:

Case	450 tokens, depth 3	h2-only
3 · arrays of objects flattened into metadata	R 2, rank 1	**R 0, rank —**
7 · ECS service restart loop	R 1, rank 1	**R 0, rank —**
8 · inbox cursor advances	R 2, rank 1	**R 0, rank —**
11 · the rebalance law	R 1, rank 1	**R 0, rank —**
9 · empty orders tab	R 3, rank 1	R 1, rank 4

Four cases have R = 0 under h2-only. No chunk in the corpus satisfies them, so no pipeline could score on them. The reason is visible in the heading paths. At depth 2, the SQLite forum report's subsections are folded into their parents, and its chunks carry paths like `Implementation analysis` and `The data model`; the word "metadata" that case 3 looks for appeared only in a depth-3 heading that no longer exists as a path. The answer text is still in the corpus, inside a larger chunk, but the case's section pattern can no longer see it.

This is the most common misunderstanding about pattern-based cases: a low score is read as a retrieval failure when it is a case failure. The defence is to read R. A case with R = 0 under some pipeline is not measuring that pipeline, and its zero should be excluded from any comparison, or the pattern should be widened (for example to `{ doc: 'SQLite-Backed Forum', any: true }`) if document-level retrieval is what matters. The report includes R for every case precisely so this check takes one glance.

18.3 Writing cases from the questions users actually ask

The twelve goldens were written as questions a colleague would type, not as headings with a question mark added. Compare the question "What rule picked eight latent classes?" with the section that answers it, "What model selection actually established". They share almost no words. A case like that tests whether the pipeline can bridge vocabulary, which is the hard part of search. A case like "What does the rebalance law say?", whose answer sits under the heading `5.9 The rebalance law`, tests something easier: whether an exact phrase in a heading wins. Both kinds belong in a set, and the set should say which is which, because a pipeline that is perfect on heading echoes can still fail every real question.

Four rules make cases useful:

- **Phrase the question as a user would.** Use the user's words, including vague ones. If users say "the orders tab was empty", the case says that, even though the report calls it a failure in the architecture section.
- **Make the section pattern as wide as the set of sections that genuinely answer the question, and no wider.** Case 1 accepts `/no actions|stateless/i` because both section 11 ("Why ChatGPT still showed no actions") and section 12 ("Stateful versus stateless Streamable HTTP") answer it. A pattern that also accepted the whole OAuth chapter would reward the pipeline for returning a section that only mentions the topic.
- **Cover the corpus.** The twelve cases reach nine of the eleven reports; the Datalab and Stabilization reports have no case, so a change that breaks retrieval there would go unnoticed. A set that only asks about one report measures how that report was

written.

- **Add an answer pattern only where a fact can be checked by text.** Case 2's answer must mention the knee rule or the ten-percent criterion. Most "why" questions have no such fact, and forcing one produces a pattern that tests phrasing.

For designers, this is the place where the product is specified. The list of golden questions is the list of tasks the search page promises to support. Writing it is a design act: it decides whose questions matter, in whose words, and what counts as an answer. A page whose layout is beautiful but whose goldens are heading echoes has not been designed for its users; it has been designed for its author.

18.4 Which run a case exercises

Chapter 12 introduced triggers: `change` nodes run on every keystroke, `submit` nodes wait for Enter. An eval run chooses a trigger too, and the choice decides which part of the pipeline is measured and what the run costs.

`ev.run(target)` defaults to `trigger: 'change'`. In that mode, nodes marked `submit` (`rewrite.llm`, `rerank.llm`, `answer`, `agentic`) are skipped exactly as they are while a user types, so the run measures the lexical pipeline and makes no Claude calls. With `trigger: 'submit'`, every node runs, the answer is generated, `answerHas` can be scored, and judges can run if `judge: true` is passed. The Evals tab maps these to its two buttons: **Run 12 cases** is a `change` run, and **Run with Claude** is a `submit` run with judges.

The Judged evals preset shows the difference on the same thirteen cases (the twelve goldens plus one with an answer pattern). The Claude calls in the second run were served by a mock sampler in node, so the call count is real but the judge scores are not:

Run	Nodes measured	Claude calls	Per case
`change`	retrieve	0	0
`submit`, `judge: true`	retrieve, rerank.llm, pack, answer, 2 judges	52	4

Two details of the run matter when reading results. First, the engine asks for at least ten hits per case: the `k` signal is set to the largest of 10, the largest `@k` in the metric list, and the plan's own `k`. A metric can never be starved by a plan that returns five results.

Second, when a run produces no hits list but does produce an answer with evidence, as an agentic node does, the evidence list stands in for the hits. That is how chapter 17's agent can be scored on the same cases as a fixed pipeline.

18.5 Synthetic cases: questions Claude writes for sampled sections

Twelve cases are few. `rag.evals.generate(target, { n, seed })` produces more by asking Claude to write one question per sampled section. It works in four steps:

1. Build the target if needed, then collect every chunk that is a heading section with more than 120 tokens and a non-empty heading path.
2. Pick `n` of them (default 12) with a seeded linear congruential generator, so the same seed picks the same sections.
3. Send them to Claude in batches of eight, one quick-tier call per batch, asking for "one question a colleague might ask that this section answers well", phrased without copying the heading.
4. Turn each question into a case whose document pattern is the chunk's title and whose section pattern is its full heading path, flagged `synthetic: true`.

Run in node with seed 7 and a mock sampler (so the question text below is a placeholder), eight sections cost exactly one call, and the generated cases look like this:

```
{ "q": "MOCK question 3",
  "expect": { "doc": "PBUI Workbench Core: A Headless Engine, a Pure Planner,
and the Hard Cutover of the React Shell",
              "section": "5. The core's modules › 5.9 The rebalance law" },
  "synthetic": true }
```

Synthetic cases are cheap and plentiful, and they have two biases you should name when you report them. Because Claude writes the question while reading the section, the question tends to reuse the section's vocabulary, which favours lexical retrieval and flatters BM25. And because the section pattern is that one heading path, a different section that answers the question equally well counts as a miss. Both biases push in known directions, which is why the Synthetic evals preset scores generated cases in their own table and never mixes them into the goldens. Use them to find sections nobody can reach, and use the goldens to decide whether a change shipped.

Key points

- A golden case is a question plus a predicate over chunks; relevance is decided by matching document and heading-path patterns, not by stored ids.
- Every case report carries a rank (first relevant position) and R (relevant chunks in the corpus); read R before reading a score.
- A case with R = 0 under some pipeline measures the case, not the pipeline; the h2-only variant makes four goldens unanswerable by construction.
- Lexical eval runs use the `change` trigger and cost nothing; `submit` runs exercise Claude-backed nodes and judges and cost about four calls per case in the Judged evals preset.
- Synthetic cases are useful for coverage but are biased toward the section's own vocabulary and its exact heading, so they are kept separate from the goldens.

With cases in hand, the next question is how to turn a list of ranks into a number. Chapter 19 defines the four metrics the engine computes and shows what each one rewards.

Lab · Reports · hybrid + rerank + answer, Synthetic evals. Load the Reports preset, open the Evals tab and press **Run 12 cases**. Every row shows a rank; click a miss, if any, to run its question in the search sheet and see what came first instead. Then load Chunk sizes, tick `h2-only` and run again: find the rows whose rank is "—" and confirm in the Corpus tab that no chunk carries the heading the case expects. Finally, load Synthetic evals, generate eight questions, and read which sections the generated questions failed to reach.

19. Ranking metrics

A golden case produces a ranked list and a predicate that marks each entry relevant or not. A metric turns that pattern of marks into one number, and the mean over cases into one number for the pipeline. This chapter defines the four metrics the engine computes (hit@k, MRR, nDCG@k and recall@k), shows the eleven lines of code that implement them, works all four by hand for one real case, and ends with the question every reader of an eval table must ask: how large does a difference have to be before it means anything when there are only twelve cases?

The metrics are not interchangeable. Each one encodes an assumption about how far down the list a reader looks and whether a second relevant result is worth anything. Choosing a metric is choosing that assumption, so the definitions below are stated together with the reader they model.

All numbers in this chapter come from one run: the Minimal search pipeline (one BM25 index, one retriever) over the twelve goldens, with five metrics requested.

```
const ev = kit.goldens().metrics(['hit@1', 'hit@5', 'mrr', 'ndcg@10',
'recall@10']);
const report = await ev.run(plan);
console.table(report.table());
```

#	Question	rank	hit@1	hit@5	mrr	ndcg@10	recall@10
1	Why did ChatGPT show no actions after OAuth worked?	1	1	1	1	0.61	0.5
2	What rule picked eight latent classes?	1	1	1	1	1	1
3	How are arrays of objects flattened into metadata terms?	1	1	1	1	0.83	1
4	Why is a content hash not enough for an operation id?	1	1	1	1	1	1
5	What happens to product flags when a product is deleted?	1	1	1	1	0.77	0.67
6	How does the eval tool report line numbers in errors?	5	0	1	0.2	0.4	0.75
7	Why did the browser ECS service restart in a loop …?	1	1	1	1	1	1
8	Why does the inbox cursor advance past events …?	1	1	1	1	0.88	1
9	Why was the orders tab empty when the explorer loaded?	1	1	1	1	0.97	1
10	How does protojson serialize int64 values?	1	1	1	1	0.77	0.67
11	What does the rebalance law say?	1	1	1	1	1	1
12	How are generative UI lambdas stored so they survive a reload?	1	1	1	1	0.94	0.75

The summary is the mean of each column: hit@1 0.917, hit@5 1, mrr 0.933, ndcg@10 0.847, recall@10 0.861, with a median of 0.4 ms per case and zero Claude calls. The engine also records `p50ms` and `calls` in every summary, so cost sits beside quality in the same row.

In every formula below, `rel` is the list of relevance marks for the returned hits in order, `rank` is the position of the first `true` (or none), `R` is the number of relevant chunks in the corpus (chapter 18), and `k` is the cutoff named after the `@`. This is the engine's code,

lightly reformatted:

```
if (name === 'hit')    m = rank && rank <= k ? 1 : 0;
else if (name === 'mrr')    m = rank ? 1 / rank : 0;
else if (name === 'recall') m = R ? rel.slice(0, k).filter(Boolean).length /
R : 0;
else if (name === 'ndcg') {
  const dcg  = rel.slice(0, k).reduce((a, r, j) => a + (r ? 1 / Math.log2(j +
2) : 0), 0);
  const idcg = Array.from({ length: Math.min(R, k) }, (_, j) => 1 /
Math.log2(j + 2))
                    .reduce((a, b) => a + b, 0);
  m = idcg ? dcg / idcg : 0;
}
```

19.1 hit@k: whether a relevant section was on the first screen

hit@k is 1 when the first relevant chunk is at position k or better, and 0 otherwise. It models a reader who scans the first k results and is satisfied by any one relevant section. The model fits a results page well: if the page shows five results above the fold, hit@5 says whether the reader could find the answer without scrolling. Designers can choose k directly from the layout.

The weakness of hit@k is that it saturates. Minimal search scores hit@5 = 1 on all twelve cases. From that point on, no change to the pipeline can raise hit@5, and a change that moves case 6 from rank 5 to rank 1 is invisible. A saturated metric is still useful as a floor, since a drop below 1 is news, but it cannot rank two pipelines that both clear it. The usual response is to tighten the cutoff. hit@1 is not saturated here (0.917), because case 6 puts its first relevant chunk fifth.

19.2 MRR: how far down the first relevant section was

The reciprocal rank of a case is 1 / rank, or 0 when nothing relevant was returned; MRR is its mean. It models a reader who reads down the list until the first relevant result and stops, and it charges for every position they had to read past. The charge is steep at the top and flat lower down: rank 1 scores 1, rank 2 scores 0.5, rank 5 scores 0.2, rank 10 scores 0.1.

For the run above, eleven cases score 1 and case 6 scores 0.2, so MRR = (11 × 1 + 0.2) / 12 = 11.2 / 12 = 0.933, which is the engine's summary. MRR ignores everything after the first relevant hit. A pipeline that returns one relevant section at rank 1 and nine irrelevant ones

scores the same as one that returns ten relevant sections. For a question with a single right section, which most of the goldens have, that is the correct behaviour. For a question whose answer is spread over several sections, it is not, and the next two metrics cover that case.

19.3 nDCG@k: credit discounted by position and normalised by the best possible list

Discounted cumulative gain adds a credit for every relevant hit in the top k, divided by $\log_2(\text{position} + 1)$. Position 1 earns 1, position 2 earns 0.631, position 3 earns 0.5, position 10 earns 0.289. The discount falls slowly, so nDCG rewards a list for putting several relevant sections high, not only the first. Normalised DCG divides by the DCG of the best possible list: all `min(R, k)` relevant chunks at the top. A list that cannot be improved scores 1.

In this engine relevance is binary, so every relevant hit has gain 1. nDCG is often defined with graded gains (highly relevant 3, partly relevant 1); the golden predicate has no grades, so nDCG here measures how many relevant sections came back and how high.

Case 1 shows what nDCG sees that MRR does not. Its pattern accepts two sections, "11. Why ChatGPT still showed no actions" and "12. Stateful versus stateless Streamable HTTP", so R = 2. The pipeline returns section 11 first and section 12 nowhere in the top ten. MRR is 1. nDCG is 1 / (1 + 0.631) = 0.613, because the ideal list had a second relevant section at position 2. The engine reports 0.6131471927654584.

19.4 recall@k: how many of the relevant sections were found

recall@k is the fraction of the R relevant chunks that appear in the top k, with no regard to order. It models a consumer that reads all k results, which is what the `pack` node does when it fills a context for Claude. For an answer pipeline, recall@k at the packing depth is the metric closest to "did Claude get to see the evidence".

recall has a dependency that the others lack: its denominator is R, and R depends on chunking. The Chunk sizes preset's 200-token variant splits sections into more pieces, and R grows with it; for case 12, R is 12 at 450 tokens and 20 at 200 tokens. With R = 20, recall@10 cannot exceed 0.5 even for a perfect list. Comparing recall across chunking variants therefore compares denominators as much as pipelines. Compare recall only between pipelines that share a chunker, or read it per case beside R.

19.5 All four by hand for one case

Case 6, "How does the eval tool report line numbers in errors?", is the one case where the metrics disagree, which makes it the best one to compute by hand. Its pattern is `{ doc: 'Agent runtime spec', section: /eval tool|execution/i }`, and four chunks in the corpus satisfy it: 5.1 Definition, the two pieces of 5.2 Execution (ids `…#5-2-execution` and `…#5-2-execution-2`), and 5.3 What good eval code looks like. So R = 4. Here is the top ten that Minimal search returned, all from the agent runtime spec:

Position	Score	Section	Relevant
1	12.4732	21. Generative UI: gui › 21.6 Canvas	
2	12.2952	7. State introspection: bench	
3	11.2416	12. Transcripts API	
4	11.1063	18. Annotations › 18.1 API	
5	10.9849	5. The eval tool › 5.2 Execution	yes
6	10.9130	5. The eval tool › 5.2 Execution (first piece; position 5 is the second)	yes
7	10.5125	9. Chat data model and rendering › 9.2 Rendering rules	
8	10.4692	9. Chat data model and rendering › 9.1 Data model	
9	10.1883	25. Testing without a live model › 25.1 Stand-ins	
10	10.1657	5. The eval tool › 5.1 Definition	yes

The four metrics follow directly:

- **hit@5.** The first relevant chunk is at position 5, and $5 \leqslant 5$, so hit@5 = 1. hit@1 = 0.
- **MRR.** 1 / 5 = 0.2.
- **recall@10.** Three of the four relevant chunks are in the top ten, so 3 / 4 = 0.75. The fourth, 5.3, is not.
- **nDCG@10.** DCG = $1/\log_2 6 + 1/\log_2 7 + 1/\log_2 11$ = 0.38685 + 0.35621 + 0.28906 = 1.03212. The ideal list puts min(4, 10) = 4 relevant chunks first: IDCG = 1 + 0.63093 + 0.5 + 0.43068 = 2.56161. nDCG = 1.03212 / 2.56161 = 0.40292.

The engine reports `{"hit@5":1,"mrr":0.2,"ndcg@10":0.4029209390875161,"recall@10":0.75}` for this case, which agrees with the hand computation to every printed digit.

The trace explains the ranking once the per-term contributions (`why`, chapter 13) are visible. Every section of this document mentions eval and tool, so those terms barely separate them. The sections above Execution win on the stem `number`: it contributes 4.42 to "18.1 API" and 4.12 to "12. Transcripts API", against 2.21 for the second Execution piece, whose own strengths are `eval` 3.31 and `line` 2.88. The section that answers the question wins on no single word by a wide margin, so it lands at 10.98, about one and a half points behind a section on canvas drawing. This is the "misranked" failure from chapter 1, and the metrics locate it: hit@5 says the answer was on screen, MRR says it was fifth, and nDCG and recall say two more answering sections were close behind.

19.6 Twelve cases, and how large a difference has to be before it means anything

Every summary number is a mean over twelve cases, and that fixes the smallest possible change in each metric. The resolution is worth working out once:

Event in one case	Change in the mean
hit@k flips between 0 and 1	1/12 = 0.083
first relevant moves from rank 1 to rank 2	MRR − 0.5/12 = −0.042
first relevant moves from rank 5 to rank 4	MRR + 0.05/12 = +0.004
one more relevant chunk enters the top 10, R = 4	recall@10 + 0.25/12 = +0.021

The parameter sweep in chapter 20 produces twelve configurations whose MRR ranges from 0.931 to 0.938. That entire spread is case 6 moving between ranks 4, 5 and 6; the other eleven cases sit at rank 1 in every configuration. A difference of 0.007 in MRR is a single case moving one position, in a case that is in the top five either way.

Two habits follow from this. First, read differences in units of cases, not decimals: before calling a change an improvement, find the cases whose rank changed, using `compare(…).cases`, which prints the rank of every case under every variant. Second, ask whether the count of cases that improved against the count that got worse could have happened by chance. Because every variant runs the same cases, the comparison is paired, and a sign test is the simplest honest check. Under the null hypothesis that a change is equally likely to help or hurt any case it affects, six improvements out of six

affected cases has a two-sided probability of $2 \times 0.5^6 = 0.031$. One improvement out of one has probability 1. With twelve goldens, most single changes affect one or two cases, and they cannot be distinguished from noise by the numbers alone. What they can do is send you to the cases, where a trace (chapter 13) shows why the rank moved, and a reason you can state is worth more than a decimal you cannot.

The remedy for small sets is more cases, not more decimals. Synthetic cases (section 18.5) add volume with known biases; additional hand-written cases from real user questions add volume without them.

Metric	Reader it models	Uses R	Saturates when
hit@k	scans the first k, any one relevant result is enough	no	every case has a relevant hit in the top k
MRR	reads down to the first relevant result and stops	no	every case is at rank 1
nDCG@k	reads the top k, values every relevant result, more near the top	yes, capped at k	every list is ideal
recall@k	consumes all k (as `pack` does)	yes	every relevant chunk is in the top k

Key points

- hit@k, MRR, nDCG@k and recall@k each model a different reader; pick the one whose reader matches the page or the consumer, and report a second one to cover what the first ignores.
- A metric at its maximum on every case can no longer detect improvement; tighten k or switch metric before comparing pipelines.
- nDCG and recall depend on R, so they change when chunking changes even if retrieval does not; compare them only across pipelines with the same chunker.
- The engine's metric values for case 6 match a hand computation exactly: hit@5 1, MRR 0.2, nDCG@10 0.403, recall@10 0.75.
- On twelve cases, one case is 0.083 of hit@k and as little as 0.004 of MRR; read differences as cases that moved, and look at those cases before believing the summary.

Metrics turn one run into numbers. Chapter 20 turns several runs into a comparison: variants of one plan, grids of parameters, and judges for the answers themselves.

Lab · Field weights. Load the Field weights preset, open the Evals tab and run the twelve cases. Find case 6 in the table and confirm its rank, MRR and nDCG against section 19.5. Then click the question to run it in the search sheet, and use the Trace tab to see the `why` terms of the sections ranked above 5.2 Execution. Change the metric list in the script to include `hit@1` and `recall@10`, press Build, and run again to see which of the four numbers moves when you change a field weight.

20. Variants, sweeps and judges

Chapters 18 and 19 produced a number for one pipeline. This chapter produces comparisons: several pipelines scored on the same cases with the same metrics, a grid of parameter settings scored automatically, and judges that score what string patterns cannot, the answers themselves. It ends with the cost of each kind of run, because the decision of what to measure is also a decision of what to spend.

The design rests on two earlier results. A variant of a plan is a new immutable value made by applying ops (chapter 7), so a comparison never mutates the pipeline under test. And the build engine names every build node by a digest (chapter 11), so ten variants that share a parser parse the corpus once. Together they make comparison cheap enough to run after every edit.

20.1 Variants as patched plans; `compare` and its deltas

`plan.variant(id, ops)` returns a copy of the plan with a new id and the ops applied. The Field weights preset defines three variants of one plan, each changing one parameter of the BM25 index:

```
const textOnly = plan.variant('text-only',  [{ op: 'set', node: 'ix-text',
path: 'fields', value: { text: 1 } }]);
const headings = plan.variant('headings-x6', [{ op: 'set', node: 'ix-text',
path: 'fields.headingPath', value: 6 }]);
const flatB    = plan.variant('b-0.3',       [{ op: 'set', node: 'ix-text',
path: 'b', value: 0.3 }]);
```

`ev.compare(targets)` runs the eval set against each target in turn and returns four things: `rows`, one summary per target; `cases`, the rank of every case under every target; `reports`, the full per-target reports with their misses; and `delta`, the difference

between the last target's summary and the first's. The delta is only last-minus-first, not a matrix, so put the baseline first and the candidate last when you want it to mean "what the change did".

Here is the real comparison for Field weights, with five metrics requested:

variant	hit@1	hit@5	mrr	ndcg@10	recall@10
fields (baseline)	0.917	1	0.933	0.847	0.861
text-only	1	1	1	0.809	0.792
headings-x6	0.917	1	0.938	0.842	0.861
b-0.3	0.917	1	0.938	0.829	0.840

The `cases` table shows that only one case changed rank in any variant. Case 6 ("How does the eval tool report line numbers in errors?") sits at rank 5 under the baseline, rank 1 under text-only, and rank 4 under the other two. Every other case is at rank 1 everywhere. The headline therefore has to be read with chapter 19's arithmetic in mind. text-only's perfect hit@1 and MRR are one case moving from 5 to 1. Its lower nDCG and recall are the same mechanism working against it on other cases: with heading and title weights removed, the first relevant section still wins, but the second and third relevant sections of multi-section answers fall out of the top ten. Whether text-only is better depends on which reader you serve. A results page judged by its first result prefers it; a `pack` node filling Claude's context prefers the baseline.

The Chunk sizes preset gives a comparison with larger movements:

variant	hit@1	hit@5	mrr	ndcg@10	recall@10
chunks (450 tokens, depth 3)	0.917	0.917	0.931	0.867	0.896
200-tok	0.833	1	0.903	0.746	0.757
900-tok	0.917	1	0.958	0.903	0.931
h2-only	0.5	0.667	0.563	0.509	0.579
overlap-40	1	1	1	0.871	0.889

Two cautions from earlier chapters apply directly. h2-only's collapse is partly real and partly the four cases with R = 0 that section 18.2 showed are unanswerable under that chunker; four zeros out of twelve cap its hit@5 at 0.667 before retrieval is considered. And 200-tok's lower recall is partly its larger R: more, smaller relevant chunks make a perfect recall@10 harder to reach. Of the remaining variants, overlap-40 and 900-tok improve the

first-result metrics, and 900-tok improves nDCG and recall as well. The cases table attributes overlap-40's perfect hit@1 to one case, case 6 again, which moves from rank 6 to rank 1.

In the workbench, variants appear as chips in the Evals tab. Ticking chips and pressing **Run 12 cases** calls `compare` with the current workspace first and the ticked variants after it, and the tab shows the rows and the per-case ranks.

20.2 A sweep is a grid of `set` ops, and each configuration rebuilds only what it changed

A variant is one hand-chosen change. A sweep is every combination of a few parameter values. `rag.sweep(target, grid)` takes a grid whose keys are `node.path` strings and whose values are lists, forms the Cartesian product, and for each configuration:

1. turns each key into a `set` op on that node and path,
2. applies the ops to the base IR with `rag.applyOps`,
3. builds the result with the target's engine, recording which build nodes were not reused,
4. runs the eval set and stores the summary with the configuration and the rebuilt list.

The Parameter sweep preset sweeps a live parameter, the RRF constant `fuse.k`, against a build parameter, the heading-path weight of the text index:

```
const res = await rag.sweep(ws, {
  'fuse.k': [5, 20, 60, 120],
  'ix-text.fields.headingPath': [1, 3, 6],
}).run(kit.goldens().metrics(['mrr', 'hit@1', 'hit@5', 'ndcg@10']), { metric:
'mrr' });
```

The real result, twelve configurations in 590 ms in node, sorted best first by MRR:

fuse.k	heading weight	mrr	hit@1	hit@5	ndcg@10	rebuilt
5	6	0.938	0.917	1	0.861	ix-text
20	6	0.938	0.917	1	0.859	—
60	6	0.938	0.917	1	0.849	—
120	6	0.938	0.917	1	0.845	—
5	3	0.933	0.917	1	0.860	ix-text
20	3	0.933	0.917	1	0.860	—

fuse.k	heading weight	mrr	hit@1	hit@5	ndcg@10	rebuilt
60	3	0.933	0.917	1	0.851	—
120	3	0.933	0.917	1	0.849	—
5	1	0.931	0.917	0.917	0.843	files parse chunks ix-text ix-meta
20	1	0.931	0.917	0.917	0.844	—
60	1	0.931	0.917	0.917	0.840	—
120	1	0.931	0.917	0.917	0.838	—

The rebuilt column is the build engine at work. The configurations ran in grid order, (5, 1), (5, 3), (5, 6), (20, 1) and so on. The first had nothing built yet, so it built every node. The second and third changed only the heading weight, a parameter of `ix-text`, so only `ix-text` was rebuilt; the parser and chunker were reused by digest. Every later configuration reused an `ix-text` it had already built for that heading weight, because the digest of the index with weight 3 is the same no matter which `fuse.k` accompanies it. `fuse.k` is a parameter of a live node and never causes a rebuild. Twelve configurations cost one full build and two index builds.

20.3 Reading a sweep: the best configuration, ties, and overfitting to twelve questions

The sweep reports `best`, which is the first row after sorting by the chosen metric. Here it is `{ 'fuse.k': 5, 'ix-text.fields.headingPath': 6 }`. Three things about that answer should be checked before it is adopted.

First, ties. Four configurations share the top MRR of 0.938. The sort is stable, so among ties the winner is whichever ran first in grid order. `best` is a ranking artefact among equals, not a finding that `fuse.k` should be 5.

Second, what moved. The whole MRR spread, 0.931 to 0.938, is one case. Per-case ranks with heading weight 3 and with heading weight 6 are identical except for case 6:

```
heading 3: 1,1,1,1,1,5,1,1,1,1,1,1
heading 6: 1,1,1,1,1,4,1,1,1,1,1,1
```

With heading weight 1, case 6 falls to rank 6, which is why hit@5 drops to 0.917 in the bottom four rows. The secondary metric looks like a trend: within each heading weight, ndcg@10 mostly falls as `fuse.k` rises. Per-case values show what the trend is made of. Between `fuse.k` 5 and 120 at heading weight 6, three cases move: case 1 rises from 0.798 to 0.818, case 8 rises from 0.850 to 0.877, and case 12 falls from 1.000 to 0.766. The

downward slope is one case losing a secondary relevant section, partly offset by two small gains. A smooth line in a summary chart can still be the movement of a single case, and the per-case table is the only way to tell.

Third, overfitting. A sweep searches for the configuration that scores best on these twelve questions. With twelve configurations and twelve cases, the winner is partly selected for the particular wording of case 6. The protection is the same as in any model selection: keep cases the sweep did not see. In practice that means running the chosen configuration against synthetic cases (section 18.5) or newly written goldens before adopting it, and preferring the simplest configuration within one case of the best. Here, heading weight 3 with the default `fuse.k` of 60 is within one case-position of the winner on every metric.

20.4 `answerHas`: the cheapest check on an answer

Ranking metrics stop at the hits. An answer can be wrong even when the right section was retrieved, because Claude can misread it or the packer can drop it; that is the "misread" failure from chapter 1. The cheapest answer check is a text pattern. A case with `answerHas` is scored on whether the generated answer's text matches the pattern, by the same matching rules as document and section patterns.

The Judged evals preset adds one such case to the goldens:

```
kit.goldens()
  .case('Which tier of the explorer verified KPIs in a browser, and what did
it check?',
        { doc: 'Tier B', section: /customer page/i, answerHas:
/282,180|77,187/ })
```

The pattern asks for one of two numbers that appear in the evidence. It costs nothing beyond the answer itself, and it is precise where it applies. Three properties limit it. It is scored only in `submit` runs, because only those produce an answer. Its summary is averaged over the cases that define it, so with two such cases the `answerHas` column moves in steps of 0.5. And it tests spelling, not meaning: an answer that writes 282180 or "about 282 thousand" fails the pattern while being correct. Use it for facts with one canonical form (identifiers, numbers as the reports print them, named rules) and use a judge for everything else.

20.5 Judges: groundedness and length as lambdas that call Claude

A judge is a named lambda attached to the eval set with `.judge(name, fn)`. In a run with `judge: true`, after each case's answer is produced, the engine calls every judge with three arguments: the run's signals, a context whose `sample` is Claude, and a record `{ q, answer, evidence }`, where `evidence` is the text of every chunk the answer was given. The judge returns a number, normally 0 or 1, which becomes a metric column. If it throws, the value is `null` and the case records `judgeError`; a failing judge never fails the run. Judges are lambdas like every other user function in the system (chapter 9), so they are stored as source and run closed.

The Judged evals preset defines two:

```
.judge('grounded', async (s, ctx, c) => {
  const v = await ctx.sample.json('Is every claim in the answer supported by
the evidence? Reply with only JSON {"grounded": boolean}.\n\nAnswer:\n' +
c.answer + '\n\nEvidence:\n' + c.evidence.join('\n---\n').slice(0, 20000), {
modelTier: 'quick' });
  return v && v.grounded ? 1 : 0;
})
.judge('concise', async (s, ctx, c) => {
  const v = await ctx.sample.json('Does this answer address the question
directly in under 150 words without filler? Reply with only JSON {"ok":
boolean}.\n\nQuestion: ' + c.q + '\n\nAnswer:\n' + c.answer, { modelTier:
'quick' });
  return v && v.ok ? 1 : 0;
})
```

The run below was made in node with a mock sampler that returns fixed replies: every answer is the same placeholder sentence, every `grounded` verdict is true and every `concise` verdict is false. The numbers in the judge columns therefore show the mechanism, not the quality of any pipeline, and the ranking metrics in a mocked `submit` run reflect the mock reranker, not Claude's:

```
submit + judge: { n: 13, grounded: 1, concise: 0, answerHas: 0, …, calls: 52
}
```

The call count is real arithmetic: 13 cases × (one `rerank.llm` call + one `answer` call + two judge calls) = 52. Every judge adds one call per case.

A judge is a model grading a model, and it deserves the same scepticism as any other measurement instrument. Three practices keep it honest. Ask binary questions with a stated criterion, as both judges above do, rather than asking for a score from 1 to 10, whose scale drifts. Calibrate before trusting: run the judge on a handful of answers you have labelled yourself and count how often it agrees. And keep judges narrow. "Grounded" asks whether claims are supported by the evidence, which a model can check by reading; "correct" would ask the judge to know the answer, which reintroduces the problem the pipeline exists to solve.

20.6 What an eval run costs, and when to spend it

The costs below are measured on the bundled corpus. Lexical latencies are per case in node; Claude call counts are exact and do not depend on the sampler.

Run	Claude calls	Time
12 goldens, `change` trigger (Run 12 cases)	0	median 0.4 ms per case
compare, baseline + 3 variants, `change`	0	under a second
sweep, 12 configurations, `change`	0	590 ms including 3 builds
13 cases, `submit` with rerank.llm + answer	26	dominated by Claude latency
same, plus 2 judges (Run with Claude)	52	dominated by Claude latency
`evals.generate`, 8 synthetic questions	1	one quick-tier call

The table suggests a policy. Lexical runs are free, so run them after every edit; the Evals tab makes that one click, and the agent in Part IX can run them from its eval tool. Variant comparisons and sweeps are also free when the parameters they change are lexical, so explore widely there. Spend Claude calls when the change touches something only a `submit` run can see: a rewrite or rerank prompt, the packing budget, the answer instructions. Spend judge calls last, when retrieval is stable and the question is whether the answers are good enough to ship. Each step in that order costs more and changes less often, which is the order in which a pipeline settles.

Key points

- `plan.variant(id, ops)` makes an immutable patched copy, and `compare` scores every target on the same cases; its `delta` is the last target minus the first.

- The per-case ranks in `compare(…).cases` show which cases moved; in Field weights, the only case whose first relevant rank changes between variants is case 6.
- A sweep is a Cartesian grid of `set` ops; the digest-addressed build memo means twelve configurations cost one full build and one index build per distinct heading weight.
- The sweep's `best` is the first row after a stable sort, so ties are broken by grid order; check what moved and confirm the winner on cases the sweep did not see.
- `answerHas` checks an answer's text for a canonical fact at no extra cost; judges check what text patterns cannot, at one Claude call per judge per case, and must be calibrated against hand labels.

Everything in this part scored a pipeline that stood still. Part VII turns to pipelines that change while they run: edits as operations, workspaces with revisions, and a history that lets any change be undone.

Lab · Chunk sizes, Parameter sweep, Judged evals. Load Chunk sizes, open the Evals tab, tick all four variant chips and press **Run 12 cases**. In the per-case table, find the four dashes in the h2-only column and confirm they are the R = 0 cases from section 18.2. Then load Parameter sweep and press **Run sweep · 12 configs**; check the rebuilt column, and read the line chart to see that only the heading weight separates the lines. Finally, if your session can call Claude, load Judged evals and press **Run with Claude**. It spends about four calls per case, roughly 52 in all. Read one answer that the `concise` judge failed and decide whether you agree with it.

Part VII · Changing it safely

A pipeline that a person and an agent both edit while it serves queries needs edits that are small, checked and reversible. This part makes every change an operation on the IR and gives the workspace that applies them a history.

21. Edits as operations

This chapter explains how a running pipeline is changed. The answer is a small vocabulary of **operations**, or ops: plain JSON objects, each naming one node of the IR and one change to it. A list of ops is applied by copying the IR, applying each op in order, and compiling the result again with every check from chapter 10. If any op fails, or the resulting graph fails a check, the whole list is refused and nothing changes. By the end of the chapter you should be able to read any op list the agent or the editor produces, predict what it will rebuild, and explain why the workbench edits pipelines this way rather than by editing the script.

The running example is a four-node live plan over the standard corpus: two retrievers, one over the BM25 index and one over the frontmatter terms index, fused by reciprocal rank and cut to `s.k` hits.

```
const corpus =
rag.corpus('reports').from(rag.files('*.md')).parse(rag.markdown())
  .chunk(rag.sections({ depth: 3, maxTokens: 450 }))
  .index('text', rag.bm25({ fields: { title: 2, headingPath: 3, callout: 2,
text: 1 } }))
  .index('meta', rag.terms({ keys: ['type', 'tags', 'status', 'aliases'] }));

const plan = rag.plan('demo').over(corpus).signals({ q: 'rebalance law', k: 8
})
  .retrieve('text', { k: 30 })
  .retrieve('meta', { k: 30 })
  .fuse.rrf()
  .limit(s => s.k);

const ws = rag.workspace(plan);
await ws.rebuild();
```

Compiled, this is the IR below. Every op in this chapter names one of these ids.

```
files:source.files → q:signal → k:signal → parse:parse.markdown →
chunks:chunk.sections
  → ix-text:index.bm25 → ix-meta:index.terms → text:retrieve → meta:retrieve
→ fuse:fuse.rrf → limit:limit
```

21.1 The ten ops

An op is an object with an `op` field and whatever fields that kind of op needs. The library defines exactly ten kinds. Anything else is refused with an error that lists them:

```
[op] op 1 (move): unknown op 'move'. Ops: set unset insert remove replace
wire signal trigger output rename
```

The function that applies them is short enough to state in full as pseudocode:

```
// pseudocode of rag.applyOps(ir, ops)
raw = a copy of ir's nodes (without signal nodes), signals, pinned outputs
for each op at index i:
    raw = applyOp(raw, op)          // throws "op i+1 (kind): reason"
return finalize(raw)                // the chapter 10 compiler: ports, ids,
cycles, order, digests
                                    // throws "after ops: N problems: ..."
```

Two properties follow from this shape. First, ops are applied to a copy, so a refused list leaves the original IR untouched; a workspace that receives a bad list keeps its revision and its graph. Second, the checks run once, on the final graph, so an op list may pass through an illegal intermediate state as long as it ends in a legal one. Removing a node that something else still depends on is the usual case: the list `[wire, remove]` succeeds where `[remove]` alone fails.

Op	Fields	Effect
`set`	`node`, `path`, `value`	Sets one param, by dotted path into `p`.
`unset`	`node`, `path`	Deletes one param, by dotted path.
`insert`	`node: {t, p}` plus `after`, `before` or `in` (+ `feeds`)	Adds a node and wires it into the graph.
`remove`	`node`	Deletes a node and rewires its consumers around it.

Op	Fields	Effect
`replace`	`node` plus `fn` (and optional `param`) or `with: {t, p}`	Swaps a lambda, or swaps the component type while keeping the id and wiring.
`wire`	`node`, `in`	Sets a node's input list.
`signal`	`key`, `value`	Sets a signal's default value.
`trigger`	`node`, `value`	Sets a live node's trigger to `'change'` or `'submit'`.
`output`	`node`, `name`	Names a node as a pinned output of the plan.
`rename`	`node`, `to`	Changes a node's id and every reference to it.

The workspace's `apply` method runs an op list and returns a **receipt**, which chapter 22 describes field by field. For now it is enough to know that `live` lists live nodes and signals whose definition changed, and `dirty` lists build nodes that must be rebuilt. Here is every op applied in turn to the demo workspace, with the receipt each returned and the structural difference reported by `rag.util.diffIr`:

```
## set        {op:'set', node:'text', path:'k', value:50}
   receipt {"revision":1,"live":["text"],"dirty":[],"changed":[],"removed":
[]}
   diff    [{"change":"param","node":"text","path":"k","from":30,"to":50}]

## set nested {op:'set', node:'ix-text', path:'fields.headingPath', value:5}
   receipt {"revision":2,"live":[],"dirty":["ix-text"],"changed":["ix-
text"],"removed":[]}
   rebuild computed [ 'ix-text' ] reused 4 ms 69

## unset      {op:'unset', node:'ix-text', path:'fields.callout'}
   receipt {"revision":3,"live":[],"dirty":["ix-text"],"changed":["ix-
text"],"removed":[]}

## insert     {op:'insert', after:'fuse', node:{t:'rerank', p:{fn:
<lambda>}}}
   receipt {"revision":4,"live":["rerank","limit"],"dirty":[],"changed":
[],"removed":[]}
```

```
   diff    [{"change":"added","node":"rerank","t":"rerank"},
            {"change":"wiring","node":"limit","from":["fuse"],"to":
["rerank"]}]

## replace fn {op:'replace', node:'rerank', fn:(s, ctx, hits, p) =>
hits.filter(h => h.tokens > 40)}
   receipt {"revision":5,"live":["rerank"],...}

## replace with {op:'replace', node:'fuse', with:{t:'fuse.max', p:{}}}
   receipt {"revision":6,"live":["fuse"],...}
   diff
[{"change":"type","node":"fuse","from":"fuse.rrf","to":"fuse.max"}]

## signal     {op:'signal', key:'k', value:5}
   receipt {"revision":7,"live":["k"],...}

## trigger    {op:'trigger', node:'rerank', value:'submit'}
   receipt {"revision":8,"live":["rerank"],...}

## wire       {op:'wire', node:'fuse', in:['text']}
   receipt {"revision":9,"live":["fuse"],...}
   diff    [{"change":"wiring","node":"fuse","from":["text","meta"],"to":
["text"]}]

## output     {op:'output', node:'text', name:'lexical'}
   receipt {"revision":10,"live":[],"dirty":[],"changed":[],"removed":[]}

## rename     {op:'rename', node:'text', to:'bm25'}
   receipt {"revision":11,"live":["bm25","fuse"],"dirty":[],"changed":
[],"removed":["text"]}

## remove     {op:'remove', node:'rerank'}
   receipt {"revision":12,"live":["limit"],"dirty":[],"changed":[],"removed":
["rerank"]}
   diff    [{"change":"wiring","node":"limit","from":["rerank"],"to":
["fuse"]},
            {"change":"removed","node":"rerank"}]
```

Read the receipts against the phases. Edits to live nodes (`text.k` , the rerank lambda, the fusion rule, a trigger) never make anything dirty, because live nodes have no build digest; they take effect on the next query run. Edits to build nodes (`ix-text.fields`) make

exactly that node dirty, and the rebuild that follows reuses the other four build nodes. That is the digest rule of chapter 10, now seen from the editing side.

The positional ops deserve a closer look, because they are where most mistakes happen.

- **`insert` with `after`** puts the new node directly downstream of the named node and moves every consumer of that node onto the new one. Inserting a reranker `after: 'fuse'` therefore rewired `limit` from `fuse` to `rerank`. An `only: [...]` list restricts which consumers move. If the named node was a plan output, and the new node produces the same type, the output moves too.
- **`insert` with `before`** takes over the named node's single input. It refuses nodes with several inputs, because it cannot know which one you meant.
- **`insert` with `in`** wires the new node's inputs explicitly and, with `feeds`, appends it to the inputs of other nodes. This is how a second retriever is added beside an existing fusion.
- **`remove`** connects each consumer to the removed node's own input. That only has an unambiguous answer when there is exactly one data input to pass through. Removing `fuse`, which has two, is refused: `cannot rewire 'limit' around 'fuse' (it has 2 inputs); use wire`.
- **`rename`** rewrites every edge and output that mentions the old id. The graph is the same, but anything keyed by node id sees a removal and an addition, which is why the receipt above lists `text` as removed and `bm25` as live.

One rewiring rule produces a surprising error. When the consumer of a removed node is variadic, like a fusion node whose port is `hits*`, `remove` splices the removed node's inputs into the consumer's list. A retriever's input is the query signal, so removing `meta` while `fuse` still reads it would wire the signal `q` into the fusion:

```
[op] after ops: 1 problem:
  fuse: input 'q' gives signal but this port takes hits
```

The compiler catches it, as it should, but the fix is to say what you mean in two steps: `[{op:'wire', node:'fuse', in:['text']}, {op:'remove', node:'meta'}]`. Because checks run on the final graph, the pair is accepted as one atomic edit.

21.2 Paths into params, and setting a lambda from source text

The `path` of `set` and `unset` is a dotted path into the node's params object, `p`. `'k'` addresses `p.k`; `'fields.headingPath'` addresses one weight inside the BM25 field map. Paths keep ops small and keep unrelated params out of the edit: changing one field weight does not restate the other three, so two edits to different weights do not overwrite each other. Values are normalized before they are stored, the same way the builder normalizes them (chapter 9): a function becomes a lambda `{ $fn: '<source>' }` and a regular expression becomes `{ $re, flags }`. The IR therefore stays JSON after any `set`.

Lambdas can enter a pipeline through three ops, and they do not all behave the same today.

Op	How the lambda is written	Stored in history as
`set` with a function `value`	`value: s => s.k * 2`	`{"$fn":"s => s.k * 2"}`
`replace` with `fn`	`fn: (s, ctx, hits) => hits.slice(0, 5)`	`"fn":"(s, ctx, hits) => hits.slice(0, 5)"`
`insert` or `replace` with a node whose `p` holds a function	`node: { t: 'rerank', p: { fn: (s, ctx, hits) => ... } }`	`{"t":"rerank","p":{}}`

The first two convert the function to source text before the op is recorded. The third reaches the IR correctly, because `applyOps` normalizes node params, but the copy kept in the workspace history is made by a JSON clone, which drops function values. The live graph is right; the record of how it got there is not. The difference matters as soon as the history is replayed, which happens when a branch is merged (chapter 23) or when ops are written back into the script. A merge of a branch whose only change was such an insert produced:

```
branch p   {"fn":{"$fn":"(s, ctx, hits) => hits.slice().reverse()"}}
merged p   {}
run errors [ 'rerank: fnOf(...) is not a function' ]
```

The rule that avoids this is the one the agent's instructions already follow: inside an op, write a lambda as source text. Either form below is stored, replayed and merged correctly, because it is JSON from the start.

```
{ op: 'insert', after: 'fuse', node: { t: 'rerank', p: { fn: '(s, ctx, hits)
=> hits.slice().reverse()' } } }
{ op: 'insert', after: 'fuse', node: { t: 'rerank', p: { fn: { $fn: '(s, ctx,
hits) => hits.slice().reverse()' } } } }
```

The `replace` op has two forms. With `fn` it replaces a lambda param in place, `fn` by default or the one named by `param`; it refuses a node that has no lambda, with an error that names the other form: `fuse (fuse.rrf) has no lambda; use {op:'replace', node, with:{t, p}}`. With `with` it replaces the component type and params while keeping the node's id and every edge, which is how `fuse.rrf` became `fuse.max` above without disturbing the retrievers or the limit.

21.3 `plan.patch(ops)`: the same ops at authoring time

Ops are not only for the running workspace. A plan spec has a `patch` method that appends ops to the spec; `rag.compile` applies them after it has built the IR from the builder calls. Because specs are immutable (chapter 7), `patch` returns a new spec:

```
const p2 = plan.patch([{ op: 'set', node: 'text', path: 'k', value: 50 }]);
rag.compile(p2).node('text').p.k      // 50
rag.compile(plan).node('text').p.k    // 30, the original is unchanged
```

A bad op in a patch fails at compile time, with the same message the workspace would give: `op 1 (remove): 'ix-text' is used by text; remove those first`.

Every preset ends with the same two lines, and this is why:

```
const ops = []; // end ops
return { plan: plan.patch(ops), evals: kit.goldens(), app };
```

The builder calls describe the pipeline as its author designed it. The `ops` array holds the edits made to it afterwards, in the workspace, by a person or by the agent. Keeping them separate means the script stays readable and the edits stay reviewable as a list. Chapter 23 shows the button that writes workspace ops into that line. Variants use the same mechanism: `plan.variant('wide', ops)` is `plan.as('wide').patch(ops)`, so a variant in the Evals tab is literally the base plan plus an op list.

21.4 Dry runs: which nodes go live, which rebuild, and what Claude would cost

`ws.dryRun(ops)` applies the list to a copy of the workspace's IR and classifies the result exactly as `apply` would, but commits nothing: the revision does not move and no event is emitted. It never throws; a refused list comes back as `{ ok: false, error }`.

```
ws.dryRun([{op:'set', node:'chunks', path:'maxTokens', value:200}])
{"ok":true,"live":[],"dirty":["chunks","ix-text","ix-meta"],"changed":
["chunks","ix-text","ix-meta"],"removed":[],"estimate":{"llmCalls":0}}

ws.dryRun([{op:'set', node:'text', path:'k', value:50}])
{"ok":true,"live":["text"],"dirty":[],"changed":[],"removed":[],"estimate":
{"llmCalls":0}}

ws.dryRun([{op:'remove', node:'ix-text'}])
{"ok":false,"error":"op 1 (remove): 'ix-text' is used by text; remove those
first"}
```

The first result says that changing the chunk size costs a rebuild of the chunker and both indexes, because both indexes are downstream of the chunks. The second says that changing a retriever's `k` costs nothing at build time. This is the information a person wants before pressing Apply on a large corpus, and the information the agent should check before an edit that could spend the user's Claude usage.

That cost is the `estimate.llmCalls` field. It is computed as the number of dirty build nodes that spend Claude calls (components marked `llm`, such as `enrich.llm`, and build lambdas that call `ctx.sample`), multiplied by the number of chunks in the current build. It is an upper bound, and on the Doc cards pipeline it is a loose one. Changing the enrichment's model tier there gives:

```
dryRun tier {"ok":true,"live":[],"dirty":["enrich_llm","ix-cards"],"changed":
["enrich_llm","ix-cards"],"removed":[],"estimate":{"llmCalls":473}}
pending     [{"node":"enrich_llm","items":11}]
```

The estimate multiplies by all 473 chunks in the build, but the enrichment only runs over the 11 document cards, so the real number is 11. The engine's `pending()` (chapter 11) reports that number exactly, once a build has run without calls. Treat the estimate as a signal that Claude calls are involved at all, and `pending()` as the count.

21.5 Why operations and not a text diff of the script

The workbench has a script, `pipeline.js`, and an agent that can write JavaScript. It would be possible to make every change by editing the script's text and rebuilding. The workbench does keep that path, for changes ops cannot express, such as defining a new component, changing the search page, or adding an eval set. For changes to the pipeline graph, it uses ops instead, for five reasons.

Ops address the graph, not the text. An op names a node by its id, and the agent finds ids by reading `ws.ir`, a JSON object. A text edit has to locate the right builder call among chained method calls and reproduce valid JavaScript around it. Two builder chains that produce the same graph can look nothing alike, so a text diff says little about what changed in the pipeline.

Ops are checked individually and refused atomically. Every refusal above names the op by index and kind, and names the node involved. A broken script edit fails as a syntax error or a compile error for the whole script, and in the workbench it also leaves the last good build in place with an error panel. For a model that must correct itself, `op 2 (set): no node 'nope'. Nodes: files, parse, chunks, ix-text, ix-meta, text, meta, fuse, limit` is a better error than a stack trace.

Ops make history granular. Each applied list becomes one history entry with an author (`user`, `agent` or `merge`) and a time. Rebuilding from the script, in contrast, is recorded as a single `reset` entry, `build from pipeline.js`, which replaces the whole IR.

Ops replay. The same list can be applied to a branch, replayed onto the main workspace by a merge, written into the script's `ops` line, or attached to a plan as a variant. A text diff of a script can only be applied to that script.

Ops can be priced before they run. `dryRun` tells you what an op list will rebuild and whether it can spend Claude calls. A script edit tells you nothing until you build it.

	Edit the script	Apply ops
Addressed by	Position in source text	Node id in the IR
Checked	Whole script, on build	Each op, then the final graph
Failure	Syntax or compile error; last good build stays up	Named op refused; IR unchanged
History	One `reset` entry per build	One entry per op list, with author

	Edit the script	Apply ops
Cost known before running	No	Yes, with `dryRun`
Can express	Anything, including new components, apps and evals	Changes to nodes, wiring, params, signals, triggers and outputs

The two paths are reconciled rather than kept apart: the `ops` line in every preset is where workspace edits go when they are worth keeping.

The key points to take from this chapter:

- An op is a JSON object naming one change to one node, signal or output, and there are exactly ten kinds.
- An op list is applied to a copy and compiled once at the end, so it is atomic, and it may pass through an illegal intermediate graph if it ends in a legal one.
- Positional ops rewire for you only when the rewiring is unambiguous; otherwise they refuse and tell you to use `wire`.
- Live edits never cause a rebuild; build edits rebuild only the changed node and what lies downstream of it.
- Lambdas inside op node specs should be written as source text, because function values are dropped from the recorded history and lost on replay.
- `dryRun` reports the cost of an edit without making it; its Claude-call estimate is an upper bound, and `pending()` gives the real count.

An op list describes an edit, but something has to hold the IR it edits, number the versions, and tell the editor, the search page and the agent that the graph has changed. That is the workspace, the subject of chapter 22.

Lab: Minimal search. Load the preset and open the DAG tab. Select `text` and change `k` in its params editor: the editor turns your JSON edit into `set` and `unset` ops and applies them, and the toast reports the new revision and any rebuild. Use **Run on Enter only** on a live node to apply a `trigger` op, and **Remove** to apply a `remove` op. Then open the History tab and read the list of entries, each with its author. Finally, ask the agent to change a field weight with `ws.dryRun` first and report what would be rebuilt.

22. Workspaces and revisions

Chapter 21 defined op lists as pure functions from one IR to another. A running workbench needs more than that. It needs one place that holds the current IR, applies op lists to it, numbers each version, and tells everyone who depends on the graph that it changed: the build engine, the search page, the inspector, and the agent. That place is the **workspace**. This chapter describes what a workspace holds, how it refuses edits planned against a version that no longer exists, what its receipts mean and who reads them, and how it refuses edits in Read-only mode. The design goal throughout is that a person and an agent can edit the same pipeline in the same page without either one silently overwriting the other.

22.1 A workspace: IR, engine, revision, listeners

`rag.workspace(target, opts)` compiles its target, which can be a plan spec, a corpus spec or an IR, and wraps the result:

```
const ws = rag.workspace(plan, { engine, name: 'main' });
ws.ir          // the current compiled IR (read-only to callers; changed only
by apply, undo, redo, reset, restore)
ws.revision    // 0, then 1, 2, 3, ... one per change
ws.history     // [{ revision, ops, by, note, at, before }]: what was
applied, by whom, and the IR it replaced
ws.future      // entries that undo has stepped back over, for redo
ws.branches    // named child workspaces (chapter 23)
ws.engine      // the build engine; shared with branches, so built artifacts
are shared too
```

The workspace does not own the engine. The engine is passed in, or the default engine is used, and its build memo is keyed by digest (chapter 11), not by workspace. Two workspaces, or a workspace and its branch, that need the same index get the same built index. That is what makes the experiments in chapter 23 cheap.

Every state change emits an event to the listeners registered with `ws.on(fn)`. Here are the events from a short session: one build, one applied edit, two undos, two redos, and a restore of a snapshot taken a few edits later.

```
event {"type":"build","report":{"ok":true,"ms":191.4,"nodes":{...}}}
event {"type":"apply","by":"user","revision":1}
event {"type":"undo","revision":3,"undone":2}
event {"type":"undo","revision":4,"undone":1}
event {"type":"redo","revision":5}
event {"type":"redo","revision":6}
event {"type":"restore","revision":11}
```

In the workbench there is exactly one main workspace, created the first time the script builds. Everything that edits the pipeline goes through it:

Source of the edit	What it calls	by
The script editor's Build	`ws.reset(ir, 'build from pipeline.js')`	`user`
The node params editor in the DAG tab	`ws.apply(set / unset ops)`	`user`
The node buttons Run on Enter only, Remove	`ws.apply(trigger / remove op)`	`user`
The agent's eval code	`ws.apply(ops, { by: 'agent' })`	`agent`
A merge of a branch	`ws.apply(branch ops, { by: 'merge' })`	`merge`
The History tab's Undo and Redo	`ws.undo()`, `ws.redo()`	(not an entry)

The page has one listener. It updates the header immediately, ignores build events, and otherwise waits 30 ms before acting, so that several edits made in one burst cause one reaction. Then it rebuilds if the engine reports dirty nodes, reruns the search with the `change` trigger, and redraws the inspector. In pseudocode:

```
// pseudocode of the workbench's onWsChange(event)
update the header (revision, dirty count)
if event is a build: stop
after 30 ms of quiet:
    if engine.dirty(ws.ir) is not empty: rebuild those nodes
    rerun the search page's session with trigger 'change'
    redraw the inspector tab
```

The consequence for a person watching the page is that an edit by the agent looks the same as an edit of their own. The DAG redraws, the results change, and the History tab gains an entry marked `agent`. Nothing about the agent's edits is hidden in a separate copy of the pipeline.

22.2 `expect`: optimistic concurrency between the agent and the editor

A person and the agent can both edit the workspace, and JavaScript gives them no locks to take. They do not need locks in the usual sense: the page runs on one thread, so two `apply` calls never interleave inside the workspace. The problem is a different one. The agent reads `ws.ir`, reasons about it, possibly across several tool calls and several seconds, and then applies ops. If the person changed the pipeline in the meantime, the agent's ops were planned against a graph that no longer exists. They may still apply cleanly and still be wrong, for instance by setting a weight back to a value the person just changed on purpose.

The workspace handles this with **optimistic concurrency**. A caller that wants protection passes the revision it planned against as `expect`. If the workspace has moved on, `apply` refuses:

```
const seen = ws.revision;
// 0
await ws.apply([{ op: 'set', node: 'text', path: 'k', value: 50 }], { by:
'user', expect: seen });
await ws.apply([{ op: 'set', node: 'text', path: 'k', value: 60 }], { by:
'agent', expect: seen });
```

```
conflict [conflict] workspace 'main' is at revision 1; these ops were planned
against 0. Re-read ws.ir and plan again.
```

The scheme is called optimistic because it assumes conflicts are rare and checks only at the end, instead of preventing other edits while a plan is being made. There is no automatic resolution. The workspace could try to rebase the agent's ops onto the new IR, but whether `k: 60` should win over `k: 50` is a question about intent, and only the caller can answer it. The error message therefore tells the caller what to do: read the IR again and plan again.

The guard is only as good as the revision it is given. The agent's instructions show the pattern `ws.apply(ops, { expect: ws.revision })`. Written that way, inside a single expression, the revision is read at the moment of applying, so the check always passes. It

protects nothing. The guard does its job when the revision is recorded where the plan was made: read `ws.revision` alongside `ws.ir` in one step, and pass that number to `apply` in a later step, after the person has had a chance to act. The same reasoning applies to code that awaits between reading and writing, such as a rebuild or an eval run, because the person's edits can land during the await.

22.3 The receipt: live changes, nodes to rebuild, removals

Every successful `apply` returns a receipt, and `dryRun` returns the same fields without the revision. The fields are computed by comparing the IR before the ops with the IR after them.

Field	Contains	Computed as	Who acts on it
`revision`	The new revision number	Previous revision + 1	Callers that pass `expect` next time
`live`	Live nodes and signals whose definition changed	Live nodes whose type, inputs, params or trigger differ; signals whose default differs	The search page reruns on `change`
`dirty`	Build nodes with no built result	Build nodes whose new digest is not in the engine's memo	The listener rebuilds them
`changed`	Build nodes whose digest changed	New digest differs from the old one	Reports and the agent
`removed`	Node ids that no longer exist	Ids in the old IR missing from the new one	The inspector clears a selection

The distinction between `dirty` and `changed` is the one that most often confuses readers. A changed build node has a different digest than before the edit. A dirty build node has a digest the engine has not built in this session. Most of the time the two lists are equal. They differ when an edit returns a node to a configuration that was built earlier. Changing `maxTokens` from 450 to 200 and back makes the chunker changed both times but dirty only the first time, because the second digest is still in the memo. The receipt reports both so that a caller can tell "the graph changed here" apart from "work is needed here".

Some receipts look empty and are not. An `output` op changes the IR's pinned outputs, but outputs are neither live nodes nor build nodes, so its receipt lists nothing. A `signal` op appears under `live` by signal name (`"live":["k"]`) although `diffIr`, which compares nodes only, reports no difference. A `rename` appears as a removal of the old id and a live change under the new one, because every list is keyed by id.

In the workbench, the receipt becomes the toast after an edit in the params editor, in this format:

```
rev 2: 1 op · rebuilding ix-text
```

The agent receives the receipt as the return value of its code and sees it as YAML in its tool result. A receipt with a non-empty `dirty` list tells it to expect a rebuild before the next search reflects the change; a receipt with only `live` entries tells it the next run already does.

22.4 Read-only workspaces, and errors written for the agent that will read them

The agent runs in one of three modes, which chapter 29 describes from the agent's side. In Read-only mode the agent may inspect and measure but not change the pipeline. The library supports this with a flag, `ws.readonly`, checked by every method that changes state:

```
readonly [readonly] ws.apply is blocked in Read-only mode. Describe the
change instead, or ask the user to switch to Auto or Ask.
readonly [readonly] ws.undo is blocked in Read-only mode. Describe the change
instead, or ask the user to switch to Auto or Ask.
```

The workbench does not set the flag on its main workspace, because the person must still be able to edit while the agent is restricted. Instead it gives the agent a proxy of the workspace that blocks seven methods (`apply`, `rebuild`, `undo`, `redo`, `merge`, `reset`, `restore`) and passes everything else through. Its message differs in one clause:

```
ws.apply is blocked in Read-only mode. Describe the change instead, or
measure it on a branch (ws.branch).
```

That clause is deliberate. `ws.branch` is not blocked, and a branch is an ordinary workspace without the restriction. In Read-only mode the agent can therefore make a branch, apply ops to it, rebuild it with the shared engine, run the evals against it, and

report the measured effect of a change it is not allowed to make. Only `merge` would bring the change into the main workspace, and `merge` is blocked. Read-only mode thus forbids changing what the person sees, not experimenting.

The error messages throughout the workspace follow one rule: the reader of an error is often a model, and a model corrects itself best when the error names the thing that was wrong and the action that would fix it. Compare the messages the library actually produces with the minimal message each could have been:

Minimal message	Message the library gives
`node not found`	`op 1 (set): no node 'txt'. Nodes: files, parse, chunks, ix-text, ix-meta, text, meta, fuse, limit`
`invalid remove`	`op 1 (remove): 'ix-text' is used by text; remove those first`
`cannot remove node`	`op 1 (remove): cannot rewire 'limit' around 'fuse' (it has 2 inputs); use wire`
`type error`	`after ops: 1 problem: limit: input 'chunks' gives chunks but this port takes hits`
`cycle`	`after ops: 1 problem: the graph has a cycle through: fuse, limit`
`wrong op`	`op 1 (replace): fuse (fuse.rrf) has no lambda; use {op:'replace', node, with:{t, p}}`
`stale`	`workspace 'main' is at revision 1; these ops were planned against 0. Re-read ws.ir and plan again.`

Each message carries an error code (`op` , `conflict` , `readonly`) that code can branch on, the index and kind of the failing op so a long list can be corrected in place, the ids involved, and, where there is one, the op that would have worked. The list of valid node ids in the first message is there because the most common cause of that error is a guessed id. Printing the real ones turns a second failed attempt into a successful one.

The key points to take from this chapter:

- A workspace holds the current IR, a revision counter, a history of applied op lists with the IR each replaced, and a set of listeners; the build engine is shared, not owned.
- In the workbench, the editor, the DAG buttons, the agent and merges all change the pipeline through the one main workspace, so every change is visible and recorded with its author.
- `expect` refuses an op list planned against an older revision; it protects only when the revision is recorded at planning time, not read in the same expression as the apply.
- The receipt separates live changes, which take effect on the next run, from dirty build nodes, which need a rebuild, and from changed build nodes, which may already be built.
- Read-only mode blocks the seven methods that change the main workspace but leaves branches open, so the agent can still measure a change it may not make.
- Errors name the failing op, the ids involved and the fix, because their most frequent reader is the agent.

The workspace's history has so far been a list that only grows. Chapter 23 walks it backwards: undo and redo, branches that hold an experiment until it is measured, and snapshots that cover what ops do not.

Lab: Reports · hybrid + rerank + answer, with the agent. Load the preset and switch the agent to Read-only mode. Ask it to try a higher heading weight in `ix-text` and tell you whether the goldens improve. Watch it make a branch, apply ops to the branch, and report a comparison while the History tab of the main workspace stays unchanged. Then, in Auto mode, ask it to record `ws.revision` in one step and apply a change in the next; between the two steps, change a param yourself in the DAG tab, and read the conflict error it receives.

23. History: undo, branches and snapshots

Chapter 22 recorded every applied op list in the workspace's history together with the IR it replaced. This chapter puts that record to use in four ways. Undo and redo walk back and forth through it. Branches hold an experiment in a separate workspace until it has been measured, then replay it onto the main one. Snapshots save and restore states that ops do not cover, which is what lets a person revert any step the agent took. Exports take the history out of the workspace, either into the pipeline script or into a standalone search page. The design question in each case is the same: what exactly is stored, so that going back is exact rather than approximate.

23.1 Undo and redo restore stored IRs; revisions only count up

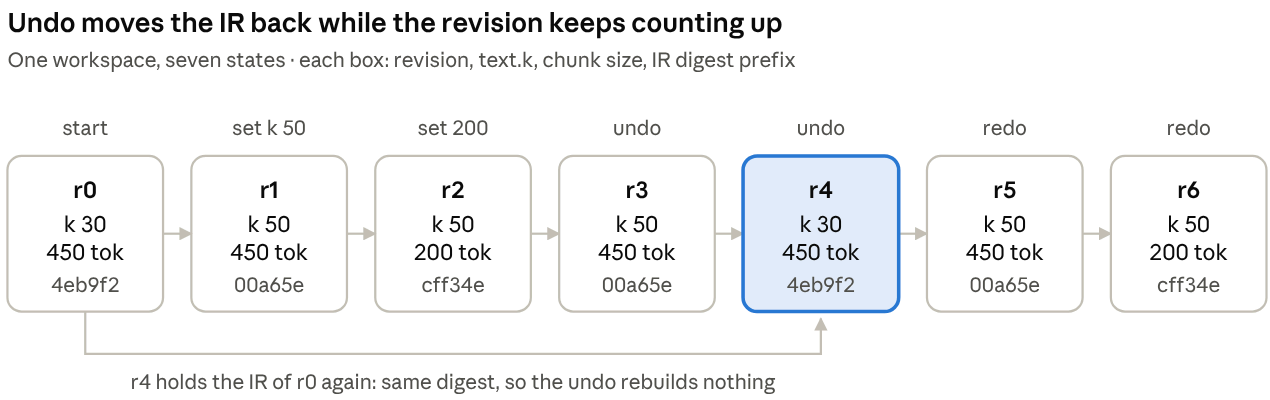
A history entry is a record with six fields:

```
{ revision: 2, ops: [{ op: 'set', node: 'chunks', path: 'maxTokens', value:
200 }],
  by: 'user', note: undefined, at: Date.now(), before: { /* the whole IR
before these ops */ } }
```

`undo` pops the last entry, sets the workspace's IR to that entry's `before`, and pushes the entry onto `future` together with the IR it is leaving, as `after`. `redo` reverses that: it pops from `future`, records the current IR as the new `before`, and sets the IR to `after`. Applying a new op list empties `future`, because the states it held no longer follow from the current one. Both return `null` when there is nothing to undo or redo.

Here is the revision counter, the two params involved and the IR digest through two edits, two undos and two redos. The history and future columns list the revision numbers of the entries on each stack.

```
step               rev  text.k  maxTokens  IR digest        history  future
dirty
start              0    30      450        4eb9f2b953e869
set text.k 50      1    50      450        00a65e4a98a8c9   1
set maxTokens 200  2    50      200        cff34eb3954775   1,2
chunks,ix-text,ix-meta
rebuild            2    50      200        cff34eb3954775   1,2
undo               3    50      450        00a65e4a98a8c9   1        2
undo               4    30      450        4eb9f2b953e869            2,1
redo               5    50      450        00a65e4a98a8c9   1        2
redo               6    50      200        cff34eb3954775   1,2
```



revisions of one workspace · two edits, two undos, two redos

Two decisions in this table need explaining.

The first is that undo restores a stored IR rather than applying an inverse op. An inverse is easy to write for `set`, where the old value is known, but not for the positional ops. Undoing an `insert after` must restore every consumer that was moved onto the new node and every output that moved with it. Undoing a `remove` must restore the node and the rewiring that `remove` performed around it. Each inverse would be a second implementation of the op's rewiring rules, with its own bugs. Storing the IR is exact by construction, and cheap: the demo pipeline's IR is 1,806 bytes of JSON, and a history of a hundred edits costs well under a megabyte.

The second is that undo increments the revision instead of decrementing it. A revision number exists to answer one question for `expect` (chapter 22): is the IR now the same one the caller planned against? If undo moved the counter back from 2 to 1, revision 1 would name two different moments, and a caller who planned at the first one would pass the check at the second even if the person had done something in between. With a counter that only grows, every revision names one moment. Equal content is a separate question, answered by the IR digest: revisions 0 and 4 hold the same IR and have the same digest, `4eb9f2b953e869`.

That equality is also why undo is free. The `dirty` column is empty after every undo and redo. The IRs they restore were built before, so every build digest they contain is still in the engine's memo, and the search page reruns immediately without rebuilding anything.

The workbench adds one rule. Its Undo button is disabled when the most recent history entry is a `reset`, the entry that a Build from `pipeline.js` writes. The library would happily undo a reset and restore the IR from before the build, but the editor would then

show a script that no longer describes the running pipeline. Undo in the History tab therefore goes back as far as the last build and no further; going back past a build means editing the script.

23.2 Branches, and why a merge replays ops instead of merging graphs

A **branch** is a second workspace that starts from the current IR of the first:

```
const b = ws.branch('headings');   // a new Workspace: same IR, same engine,
revision 0, baseRevision 0
```

It is stored in `ws.branches` under its name, it records the parent's revision at the moment it was made as `baseRevision`, and it shares the parent's engine. Edits to the branch leave the parent untouched. That makes a branch the right place for a change whose value is not yet known. Here is a full experiment: raise the heading weight and lower the title weight in the BM25 index, rebuild, and score both workspaces on the twelve golden questions of chapter 18.

```
const ev = kit.goldens();
await ev.run(ws);                                  // main
await b.apply([{ op: 'set', node: 'ix-text', path: 'fields.headingPath',
value: 6 },
               { op: 'set', node: 'ix-text', path: 'fields.title', value: 1
}], { by: 'agent' });
await b.rebuild();
await ev.compare([ws, b]);
```

```
main     {"n":12,"hit@5":1,"mrr":0.892,"ndcg@10":0.818,"p50ms":0.7,"calls":0}
branch receipt {"revision":1,"live":[],"dirty":["ix-text"],"changed":["ix-
text"],"removed":[]}
branch rebuild computed [ 'ix-text' ] reused 4
branch   {"n":12,"hit@5":1,"mrr":0.917,"ndcg@10":0.838,"p50ms":0.7,"calls":0}
main unchanged rev 0 fields {"title":2,"headingPath":3,"callout":2,"text":1}
diff main→branch [{"change":"param","node":"ix-text","path":"fields",
                   "from":{"title":2,"headingPath":3,"callout":2,"text":1},
                   "to":{"title":1,"headingPath":6,"callout":2,"text":1}}]
delta    {"hit@5":0,"mrr":0.025,"ndcg@10":0.02}
cases that moved [{"q":"How does the eval tool report line numbers in
errors?","main":5,"headings":2}]
```

The branch rebuilt one index and reused the other four build nodes from the shared engine. The comparison is specific: one question moved from rank 5 to rank 2 and nothing else changed, which accounts for the whole MRR difference of 0.025 (from 1/5 to 1/2 on one of twelve cases). Whether that is worth keeping is a judgement chapter 19 discusses. If it is, the branch is merged:

```
merge {"revision":1,"live":[],"dirty":[],"changed":["ix-text"],"removed":
[],"merged":"headings","ops":2}
history [{"revision":1,"by":"merge","ops":"set ix-text.fields.headingPath,
set ix-text.fields.title"}]
```

A merge does two things. It checks that the parent has not moved since the branch was made, that is, that `baseRevision` still equals the parent's revision. Then it concatenates the ops of every entry in the branch's history and applies them to the parent as one list, by `merge`. The result is a single history entry on the main workspace, undoable as a unit, and the branch is deleted. The receipt shows `changed` but not `dirty`: the branch already built that index digest in the shared engine, so the merged main workspace needs no rebuild at all. Ops the branch undid are not replayed, because undo removed their entries from the branch's history.

If the parent has moved, the merge is refused:

```
merge conflict [conflict] 'main' moved from revision 1 to 2 since branch
'k60' was made. Re-apply the branch ops on a fresh branch.
```

The remedy the message names is mechanical, and it works:

```
const ops = c.history.flatMap(h => h.ops);   // the stale branch's ops
const c2 = ws.branch('k60-again');
await c2.apply(ops);                         // fails here, with a named op,
if they no longer fit
await ws.merge(c2);                          // {"revision":3,"live":
["text"],...,"merged":"k60-again","ops":1}
```

Why refuse rather than merge? A version control system for text merges two edited copies against their common ancestor, and resolves each region by rules about which side changed it. The same could be done for IRs, field by field. The workbench refuses for a reason specific to its purpose. A branch exists to be measured, and its eval numbers were measured on the base revision plus the branch's ops. If the parent has moved, the merged pipeline is base plus the parent's new ops plus the branch's ops, a graph nobody has measured. Refusing forces the experiment to be replayed on the current state, where it

can be measured again before it is kept. Replaying ops also reuses every check of chapter 21: if the branch's ops no longer fit the moved parent, for example because a node they name was removed, the fresh branch reports which op failed.

Replay has one weakness, inherited from section 21.2. A merge applies the ops recorded in the branch's history, and a function value inside an `insert` or `replace ... with` node spec is not recorded. A branch that inserted a reranker as `p: { fn: (s, ctx, hits) => ... }` works on the branch and merges into a reranker with no lambda, which fails on the first run with `rerank: fnOf(...) is not a function`. Writing lambdas in ops as source text avoids this.

23.3 Snapshots: what the agent saves before each step

Undo covers ops. Some changes are not ops. The agent can replace the script's text, rebuild from it, add or remove a corpus file, load a different preset, or mount a new search page, and a single tool step can do several of these at once. To make every step reversible, the workbench needs a record of state that is larger than the workspace's history.

The workspace contributes its part with `snapshot()` and `restore(snap)`:

```
const snap = ws.snapshot();   // { ir, revision, history: <number of
entries>, future }
```

```
snapshot {"revision":8,"history":2,"future":0,"digest":"229dc7426bd87c"}
after two more ops   rev 10   ... → text:retrieve → fuse:fuse.rrf →
limit:limit           (meta removed)
after restore        rev 11   digest 229dc7426bd87c   history 2   ... →
text:retrieve → meta:retrieve → fuse:fuse.rrf → limit:limit
```

A snapshot stores the IR, the revision, the length of the history and the redo stack. Restoring sets the IR back, truncates the history to its old length, and restores the redo stack. Like undo, it increments the revision, to 11 here, so that a caller holding revision 8 or 10 cannot mistake the restored state for the one it saw.

The workbench's own snapshot wraps this one with everything else a step can change:

Field	Holds
`ws`	The workspace snapshot above
`script`, `scriptBuilt`	The editor's text, and the text of the last successful build

Field	Holds
`uploaded`, `disabled`	The user's uploaded files, and which corpus files are switched off
`appIr`	The gui app currently mounted in the App tab
`preset`, `presetText`	The loaded preset and its original text

Before every tool step the agent runs, outside Read-only mode, the agent panel takes one of these snapshots and keeps it with the step. Each completed step then shows **Revert to before this step**, and each answer shows **Undo this answer**, which reverts to before its first step. Restoring a snapshot puts back the uploaded files, rebuilds the script if its text differs, restores the workspace, and remounts the app; the step and every step after it are marked as reverted. Read-only steps take no snapshot, because they cannot change anything.

The two mechanisms do different jobs. Workspace undo is fine-grained and covers only the pipeline graph; it is what a person uses after a param edit they did not like. Snapshots are coarse and cover the whole page; they are what a person uses after an agent step that did several things at once.

23.4 Exporting a workspace

The workspace's state can leave it in three forms.

Back into the script. The History tab's **Keep in script** button collects the ops applied since the last build, meaning the entries after the most recent `reset`. It appends them to the ops the script already had, writes the result into the script's `const ops = [...]; // end ops` line, and builds the script again. Running that logic on the Minimal search preset after two edits, one by the user and one by the agent, leaves this at the end of the script:

```
const ops = [
  { "op": "set", "node": "text", "path": "k", "value": 40 },
  { "op": "insert", "after": "text", "node": { "t": "limit", "p": { "k": 10 }
} }
]; // end ops
return { plan: plan.patch(ops), evals: kit.goldens(), app };
```

The edits are now part of the script and survive a reload, and the builder calls above that line are untouched. A script without the marker line cannot receive them, and the button says so: `pipeline.js needs a "const ops = […]; // end ops" line to write into`.

As the IR. The IR tab saves the current IR as `<plan id>-rev<revision>.json` and copies it as Mermaid text. This is the exact graph, and `rag.ir(json)` reads it back.

As a standalone search page. `rag.export(ws, { title })`, the IR tab's **Export search page** button, writes one HTML file that runs without the workbench and without Claude. It first removes every live node that runs only on submit, except answers, and then every answer and context node, which leaves the lexical part of the pipeline:

```
full      ... → fuse:fuse.rrf → limit:limit → rerank:rerank.llm! → pack:pack
→ answer:answer!
exported  ... → fuse:fuse.rrf → limit:limit
```

The file embeds the corpus files, the library source and the pruned IR, and runs a search 150 ms after each keystroke. It is a way to hand someone the search experience the workbench measured, without the workbench.

The key points to take from this chapter:

- Each history entry stores the IR it replaced, so undo and redo restore exact states instead of computing inverse ops.
- Revisions only count up, so a revision names one moment; equal content is recognized by equal IR digests, which also makes undo free of rebuilds.
- A branch is a separate workspace that shares the engine, so experiments rebuild only what they change and can be measured before they are kept.
- A merge replays the branch's ops onto the parent and refuses if the parent moved, because the branch's measurements describe only the base it was built on.
- Snapshots cover the whole page (script, files, app and workspace), which is what lets every agent step be reverted.
- Workspace edits can be kept in the script's `ops` line, saved as an IR, or exported as a standalone lexical search page.

Parts III to VII have treated the pipeline as data to compile, run, measure and change. Part VIII turns to the page the pipeline runs in, and shows that the page is described the same way: a grammar of components, compiled once, with closed lambdas for anything that changes.

Lab: the History tab. Load Reports · hybrid + rerank + answer. Change two params in the DAG tab and watch both entries appear in the History tab, marked `user`. Press Undo twice and Redo once and watch the revision count up while the results switch back and forth without a rebuild. Then ask the agent, using its suggestion chip, to fix the eval-tool

miss on a branch, compare it with main and keep it if it helps; afterwards, find the single `merge` entry it produced. Finally, press **Keep in script** and read the `ops` line it wrote, then use **Revert to before this step** on the agent's last step and confirm the script and the History tab return to where they were.

Part VIII · Interfaces

The page a search runs in is described the same way as the pipeline: a tree built once from a grammar, with props that are values or closed lambdas. This part builds that grammar, the runtime that keeps it current, and the decisions a search page is made of.

24. A grammar for pages

This chapter explains how a search page is described as data. By the end you should be able to read any preset's `gui.app(...)` block and say what it will draw, which parts of it can change after it is drawn, and why the description survives being saved, reloaded and edited by the agent. The design follows the pipeline grammar of Part III closely, on purpose: a page is a tree of typed nodes with props, the props are values or closed lambdas, and the whole tree is compiled once into JSON.

The alternative would have been to let preset authors write DOM code: a function that builds elements and wires event handlers. That is more flexible, and it is what most front-end code looks like. It also fails every requirement this workbench has. A function cannot be stored in the database and reloaded, cannot be shown to the agent as text it can edit, cannot be diffed between two presets, and cannot be re-rendered selectively, because nothing outside the function knows which part of the page depends on which value. A tree of nodes can do all four.

24.1 `gui.app`: title, `use(plan)`, state, derive, watch, view

An app is built with a fluent builder, like a plan. Here is the complete page of the Faceted search preset:

```
const app = gui.app('facets').title('Faceted search').use(plan).state({
selected: null })
  .view(v => v.row(r => r
    .col(c => c
      .searchBox('q')
      .toggle('codeOnly', 'Only sections with code')
      .status()
      .facets('filters', { fields: ['type', 'tags', 'status'], from: 'text',
max: 10 })
      .results({ limit: 15 }), { grow: 3 })
    .col(c => c.chunk(s => s.selected), { grow: 2 })));
```

Each builder method sets one part of the app's description:

Method	What it records	Example
`title(t)`	The name shown above the page.	`'Faceted search'`
`use(plan)`	The id of the plan whose signals and outputs the page is bound to.	`{ plan: 'facets' }`
`state(obj)`	Initial values of keys the page owns, beyond the plan's signals.	`{ selected: null }`
`derive(key, f)`	A key computed from other keys, recomputed after every change.	`derive('n', s => s.out.limit?.length)`
`watch(key, f)`	A lambda called with the new value whenever `key` changes.	see the Document browser preset
`view(b)`	The tree of components, built by calling `b` once.	the `row` / `col` tree above

Two of these deserve a closer look. `use(plan)` does not copy the plan into the app. It records only the plan's id, and at mount time the host hands the runtime a live connection to whichever workspace holds that plan. This is why the agent can change the pipeline with an op and the page keeps working: the page never had its own copy of the pipeline.

`watch` is for the cases where one key should drive another. The Document browser preset keeps a plain `doc` key for its dropdown and turns it into the plan's `filters` signal with a watcher:

```
.state({ doc: '', selected: null })
.watch('doc', (s, ctx, v) => ctx.set({ filters: v ? { doc: v } : {},
selected: null }))
```

The watcher receives the new value as its third argument. It is a lambda like any other, so it is stored as source text and run without its closure (chapter 9).

24.2 Layout: row, col, grid, card, tabs, when

Six components arrange other components. They take a builder callback for their children and an optional props object.

Component	Renders as	Props
`col(b, o)`	A vertical stack with a 12 px gap.	`grow`, `gap: 'tight'`
`row(b, o)`	A horizontal flex row that wraps. Each child takes at least 280 px.	`grow` on each child
`grid(cols, b, o)`	A CSS grid with `auto-fit` columns at least `560 / cols` px wide.	`cols`
`card(title, b, o)`	A titled section.	`title`
`tabs(b, o)`	A tab bar; the open tab's index is kept in a state key.	`key`
`when(cond, b)`	Its children when `cond` is truthy, otherwise an empty hidden element.	`cond`

The row rule is the one that matters most for designers. A `.g-row` is `display: flex; flex-wrap: wrap`, and every child has `flex: 1 1 280px`. A row of two columns therefore sits side by side when there is room for two 280 px columns and stacks into one column when there is not. No breakpoint is written anywhere in a preset; the layout adapts because the grammar has one opinion about minimum width. Chapter 27 returns to what this does on a phone.

`tabs` is the only layout component that owns state. The open tab is stored under a key the builder generates (`tab_12`, say) unless you pass one, so switching tabs is an ordinary state change and re-renders only the tab component:

```
.tabs(t => t
  .tab('Answer', b => b.answer({ title: 'Answer' }).chunk(s => s.selected))
  .tab('Sources', b => b.facets('filters', { fields: ['type', 'tags']
}).results({ limit: 20 }))
  .tab('Trace', b => b.trace().dag()))
```

`when` takes a condition, which is almost always a lambda. The Document browser uses three of them to show an outline, a hint, or results depending on whether a document is chosen and whether anything is typed:

```
.when(s => s.doc && !s.q, w => w.table(/* the outline */))
.when(s => !s.doc && !s.q, w => w.note('Choose a document to see its outline,
or type to search everything.', 'info'))
.when(s => s.q, w => w.status().results())
```

24.3 Content: text, note, stat, table, list, chart, code

Content components show values. Each takes its value as a prop, and every prop may be a lambda.

Component	Shows	Notes
`h(text, level)`	A heading.	
`text(md)`	Markdown, with `[E1]` citations turned into buttons.	Supports `wb:` links.
`note(md, tone)`	A tinted callout.	`info`, `ok`, `warn`, `bad`
`badge(text, tone)`	A small label.	
`stat(label, value, o)`	One number with a label, an optional unit and a signed delta.	Numbers ≥ 1,000 print as `1.2k`.
`stats(items)`	A row of stats from an array or an object.	
`kv(obj)`	A two-column key/value table.	
`table(rows, o)`	A table. Columns are inferred from the first 50 rows unless given.	`columns`, `max` (200), `onRowClick`
`list(items)`	A bulleted list of markdown strings.	
`chart(kind, data, o)`	An SVG `line`, `bar` or `scatter` chart.	`bar`, `line`, `scatter` are shortcuts.
`code`, `json`, `link`, `divider`	What their names say.	

The Corpus dashboard preset is built only from these. Its first component computes six statistics from the built corpus:

```
.stats((s, ctx) => {
  s.out;
  const ch = ctx.rag.chunks(), tok = ch.reduce((a, x) => a + x.tokens, 0);
  return [
    { label: 'documents', value: ctx.rag.docs().length }, { label: 'chunks',
value: ch.length },
    { label: 'tokens', value: tok }, { label: 'avg tokens', value:
Math.round(tok / Math.max(1, ch.length)) },
    { label: 'with code', value: ch.filter(x => x.hasCode).length }, { label:
'with tables', value: ch.filter(x => x.hasTable).length },
  ];
})
```

The bare `s.out;` on the second line looks like a mistake. It is a read, and chapter 25 explains why a read with no use is how this lambda asks to be recomputed after every query run.

24.4 Controls bound to state keys: slider, number, select, toggle, input, button

A control is bound to a state key by name. It reads the key to draw itself and writes the key when the user acts.

Control	Writes	When
`slider(key, { min, max, step, label })`	a number	on every `input` event, while dragging
`number(key, o)`	a number	on `change` (blur or Enter)
`select(key, options, o)`	the chosen option's `value`	on `change`
`toggle(key, label)`	`!current`	on click
`input(key, o)`	the text	on every keystroke
`button(label, onClick, o)`	nothing by itself	calls `onClick(s, ctx)`

The key may be a plan signal or a page-only key, and the control does not know or care which. That single rule is what makes the Ranking playground possible. Its four sliders are bound to `k`, `headingBoost`, `failureBoost` and `rrfK`, which are signals of its plan, so dragging one is enough to rerun the search:

```
.card('Knobs', b => b
  .slider('k', { min: 3, max: 20, step: 1, label: 'results (k)' })
  .slider('headingBoost', { min: 0, max: 4, step: 0.25, label: 'heading
boost' })
  .slider('failureBoost', { min: 0, max: 4, step: 0.25, label: 'failure-row
boost' })
  .slider('rrfK', { min: 5, max: 120, step: 5, label: 'rrf k' }))
```

A `select` takes its options as a list of values or of `{ value, label }` objects, and the options may themselves be a lambda. The Document browser builds its list of reports at run time from the corpus, so an uploaded file appears in the dropdown after the next build without any change to the page:

```
.select('doc', (s, ctx) => [{ value: '', label: 'All documents' },
  ...ctx.rag.docs().map(d => ({ value: d.id, label: d.title.replace(/^PROJECT
REPORT - |^PROJ - /, '') }))],
  { label: 'Document' })
```

24.5 Props as lambdas, and why the tree is built only once

Every prop is one of three things: a plain value, a lambda, or a bake. The difference is when it is evaluated.

Kind	Written as	Evaluated	Stored in the tree as
Value	`'Faceted search'`, `15`, `['type', 'tags']`	never; it is already a value	the value
Lambda	`s => s.selected`	every time the component renders	`{ "$fn": "s => s.selected" }`
Bake	`gui.bake(() => …)`	once, when the view is built	the result

The builder callback passed to `view` runs exactly once. It does not draw anything. It appends nodes to a list, and `compile()` turns that list into JSON. Here is the compiled tree of the Faceted search page, produced by running the preset's code in node:

```
{
  "kind": "gui", "version": 1, "id": "facets", "title": "Faceted search",
  "state": { "selected": null }, "derived": {}, "watchers": [],
  "uses": { "plan": "facets" },
  "root": { "id": "n9", "t": "row", "p": {}, "c": [
    { "id": "n6", "t": "col", "p": { "grow": 3 }, "c": [
      { "id": "n1", "t": "searchBox", "p": { "key": "q" } },
      { "id": "n2", "t": "toggle", "p": { "key": "codeOnly", "label": "Only
sections with code" } },
      { "id": "n3", "t": "status", "p": {} },
      { "id": "n4", "t": "facets", "p": { "key": "filters", "fields":
["type", "tags", "status"], "from": "text", "max": 10 } },
      { "id": "n5", "t": "results", "p": { "limit": 15 } } ] },
    { "id": "n8", "t": "col", "p": { "grow": 2 }, "c": [
      { "id": "n7", "t": "chunk", "p": { "id": { "$fn": "s => s.selected" } }
} ] } ] },
  "digest": "04b0e5487dc484"
}
```

Three details in this output are worth reading slowly. Every node has the same three fields, `id`, `t` (its type) and `p` (its props), plus `c` for children; there is nothing else to learn. The only lambda in the page is the chunk viewer's `id`, stored as its source text. And the ids are numbered in the order nodes were finished, not the order they appear: the search box is `n1` and the row that contains everything is `n9`, because a container's children are built inside its callback before the container itself is appended.

A lambda prop is stored as text and compiled with the same guard as a pipeline lambda, so it cannot see the variables around it. This is the rule from chapter 9, and it applies with the same error message. Compiling a page whose lambda refers to a local `limit` and then rendering it gives:

```
gui bad references free identifier `limit`. Lambdas are stored as source and
run without their closure:
use their arguments, a signal, or rag.bake.
```

When a value is known at build time and should not change, a bake avoids the question. `gui.bake(f)` calls `f` while the view is being built and stores the result, so the compiled tree holds a plain value:

```
gui.app('b').state({ n: 1 }).view(v => v
  .text(gui.bake(() => 'built at ' + 'compile'))
  .text(s => 'n is ' + s.n))
// root.c[0].p.md → "built at compile"
// root.c[1].p.md → { "$fn": "s => 'n is ' + s.n" }
```

Lambda props receive `(s, ctx)`. `s` is the page's state, which includes every plan signal, every page key, every derived key, and four keys the runtime maintains: `out` (each node's output from the last run, by node id), `result` (the whole run result), `run` (status, trigger, time and Claude calls of the last run) and `stream` (partial answer text while Claude is writing). `ctx` carries the actions a lambda may take:

`ctx` member	Does
`ctx.set(patch)`, `ctx.get(key)`	Write or read state.
`ctx.run(trigger)`, `ctx.submit(patch)`	Start a query run, as typing or as Enter.
`ctx.rag.chunk(id)`, `.doc(id)`, `.chunks(f)`, `.docs()`	Read the built corpus.
`ctx.rag.explain(id)`, `.result()`, `.ir()`	Read the last run and the plan.
`ctx.toast`, `ctx.copy`, `ctx.download`, `ctx.log`	Talk to the host page.
`ctx.ws`, `ctx.evals`, `ctx.ui`, `ctx.lib`	The workspace, the eval set, the workbench UI, and the `rag` and `gui` libraries.

Handler props receive a third argument describing the event: `onRowClick(s, ctx, row)`, `onClick(s, ctx, ev)` on a canvas, `item(s, ctx, hit)` on results, `onCite(s, ctx, evidence)` on an answer.

Why build the tree only once and evaluate lambdas on every render, instead of rerunning the whole `view` callback? Because a tree built once has stable node ids, and stable ids are what let the runtime replace one component without touching its neighbours. A callback rerun on every change would produce a new tree each time, and the runtime would have to compare two trees to find what changed. The workbench avoids that comparison entirely: structure is fixed at build time, and only lambdas vary.

24.6 `canvas`: drawing with `(s, ctx, g)`

When no component draws what you need, `canvas` hands you a drawing surface. Its `draw` lambda receives a third argument, `g`, with a small set of primitives: `clear`, `line`, `polyline`, `rect`, `circle` and `text`, plus `g.width`, `g.height` and `g.theme`, the page's colours read from its CSS variables. Coordinates are in the canvas's own units (the `width` and `height` props), and the canvas scales to its column.

The Result map preset draws one row per document and one dot per hit, further left for better ranks:

```
.canvas((s, ctx, g) => {
  g.clear();
  const docs = ctx.rag.docs();
  const hits = ((s.out || {}).text || []).slice(0, 40);
  const L = 150, T = 14, rowH = (g.height - T - 8) / Math.max(1,
docs.length);
  docs.forEach((d, i) => { /* label and a rule per document */ });
  hits.forEach((h, r) => {
    const ch = ctx.rag.chunk(h.id);
    const i = docs.findIndex(d => d.id === ch.docId);
    const y = T + i * rowH + rowH / 2, x = L + 8 + (r / 40) * (g.width - L -
30);
    const on = s.selected === h.id || s.hover === h.id;
    g.circle(x, y, Math.max(3, 8 - r / 6), { fill: on ? g.theme.accent :
g.theme.ink, id: h.id });
  });
}, { width: 720, height: 300, label: 'Hits by document and rank',
  onClick: (s, ctx, ev) => { if (ev.hit) ctx.set({ selected: ev.hit }); },
  onHover: (s, ctx, ev) => ctx.set({ hover: ev.hit }) })
```

A shape drawn with an `id` becomes a target: the canvas remembers its geometry, and a click or a hover reports `ev.hit`, the id of the topmost shape under the pointer. That is the whole of the canvas's interaction model, and it is enough to make the map clickable without any DOM per dot.

The primitives check their arguments. A coordinate that is not a finite number throws `g.circle(x, y, r, o): coordinates must be finite numbers`, and the error is painted on the canvas itself in the alert colour rather than thrown into the page. The canvas is also the one component that redraws in place: when state it read changes, it calls `draw` again on the same element instead of being replaced.

The key points of this chapter:

- A page is a JSON tree of `{ id, t, p, c }` nodes, built once by the `view` callback and compiled with a content digest, exactly like a plan.
- A prop is a value, a lambda evaluated on every render, or a bake evaluated once at build time; lambdas are stored as source and cannot see local variables.
- `use(plan)` records only the plan's id, so the page follows whatever pipeline the workspace holds now.
- Controls write state keys by name, and a key that is also a plan signal reruns the search; the control does not need to know which kind of key it writes.
- A row wraps its children at 280 px, which is the only layout rule a preset author has to know.

The tree says what the page is. It does not say when any part of it is drawn again. That is the runtime's job, and the next chapter shows how it decides, component by component, what a change affects.

Lab: Corpus dashboard, Result map. Load Corpus dashboard and read its code next to the page: every chart is one `bar` call over `ctx.rag.chunks()`. Delete the bare `s.out;` line from the "Chunks per document" chart, upload a markdown file in the Files tab, and notice that this chart, alone, no longer updates. Then load Result map, search for something, and click a dot; change `Math.max(3, 8 - r / 6)` to a constant radius and see how much of the rank signal the dot size was carrying.

25. A reactive runtime

This chapter explains how the page decides what to draw again. A view tree from chapter 24 is drawn once when it is mounted. After that, every change to state, whether it comes from a keystroke, a slider, a facet click or a finished query run, has to reach the parts of the screen that show it and no others. By the end of the chapter you should be able to look at any component and say exactly which state keys make it redraw, and to explain why typing a sentence into the search box starts one query run rather than eighteen.

The runtime has one rule, and the rest of the chapter is consequences of it: **a component redraws when a key it read during its last render changes.** It does not redraw because its parent redrew, because a key it might read changed, or because time passed. The runtime knows which keys a component read because it watched the component read them.

25.1 Read tracking through a proxy

When the runtime renders a component, it creates an empty set of dependencies for it and hands every lambda prop a state object that records reads. The state object is a JavaScript `Proxy` whose `get` trap adds the key's name to that set before returning the value. This is the code, from `Runtime.view` in `gui.js`:

```
view(deps) {
  return new Proxy({}, {
    get: (t, k) => { if (typeof k !== 'string') return undefined; if (deps)
deps.add(k);
                     return k in this.der ? this.der[k] : this.state[k]; },
    // has, ownKeys and getOwnPropertyDescriptor make `in` and spreading work
  });
}
```

The chunk viewer's prop `s => s.selected` therefore leaves `{ selected }` in its dependency set after it renders, and nothing else. The Corpus dashboard's `stats` lambda reads `s.out` on purpose, without using the value, so that `out` lands in its set; that is the whole meaning of the bare `s.out;` line in chapter 24. A lambda that reads nothing from `s` never redraws, which is correct for a lambda whose output cannot change and surprising for one that reads the corpus through `ctx.rag` and expects to follow rebuilds.

Built-in components that do not take lambdas declare their reads directly. The status line adds `run` and `out`; the results list adds `run`, `q`, `out` and its selection key; a slider adds its own key. The table below lists them, because a designer who wants to know why a panel flickers needs exactly this list.

Component	Keys it reads, besides its lambda props
`searchBox`	`run`, its key (default `q`)
`status`	`run`, `out`
`results`	`run`, `q`, `out`, its selection key (default `selected`)
`facets`	its key (default `filters`), `out`
`answer`	`out`, `stream`, `run`
`trace`, `dag`	`result`

Component	Keys it reads, besides its lambda props
`slider`, `number`, `select`, `toggle`, `input`	their key
`tabs`	its tab key

Read tracking is dynamic, not static. A lambda such as `s => s.doc && !s.q` reads `doc` always but reads `q` only when `doc` is truthy, so its dependency set changes from render to render. This is the same distinction chapter 9 drew for pipeline lambdas, where a static scan of the source finds the signals a lambda might read and runtime tracking records the ones it did read. The page uses only the runtime half, because a page component is cheap to redraw and a missed dependency would show stale data.

25.2 From a state change to a redraw: set, derive, refresh

Every change goes through one method, `set(patch, source, opts)`. It does five things in order:

1. It applies the patch to state and remembers the old values.
2. It recomputes every derived key, repeating up to four passes until no derived value changes, so a derived key may depend on another derived key.
3. It collects the set of keys whose values actually changed, by identity for state keys and by identity or JSON equality for derived keys. If the set is empty it stops.
4. It calls `refresh(changed, source)`, then every watcher whose key changed.
5. If any changed key is a plan signal, it schedules a query run (25.4).

`refresh` walks the mounted components. For each one still in the page, it skips the component that caused the change (the `source`, usually the input you are typing into, so that your cursor is not destroyed), and it redraws the component if any of its dependencies is in the changed set. Redrawing means rendering the node again from the same tree and replacing its element. Two components instead update their existing element in place: the search box, which must keep focus while results stream in, and the canvas, which repaints rather than being replaced.

Here is what that looks like when measured. The script for this chapter loads the Ranking playground in a headless browser with a mocked Claude, wraps `refresh` to record which components it redraws, and moves the heading-boost slider from 1 to 2. Three refreshes are recorded:

```
changed: [headingBoost]          redraw: []
source: n3 (the slider)
changed: [run]                   redraw: [searchBox (in place), status,
results]
changed: [out, result, run]      redraw: [searchBox (in place), status,
results, dag]
```

The first refresh redraws nothing. The slider is the only component that read `headingBoost`, and it is the source, so it is skipped; its number label was already updated by its own event handler. The second refresh comes from the runtime marking the run as started, and redraws the three components that show run status. The third comes from the finished run and adds the DAG, which reads `result` to colour the nodes that ran. The knobs card, the other three sliders and the section viewer are not touched at any point.

The same trace shows a cost worth knowing about. `results` reads `run` so that it can say “Searching…” when the list is empty, and that read makes it redraw at the start of every run as well as at the end. On a list of twelve results that costs well under a millisecond. On a component that does real work in its render, it is worth checking which of its reads are necessary.

25.3 Plan signals are state keys: the bridge between the page and the session

A page bound to a plan with `use(plan)` gets the plan’s signals as its initial state. The runtime copies `ir.signals` into state when it is constructed, then layers the page’s own `state({...})` on top, then adds `out`, `run`, `stream` and `result`:

```
this.state = { ...signals, ...this.ir.state, out: {}, run: { status: 'idle'
}, stream: null, result: null };
this.signalKeys = new Set(Object.keys(signals));
```

From then on, there is no separate signal store. The search box writes `q`, a facet writes `filters`, a slider writes `headingBoost`, and each of those writes is an ordinary `set`. The only thing that marks them as signals is membership in `signalKeys`, which `set` checks at step 5. When a run starts, the runtime collects the current values of exactly those keys and passes them to the session:

```
signals() { const o = {}; for (const k of this.signalKeys) o[k] =
this.state[k]; return o; }
```

When the run finishes, it writes the session's per-node outputs to `out`, the run result to `result`, and a summary to `run`. Any component can then show any node's output by reading `s.out[nodeId]`, which is how the Compare stages preset puts three nodes' results side by side.

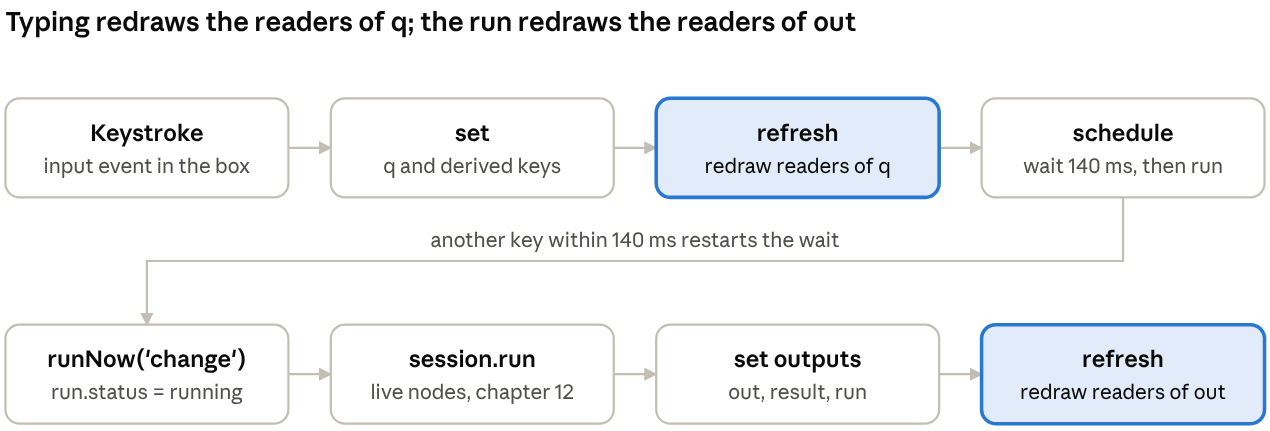
The bridge works in the other direction too. Because the session remembers which signals each pipeline node read (chapter 12), the slider move above reran only the nodes that read `headingBoost`. The trace of that run, read from `state.result`, is:

```
text        cached   in 1   out 40
failures_2  cached   in 1   out 6
fuse        cached   in 2   out 46
rerank      ran      in 46  out 46
limit       ran      in 46  out 8
```

The page redrew four components and the pipeline reran two nodes. Neither side had to be told about the other's bookkeeping; each tracked its own reads.

25.4 Scheduling query runs by trigger

Chapter 12 defined the two triggers: a `change` run happens while the user types and runs only nodes whose trigger is `change`, and a `submit` run happens on Enter and runs everything. This section covers how the page decides when to start each kind.



one keystroke · set, refresh, debounce, run, refresh

When `set` sees a changed signal, it calls `schedule`. A `submit` is started immediately. A `change` waits: `schedule` clears any pending timer and sets a new one for 140 ms. A key pressed within that window cancels the previous timer, so a burst of typing produces one

run after the user pauses. Measured in the browser, typing “why no app actions” (18 characters, 40 ms apart) into the Reports preset started exactly one run:

```
runs: [ { trigger: 'change', t: 3245 } ]
```

Enter bypasses the timer. The search box handles the key itself: it writes `q` with `noRun: true`, so the write does not schedule a `change` run of its own, then calls `runNow('submit')`. Other ways to start a run follow the same pattern. `ctx.run(trigger)` starts one directly, and `ctx.submit(patch)` writes a patch without scheduling and then submits.

Runs can overlap: the user may press Enter and keep typing while Claude is answering. The runtime handles this with two mechanisms. Each run gets an increasing sequence number, and a finished run writes its outputs only if its number is still the latest, so a slow early run cannot overwrite a fast later one. And starting a run aborts the previous one through an `AbortController` whose signal is passed to the session and from there to every Claude call, so an abandoned answer stops spending. While a `submit` run is in flight, the search box’s button reads Stop, and clicking it aborts the run and marks the page as done.

While Claude streams an answer, the session reports partial text through `onText`. The runtime writes it to the `stream` key at most once every 60 ms, which redraws the answer component and nothing else, because only the answer reads `stream`.

25.5 Errors that stay inside the component that raised them

A page made of lambdas will contain lambdas that throw. The runtime’s position is that one broken component must not take the page down with it, and it implements that position separately for each place a lambda can run.

Where the error happens	What the user sees
A prop lambda during render	That component is replaced by one red line, `× text: <message>`. Its neighbours render normally.
A canvas `draw` lambda	The message is painted on the canvas. The element stays, so later state changes can repaint it.

Where the error happens	What the user sees
A handler (`onClick`, `onRowClick`, a watcher)	The message appears in the app's error bar at the top, `× <message>`. The page state is unchanged.
A derived key	The key becomes `undefined`, and the error is kept on the runtime.
A query run	The run's status becomes `error` or `partial`, and the status line shows the message.

The evidence is a small app mounted in the browser, with one healthy `text`, one `text` whose lambda reads a missing property, a `stat`, a canvas that draws a circle with an undefined radius, and a button whose handler throws:

```
gui.app('errs').state({ n: 1 }).view(v => v.col(c => c
  .text(s => 'n is ' + s.n)
  .text(s => s.missing.deep)
  .stat('n', s => s.n)
  .canvas((s, ctx, g) => { g.circle(10, 10, s.nope); }, { width: 100, height:
40 })
  .button('Break', (s, ctx) => { throw new Error('handler failed'); })));
```

After mounting, clicking Break, and setting `n` to 2, the page's components read:

```
n1 g-md:          n is 2
n2 g-err-inline:  × text: Cannot read properties of undefined (reading
'deep')
n3 g-stat:        2 n
n4 g-canvas:      (message painted: g.circle(x, y, r, o): coordinates must be
finite numbers)
n5 btn:           Break
error bar:        × handler failed
```

The healthy text and the stat both followed `n` from 1 to 2. The broken text stayed broken and stayed small. The button still works and can be clicked again. Nothing in the page was blank.

The key points of this chapter:

- A component redraws when, and only when, a key it read during its last render changes; the runtime learns the reads by handing lambdas a recording proxy.

- Every change goes through `set`, which recomputes derived keys, redraws the readers of the changed keys except the component that caused the change, calls watchers, and schedules a run if a signal changed.
- Plan signals are ordinary state keys that the runtime also passes to the session, so a control does not need to know whether it is driving the page or the pipeline.
- Typing schedules one `change` run 140 ms after the last key; Enter starts a `submit` run at once; a newer run aborts and outranks an older one.
- An error is shown at the smallest scope that contains it: the component, the canvas, the error bar, or the status line.

The runtime is general: nothing in this chapter is specific to search. The next chapter turns to the components that are, and shows how each one reads the plan's outputs.

Lab: Ranking playground. Load the preset, open the Inspect panel's Trace tab, and drag one slider at a time. For each slider, predict from the plan which pipeline nodes will show `ran` and which `cached`, then check. `rrfK` is read by `fuse`, so moving it reruns three nodes; `headingBoost` and `failureBoost` are read by the rerank lambda, so they rerun two. Then type a sentence quickly and watch the status line change once, not once per key.

26. Search components

This chapter covers the nine components that know about search: `searchBox`, `results`, `hits`, `facets`, `answer`, `chunk`, `trace`, `dag` and `status`. They are not special to the runtime. Each is an ordinary component that reads state keys, and the keys it reads are the plan's signals and the run's outputs described in chapter 25. What makes them search components is what they read: `s.q`, `s.filters`, `s.out[nodeId]`, `s.run`, `s.result`. By the end of the chapter you should know, for each one, which node's output it shows by default, how to point it at a different node, and what it writes back.

The central fact is that **every node's output is in `s.out`, keyed by node id.** A results list does not show "the results"; it shows the hits output of one particular node, chosen by a rule you can override. Once that is clear, layouts such as the Compare stages preset, which puts three nodes of the same plan side by side, stop looking like special features:

```
.grid(3, g => g
  .card('text · BM25', b => b.results({ from: 'text', limit: 8 }))
  .card('fuse · RRF with meta', b => b.results({ from: 'fuse', limit: 8 }))
  .card('rerank · heading λ', b => b.results({ from: 'limit', limit: 8 })))
```

26.1 `searchBox`: a query signal, a submit action, and a box that keeps focus while results redraw

`searchBox(key, o)` draws a text field and a button. It is bound to a state key, `q` by default, and takes `placeholder` and `submitLabel` (default "Ask"). It has three behaviours, each a direct consequence of chapter 25:

- Typing writes the key on every input event, with the box itself as the change's source. The write is a signal change, so it schedules a `change` run 140 ms later, and the box is not redrawn by its own write.
- Enter, or a click on the button, writes the key with `noRun: true` and then starts a `submit` run. This is the only path that runs nodes with a `submit` trigger, which in the bundled presets means the only path that calls Claude.
- While a `submit` run is in flight, the button reads Stop. Clicking it aborts the run through its `AbortController` and marks the page idle.

The search box reads `run` (to switch between Ask and Stop) and its key, so it is affected by every run. If it were redrawn like other components, its `<input>` element would be replaced twice per run, and the user would lose focus and cursor position in the middle of typing. It is the one component besides the canvas that redraws in place: its `_redraw` updates the button's label and, only if the input does not have focus, copies the key's value into the field. The browser test confirms the effect: after pressing Enter and waiting for the streamed answer to finish, `document.activeElement` was still the search field.

The "only if not focused" condition matters in one case. When something other than the box writes `q`, such as the agent's `ui.query(...)` or a lambda calling `ctx.set({ q })`, the field shows the new value only when the user is not typing in it. The user's keystrokes win.

26.2 `results` and `hits`: showing any hits node, not only the last one

`results(o)` shows a ranked list from `s.out[from]`. When `from` is not given, it uses `ir.outputs.hits`, which the compiler derives (chapter 10) by this rule, in order:

1. an output the plan pinned with `.output('hits', node)`;
2. otherwise the last `rerank` node;
3. otherwise the node that feeds `pack`;
4. otherwise the last node whose output type is `hits`.

The rule is chosen so that the default list shows what the answer will be built from. It has a consequence that is easy to miss, and the Ranking playground shows it. Its plan ends in `rerank(…)` followed by `limit(s => s.k)`, so the derived output is `rerank`, not `limit`. Setting the `k` slider to 3 changes the limit node's output and nothing the page shows:

```
outputs:            { hits: 'rerank' }
before:  k = 8   results drawn: 12   out.limit.length: 8
after:   k = 3   results drawn: 12   out.limit.length: 3   status: “46
results · lexical · 0.1 ms”
```

The list shows the first 12 of rerank's 46 hits, because 12 is the component's own default `limit`. The fix is one prop, `results({ from: 'limit' })`, or one line in the plan, `.output('hits', 'limit')`. The lesson for a page author is to name `from` whenever the plan's last hits node is not a rerank.

Each result is drawn from the hit and from the chunk it points to:

Part	Source
Rank, zero-padded	position in the list
Title	the last heading of the chunk's heading path, or the document title
Chips	the nodes that contributed the hit: the keys of `hit.parts` after fusion, or `hit.via`; plus the chunk's kind if it is not a section
Location line	document title › the rest of the heading path
Snippet	260 characters of the chunk's text, starting near the first query word, with query words marked
Reason	`hit.reason`, prefixed with ↳, when a rerank gave one

The first result of the playground after a search for “Why was the orders tab empty?” shows every part:

```
01  Empty orders tab on load                                    RERANK
FAILURE
TTC Customer Clustering — Order-Grain Segmentation and the Shiny Explorer ›
The Shiny explorer › Failure modes encountered
Failure: Empty orders tab on load Mechanism: Event reactive never evaluates
before first button click Fix: Plain reactive with debounce
↳ heading 1.00 · failure
```

The `FAILURE` chip says this chunk came from the failure-table chunk stream (chapter 4), and the reason line is the string the rerank lambda returned. When the defaults are wrong for a corpus, the `item` prop replaces the title, location line or snippet: `item: (s, ctx, hit) => ({ title, sub, snippet })`. Clicking a result writes its id to `selected`, or to `selectedKey` if given, unless an `onSelect` handler takes over. The list shows `limit` results (default 12) and a count of the rest.

`hits(list, o)` is the compact sibling. It takes a list rather than a node id, so it can show hits that never passed through the plan: a list a lambda computed, or a list the agent wants to put in the chat. Each row shows rank, title, document, score and reason, and a click opens the section in the workbench.

26.3 `facets`: counts from the result set, written back as a filter signal

`facets(key, o)` draws one row of chips per metadata field, each chip a value with a count. It is bound to a state key, `filters` by default, holding an object from field name to a list of chosen values. The plan reads that key in `filter.where()` (chapter 6), so a click on a chip is a signal change and reruns the plan.

The question a facet component has to answer is: counts of what? If it counted the hits after filtering, choosing `tags: architecture` would make every other tag's count drop to zero, and the user could no longer see what choosing another tag would give. So by default `facets` counts the input of the plan's first filter node, the hits before filtering. The Faceted search preset names that node explicitly with `from: 'text'`. Before and after clicking the `architecture` chip, the chip rows read:

```
type    project-report 79   spec 1
tags    architecture 79   pbui 79   project-report 79   typescript 79
workbench 79   testing 61   sync 60 …
status  complete 79   proposed 1

after the click:   filters = { tags: ['architecture'] }   (counts unchanged)
```

The trace of the run the click caused shows the other half of the design: `retrieve` was not rerun, because its inputs had not changed, and only the nodes after it ran.

```
text       cached   in 80  out 80
filter     ran      in 80  out 79
code_only  ran      in 79  out 79
limit      ran      in 79  out 30
```

Three more details complete the component. Choices within one field are alternatives, and choices across fields must all hold, because `filter.where` keeps a hit when, for every field in the filter, the hit's value is one of the chosen values. A chosen value that no longer appears in the counted hits is still drawn, with a count of 0, so the user can always unchoose it. And clicking the last chosen value of a field removes the field from the object, so `filters` returns to `{}` rather than to `{ tags: [] }`. Each field shows at most `max` values (default 8); with no hits, the component says "Facets appear once there are results."

26.4 `answer` and `chunk`: streamed text, citation links, the section viewer

`answer(o)` shows `s.out[from]`, by default the plan's answer output. While a `submit` run is streaming, it shows `s.stream` instead, the partial text written every 60 ms (25.4). The header counts citations, and the body renders the answer's markdown with every `[E1]`-style label turned into a button. Below the text, a list shows the evidence that was cited, with its section and document. In the browser test, with Claude replaced by a mock that returns a fixed two-sentence answer, the component read:

```
ANSWER                                                    2 of 4 sources
cited
ChatGPT sent sessionless server/discover with MCP 2026-07-28, which the
stateful transport did not accept E1.
Making Streamable HTTP stateless fixed it E2.
E1  12. Stateful versus stateless Streamable HTTP · PROJECT REPORT -
CoinVault MCP - From Local Con…
E2  14. Evidence returned by knowledge_search · PROJECT REPORT - CoinVault
MCP - From Local Con…
```

"2 of 4" says the answer cited two of the four sections it was given. Clicking E1 wrote that section's id to `selected`, and the section viewer next to it showed `… › 12. Stateful versus stateless Streamable HTTP`. An `onCite` handler replaces that behaviour.

The answer has four other states, each with its own text. Before any `submit` run it says “Press Enter in the search box for a cited answer. Typing only runs the lexical nodes.” While the run has started but no text has arrived, it says “Thinking…”. When the run reported errors, they appear in a red line under the body. And when the plan has no answer node, it says so.

One behaviour follows from chapter 12 and surprises users. The answer node's trigger is `submit`, so on a `change` run it is skipped and its output is `null`. Typing one more character after an answer has appeared therefore clears it: in the test, after the answer above, typing “ x” returned the component to “Press Enter…”. The answer belongs to the question it answered, and the page does not show it under a different question. Chapter 27 discusses whether that is the right choice.

`chunk(id, o)` shows one section. Its `id` prop is almost always the lambda `s => s.selected`, which makes the viewer follow whatever was last clicked in a result list, an answer citation, a table row or a canvas dot. It shows the section's token count and id, its full heading path, its text rendered as markdown, and three buttons: previous and next section of the same document, which rewrite `selected`, and “Why this?”, which opens the inspector's Trace tab on `rag.explain` for this chunk (chapter 13). With nothing selected it says “Select a result to read the section.”

26.5 `trace`, `dag` and `status`: the pipeline as part of the page

The last three components show the pipeline itself. They exist because chapter 2's argument, that a search pipeline should be something you can see into, applies to the people using a search page as much as to the people building it.

`status(o)` is one line. It reads `run` and `out` and prints one of four things:

State	Text
no run yet	(empty)
running	`Searching…` on a `change` run, `Answering…` on a `submit` run
failed	`x` and the message, in the alert colour
done	`<n> results · lexical · <ms> ms` after typing; `reranked` in place of `lexical` after Enter; `· <c> Claude calls` appended when the run made any

The browser test recorded the status line through one typed question and one Enter:

```
42 results · lexical · 1.3 ms
Answering…
3 results · reranked · 406.6 ms · 3 Claude calls
```

“Searching…” never appeared, because a lexical run over these reports finishes within one animation frame. The word after the count is chosen by trigger, not by what ran: a `submit` run is labelled “reranked” even if the plan has no rerank node. The Claude-call count is the run's own count from the session, so it is exact; in this test the three calls went to the mock.

`trace(o)` is a table of the last run's trace: node, status, items in, items out, milliseconds, one row per live node, with each row styled by its status. It is the same data as the inspector's Trace tab, small enough to put in a page. `dag(o)` draws the plan's graph using the workbench's own renderer in compact form, coloured by the last run's statuses. By default it shows only live nodes, because the build half of a pipeline does not change while someone searches; `dag({ all: true })` shows both. The Answer first preset puts the two together in a tab:

```
.tab('Trace', b => b.trace().dag())
```

The key points of this chapter:

- Every search component reads ordinary state: `q`, `filters`, `selected`, `run`, `result`, and `out[nodeId]`.
- `results`, `facets` and `answer` take a `from` prop; without it they use the plan's derived outputs, and the derived hits output is the last rerank, not the last node.
- The search box schedules `change` runs while typing, starts a `submit` run on Enter, offers Stop during a submit, and keeps focus because it updates in place.
- Facets count the hits before filtering and write an object of chosen values that `filter.where` reads; values within a field are alternatives, fields are combined.
- The answer streams, turns `[E#]` labels into buttons that select the cited section, and clears when the question changes.
- `status`, `trace` and `dag` put the run's cost and path on the page itself.

These components are mechanisms. Whether a page should show the answer first, what a result should say, and what to do when nothing is found are decisions about people, not code, and the next chapter takes them up.

Lab: Compare stages, Answer first, Document browser. In Compare stages, type “cursor semantics tension” and watch sections move between the three columns as each node reorders them; then change the third card to `from: 'rerank'` and confirm it shows the same list as `from: 'limit'` up to the limit. In Answer first, press Enter, open the Trace tab, and match the `trace` rows against the `dag`'s colours. In the Ranking playground, set k to 3 and see the list stay at twelve; add `from: 'limit'` to its `results()` call and set it again.

27. Designing search experiences

This chapter is about the decisions a search page is made of, and it is written first for designers. The previous three chapters described mechanisms: a grammar, a runtime, a set of components. None of them says what a search page should do when the user presses Enter, what a result should tell them, or what to show when nothing matches. Those are decisions about time, trust and cost, and every search page makes them, deliberately or by default. The workbench makes each one in a specific way, and the point of this chapter is to name each decision, show what the workbench does, show the evidence, and say plainly where it does not yet handle a case.

Three quantities shape almost every decision below, so it is worth fixing them first.

Quantity	Typing (`change` run)	Enter (`submit` run)
Time, measured in the browser	1.3 ms for 42 results	406.6 ms, of which about 350 ms is the mock's artificial delay; real Claude calls take seconds
Claude calls	0	3 in the Reports preset: rewrite, rerank, answer
What the user gets	a ranked list of sections	a reranked list and a cited answer

One interaction is a thousand times cheaper and faster than the other. A design that treats them alike will either make typing slow or make answers feel free.

27.1 Type versus Enter: instant ranking, deliberate answers

The first decision is what typing does. The workbench's answer is that typing searches and Enter answers. While the user types, the page reruns the lexical part of the pipeline 140 ms after they pause, and the result list follows their words with no visible delay. Nothing that calls Claude runs until they press Enter or the Ask button. Chapter 12 described the mechanism, per-node triggers; this section is about why it is the right default.

Typing is exploration. A user typing "orders tab" is not yet asking a question; they are watching the list to see whether the corpus uses their words. Lexical results are the right feedback for that, because they respond to exactly the words typed and they cost nothing. An answer is a commitment: it takes seconds, it spends the user's Claude usage, and it is only useful for a finished question. Running it on every pause would spend calls on fragments such as "why was the ord" and would replace the answer under the user's eyes while they are still writing.

The page tells the user the rule in two places. The answer panel's empty state says "Press Enter in the search box for a cited answer. Typing only runs the lexical nodes.", and the Answer first preset's placeholder says "Ask a question and press Enter…". The status line then confirms which kind of run happened: `lexical` after typing, `reranked` and a call count after Enter.

A designer can move the line between the two by changing triggers, not pages. A rerank lambda with no Claude call runs on every keystroke (the Ranking playground's rerank does); setting a node's trigger to `submit` with a `trigger` op holds it back for Enter. The page needs no change, because the search box only ever says "this is typing" or "this is Enter".

27.2 What a result shows: title, heading path, snippet, score, reason

The second decision is what one result says. A result is a claim that this section is relevant, and the user's job is to judge the claim in a second or two. Every part of the workbench's result row exists to help with that judgement, and each is shown or omitted for a reason.

Part	Shown	Why
Section heading as the title	yes	The unit of retrieval is a section (chapter 4), so the section's own heading names it better than the document's title does.
Document and heading path	yes	Users trust a section more when they can see where it lives. "The Shiny explorer › Failure modes encountered" tells them what kind of text to expect.
Snippet near the first query word, with matches marked	yes	It shows the words that matched in context, which is the evidence for the lexical claim.
Retriever chips (`TEXT`, `RERANK`, `FAILURE`)	yes	They tell a pipeline builder which node found the hit. For end users they are noise, and the workbench does not yet have a prop to hide them.
Rerank reason	when present	A rerank that explains itself ("names the discover/stateless cause") gives the user a second, semantic claim to judge.
Score	no, in `results`; yes, in `hits`	A BM25 score of 12.68 means nothing to a reader and invites comparisons across queries that are not valid. The compact `hits` list, used mostly by the agent, shows it.

When the defaults are wrong for a corpus, the `item` prop replaces the title, the location line or the snippet with a lambda over the hit. A corpus of short notes might use the document title as the title; a corpus of code might show the first line of the function instead of a snippet. The row's layout and order stay the same, which keeps lists from different presets readable in the same way.

27.3 Answer first or results first

The third decision is which comes first on the page. The workbench offers both arrangements, and neither is the default for every corpus.

The standard layout, `kit.searchApp`, puts results first: a search column that takes three fifths of the width, with the answer and the section viewer in a column beside it. It suits users who are exploring, who will read sections themselves, and who press Enter only for some questions. The answer is there when asked for and out of the way when not.

The Answer first preset puts the answer first: a search box, a status line, and three tabs, Answer, Sources and Trace. It suits users who came with a question and want a sentence back. Sources are one click away rather than always visible, which is a statement about how much the designer expects users to check.

	Results first (`kit.searchApp`)	Answer first (preset)
First thing seen	ranked sections, instantly	an empty answer panel until Enter
Cost of a visit with no Enter	0 Claude calls, useful page	0 Claude calls, nearly empty page
Checking a claim	citation click shows the section beside the answer	citation click shows the section under the answer; the list is on another tab
Best for	exploring a corpus	asking it questions

One behaviour applies to both layouts and is a decision in its own right. Chapter 26 showed that the answer clears as soon as the user types again, because the answer node is skipped on `change` runs and its output becomes empty. The benefit is that the page never shows an answer beneath a question it did not answer. The cost is that a user refining a question loses the answer they were refining it from. A third option, keeping the previous answer visible but marked as an answer to the previous question, is not implemented in the workbench; it would need the answer component to remember the question each answer was given for.

27.4 Citations the reader can check in one click

The fourth decision is how a reader checks an answer. An answer generated from retrieved text is only as trustworthy as the reader's ability to see that text, so the workbench treats every citation as a link to its section.

The pieces were described in chapter 26: `[E1]` labels in the answer become buttons, a click writes the cited section's id to `selected`, and the section viewer shows that section with its full heading path. In the browser test, clicking E1 in the mocked answer showed `… › 12. Stateful versus stateless Streamable HTTP` in the viewer, one click from claim to source. The header's "2 of 4 sources cited" adds a second signal: the reader knows the answer was built from four sections and relied on two, and can open the other two from the Sources list if they want to know what was left out.

Two gaps are worth naming. The answer does not mark sentences that carry no citation, so an uncited claim looks the same as a cited one; the answer prompt asks Claude to cite, and the `answerHas` and judge checks of chapter 20 measure how well it does, but the page does not show it. And the section viewer shows the whole section, not the sentences that support the claim, so on a long section the reader still has to find the passage.

27.5 Empty, loading, partial and failed states

The fifth decision is what each part of the page says when it has nothing normal to say. Most of a search page's states are not "showing results". The table lists every such state the workbench's components draw, with the exact text, collected from the code and confirmed in the browser where marked.

Component	State	Text
results	nothing typed	"Type a question about the reports." (confirmed)
results	no matches	"No sections match. Try other words, or clear a filter." (confirmed)
results	running, list empty	"Searching…"
facets	no hits	"Facets appear once there are results."
answer	before Enter	"Press Enter in the search box for a cited answer. Typing only runs the lexical nodes." (confirmed)

Component	State	Text
answer	waiting for the first token	“Thinking…”
answer	nothing retrieved	“No evidence was retrieved for this question.” (written by the answer node, without a Claude call)
answer	run errors	the errors, in a red line under the body
chunk	nothing selected	“Select a result to read the section.”
trace	no run yet	“No run yet.”
status	running	“Searching…” or “Answering…”
status	failed	“✗” and the error, in the alert colour

The no-match text is the most important line in the table. It tells the user what to do next, and it names the two causes a user can act on, their words and their filters. The empty-query text does a different job: it tells a first-time user what the box is for.

Three cases are not handled well, and a designer adapting the workbench should know them.

- The empty-state texts mention “the reports” and are built into the components, so a page over a different corpus shows the wrong noun. Only the search box’s placeholder and button label are props.
- A run that finishes with errors in some nodes has status `partial`, and the status line prints it exactly like a successful run. The errors are visible only in the answer panel, so a page without an answer component does not show them at all.
- “Searching…” is almost never seen. Lexical runs over this corpus finish within one animation frame, which is good, but on a larger corpus there is no progress indicator between the old list and the new one.

27.6 Marking the actions that cost Claude calls

The sixth decision is how much the page says about cost. Claude calls spend the user’s usage, and a user cannot make sensible choices about an action whose cost is hidden. The workbench marks cost in several places, before and after the fact.

Where	What it says	When
Preset picker	an `enter` or `build` tag on presets with Claude steps, and “Enter spends Claude calls.” or “Build spends Claude calls.” in the description	before loading
DAG	an `llm` tag on every node that runs on submit	while building the pipeline
Build status	“<node>: <n> items wait for Claude · press Build to spend those calls”	before a build with deferred calls
Evals tab	a separate “Run with Claude” button, whose tooltip says it spends calls	before running evals
Status line	“· 3 Claude calls”	after each run
Search box	Stop, while a submit run is in flight	during a run

The pattern is that the expensive action is always a separate, explicit act (Enter, Build, Run with Claude) and the page says what it spent afterwards. What the workbench does not yet do is say what Enter will cost before it is pressed. The Ask button looks the same on a plan with no Claude steps as on one with three, and the user learns the number only from the status line after the run. An estimate is available in principle, because the plan knows which nodes have a `submit` trigger and which of them call Claude; it is not drawn.

27.7 Phone width: one column, no horizontal scroll

The last decision is what the page does on a phone. The workbench's answer comes from one CSS rule in the grammar (chapter 24): a row's children are at least 280 px wide and wrap. Below about 600 px, every row becomes a column, in source order, and nothing scrolls sideways.

The browser test loaded the Faceted search preset at 390 × 844, the size of a current phone, and measured the page:

```
scrollWidth 390   clientWidth 390        (no horizontal scroll)
column 1 (search, facets, results):  x 46   y 149    width 298
column 2 (section viewer):           x 46   y 3273   width 298
```

At this width the workbench also collapses its own panels into four tabs, Search, Pipeline, Inspect and Agent, so the search page has the screen to itself.

The numbers also show the problem. The section viewer, which sits beside the results on a laptop, is 3,124 pixels below the top of the results on a phone, under fifteen results. Clicking a result writes `selected`, and the viewer updates, but the page does not scroll to it, so on a phone the click appears to do nothing. The workbench does not handle this case. A page designed for phones can avoid it within the grammar: put the viewer in a `when(s => s.selected, …)` above the results, or put results and section in two tabs, so that selection changes what is on screen rather than what is far below it.

The key points of this chapter:

- Typing and Enter differ in cost by orders of magnitude, so the workbench lets typing search and makes answering a separate, explicit act.
- Each part of a result row exists to help the user judge one claim of relevance; scores are left out of the main list because they mean nothing to readers.
- Results first suits exploring and answer first suits asking; both clear the answer when the question changes, which keeps answers honest and loses context.
- Citations are buttons that open the cited section, and the answer says how many of its sources it used.
- Every component has explicit empty, waiting and error texts, but the texts name “the reports”, partial failures look like success in the status line, and there is no progress indicator for slow lexical runs.
- Cost is marked before every expensive act except Enter, where it is reported only afterwards.
- One wrapping rule gives a single column on phones with no horizontal scroll, but selection does not scroll the section viewer into view.

This completes the account of the page. Part IX turns to the other way of changing the workbench: an agent that edits the same pipeline and draws the same components, using the operations of Part VII and the grammar of this part.

Lab: Faceted search, Answer first, Document browser, Result map. Open Faceted search on a phone, or narrow the browser below 600 px, click a result, and find where the section went. Then edit the page so the section appears above the results when one is selected, using `when`. In Answer first, ask a question, then type one more word and watch the answer disappear; decide whether your users would want that. In Document browser, choose a report without typing and notice that the page shows an outline instead of an

empty list, an empty state that is useful rather than apologetic. In Result map, compare how quickly you can see which report dominates a query against reading the same information from the result list.

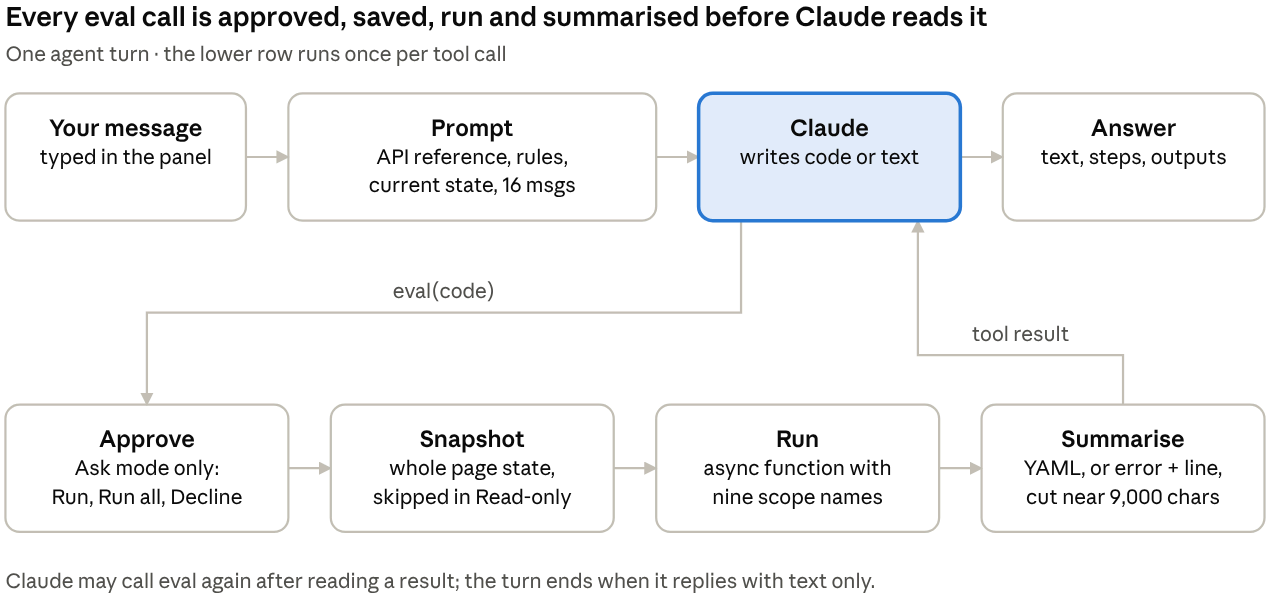
Part IX · The agent

The agent is Claude with one tool that runs JavaScript in the page, so it works through the same `rag`, `gui` and workspace objects as a programmer and is held to the same checks. This part explains that design and the controls around it.

28. One tool: eval

This chapter explains how the agent acts on the workbench. The agent is Claude, called through the page's sampler with a list of tools. Claude reads the prompt and replies either with text or with a request to call one of those tools; the runtime runs the tool, hands the result back to Claude, and repeats until Claude replies with text alone. The workbench gives Claude exactly one tool, `eval`, which runs a piece of JavaScript in the page. Everything the agent does, from running the golden questions to drawing a chart in the conversation, is code passed to that one tool. By the end of the chapter you should be able to read any agent step, predict what it returns, and explain why the tool's output and error messages are shaped the way they are.

A note on the evidence. Every tool result, error message and prompt excerpt below was produced by the real page in a headless browser. The code inside each step, however, was written by the author and fed to the page through a scripted stand-in for Claude, so the steps show what the workbench does with code, not what Claude would choose to write.



one agent turn · prompt, Claude, and the per-call path of the eval tool

The picture shows one turn. The upper row runs once: your message is wrapped in a prompt and sent to Claude. The lower row runs every time Claude calls `eval`, and the turn ends when Claude stops calling it and answers in text. The rest of this chapter follows that path from left to right: why there is one tool (28.1), what the code can reach (28.2), what comes back (28.3 and 28.4), and what Claude was told before it wrote anything (28.5).

28.1 Why one eval tool rather than many narrow tools

The usual way to give a model abilities is one tool per action: `set_param`, `run_evals`, `search`, `show_table`, each with its own JSON schema. The workbench could have done that, and it pays to see why it did not.

Count what the agent can reach. The workbench API `wb` has 28 entry points, the workspace `ws` has 17, the page driver `ui` has 8, the conversation outputs `chat` have 11, and the link builders `ref` have 4. Behind them sit the whole `rag` and `gui` libraries from Parts III to VIII. One tool per action would mean some 70 schemas, each of which has to be written, described to the model and kept in step with the code, and none of which could express a loop, a comparison or a condition. A question such as "which of these five weight changes fixes the miss without losing another case?" would take five round trips per candidate through narrow tools; as code it is one step.

The `eval` tool, by contrast, is defined once and never changes. This is its definition as the page sends it to the sampler:

```
{
  "name": "eval",
  "description": "Run JavaScript in the workbench page as the body of an
async function. Scope: rag, gui, wb, ws, ui, chat, ref, log, console. Use
await; end with `return <small value>`. Returns \"ok\" plus a YAML summary of
the returned value (strings verbatim) and captured logs, truncated near 9,000
characters; or \"error\" with the message and line.",
  "inputSchema": { "type": "object",
    "properties": { "code": { "type": "string", "description": "JavaScript
statements; use await; end with return <small value>" } },
    "required": ["code"] }
}
```

The abilities live in the API, not in the tool list, so every component, op and gui element added to the library is available to the agent the moment it exists. The same code a programmer types into the pipeline script works in an agent step, and an agent step can

be pasted into the script. That is the design goal of the whole book carried one layer up: the pipeline is data that code can read and change, and the agent is one more writer of that code.

	One tool per action	One `eval` tool
Tool definitions	One schema per action, ~70 here	One, fixed
New library feature	A new schema and description	Available at once
Loops, comparisons, conditions	A round trip per call	Ordinary JavaScript in one step
What a mistake looks like	A schema validation failure	A JavaScript or library error with a line number
Where limits are enforced	In the tool list	In the objects the code receives (chapter 29)

The last row is the price. A narrow tool can only do what its schema allows; code can do anything the page can. Chapter 29 is about how the workbench limits that power, and where those limits stop.

28.2 The scope: `rag`, `gui`, `wb`, `ws`, `ui`, `chat`, `ref`, `log`, `console`

When Claude calls `eval`, the agent panel wraps the code in an async function and calls it with nine named arguments:

```
const names = ['rag', 'gui', 'wb', 'ws', 'ui', 'chat', 'ref', 'log',
'console'];
const fn = new Function(...names,
  `"use strict";return (async () => {\n${code}\n})();\n//# sourceURL=agent-
eval.js`);
const res = await fn(scope.rag, scope.gui, scope.wb, scope.ws, scope.ui,
chat, scope.ref, log, cons);
```

Three consequences follow from these lines. The code may use `await` anywhere, because it is the body of an async function. It runs in strict mode, so assigning to an undeclared name is an error rather than a new global. And the `sourceURL` comment

names the code `agent-eval.js` in stack traces, which is how errors get line numbers (28.4).

The nine names are the whole intended interface:

Name	What it is	Typical use
`rag`	The pipeline library of Parts III–VII	`rag.evals(…)`, `rag.explain(result, id)`
`gui`	The page grammar of Part VIII	`gui.app('id').use(ws).view(…)`
`wb`	The workbench: script, files, presets, last result, last eval	`await wb.runEvals()`, `wb.chunks(f)`, `wb.presets.load(id)`
`ws`	The live workspace of chapter 22	`ws.apply(ops)`, `ws.branch(name)`, `ws.ir`
`ui`	Drives the visible page	`ui.query(q)`, `ui.tab('evals')`, `ui.chunk(id)`
`chat`	Puts outputs into the conversation	`chat.table(rows)`, `chat.ui(app)` (chapter 30)
`ref`	Builds `wb:` links for the answer's prose	`ref.chunk(id)`
`log`, `console`	Capture text into the step's logs	`log('k is', ws.ir.node('fuse').p.k)`

The objects are the page's own, not copies. `ws` is the same workspace the DAG tab edits, so an op the agent applies appears in the History tab and re-runs the search box exactly as a person's edit would. The one exception is Read-only mode, in which `ws` and `wb` are replaced by guarded versions (29.3).

A real step that uses three of the names: it runs the golden questions through `wb`, puts the report in the conversation with `chat`, and returns the summary.

```
const r = await wb.runEvals();
chat.evals(wb.lastEval);
return r.summary;
```

The function body runs in the page's global scope, which matters later: the nine names are what the code is given, not all it can reach.

28.3 Results as short YAML, truncated near 9,000 characters

Whatever the code returns is converted to text before Claude sees it. A string is passed through verbatim. Anything else goes through two functions in `agent.js`: `plain`, which turns an arbitrary JavaScript value into plain data, and `yaml`, which prints that data. The step above returned the eval summary, and this is the complete tool result Claude received:

```
ok
n: 12
hit@5: 0.917
mrr: 0.926
ndcg@10: 0.79
p50ms: 1.3
calls: 0
```

YAML was chosen over JSON for the reader, and here there are two readers: Claude, for whom every brace and quote is a token, and the person, who sees the same text in the step's expanded view. The receipt of an applied op shows the difference in shape:

```
ok
before: 0
rec:
  revision: 1
  live: [fuse]
  dirty: []
  changed: []
  removed: []
```

The `plain` pass is where the limits are enforced. Its rules are worth knowing, because they decide what a step should return:

Value	Becomes
String, number, boolean, null	Itself
Object with `toJSON()`	Its `toJSON()` result, so an IR or report prints its summary form
Function	`ƒ name`
DOM node	`<tagname>`

Value	Becomes
Error	`{ error: "Name: message" }`
Map, Set	An object or list of the first 40 entries
Array	The first 40 items, then `… N more`
Object	The first 40 keys not starting with `_`, then `…: N more keys`
Anything nested deeper than 6 levels	`"{…}"` or `"[N items]"`

The depth limit, from a real step that returned a deliberately deep object:

```
ok
n: 490
deep:
  a:
    b:
      c:
        d:
          e: "{…}"
```

After printing comes a length limit. A result longer than 9,000 characters is cut there and ends with a line saying how much was dropped. A real step that returned the id and text of every chunk produced a 9,040-character result ending:

```
      The *signature* of an order is the sorted concatenation of its present
dimensions. Signatures make bask
# … truncated: 37750 more characters
```

Captured `log` and `console` output is appended under a `logs:` line, capped at 6,000 characters, and the whole tool result is capped at 28,000. These limits exist because every tool result stays in the conversation Claude reads on every later call of the turn. A step that returns the corpus wastes the budget of every step after it. The prompt says so in its first rule: "Return specific small values (a few rows, ids, numbers), never whole objects." Large evidence belongs on the screen through `chat` (chapter 30), where the person can read it and Claude only sees that it was shown.

28.4 Errors with line numbers, and how the agent recovers from them

A step that throws produces a tool result that starts with `error` instead of `ok`, followed by the error's class and message and, when the stack trace allows it, the line of the agent's code where it was thrown. This step reads a weight that does not exist:

```
const ir = ws.ir;
const fuse = ir.node('fuse');
return fuse.p.weights.missing.toFixed(2);
```

```
error
error: "TypeError: Cannot read properties of undefined (reading 'toFixed')"
line: 3
```

The line number comes from the stack trace. Because the function was created with `//# sourceURL=agent-eval.js`, the browser reports frames in it as `agent-eval.js:N`. That `N` counts lines of the generated function, which has a header the agent never wrote, so the page subtracts an offset. The offset is not hard-coded: at boot the page builds a probe function the same way, throws on a known line and measures the difference. In Chromium it is 3. The same measurement serves the script editor, which marks the failing line of a pipeline script in the gutter.

A JavaScript error tells Claude what went wrong in its own code. A library error should also tell it what to do instead, and the library's errors are written for that reader. A step that sets a param on a node that does not exist gets this:

```
error
error: "RagOpError: op 1 (set): no node 'nope'. Nodes: files, parse, rewrite,
chunks, failures, synonyms, ix-text, ix-meta, ix-failures, text, meta,
failures_2, fuse, filter, rerank, neighbors, pack, answer"
line: 1
```

The message names the failing op by position and kind, states the reason, and lists the valid ids, so the next step can be written from the error alone. The line is 1 because the error was thrown inside `rag.js`, and the reported line is the agent's line that called into it. Errors met elsewhere in this book follow the same pattern:

Error	Raised by	What the message carries
`RagOpError`	`ws.apply`, `applyOps`	The op's index and kind, the reason, the valid choices

Error	Raised by	What the message carries
`RagCompileError`	Compiling a plan or op result	Every problem found, with node ids
`RagConflictError`	`ws.apply(ops, { expect })`, `ws.merge`	The current and expected revisions, and what to do
`RagReadonlyError`	Guarded `ws` and `wb` in Read-only mode	The blocked call and the allowed alternative
Any JavaScript error	The agent's own code	The message and the agent's line

Recovery needs no special machinery. The error is an ordinary tool result, so Claude reads it and writes the next step. The person sees the failed step in the panel, marked `error` and expanded by default, with its code and the error beneath it. The step's snapshot still exists, so even a step that failed halfway through a change can be reverted (chapter 29).

28.5 The system prompt: an API reference plus the current state of the page

Before Claude writes any code it has read a long instruction text, built fresh for every message. Strictly, it is not a system prompt: the agent panel sends it as the first user turn of the request, followed by the recent conversation and then the new message. In the session used for this chapter it was 11,994 characters long, in four parts.

The first part is one sentence of role: "You are the agent inside a RAG workbench: a page where a person builds and tunes search over a corpus of markdown reports. You act only through the eval tool, which runs JavaScript in the page."

The second part is the API reference, a hand-written markdown text kept in `agent.js` as `DOCS`. It covers the lambda phases, the corpus and plan builders, compile, run and explain, workspace ops, evals and sweeps, `gui`, `kit`, and the agent-only objects `wb`, `ui`, `chat` and `ref`. The same text is shown to the person in the editor's Reference tab, followed by a component table generated from the registry. The agent and the person therefore read the same reference, and an error in it misleads both.

The third part is a list of working rules. Four of them, verbatim:

```
- Prefer one eval that looks, acts, shows and returns a small answer. Return
specific small values (a few rows, ids, numbers), never whole objects. Read
errors: they name the right call.
- Change the running pipeline with ws.apply(ops) (it is undoable and shows in
History). Edit the script (wb.script.set + build) only when the person asks
for a code change.
- Try risky changes on a branch (ws.branch), measure with an eval set, show
the comparison, then merge if it helps. Say what changed and the measured
effect.
- Cost: nodes marked submit (rewrite.llm, rerank.llm, answer, agentic,
enrich.llm, judges) spend the person's Claude usage. Run evals with trigger
'change' (lexical, free) unless asked. Ask before building enrich.llm over
the whole corpus (one call per chunk).
```

The last rule states the current run mode, in words that change with the mode (29.1 to 29.3).

The fourth part, `## Current state`, is the page's state printed as YAML at the moment the message was sent. An excerpt:

```
corpus:
  files: 11
  docs: 11
  chunks: 490
  indexes: [text, meta, failures]
plan: ask
revision: 0
nodes: files:source.files → q:signal → … → fuse:fuse.rrf →
filter:filter.where → rerank:rerank.llm! → neighbors:expand.neighbors →
pack:pack → answer:answer!
signals:
  q: Why did ChatGPT show no app actions after OAuth worked?
  k: 8
evals:
  id: goldens
  cases: 12
  last: not run yet
script:
  lines: 84
```

```
  built: true
  preset: reports
pendingClaudeBuild: []
claude: available
```

The `!` after a node type marks a node that calls Claude. With this section the agent's first step can act rather than look around: it already knows the plan's node ids, the revision, whether the evals have run and whether a Claude build is pending.

After the instructions come up to 16 earlier messages. Earlier agent answers are not replayed as tool calls; each is flattened into text, with every step rendered as its code (clipped to 1,500 characters), its status and its result (clipped to 900), and every output as a one-line marker. This is how the first answer of the session looked to Claude on the next message:

```
[eval]
const before = ws.revision;
const rec = await ws.apply([{ op: 'set', node: 'fuse', path: 'k', value: 20
}]);
return { before, rec };
→ ok:
before: 0
rec:
  revision: 1
  live: [fuse]
  …
[the person reverted this step and everything after it]
[shown: hits "Results for “Why did ChatGPT show no actions after OAuth
worked?”"]
```

The bracketed note on the reverted step matters: without it, Claude would believe its change was still in place. If the whole request exceeds 230,000 characters, the oldest messages are dropped until it fits, which keeps it under the sampler's 256 KiB prompt limit described in chapter 31.

One weakness of this design deserves a plain statement. `DOCS` is prose, and nothing checks it against the code. It documents `ws.apply(ops, { expect: ws.revision })` as the way to guard an edit, and as chapter 29 shows, that call can never conflict. A reference the model trusts needs the same review as the code it describes.

The key points to take from this chapter:

- The agent has one tool, `eval`, which runs JavaScript in the page; its abilities are the page's API, not a list of tool schemas.
- Agent code runs as the body of a strict async function with nine named arguments, and those objects are the page's own, so the agent's edits are indistinguishable from a person's in effect.
- Results come back as YAML, reduced by fixed rules to 40 items, 40 keys and 6 levels, and cut near 9,000 characters, because every result stays in Claude's context for the rest of the turn.
- Errors come back as ordinary tool results with the agent's line number, and the library's messages name the valid alternatives so the next step can be written from the error alone.
- Each message is preceded by a fresh prompt containing the API reference, working rules, the run mode and the current state of the page, followed by up to 16 earlier messages with their steps flattened into text.

One tool that can do anything the page can do needs controls around it. Chapter 29 describes the three run modes, the snapshots that make every step reversible, and the limits of both.

Lab: the suggestion chips in the agent panel. Open the agent panel on a new conversation and pick "Which golden questions does the current plan miss, and why? Show the ranks." Expand each step as it finishes and compare its code with its YAML result. Then open the editor's Reference tab and find, in the text the agent was given, the calls its steps used. Finally, ask the agent to read a node that does not exist, and read the error it gets back and the step it writes next. This step spends Claude usage.

29. Run modes and reversibility

Chapter 28 gave the agent one tool that can do anything the page can do. This chapter explains the controls around that tool. There are two kinds. A **run mode**, chosen by the person in the agent panel, decides what happens before and during each step: whether it runs at once, waits for approval, or runs with the mutating calls taken away. A **snapshot**, taken before each step, decides what can happen afterwards: any step, and everything after it, can be undone. By the end of the chapter you should know what each mode guarantees, what it does not, and how an agent's edit and a person's edit interact when both change the same pipeline.

As in chapter 28, the step code below was written by the author and fed through a scripted stand-in for Claude; the panel behaviour, results and errors are the real page's.

The three modes differ in four ways:

	Auto	Ask	Read-only
Step waits for the person	No	Yes, every step	No
Snapshot before the step	Yes	Yes, after approval	No
`ws.apply`, `wb.script.set`, …	Work	Work	Throw `RagReadonlyError`
Revert afterwards	Yes	Yes, for steps that ran	Nothing to revert

The mode is also stated to Claude, in the last line of the working rules (28.5), so it can plan for it. The three lines, verbatim:

```
- Run mode is Auto. Every step is snapshotted and can be reverted by the
person.
- Run mode is Ask. Each eval waits for the person to approve it, so batch
work into fewer, clearer steps.
- Run mode is Read-only. Mutating calls (ws.apply, merge, rebuild,
wb.script.set, wb.addFile) throw; explain what you would change instead.
Branches you create are fine to edit and measure.
```

The mode is read when an answer starts and recorded on the answer, so switching modes mid-answer affects the next message, not the steps already planned.

29.1 Auto: run, snapshot, report

In Auto mode each step runs as soon as Claude asks for it. The order of events inside `runEval` is the whole contract:

```
// pseudocode of Agent.runEval(code) in Auto mode
add a step part to the answer, status 'run'; repaint
snapshot = host.snapshot()            // chapter 23.3: workspace, script,
files, app, preset
scope = host.scope({ readonly: false, chat, log })
result = run code (28.2); format it (28.3, 28.4)
step.status = ok | err; step.result = result
host.afterStep()                      // re-render inspector and top bar
return result to Claude
```

The snapshot is taken before the code runs, so it records the state the step started from. The page re-renders after every step, so the DAG, the History tab and the search results show the step's effect before Claude has decided what to do next. A person watching an agent work sees each change land.

The panel shows each step as a collapsed line with its status, the first line of its code and its time. The five steps of the session used in chapter 28 appeared like this:

```
eval  ok     const r = await wb.runEvals();
58 ms
eval  error  const ir = ws.ir;
2 ms
eval  error  return ws.apply([{ op: 'set', node: 'nope', path: 'k', value: 1…
3 ms
eval  ok     const r = await ui.query('Why did ChatGPT show no actions after…
25 ms
eval  ok     const before = ws.revision;
3 ms
```

Expanding a step shows its full code, its result and its logs, and a **Revert to before this step** link. Under the whole answer, a footer reads `5 steps · 0.2 s` and offers **Undo this answer**, which reverts to before the answer's first step that still has a snapshot.

An edit made by the agent is an ordinary workspace edit. The last step applied one op and got this receipt:

```
ok
before: 0
rec:
  revision: 1
  live: [fuse]
  dirty: []
  changed: []
  removed: []
```

The receipt is the one chapter 22 describes: revision 1, one live node changed, nothing to rebuild. There is one gap here. The workspace records an author with every history entry, taken from `apply(ops, { by })` and defaulting to `user`. The agent's `ws` is the page's workspace and the agent's code does not pass `by`, so its edits are recorded as `user`. In a session where the agent set `fuse.k` to 12 and 25 and the person set it to 40 in between, the History tab read:

```
3  user  set fuse.k=12     ← the agent
4  user  set fuse.k=40     ← the person
5  user  set fuse.k=25     ← the agent
```

The conversation shows which steps were the agent's; the History tab cannot. Passing `{ by: 'agent' }` fixes it for one call, as the worked session in chapter 30 does, and the agent panel could do it for every call by wrapping `ws.apply`.

29.2 Ask: approve one step, or the rest of the turn

In Ask mode the step is shown before it runs. `runEval` sets the step's status to `ask` and waits on a promise that only the person can resolve:

```
Run this step?                                          Ask mode
return ws.apply([{ op: 'set', node: 'fuse', path: 'k', value: 12 }]);
[Run]  [Run all in this answer]  [Decline]
```

The three buttons resolve the promise three ways. **Run** runs this step and leaves the next one to be asked about. **Run all in this answer** runs this step and marks the answer as approved, so the remaining steps of the same answer run without asking; the next message asks again. **Decline** does not run the code and does not take a snapshot; instead the tool returns this text to Claude:

```
The person declined this step. Ask what they would prefer, or explain without
running it.
```

In a real run of three scripted steps, the person pressed Run, Decline, then Run all. The three tool results were the receipt of the first edit, the declined message, and `ok 12` from the third step, which read `fuse.k` and found the first edit in place and the second never made. The step statuses were `ok`, `declined`, `ok`.

Two details make Ask mode safe to leave unattended. Pressing **Stop** while a step waits declines it, because the approval promise listens to the turn's abort signal. And the snapshot is taken after approval, so a declined step leaves nothing to revert.

Ask mode asks about every step, including steps that only read. This is why its line in the prompt tells Claude to "batch work into fewer, clearer steps": an agent that looks around in five small steps costs the person five clicks.

29.3 Read-only: mutating calls throw, and branches stay open for experiments

Read-only mode does not stop code from running. It changes the objects the code receives. When the mode is Read-only, the scope is built with guarded versions of `ws` and `wb`:

Object	Blocked members
`ws`	`apply`, `rebuild`, `undo`, `redo`, `merge`, `reset`, `restore`
`wb`	`addFile`, `removeFile`, `script.set`, `script.build`, `presets.load`, `build`

The guarded `ws` is a proxy over the real workspace: reading `ws.ir`, `ws.revision` or `ws.history` works, and calling a blocked method throws. The messages are written for the agent, and say what to do instead:

```
error
error: "RagReadonlyError: ws.apply is blocked in Read-only mode. Describe the
change instead, or measure it on a branch (ws.branch)."
line: 1
```

```
error
error: "RagReadonlyError: wb.script.set is blocked in Read-only mode.
Describe the change instead, or ask the person to switch to Auto or Ask."
line: 1
```

`ws.branch` is not blocked, and the branch it returns is an ordinary workspace (chapter 23.2), not a guarded one. An agent in Read-only mode can therefore try a change and measure it without touching the live pipeline. A real step:

```
const b = ws.branch('try-k30');
await b.apply([{ op: 'set', node: 'fuse', path: 'k', value: 30 }]);
return { branch: b.revision, main: ws.revision, mainK: ws.ir.node('fuse').p.k
};
```

```
ok
branch: 3
main: 2
mainK: 60
```

The branch moved to revision 3; the main workspace stayed at revision 2 with `k` at 60. This is the most useful way to run an agent on a pipeline you care about: it can explore and report numbers, and the decision to merge stays with you.

Read-only mode takes no snapshots, because nothing it can change through `ws` and `wb` needs undoing. That assumption has edges, and they should be stated plainly:

- `ui` is not guarded. `ui.query` changes the search box and runs a query, `ui.filters` changes the filters, and `ui.app` replaces the App tab. These change what the person sees, not the pipeline.
- `rag` is not guarded. `rag.configure` changes library-wide settings.
- The code runs in the page's global scope. A Read-only step that returned `typeof window.__wb` got `object`: the workbench's internal state, including the unguarded workspace, is reachable.

Read-only is therefore a guard against mistakes by a cooperating agent, not a sandbox against code written to get around it. That is the right strength for this page, where the agent and the person act for the same user on the user's own data. A page that ran untrusted code would need an isolated realm, such as a worker or an iframe that receives only the guarded objects.

29.4 Reverting to before any step

Chapter 23.3 described what a snapshot holds: the workspace's IR, revision and history length, the script and its last built text, the uploaded and disabled files, the App tab and the loaded preset. This section is about how the agent panel uses snapshots.

Each step that ran in Auto or Ask mode keeps its snapshot, and **Revert to before this step** restores it. In the real session, the fifth step had set `fuse.k` from 60 to 20, taking the workspace from revision 0 to 1. Reverting it gave:

```
before revert   k 20   revision 1   history 1
after revert    k 60   revision 2   history 0
```

The value returned to 60 and the history entry was removed, while the revision went up rather than back, for the reason chapter 23.1 gives: a revision number names a state that existed, and a caller holding revision 1 must not mistake the restored graph for it. The reverted step and every later step of the answer are marked as reverted, and the panel says so under each.

Claude is told. On the next message, the flattened history (28.5) ends that step with:

```
[the person reverted this step and everything after it]
```

Without this line, the agent would describe a pipeline that no longer exists.

A snapshot restores the whole page to a point in time, so reverting has three consequences worth knowing:

- Everything after the snapshot is discarded, including edits the person made by hand after the agent's step. Revert is for undoing an agent step soon after it ran; for a single param edit, the workspace's own Undo is the finer tool.
- Snapshots live in memory, keyed by the step objects of the current page. Conversations are saved (chapter 30.4) but snapshots are not, so a conversation reopened after a reload shows its steps without revert links.
- Read-only steps and declined steps have no snapshot and no revert link.

29.5 Editing the live pipeline with ops versus editing the script

The agent can change the pipeline in two ways, and the prompt tells it which to prefer: "Change the running pipeline with ws.apply(ops) (it is undoable and shows in History). Edit the script (wb.script.set + build) only when the person asks for a code change."

	`ws.apply(ops)`	`wb.script.set(text)` + `wb.script.build()`
Granularity	One op list	The whole script
Rebuild	Only changed build nodes (chapter 21.4)	Every build node whose digest changed
History	One entry per op list, with receipt	One `reset` entry
Cost known in advance	Yes, `ws.dryRun(ops)`	No
Survives a reload	Only if kept in the script	Yes, the script is saved
Can express	Params, wiring, signals, triggers, outputs, node swaps	Anything, including new components and apps

Ops are the safer instrument for an agent: each is small, checked, priced in advance and visible in History. The script is the durable form, and **Keep in script** (chapter 23.4) carries ops into it when the person decides they are worth keeping.

When the agent and the person edit the same workspace, revisions are how an edit avoids overwriting a change it never saw. Chapter 22.2 introduced `apply(ops, { expect })`. For the agent the rule is concrete: read the revision in one step, keep the number, and pass it to the step that edits. A real run, in which the person changed `fuse.k` to 40 between the agent's two steps:

```
// step 1 returned 3; the person then edited fuse.k in the DAG tab
return await ws.apply([{ op: 'set', node: 'fuse', path: 'k', value: 25 }], {
expect: 3 });
```

```
error
error: "RagConflictError: workspace 'main' is at revision 4; these ops were
planned against 3. Re-read ws.ir and plan again."
line: 1
```

The conflict is the correct outcome: the agent planned against a graph that no longer exists. The API reference, however, shows the guard written as `{ expect: ws.revision }`, and that form can never conflict, because it reads the revision at the moment of the call. In the same run it succeeded, set `k` to 25 and silently replaced the person's 40:

```
return await ws.apply([{ op: 'set', node: 'fuse', path: 'k', value: 25 }], {
expect: ws.revision });
```

```
ok
revision: 5
live: [fuse]
…
```

The reference should show the two-step form, and until it does, a careful agent has to know better than its documentation. A second known issue from chapter 21 also bears on agent edits: a lambda written as a JavaScript function inside an `insert` or `replace` op reaches the IR but is dropped from the recorded history, so a merge or Keep in script that replays the ops loses it. Agent code should write such lambdas as source text, `fn: '(s, ctx, hits) => …'`.

The key points to take from this chapter:

- Auto runs each step at once after taking a snapshot of the whole page; Ask shows each step and waits for Run, Run all or Decline; Read-only runs steps with the mutating calls of `ws` and `wb` taken away.

- The run mode is stated to Claude in the prompt, so the agent can plan for it: fewer steps in Ask, branches in Read-only.
- Read-only is a guard for a cooperating agent, not a sandbox: `ui`, `rag` and the page's globals are still reachable.
- Reverting a step restores the snapshot taken before it, discards everything after it including the person's later edits, increments the revision, and tells Claude on the next message.
- Agent edits are recorded in History as `user` unless the code passes `{ by }`, so the History tab cannot tell them apart from the person's.
- A revision guard only protects an edit if the revision was read in an earlier step; `{ expect: ws.revision }` never conflicts.

The controls in this chapter decide what the agent may change. Chapter 30 turns to what it shows: the tables, charts, apps and links it puts into the conversation.

Lab: the mode switch in the agent panel. Load Reports · hybrid + rerank + answer and set the mode to Ask. Ask the agent to raise the heading weight in `ix-text` and run the goldens; approve the first step, decline the second, and read what the agent writes next. Switch to Read-only and ask for the same change: watch it create a branch, apply the op there, and report a comparison while the History tab stays unchanged. Finally, in Auto mode, ask for any edit, then use **Revert to before this step** and check the revision number in the top bar: it goes up, not back. These steps spend Claude usage.

30. Answers the agent can draw

An agent that can only reply in prose has to describe evidence: "the expected section ranked ninth". An agent that can draw puts the ranking on screen and lets the person read it. This chapter explains how the agent's code places tables, charts, hit lists, eval reports, DAGs and complete gui apps into the conversation, how its prose links into the workbench, how it drives the page directly, and how a conversation is stored. It ends with a complete worked session. By the end you should be able to predict what the person sees, and what Claude sees, for any output an agent step produces.

The step code is again the author's, run through a scripted stand-in for Claude; the outputs, rendering, storage and numbers are the real page's.

30.1 `chat` outputs: md, table, chart, hits, evals, dag, ui, download

An agent answer is not a single string. It is a record with the streamed text and a list of **parts**, and each part is either a tool step (chapter 28) or an output. Every part stores `at`, the length of the answer's text at the moment it was created, and the panel renders the answer by interleaving text and parts in that order. An output appears where Claude was in its prose when the step that made it ran.

The `chat` object creates output parts. Each method adds a part and returns a short string to the agent's code:

```
// pseudocode of makeChat(msg)
push(kind, data) = msg.parts.push({ type: 'out', kind, data, at:
msg.text.length }); repaint soon
                   return 'shown: ' + kind
```

The eleven kinds, and what each stores:

Method	Stores	Rendered as
`chat.md(text)`	The text	Markdown
`chat.note(text, tone)`	Text and tone (`info`, `ok`, `warn`, `bad`)	A coloured note
`chat.code(text, lang)`	The text	A code block
`chat.json(value, { title })`	The value through `plain`	Pretty-printed JSON (not in the API reference)
`chat.table(rows, { title, columns })`	A compiled gui app with one `table`	A live gui runtime
`chat.chart(kind, data, opts)`	A compiled gui app with one `chart`	A live gui runtime
`chat.hits(resultOrHits, { title, k })`	A gui `hits` list of the first `k` (default 8)	A live gui runtime
`chat.evals(report)`	A gui app with summary stats and the case table	A live gui runtime
`chat.ui(app)`	The compiled app	A live gui runtime

Method	Stores	Rendered as
`chat.dag({ highlight })`	A copy of the current IR and the ids to highlight	The DAG view, compact
`chat.download(name, data)`	The data as a string	A file line with a Save button

The central design choice is visible in the middle rows: tables, charts, hit lists and eval reports are not special widgets of the agent panel. They are gui apps, compiled with the grammar of chapter 24 and mounted with the same runtime as the search page. `chat.table(rows)` is shorthand for `gui.app('table').view(v => v.table(rows)).compile()`. The panel needs one renderer for all of them, and anything the agent can show in the conversation it could also mount in the App tab.

A real step that uses most of the kinds:

```
chat.table(wb.lastEval.table().slice(0, 4), { title: 'First four goldens' });
chat.chart('bar', wb.lastEval.cases.slice(0, 6).map(c => [String(c.i), c.rank
|| 0]), { title: 'Rank per case' });
chat.dag({ highlight: ['fuse'] });
const app = gui.app('k').title('k slider').use(ws).state({ k: 8 })
  .view(v => v.slider('k', { min: 1, max: 20, step: 1, label: 'k'
}).results());
chat.ui(app);
chat.note('Ranks above 5 count as misses for hit@5.', 'warn');
chat.download('summary.json', wb.lastEval.summary);
```

The answer's parts after this step, as the panel recorded them, and the elements it rendered:

```
tool
table · First four goldens      → out out-table
chart · Rank per case           → out out-chart
dag                             → out out-dag
ui · k slider                   → out out-ui
note                            → out out-note g-note warn
download                        → out out-download
```

The app in the fourth output is live, not a picture. It declared `.use(ws)`, which binds it to the workspace's plan (chapter 25.3), so moving its slider sets the signal `k`, the session re-runs the nodes that read `k`, and its `results` list redraws. The person can tune the

pipeline from inside the conversation. Because the panel repaints the whole answer as text streams in, each mounted runtime is cached under a key made of the message, the part and the app's digest; a repaint reattaches the same runtime, so a slider keeps its position while Claude keeps writing.

What Claude sees of all this is deliberately little. The code receives `shown: table`, and on the next message the flattened history (28.5) carries one line per output:

```
[shown: hits "Results for “Why did ChatGPT show no actions after OAuth worked?”"]
```

The rows, the chart's values and the app's state are on the person's screen, not in Claude's context. This is the division of labour chapter 28.3 argued for: a step returns a few numbers to Claude and shows the full evidence to the person. If Claude needs a value to reason about, the step must also `return` it.

30.2 `ref` links: pointing at a query, a chunk, a node or a tab

Prose can point at things too. The `ref` object builds links in a `wb:` scheme, which the panel's markdown renderer turns into in-page actions instead of navigation. A real step returned four of them:

```
ok
- wb:chunk?id=project-report-coinvault-mcp-from-local-conformance-to-chatgpt-2%2311-why-chatgpt-still-showed-no-actions
- wb:node?id=fuse
- wb:query?q=rebalance%20law
- wb:tab?name=evals
```

Claude writes them into its text as ordinary markdown links, as the prompt asks: "In your text, link what you talk about with ref: the section, node, query." The markdown renderer recognises the scheme and emits `<a href="#" class="ref" data-ref="wb:…">` instead of a real link; the panel catches clicks on `a.ref` and hands the address to one handler:

Link	Action on click
`wb:query?q=…`	Puts the query in the search box and runs it on `change`; with `&submit=1`, runs it on `submit`, which spends Claude usage

Link	Action on click
`wb:chunk?id=…`	Opens the section in the search page's section viewer
`wb:node?id=…`	Selects the node in the DAG tab
`wb:tab?name=…`	Opens an inspector tab: dag, trace, evals, history, corpus or ir

A link whose target no longer exists, such as a chunk removed by rechunking, shows the handler's error as a toast rather than failing silently. Links make an answer checkable in one click: "the expected section ranked ninth" becomes a link that opens that section.

30.3 `ui` : running a query, opening a tab or a section for the person

`ref` gives the person something to click. `ui` does the click for them, during the step:

Call	Effect	Returns
`await ui.query(q, { submit, filters })`	Sets the search box and runs it	`{ q, trigger, ms, top, answer, errors }`
`ui.filters(obj)`	Sets the filter signal	`'filters set'`
`ui.signals(obj)`	Sets any signals	`'signals set'`
`ui.tab(name)`	Opens an inspector tab	`'showing ' + name`
`ui.select(id)`	Selects a node in the DAG	`'selected ' + id`
`ui.chunk(id)`	Opens a section	`'opened ' + id`
`await ui.app(app)`	Mounts an app in the App tab	`mounted '…' in the App tab`
`ui.toast(msg)`	Shows a toast	`'ok'`

`ui.query` is both an action and a measurement. It changes what the person sees and returns a summary Claude can reason about. A real step ran the running question and returned its top five, with each item's `id` , `score` , `doc` and `heading` reduced to a line:

```
ok
- "1. 11. Why ChatGPT still showed no actions · 0.02108"
- "2. 15. Failure chronology and diagnostic lesson · 0.02035"
- "3. 1. The acceptance problem · 0.01999"
- "4. 6. OAuth discovery and registration incompatibilities › 6.1 RFC 9728
path-specific metadata · 0.01988"
- "5. 1. The acceptance problem › 1.1 Exact deployed identity · 0.01862"
```

The scores are reciprocal-rank-fusion scores (chapter 14), which is why they are small and close together. With `{ submit: true }` the query also runs the Claude nodes, the reranker and the answer, and the returned `answer` is clipped to 800 characters; the prompt's cost rule tells the agent not to do this unless asked.

On a narrow screen the page shows one panel at a time, and `ui` calls switch to the panel they act on. On a wide screen they leave the layout alone, so the agent cannot move the person's view away from the conversation they are reading.

The `ui` calls validate their arguments and fail with errors that list the valid choices, in the style of chapter 28.4: `ui.tab('graph')` throws `tabs: dag trace evals history corpus ir`, and `ui.select('nope')` throws `no node 'nope'`.

30.4 Conversations saved per user, and what is clipped when they are stored

When an answer finishes, the panel saves the whole conversation to the per-user database described in chapter 31. Each conversation is one document, and an index of conversations lives in the workbench's root document:

```
data/users/<uid>/ragwb                          root: { convIndex: [{ id,
title, updated, n }], agent: { tier, mode }, preset, script, … }
data/users/<uid>/ragwb/transcripts/<id>         { id, title, created, updated,
messages: […] }
```

The stored shapes, from the real session:

```
convIndex[0]   { id: 'cmvblfn10nf4b', title: 'Run the goldens, then show me
the top results for the OAuth…', n: 16, updated: … }
user message   { role, text, at }
agent message  { role, text, parts, pending, at, tier, mode, usage }
tool part      { type: 'tool', code, status, at, t0, ms, result, logs, open }
output part    { type: 'out', kind: 'table', data: { ir: { kind: 'gui', …,
root: { … rows … }, digest }, title, n }, at }
```

The title is the first message, clipped to 60 characters. The index keeps the 60 most recent conversations, and the panel's **History** button lists them by date with their message counts. The agent's model tier and run mode are saved in the root document when the person changes them, so a returning person gets the same mode.

Two kinds of clipping keep the stored record small. At save time every step's `result` is clipped to 2,500 characters and its `logs` to 1,000; the step in 28.3 that returned 9,040 characters was stored with 2,500. Then, if the conversation as JSON still exceeds 230,000 characters, any output whose compiled app exceeds 20,000 characters is replaced by a one-line note, `(too large to keep)`, and if that is not enough the oldest message pairs are dropped. Output parts store compiled gui apps, not screenshots, so a reopened conversation re-mounts its tables, charts and apps as working gui runtimes.

What a reopened conversation lacks is its snapshots. They live in memory with the step objects (29.4), so a conversation loaded from History shows its steps without revert links.

30.5 A worked session: diagnose a miss, branch, fix, score, merge

The pieces of Parts VI, VII and IX meet in one task, the second suggestion chip in the agent panel: "Fix the eval-tool miss on a branch, compare it with main, and keep it if it helps." What follows is that task as five steps on the Reports preset, written by the author and run through the scripted stand-in. Every result is real.

Step 1: find the miss. The golden questions run on `change`, which is lexical and free.

```
const r = await wb.runEvals({ trigger: 'change' });
return { summary: r.summary, misses: r.misses };
```

```
ok
summary:
  n: 12
  hit@5: 0.917
  mrr: 0.926
  ndcg@10: 0.79
  p50ms: 1.3
  calls: 0
misses:
  - q: How does the eval tool report line numbers in errors?
    rank: 9
    got: "Agent runtime spec — a portable chat agent with a single eval()
tool › 21. Generative UI: gui › 21.6 Canvas"
    want:
      doc: Agent runtime spec
      section: /eval tool|execution/i
```

Eleven of twelve questions find a relevant section in the top five. The miss found the right document but the wrong section: the top hit is the spec's section on canvas drawing.

Step 2: see where the expected sections rank. The step runs the missed question with a deep `k` and finds every section the case would accept.

```
const c = wb.lastEval.cases.find(c => !c.m['hit@5']);
const res = await ws.run({ q: c.q, k: 40 }, { trigger: 'change' });
const want = wb.chunks(x => /Agent runtime spec/i.test(x.title) && /eval
tool|execution/i.test(x.headingPath.join(' '))).map(x => x.id);
return { q: c.q, top3: res.top(3).map(t => t.doc.slice(0, 40) + ' › ' +
t.heading),
         expected: want.map(id => ({ id: id.split('#')[1], rank:
res.hits.findIndex(h => h.id === id) + 1 || null })) };
```

```
expected:
  - id: "5-1-definition"                       rank: 16
  - id: "5-2-execution"                        rank: 11
  - id: "5-2-execution-2"                      rank: 9
  - id: "5-3-what-good-eval-code-looks-like"   rank: 22
```

All four accepted sections sit under the heading "5. The eval tool", and the best of them ranks ninth. The spec's title, "Agent runtime spec: a portable chat agent with a single `eval()` tool", contains both query words, so every section of the spec matches them through its title, the canvas section included. In the BM25F of chapter 5.5, field weights

scale term frequencies before saturation; with the title weighted 2, the title's matches account for much of each section's score for "eval" and "tool", and a section's own heading adds comparatively little. That suggests a fix: weight the title less.

Step 3: try candidates on branches. One step tries five changes, each on its own branch, and scores each with the goldens. Branches share the engine's build memo (chapter 11), so each rebuilds only `ix-text` when its op touches the index.

```
candidate     hit@5   mrr     ndcg@10  rank of the missed case
heading 5     0.917   0.929   0.797    7
heading 8     0.917   0.929   0.797    7
title 1       1       0.933   0.803    5
meta 0        0.917   0.926   0.805    9
text k 60     0.917   0.926   0.79     9
```

Raising the heading weight helps a little; halving the title weight from 2 to 1 moves the missed case to rank 5 and loses no other case.

Step 4: compare the chosen branch with main and show it.

```
const b = ws.branch('title-1');
await b.apply([{ op: 'set', node: 'ix-text', path: 'fields.title', value: 1
}], { by: 'agent' });
const built = await b.rebuild();
const d = await wb.evals.compare([ws, b], { trigger: 'change' });
chat.table(d.rows, { title: 'main vs title-1' });
return { rebuilt: built.computed, delta: d.delta, lineNumbersRank:
d.cases.find(c => /line numbers/.test(c.q)) };
```

```
ok
rebuilt: [ix-text]
delta:
  hit@5: 0.083
  mrr: 0.007
  ndcg@10: 0.013
  p50ms: -0.1
  calls: 0
lineNumbersRank:
  q: How does the eval tool report line numbers in errors?
  main: 9
  title-1: 5
```

The person sees the full comparison table in the conversation; Claude sees the deltas and the one case that moved. Note `{ by: 'agent' }` on the branch's op, which makes the History entry honest (29.1).

Step 5: merge and confirm.

```
const rec = await ws.merge('title-1');
const built = await ws.rebuild();
const r = await wb.runEvals({ trigger: 'change' });
return { rec, rebuilt: built.computed, summary: r.summary, history:
ws.history.map(h => `${h.revision} ${h.by} ${h.note || ''}`) };
```

```
ok
rec:
  revision: 1
  changed: [ix-text]
  merged: title-1
  ops: 1
  …
rebuilt: []
summary:
  n: 12
  hit@5: 1
  mrr: 0.933
  ndcg@10: 0.803
  calls: 0
history: ["1 merge merge title-1"]
```

The merge replayed the branch's one op on main (chapter 23.2), and the rebuild computed nothing: the branch had already built `ix-text` with the same digest, and main found it in the shared memo. The History tab shows one `merge` entry, and the step's snapshot means the person can revert the whole merge in one click.

An honest reading of the result belongs in the agent's answer, and in this chapter. The fix moved one case from rank 9 to rank 5, exactly the edge of hit@5, and the gains in MRR and nDCG are small. Chapter 19.6 showed how far a single case can move these numbers on twelve questions. The change is defensible, because it has a mechanism and costs no other case, but it is not proof that title weight 1 is better for this corpus in general. An agent that reports "hit@5 is now 1.0" without that sentence has reported the number and not the evidence.

The key points to take from this chapter:

- An agent answer is text plus parts, and each output part is placed at the point in the text where the step that made it ran.
- Tables, charts, hit lists, eval reports and apps in the conversation are compiled gui apps on the same runtime as the search page, so an app bound to the workspace is live inside the conversation.
- Claude sees only that an output was shown, never its contents; values Claude must reason about have to be returned as well as shown.
- `ref` builds `wb:` links that run a query, open a section, select a node or open a tab when clicked, and `ui` performs the same actions during a step.
- Conversations are saved per user with step results clipped to 2,500 characters and outputs stored as compiled apps; snapshots are not saved.
- A complete improvement cycle is five steps: find the miss, locate the expected sections, try candidates on branches, compare and show, merge and confirm.

The agent, like every other part of the workbench, works inside limits it did not choose: one sampler, a small database, a prompt cap of 256 KiB, and a page with no server of its own. Part X describes that platform and how each of its limits shaped the design.

Lab: ask the agent for a search page. Load Reports · hybrid + rerank + answer and ask the agent: "Build me a small app with sliders for the fuse weights that reruns the goldens live." When the app appears in the conversation, move a slider and press its button, and check that the History tab gains an entry. Then ask the agent to mount the same app in the App tab with `ui.app`. Finally, ask it to explain the top result for your current query with links, and click each `wb:` link in its answer to see where it leads. These steps spend Claude usage.

Part X · The platform and the page

The workbench runs as a hosted page whose only services are a Claude sampler, a small per-user database and file downloads. This part describes those limits, which shaped Parts II to IX, and the page built inside them.

31. The artifact runtime

This chapter describes the platform the workbench runs on, and connects each of its limits to a decision made earlier in the book. The workbench is one HTML page published as a claude.ai artifact. It has no server of its own. Everything it does beyond running JavaScript in a browser tab comes from four services the page asks for by name: a Claude sampler, a small database, the identity of the viewer, and a way to hand the viewer a file. Each service has a size limit or an absence case, and most of the design in Parts II to IX is a response to one of them. By the end of the chapter you should be able to say, for any feature of the workbench, which service it depends on and what happens when that service is missing.

Output in this chapter comes from three places, each labelled where it appears: node scripts that run the real `rag.js` over the eleven bundled reports, a headless Chromium session that drives the real page with a stand-in for the claude.ai runtime, and the runtime's own type definitions (contract version 0.2.84). The stand-in returns fixed text in place of Claude, so no Claude output in this chapter is real; sizes, counts and database paths are.

31.1 `claude.use`: sample, db, user, downloads

A published artifact reaches the platform through one global function, `window.claude.use(name)`. It returns a promise that resolves to the capability's namespace, or to `null` when this view cannot use it. The contract makes the reasons for `null` indistinguishable on purpose: the capability may not be declared, the viewer may not have granted it, it may have failed to load, or no viewer may be listening at all, in which case `null` arrives after ten seconds. A page cannot ask why; it can only branch on presence.

The workbench asks for all four capabilities at once and stores whatever comes back:

```
async function initCaps() {
  const c = window.claude;
  if (!c || typeof c.use !== 'function') return;
  const get = n => c.use(n).catch(() => null);
  const [sample, db, user, downloads] = await Promise.all([get('sample'),
get('db'), get('user'), get('downloads')]);
  W.caps = { sample, db, user, downloads };
  if (sample && sample.limits) { try { W.caps.limits = await sample.limits();
} catch (_) { W.caps.limits = null; } }
  if (db && user) { try { W.caps.uid = await user.id(); } catch (_) {
W.caps.uid = null; } }
}
```

The important line is not in this function but in the one that calls it. `boot()` starts `initCaps()` and does not wait for it. It loads the bundled corpus, compiles the first preset, builds it and mounts the search page, and only then, in a `.then` on the capabilities promise, configures the sampler, the cache and the database. The first search is therefore always lexical and always local. This ordering is what lets the page work at all when the runtime is absent. Opening the same files from a plain web server, where `window.claude` does not exist, the headless session recorded:

```
status   Built · 11 docs · 490 chunks · 187 ms
console  Claude is not available in this view: LLM nodes are skipped with an
error
         and the agent cannot run. Lexical search, evals and editing work.
```

The table lists what each capability is used for and what the page does without it.

Capability	What the workbench uses it for	Without it
`sample`	the components that call Claude (`rewrite.llm`, `rerank.llm`, `answer`, `agentic`, `enrich.llm`), eval judges, synthetic cases, and the agent	LLM nodes fail with *Claude is not available in this view, so LLM nodes cannot run*; Build refuses deferred steps; the agent cannot send. Lexical search, evals and editing work.
`db`	the user's settings, script and preset, disabled files, the conversation index, uploaded files, agent transcripts, and the per-item cache of Claude results	Nothing outlives the page load. Uploads work until the tab closes.

Capability	What the workbench uses it for	Without it
`user`	`user.id()`, the opaque id that names the user's private folder in the database	The same as no `db`: the store needs both.
`downloads`	Save JSON in the IR tab, Export search page, and `chat.download` from the agent	A toast: *Saving files is not available in this view*.

Two things are absent from this list. The page keeps no shared state: every database path it writes is under the viewer's private folder, so two people using the same published workbench never see each other's files or conversations. And apart from one Google Fonts stylesheet, the page uses no network beyond the files the artifact host serves (the scripts, `corpus.json`, the presets) and the four services. There is no call to an external API anywhere in the code, which is why the rest of this chapter can account for every byte the page sends to Claude.

31.2 Sampling: the prompt cap, tools, `.json`, and `limits()`

The sampler is one function with three variants of use. `sample(input, opts)` sends text and resolves to `{ text, truncated }`. `sample.json(input, opts)` sends text and resolves to a parsed JSON value. Passing `opts.tools` lets Claude call functions in the page between rounds of its answer. Every call is memoryless: the input must carry the instructions, the data and the expected format, because Claude keeps nothing between calls.

The engine never hands the raw sampler to a component. It wraps it so that every call increments a counter, which is how traces and build reports can say how many calls a step spent:

```
sampler(counter, signal) {
  const base = this.env.sample;
  if (!base) {
    const no = () => Promise.reject({ code: 'sampling_unavailable', message:
'Claude is not available in this view, so LLM nodes cannot run' });
    no.json = no;
    return no;
  }
  const f = (input, opts = {}) => { counter.calls++; return base(input, { ...
(signal ? { signal } : {}), ...opts }); };
  f.json = (input, opts = {}) => { counter.calls++; return base.json(input, {
...(signal ? { signal } : {}), ...opts }); };
  return f;
}
```

The wrapper also threads the run's `AbortSignal` into every call, which is how typing a new character cancels an answer that is still streaming (chapter 12). Components choose a model tier per call: the cheap structured steps (`rewrite.llm`, `rerank.llm`, `enrich.llm`, synthetic questions) default to `quick`, and the steps whose text the reader sees (`answer`, `agentic`) default to `default`. `answer` and `agentic` also pass `onText`, the streaming callback, which is how an answer appears word by word in the page.

Tools are not available in every view. `sample.limits()` reports what this view can do, and the workbench reads exactly one field of it:

```
const limits = this.host.limits || {};
if (limits.tools) opts.tools = [this.evalTool(msg)]; else opts.cache = false;
```

Without `tools`, the agent can still talk but cannot act, and the page says so in the console. In the headless session the stand-in reported `{ maxPromptBytes: 262144, tools: { maxCount: 16 } }`; the first number is fixed by the contract, the second was chosen for the test.

The number that shapes the most design is `maxPromptBytes`. The contract caps every sampler input at 256 KiB of UTF-8 text, 262,144 bytes, and rejects anything larger with `prompt_too_large`. The bundled corpus is 535,461 bytes, 2.04 times the cap. The largest single report is 106,755 bytes and fits; all eleven together do not. This is the measurement behind chapter 1's claim that something must choose what Claude reads.

The workbench never reads `maxPromptBytes`. Instead, every prompt it sends has a bound built into the component that sends it. The table lists each caller, what bounds its input, and the size measured for the running question where a measurement is meaningful.

Caller	What bounds the prompt	Measured
`rewrite.llm`	the question plus a fixed instruction	304 bytes
`rerank.llm`	the top 20 hits, each clipped to 700 characters	14,964 bytes
`answer`	the `pack` budget: 40,000 characters of section text by default	37,213 bytes of context at the default budget: 26 sections from 6 reports
`enrich.llm`	one section, clipped to 6,000 characters	at most about 6.3 KB per call
`agentic`	8 search results of 220 characters; each read clipped to 6,000 characters; at most 8 reads	grows with each round
the agent	history trimmed until the turns serialize to 230,000 characters; each eval result truncated near 9,000	grows with the conversation

Two details in this table are worth reading closely. First, the `pack` node counts characters of section text, not bytes, and not the labels and heading paths it adds. For the running question its own counter said 32,846 while the context it produced was 36,917 characters and 37,213 UTF-8 bytes. At the default budget the gap does not matter, because the whole prompt is about 15 percent of the cap. A pack budget raised past 230,000 characters could produce a prompt the sampler refuses. Second, the agent and the agentic node are the only callers whose input grows during a call. For those, `prompt_too_large` is a real possibility, and the agent maps it to a message rather than a crash: *The conversation is too long. Start a new one.*

The agent's other error messages come from the same table, keyed by the contract's error codes: `not_granted` asks the viewer to allow Claude for the page, `rate_limited` says to try later, `tools_unavailable` explains that the agent can talk but not act. Inside the

pipeline, a sampler error becomes a node error in the trace (chapter 13), with the rest of the run continuing where it can.

31.3 Why there are no embeddings, and what lexical retrieval plus Claude covers instead

Most retrieval systems built since 2020 rank by vector similarity: each section is turned into an embedding, a list of a few hundred numbers produced by a neural model, and a query is answered by finding the sections whose vectors are nearest to the query's vector. Embeddings find sections that share meaning with a query but not words, which is BM25's main weakness.

The runtime offers no embedding call. The sampler returns text, and text is the only thing Claude produces through it. Asking Claude to emit vectors as JSON would be slow, would spend one call per section, and would not produce vectors from a consistent model that could be compared with each other. Running an embedding model inside the browser was left out of scope for this book. The retrieval in Parts II and V is therefore lexical at its core, and the semantic gap is closed by using Claude where it is good: on text, at query time or at build time.

Embedding retrieval would supply	What the workbench does instead	Where
matching synonyms and paraphrases	synonym tables applied at search time; `rewrite.llm` writes keyword queries in the corpus's vocabulary	chapters 6, 16
matching a question to an answer that uses different words	HyDE: search with an answer Claude drafts first	chapter 16
ranking by meaning rather than term overlap	`rerank.llm` reorders the top 20 lexical hits with stated reasons	chapter 16
a document-level semantic summary	`enrich.llm` writes a summary, questions and keywords per document at build time, and a second index searches them	chapters 11, 17
following a line of inquiry across documents	the `agentic` node searches, reads and searches again	chapter 17

Each substitute costs Claude calls where an embedding index would have cost one build. The workbench makes those costs visible and optional: build-time enrichment waits for an explicit Build (chapter 11), and query-time calls wait for Enter (chapter 12). The lexical path that runs on every keystroke costs nothing and needs no service at all.

31.4 Persistence: one document per user, with transcripts, files and cache collections

The `db` capability is a document store. Paths alternate between collections and documents, so a path with an even number of segments names a document. The contract reserves `data/users/<id>/` as private to that user, where `<id>` is the value of `user.id()`. The workbench keeps everything under one document there:

```
this.root = W.caps.db.doc(`data/users/${W.caps.uid}/ragwb`);
```

Below that document are three collections. The headless session recorded every document the page wrote during one sitting: an upload, a file switched off, the Doc cards preset built with Claude calls, an edit to the script, and one agent turn. The stand-in database holds what the real one would; the paths and sizes are the page's own.

```
data/users/u1/ragwb                                      2383 bytes  disabled,
preset, script, convIndex, updated
data/users/u1/ragwb/files/note-md-8e6076                  309 bytes  name, text,
added
data/users/u1/ragwb/cache/7f1b8901a5294e:9b5611bed81a20 1921 bytes  v, at
  … 10 more cache documents, 327 to 2089 bytes …
data/users/u1/ragwb/transcripts/cmvblhnkbo407            8808 bytes  id, title,
created, updated, messages
```

Document	Holds	Written when
`ragwb`	the current preset id, the script if it differs from the preset, disabled files, agent settings, an index of the last 60 conversations	a successful build, a file toggle, a settings change, a saved conversation
`files/<slug>-<hash>`	one uploaded file: its name and full text	on upload; deleted on Remove

Document	Holds	Written when
`cache/<key>`	one Claude result for one item of a map component, under the item cache key of chapter 11	each time an `enrich.llm` item is computed
`transcripts/<id>`	one agent conversation	after each agent turn

The root document stores the script only when it differs from the preset it came from. Before the edit in this sitting, the document held `"script": ""`; after a one-line edit to the Doc cards preset it held 2,060 characters. A user who never edits a preset therefore stores a preset id and nothing else.

Every size limit in the workbench's storage code comes from one rule in the contract: a document body is at most 256 KiB serialized. The page applies it in three places. An upload over 250,000 characters is refused before it is stored (*split it first*). A conversation whose serialized form passes 230,000 characters has large generated apps replaced by a placeholder and then loses its oldest turns, two at a time. Eval results and logs inside a stored conversation are clipped to 2,500 and 1,000 characters. The margins are in characters, not bytes. A 250,000-character file written mostly in a script that needs two or three bytes per character serializes past 256 KiB; the database refuses that write, the page logs *could not store*, and the file stays available until the tab closes.

Writes to the same document are serialized by a small per-path promise chain, `store.write(ref, fn)`, because the contract allows one write at a time per document and resolves concurrent writes as last-writer-wins. Reading back happens once, at boot. The same headless session reloaded the page after the writes above and recorded:

```
status       Built · 11 docs · 453 chunks · 127 ms
console      restored preset “Doc cards” with your edits
files        11 of 12 on · 497 KB
build pass   enrich  enrich_llm 0 computed · 11 cached · 0 calls
agent        1 conversation in history: “Run the goldens and try a heading
boost on a branch”, 2 messages
```

The last two lines are the point of the cache collection. Eleven Claude calls were spent once, before the reload; after it, the same eleven document cards came back from the database without a call, because each cache key names the component, its params and the item, not the session.

31.5 Uploading reports: what a new file does to digests, caches and evals

An upload goes through the Files tab of the editor. The page accepts `.md`, `.markdown` and `.txt`, renames the last two to `.md`, stores each file under `files/`, and rebuilds. Nothing else is special about an uploaded file: the source node reads bundled and uploaded files through one function, `activeFiles()`, which also drops any file the user has switched off.

Chapter 11 explained what a change to the source node does. Here is the same mechanism run against an upload, in node, on the Reports preset, adding one short note written for this chapter:

```
after adding one file:
   files       967fb737 -> c2a46e3a CHANGED
   parse       d8c4fd64 -> dfe4f98e CHANGED
   chunks      ebe4a56e -> b208d2af CHANGED
   failures    97261a69 -> 5935aad9 CHANGED
   synonyms    9c379cd0 -> 537c8414 CHANGED
   ix-text     ed7fac9a -> d6fe9dce CHANGED
   ix-meta     5fb5ef38 -> bf8fbbf1 CHANGED
   ix-failures a803a862 -> 0e8a9dfa CHANGED

files        source.files   count   12 computed 0 cached  0
parse        parse.markdown count   12 computed 1 cached 11
chunks       chunk.sections count  464 computed 1 cached 11
failures     chunk          count   28 computed 1 cached 11
ix-text      index.bm25     count  464                    68.6 ms
```

Every build digest changes, because every build node is downstream of the files. The work done is much smaller than that suggests. The map components, parse and the two chunkers, recompute only the new item and take the other eleven from the per-item cache. The indexes are built whole, because an index is a function of all its chunks at once, and the BM25 index is where the time goes: 68.6 ms of a 143 ms rebuild. Removing the file again restored the old digests, and every node was reused.

For Claude-backed map components the per-item cache also lives in the database, so an upload costs one call per new item and nothing for the rest. On the Doc cards preset: the first Build spent 11 calls; after the upload, Build spent 1 call, with 11 items cached; a new engine with the same database, which is what a reload is, spent 0 calls and read all 12 cards from the cache.

Evals are unaffected in their definition and affected in their numbers. A golden case names its expected document by pattern (chapter 18), so a new file cannot break a case, but it can outrank the expected section, and it changes the corpus statistics every BM25 score depends on. For the Reports preset, adding the note left hit@5 at 0.917 and MRR at 0.926 and moved nDCG@10 from 0.790 to 0.792. The plan's IR digest was the same before and after, which is the correct reading: the pipeline did not change, the corpus did. An eval report records the IR digest but not a digest of the files, so two reports with the same IR digest can come from different corpora. When comparing eval runs across uploads, note the corpus readout in the top bar alongside them.

31.6 The same library in node: testing without a page

`rag.js` ends with a two-way export: it assigns `window.rag` in a browser and `module.exports` in node. It touches no DOM, so the whole grammar, compiler, engine, evals and workspace run in node unchanged. `gui.js` also loads in node; it can build and compile app specs, and only `mount` needs a document. `kit.js` attaches to the global `rag`.

A test supplies the two things the runtime would: a list of files and, if the pipeline calls Claude, a sampler with the same shape as the real one.

```
globalThis.rag = require('/home/claude/ragwb/rag.js');
require('/home/claude/ragwb/gui.js');
require('/home/claude/ragwb/kit.js');
const { files } = require('/home/claude/ragwb/corpus.json');
const sample = async (input, opts = {}) => ({ text: 'A drafted paragraph
[E1].', truncated: false });
sample.json = async input => /keyword queries/.test(input) ? { queries:
['stateless discover'] } : {};
sample.limits = async () => ({ tools: { maxCount: 8 } });
rag.configure({ files: () => files, sample, buildCalls: 'explicit' });
```

With that in place, a script can load each preset exactly as the editor does, with `new Function('rag', 'gui', 'log', 'kit', …)`, then compile it, build it, run it on `change` and on `submit`, compile its app and variants, run its evals, and spend any deferred build calls. Running that over all 29 presets took 1.5 seconds and printed one line per preset; four lines are shown here.

```
reports    328ms nodes 21 · lex top 46 (11. Why ChatGPT still showed no
actions) · submit calls 3 answer
           · app row · variants 2 · hit@5 0.917 mrr 0.926 · pending —
minimal    133ms nodes 7 · lex top 20 (5. The core's modules › 5.9 The
rebalance law) · submit calls 0
           · app row · variants 0 · hit@5 1 mrr 0.933 · pending —
doc-cards   38ms nodes 14 · lex top 8 (What was learned) · submit calls 11 ·
app row · variants 0
           · hit@5 0.75 mrr 0.769 · pending enrich_llm:11
           explicit build: computed enrich_llm calls 11; pending after 0
agentic     48ms nodes 10 · lex top 0 (—) · submit calls 1 answer · app row ·
variants 0
           · hit@5 0 mrr 0 · pending —
all presets ok
```

The numbers that come from the stand-in sampler are the call counts and the agentic preset's zero scores, which reflect a stand-in that searched for something unrelated. The lexical results, node counts, eval scores of the lexical presets and the pending counts are real. A node test checks what is deterministic: that every preset compiles, builds, runs and evaluates, and that Claude calls happen where and as often as the design says. It cannot check what Claude would write. Chapter 32 adds the browser checks the node test cannot make.

The key points of this chapter:

- The workbench uses four platform services, `sample`, `db`, `user` and `downloads`, each of which may be absent, and it builds and searches before asking whether any of them are present.
- The prompt cap of 256 KiB is half the size of the bundled corpus, which is why retrieval exists, and every caller bounds its own prompt rather than checking the cap.
- The runtime has no embedding call, so lexical retrieval is the core and Claude closes the semantic gap through rewriting, HyDE, reranking, enrichment and agentic search, each at a visible cost.
- All persistent state lives under the viewer's private folder, in one root document and three collections, and every size limit in the storage code derives from the 256 KiB document cap.
- An upload changes every build digest but recomputes only the new file's items and the indexes; Claude results for unchanged items come from the database cache, including after a reload.

- The library runs unchanged in node, where a stand-in sampler makes every preset testable in seconds.

The limits described here explain what the workbench does. The next chapter describes how it looks: the page that presents those capabilities to a person, and the checks that keep it working.

Lab: Doc cards, then the Files tab. Load Doc cards and press Build; note the number of Claude calls in the status line. Reload the page and press Build again: the status line should report no calls, because the cards come from your database. Now upload a short markdown file of your own in the Files tab, press Build, and confirm that it cost exactly one call. Switch one bundled report off and watch the corpus readout and the chunk count change; switch it back on and check in the DAG tab's build passes that nothing was recomputed. Finally, open the IR tab and press Save JSON to see the `downloads` capability ask before it saves.

32. The page itself

This chapter describes the page a person sees: its visual language, its layout, the preset picker that is the entry point to every chapter's Lab, and the checks that keep all of it working. The visual language is the Lattice design system, and it is built on one rule that this chapter returns to several times: a visual distinction exists only when it carries a meaning the reader needs. There is one accent colour, and it means *signal*. There are two broken line styles, and they mean *build time* and *runtime*. By the end of the chapter you should be able to look at any part of the workbench and say what each colour and line style is telling you, and to say which parts of the page have been checked by machine and which have not.

Measurements in this chapter come from the page's stylesheet, from a headless Chromium session that loaded the page with a stand-in for the claude.ai runtime, and from the node test of chapter 31. As before, no Claude output shown is real.

32.1 Ink on paper, with one signal blue

The stylesheet defines every colour once, as a custom property on `:root`, and redefines the same names for dark mode. The page draws nothing in a colour that is not one of these tokens.

Token	Light	Dark	Used for	Contrast on the sheet, light / dark
`--ink`	#111417	#E3E6E9	text, 1 px rules, borders, primary buttons	18.5 / 13.7
`--muted`	#5D646A	#98A0A7	secondary text, labels, build nodes and edges	6.0 / 6.5
`--faint`	#A9AEB2	#69727A	editor line numbers, console timestamps	2.2 / 3.5
`--sig`	#1F4FD1	#86A2F7	signals, live tags, links, focus, the selected node	6.8 / 7.0
`--ok`	#2A7752	#5DBB8B	a finished build, runtime edges	5.4 / 7.3
`--warn`	#9C5F00	#E0A23A	steps that spend Claude calls, dirty nodes	5.2 / 7.7
`--bad`	#C2361B	#F07158	errors and destructive buttons	5.5 / 5.9
`--paper`, `--sheet`	#FAFAF8, #FFFFFF	#121518, #181C20	the two background levels	—

The contrast ratios were computed from the token values with the WCAG formula against `--sheet`. Every token that carries text a reader must read is above 4.5:1 in both themes. `--faint` is not, and is used only for line numbers and timestamps, which are reference marks rather than content. A stricter reading of the accessibility guideline would count line numbers as text, and by that reading the editor gutter fails in light mode.

Ink on paper names the default: black 1 px rules on a near-white page, no shadows except one hatch pattern, no rounded corners, no gradients. Hierarchy comes from type rather than colour. The stylesheet uses three families with fixed roles: Arial for running text, Archivo for the brand and node names, and IBM Plex Mono for values, buttons, status lines and anything a reader might compare digit by digit. Labels are 9 px bold capitals with wide letter spacing, which is the *stencil* style of the Lattice system.

One signal blue means the accent has exactly one job. `--sig` appears 41 times in the stylesheet, and every use marks something that is live or selected: signal nodes and the edges that read them, the `live` tag, links, the focus outline, the selected node in the

DAG, and the busy status. The three state colours are kept for states. Green says a build finished or data flowed; amber says a step will spend Claude calls or a node is out of date; red says something failed. Each has a light wash (`--ok-w` , `--warn-w` , `--bad-w`) for backgrounds, so a state can be shown without large areas of saturated colour.

Dark mode is chosen by the operating system, through `prefers-color-scheme` , and can be forced either way by setting `data-theme` on the root element, which is how the artifact host switches themes. Both themes define every token, so no component has a dark-mode rule of its own. In the headless session the computed value of `--sig` was `#1F4FD1` with a light colour scheme and `#86A2F7` with a dark one, and the measured layout was identical.

32.2 Dashed for build, dotted for runtime, in the DAG, the editor and the agent panel

Chapter 11 divided every pipeline into a build phase, run once per corpus, and a live phase, run per query. The page shows that division with line style, so that a reader can see which part of a pipeline an edit will rebuild without reading a type name. The DAG renderer assigns each edge a class from the edge′s kind and its source node′s phase:

```
const cls = kind === 'signal' ? 'signal' : kind === 'artifact' ? 'artifact' :
byId.get(a).phase === 'build' ? 'build' : 'data';
paths += `<path class="${cls}${hot}" d="${d}"/>`;
if (kind === 'data' && ta && (ta.status === 'ran' || ta.status === 'cached')
&& ta.out > 0 && byId.get(a).phase === 'live')
  paths += `<path class="run" d="${d}"/>`;
```

and the stylesheet gives each class a stroke:

Class	Stroke	Meaning
`build`	muted, dashed 3 2	data flowing between build nodes
`artifact`	muted, dashed 3 2	a live node using a build output, such as an index
`data`	ink, solid	data flowing between live nodes
`signal`	blue, solid, 0.9 px	a node reading a signal
`run`	green, dotted 1 2.5, 1.6 px, round caps	an edge that carried data in the last run, drawn over its `data` edge

Build nodes get a dashed border to match their edges, and the `build` tag in the node detail panel is dashed as well. Live nodes that run a lambda are filled with a fine blue hatch, so query-time code you wrote is distinguishable from configured components at a glance; build lambdas keep the plain dashed border. Signals are drawn as pills in the blue wash.

The dotted runtime line is the only element in the DAG that changes on every keystroke. After typing the running question into the Reports preset at 1,440 px, the headless session counted the DAG's paths and nodes:

```
paths  build 7 · artifact 6 · data 11 · signal 5 · run 7
nodes  build 6 · build lam 2 · signal 3 · live ran 7 · live skipped 2 · live lam skipped 1
```

Seven live nodes ran and seven of the eleven live edges are dotted: the path the query actually took. The three skipped nodes are the ones that wait for Enter, and they are drawn at half opacity with no dotted edge into them. Pressing Enter dots the rest. A legend under the DAG states the convention in words: *data, build / artifact, signal read, carried data last run, Enter only*.

The convention follows the DAG wherever it is drawn. The agent's `chat.dag()` and the `dag` component of chapter 26 call the same renderer, so a graph in a conversation or inside a generated app reads the same way as the inspector's. The agent's links into the workbench, the `wb:` references of chapter 30, are underlined with a dotted line, marking them as live references into the running page rather than links to the web. One dashed

element does not follow the rule: the file drop zone in the editor's Files tab has a dashed border, the common convention for a drop target. It sits beside the build controls and could be read as one; a solid border in the hatch pattern would remove the ambiguity.

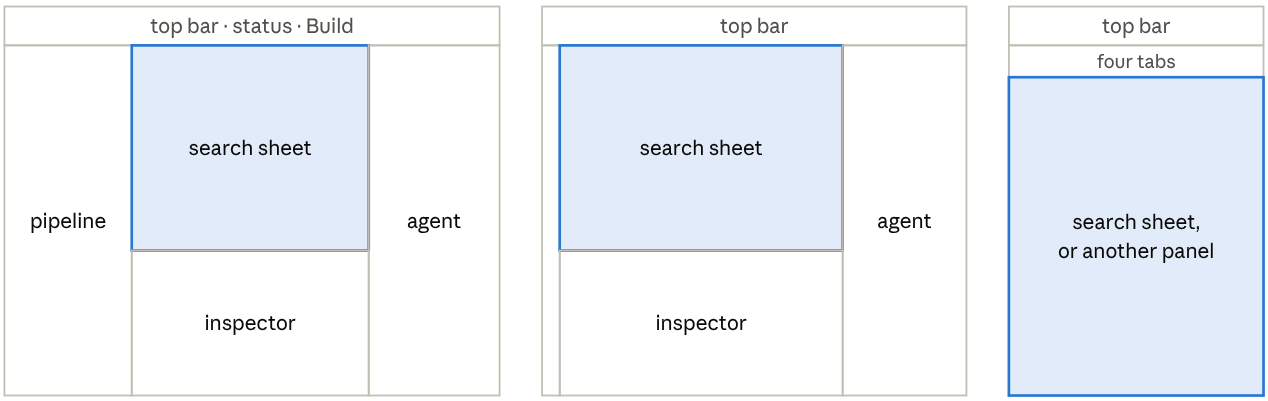
32.3 Layout: editor, search sheet, inspector, agent

The page is one CSS grid. Three rows hold a 46 px top bar, a row of panel tabs that exists only on narrow screens, and the main area. The main area has three columns: the pipeline editor, a centre column, and the agent. The centre column is itself split into the stage, which holds the search sheet, and the inspector below it.

```
.app    { display: grid; grid-template-rows: 46px auto minmax(0, 1fr); }
.main   { display: grid; grid-template-columns: var(--ide-w) minmax(0, 1fr)
var(--side-w); }
.center { display: grid; grid-template-rows: minmax(0, 1fr) var(--insp-h); }
:root   { --ide-w: 360px; --side-w: 370px; --insp-h: 340px; }
```

One page, three arrangements by viewport width

The search sheet is shaded. Measured in headless Chromium at 1440, 1200 and 390 px; none scrolls sideways.



1240 px and wider
three columns: 360 · flexible · 370 px

980 to 1240 px
pipeline starts folded to a rail

below 980 px
one panel at a time, chosen by the tabs

page layout · three width ranges

Three width ranges produce three arrangements. At 1,240 px and wider all three columns are open. Between 980 and 1,240 px the page starts with the pipeline editor folded to a 34 px rail, because the search sheet would otherwise be too narrow to read; at 1,280 px and below both side columns also shrink to 340 px. Below 980 px the page shows one panel at a time and adds four tabs, Search, Pipeline, Inspect and Agent. The headless session measured these sizes:

Viewport	Pipeline	Centre	Agent	Inspector	Page width
1440 × 900	360	676	370	676 × 340	1440
1200 × 800	34 (rail)	792	340	792 × 340	1200
980 × 800	hidden	946	hidden	hidden	980
390 × 844	hidden	356	hidden	hidden (own tab)	390

The last column is the check that matters on a phone: the document is exactly as wide as the viewport at every size, so nothing scrolls sideways. The top bar is the one element allowed to overflow, and it scrolls inside itself. The body's side padding is `max(16px, env(safe-area-inset-left))`, which keeps a 16 px gutter and moves content clear of a phone's rounded corners.

Each boundary between panels can be dragged. The drags are clamped: the editor between 260 and 720 px, the agent between 280 and 720 px, the inspector between 120 px and the centre's height minus 120. Each side panel folds to its rail with a button, so a reader can give the search sheet the whole width.

The search sheet is drawn as a sheet of drafting paper on a grid. The stage behind it has two grids, fine lines every 12 px and heavier lines every 48 px, and the sheet casts a hatched offset shadow instead of a blurred one. A title block in the stage header shows the plan's id, the workspace revision, the chunk count and the last build time, which is the same information a drawing's title block gives: what this is, which revision, and how it was produced.

One layout problem is visible at full width. The inspector's build-pass list is a four-column grid inside a 270 px panel, and a pass that built two outputs wraps one word per line (*chunks | 462 · | failures | 28*). The panel is too narrow for the summary it is asked to hold.

32.4 The preset picker: stashed edits and restoring a session

Every Lab in this book starts from the preset picker at the top of the editor. The button shows the preset's position and group, *Preset 01 of 29 · Start here*, and its title; the arrows beside it step to the previous and next preset. The menu lists the 29 presets in six groups: Start here (3), Corpus & chunking (5), Ranking (6), With Claude (6), Search experiences (7) and Evaluation (2), with a filter box that matches titles, descriptions, groups and ids.

Each preset's entry in `presets/index.json` declares whether it spends Claude calls and when. Ten are marked `enter`, one, Doc cards, is marked `build`, and eighteen are marked as free. The picker turns that field into the last sentence of the description under the

button, *Enter spends Claude calls.* or *Build spends Claude calls.*, so the cost is stated before the reader acts (chapter 27).

A preset is only a script, so editing it is ordinary editing, and the head shows an *edited* marker while the text differs from the preset. Loading another preset over edited text would lose the edit, so the picker keeps it:

```
async function loadPreset(id, opts = {}) {
  const p = findPreset(id);
  const text = await fetchPreset(p);
  const cur = src().value;
  if (!opts.silent && cur.trim() && cur !== W.presetText) W.stash = { text:
cur, label: W.preset ? W.preset.title : 'your script' };
  …
}
async function restoreStash() {
  if (!W.stash) return;
  const st = W.stash;
  W.stash = { text: src().value, label: W.preset ? W.preset.title : 'the
preset' };
  setEditor(st.text);
  …
  await buildScript();
}
```

The stash holds one script. While it is set, the menu's first entry is *↺ Back to your edits of “Parameter sweep”* (the label the headless session recorded after editing the sweep preset and stepping to the next one). Restoring swaps the stash with the current text, so pressing it twice returns to where you were.

The session itself is restored from the database (chapter 31). After each successful build the page saves the preset's id and, if the text differs from the preset, the script. On load, once the database answers, the page reloads the saved preset and applies the saved script on top of it. The headless session showed this after a reload: *restored preset “Doc cards” with your edits*, with the *edited* marker visible. A script written from scratch, with no preset, is restored the same way under the message *restored your pipeline.js*.

The stash is not saved. It lives in a page variable, and the database holds only the current preset and script. In the headless run, after editing the sweep preset and stepping to the next one, the saved state read `{"preset":"reports","script":""}`. A reader who edits a preset, switches to another, and then reloads has lost the edits; they were recoverable only until the tab closed.

32.5 Checking the page: node tests and a headless run of every preset

The workbench has two automatic checks, and they divide the work by what each environment can see.

The node test of chapter 31 checks the library: every preset compiles, builds, runs on `change` and on `submit`, compiles its app and variants, runs its evals, and spends its deferred build calls when asked. It takes 1.5 seconds and checks nothing about the page.

The browser check loads the real page in headless Chromium with a stand-in for the claude.ai runtime: a sampler that returns fixed text, a database held in memory, a fixed user id, and a downloads service that records the filename. It then drives the page the way a reader would. It opens the picker, chooses each of the 29 presets by its title, waits until the built script matches the editor, and records the number of result rows, the status line, any error box, any inline component error and any console error. For six presets it also exercises the preset's own controls. The run recorded, in part:

```
reports            results  12  Built · 11 docs · 490 chunks · 0 ms
minimal            results  12  Built · 11 docs · 462 chunks · 85 ms
table-rows         results  10  Built · 11 docs · 812 chunks · 27 ms
per-doc            results  10  Built · 11 docs · 11 chunks · 19 ms
doc-cards          results   8  enrich_llm: 11 items wait for Claude · press
Build to spend those call
   after Build: Built · 473 chunks · 203 ms · 11 Claude calls
doc-browser        results   0  Built · 11 docs · 462 chunks · 0 ms
   outline rows: 17
playground         results  12  Built · 11 docs · 490 chunks · 15 ms
   after slider: 46 results · lexical · 0.5 ms
sweep              results   6  Built · 11 docs · 462 chunks · 0 ms
   sweep rows: 12
edited marker visible: True
stash entry: 1 ↺ Back to your edits of “Parameter sweep”
mobile scrollWidth 390

ERRORS: []
```

A build time of 0 ms means every build node was reused from the memo, because an earlier preset had built the same corpus. A count of zero results is expected for the presets whose pages show something other than a result list: an outline, a canvas, a dashboard, tabs, or an agentic answer. The playground line moved the third slider, the

failure-row boost, to 3; the status still reports 46 results because a boost reorders hits without removing any. The check does not move the `k` slider, so it could not have caught the bug described in chapter 26.

Check	Node test	Browser run	Neither
every preset compiles, builds and runs	yes	yes	
eval scores of lexical presets	yes		
Claude calls happen where and as often as designed	yes (counted)	yes (Doc cards)	
the page mounts each preset without errors		yes	
preset controls: sweep, slider, outline, Build		yes, six presets	the other 23
no sideways scroll on a phone		yes, at 390 px	
what Claude actually writes; real latency			yes
permission prompts, real database limits, rate limits			yes
visual regressions		screenshots, compared by eye	yes

The last three rows are the honest boundary of this book's evidence. Everything about Claude's behaviour in Parts V, VI and IX was observed through a stand-in, and the database's 256 KiB document limit and quotas were read from the runtime's type definitions rather than triggered. Those are the first things to check on a real deployment, and the workbench makes them checkable: the trace counts every call, the status line reports every build's cost, and the agent's errors name the runtime's codes.

The key points of this chapter:

- Every colour is a token defined once for each theme, and every distinction it draws carries a meaning: blue for signals and selection, green, amber and red for states.

- Dashed lines mark the build phase and dotted lines mark the path the last query took, in every place a DAG is drawn; the file drop zone is the one dashed element that means something else.
- One grid produces three arrangements by width, and at every measured width the page is exactly as wide as the viewport.
- The preset picker states each preset's Claude cost before it is used, stashes one set of edits when switching, and restores the current preset and script after a reload, though not the stash.
- Node tests check the library and a headless run checks the page; neither checks what Claude writes.

The book began with a question about eleven reports and the observation that a prompt cannot hold them all, so something must choose what Claude reads. Every part since has been an answer to the question of how to make that choice visible and changeable. Part III turned the pipeline into data, so it could be drawn and diffed. Part IV made changing it cheap, by naming every result after what it depends on. Parts V and VI made a change judgeable, with traces that explain a ranking and evals that score it. Part VII made a change safe, as an operation that can be checked, undone and replayed. Parts VIII and IX gave two kinds of user, a person with a page and an agent with one tool, the same access to the same graph. This last part showed the limits all of that was built inside. The workbench is small enough to read in an afternoon; the habit it is meant to teach, of never changing a search system without seeing what the change did, scales to systems far larger.

Lab: the page itself. Switch your system to dark mode and back, and find the one colour that changes meaning: none should. Load Reports, open the DAG tab, type a question, and trace the dotted line from the query signal to the results; press Enter and watch the remaining edges dot. Narrow the window below 1,240 px and then below 980 px and confirm that nothing scrolls sideways. Finally, edit any preset, switch to another, use *Back to your edits*, and then reload the page: decide whether the session restored what you expected, and what you would change about the stash.

Appendices

Reference material for the whole book: the API, the presets, the data formats and the terms.

A. API reference

This appendix lists every public name in `rag.js`, `gui.js`, `kit.js` and the agent's scope, generated by enumerating the loaded modules and reading the defaults from the source (`rag.version` is `0.1.0`). Signatures use the library's own argument names. A parameter written `name = v` has default `v`; "λ" marks a parameter that may be a stored lambda. Pipelines are built from specs (chapter 7), compiled to an IR (chapter 10), built and run by an engine (chapters 11–12), changed through a workspace (chapters 21–23) and shown through `gui` (chapters 24–26).

A.1 The `rag` namespace

Name	Signature	Returns	Notes
`rag.corpus`	`(id = 'corpus')`	`CorpusSpec`	Empty build-phase spec. A.2.
`rag.plan`	`(id = 'plan')`	`PlanSpec`	Empty live-phase spec. A.3.
`rag.graph`	`(id, g => …)`	`PlanSpec`	Explicit wiring form. A.4.
`rag.fragment`	`(id, (spec, params) => spec)`	`Fragment`	Reusable spec transformation, applied with `spec.use(fragment, params)`.
`rag.evals`	`(id = 'evals')`	`Evals`	Golden-case suite. A.11.

Name	Signature	Returns	Notes
`rag.evals.generate`	`(target, { n = 12, seed = 7, tier = 'quick', id = 'synthetic' })`	`Promise<Evals>`	Claude writes one question per sampled section (sections over 120 tokens), in batches of 8.
`rag.sweep`	`(target, grid)`	`{ configs, run(evals, opts) }`	Cartesian grid of `set` ops keyed `'node.path'`. A.11.
`rag.bake`	`(ctx => value)`	`{ $bake }`	Evaluated once at build with `{ docs, chunks, tokens, slug }`.
`rag.live`	`(f)`	`{ $fn }`	Stores a function as a lambda explicitly.
`rag.c`	`(type, p = {})`	`{ $c, p }`	Component reference by type name.
`rag.compile`	`(plan or corpus or IR)`	IR	Checks, orders, digests; applies a plan's pending `patch` ops.
`rag.applyOps`	`(ir, ops)`	IR	Applies ops to a copy and recompiles; throws `RagError('op')`.
`rag.ir`	`(json)`	IR	Re-attaches IR methods to stored JSON.
`rag.build`	`(target, opts)`	`Promise<BuildReport>`	Default engine.
`rag.run`	`(target, signals, opts)`	`Promise<Result>`	Default engine, or the workspace if given one.

Name	Signature	Returns	Notes
`rag.search`	`(target, q, { k, where, signals, trigger = 'change' })`	`Promise<top[]>`	`where` becomes the `filters` signal. Returns `result.top(k or 10)`.
`rag.explain`	`(result, chunkId)`	`{ chunk, path, where }`	Rank and score of one chunk at every hits node that ran.
`rag.workspace`	`(target, opts)`	`Workspace`	A.12.
`rag.engine`	`()`	`Engine`	The shared default engine.
`rag.configure`	`(env)`	env	Merges `files`, `sample`, `cache`, `log`, `buildCalls`, `librarySource` into the default engine's environment.
`rag.define`	`(type, def)`	type	Registers a component; family methods appear on every plan. A.6.
`rag.components`	`()`	`[{ type, phase, in, out, trigger }]`	The registry.
`rag.export`	`(target, { title })`	`Promise<string>`	Standalone HTML search page: submit-only nodes and context and answer nodes removed.

Name	Signature	Returns	Notes
`rag.util`	object		`hash stable tokenize stem parseMarkdown parseFrontmatter chunkSections Bm25Index TermsIndex slug clip signalReads fnOf lam diffIr triggerOf describeNode costsCalls`
classes			`Engine Workspace Session Evals RagError`

Component factories are generated from the registry: `rag.source.files`, `rag.parse.markdown`, `rag.chunk.sections`, `rag.index.bm25`, `rag.index.terms`, `rag.enrich.llm`, and so on for every type, plus the short names `rag.files`, `rag.markdown`, `rag.sections`, `rag.bm25` and `rag.terms`. A factory returns a component reference `{ $c, p }` that builder methods accept.

A.2 Corpus builder (`CorpusSpec`)

Every method returns a new frozen spec. The builder keeps a cursor of the latest `files`, `docs` and `chunks` node so the next call can wire itself.

Method	Adds	Default id	Wiring
`.from(src)`	`source.files` (string or `{ glob }`) or `source.inline` (array of `{ name, text }`)	`files`	no inputs
`.parse(parser = rag.markdown())`	`parse.markdown`	`parse`	latest files

Method	Adds	Default id	Wiring
`.chunk(spec)`	`chunk.sections` or another chunker	`chunks`, then `chunks_2` …	latest docs; becomes the chunk cursor
`.chunk(name, (doc, ctx, p) => chunks, opts)`	`chunk` with a build lambda	slug of `name`	latest docs, or `opts.from`; a named stream after the first does not move the cursor
`.enrich(name, (chunk, ctx, p) => value, opts)`	`enrich`	`enrich_<name>`	latest chunks, or `opts.from`; options `concurrency = 3`, `budget: { calls }`
`.enrich(name, rag.enrich.llm(opts), { from })`	`enrich.llm`	`enrich_<name>`	moves the cursor only when it reads the current chunk stream
`.index(name, rag.bm25(p) or rag.terms(p), { from })`	`index.bm25` or `index.terms` with `p.name = name`	`ix-<name>`	latest chunks, or `from`
`.synonyms(table or rag.bake(…))`	`synonyms`	`synonyms`	latest docs
`.then(component, { id, from })`	any build component	type's last segment	the cursor of the component's first input port
`.use(fragment, params)`	whatever the fragment adds		
`.as(id)`	renames the spec		
`.toJSON()`	`{ kind: 'corpus', id, nodes }`		

A.3 Plan builder (`PlanSpec`)

Method	Effect
`.over(corpus)`	Attaches a corpus; its nodes are compiled into the same IR.
`.signals(obj)`	Declares signals and their initial values. Declaring `q` makes it the query cursor.
`.<family>(…)`, `.<family>.<variant>(…)`	Adds a live node of that type. Generated from the registry as prototype getters, so components defined later appear on existing plans.
`.then(component, opts)`	Adds any live component.
`.patch(ops)`	Records ops applied after compilation (chapter 21).
`.variant(id, ops or (plan => plan))`	A renamed copy, patched or transformed.
`.use(fragment, params)`	Applies a fragment.
`.as(id)`	Renames the plan.
`.toJSON()`	`{ kind: 'plan', id, signals, nodes, ops }`

Every live step accepts `{ id, trigger: 'change' or 'submit', from }` in its options. Without `from`, a node reads the cursor of each input port. A retrieve adds a parallel hit stream; any step that takes a single `hits` input while two or more streams are open throws `needs a fuse step first, e.g. .fuse.rrf()`. Default ids come from the component (`retrieve` uses the index name, `fuse.*` uses `fuse`, `filter.where` uses `filter`, `rerank.llm` uses `rerank`, `expand.neighbors` uses `neighbors`, `expand.links` uses `links`), with `_2`, `_3` appended on collision.

A.4 Graph form and fragments

`rag.graph(id, g => { … })` exposes four calls: `g.over(corpus)`, `g.signal(name, value)` returning a handle, `g.node(id, component, { from })` returning a handle, and `g.output(name, handle)`, which pins an output. `from` takes handles or ids, and the node

is wired exactly as stated. A fragment is `rag.fragment(id, (spec, params) => spec)`; `spec.use(fragment, params)` calls it, and it works on corpus and plan specs alike.

A.5 Build components

Build components run once per digest (chapter 11). `map` components run per item with a per-item cache; the rest run once over all inputs.

Type	Ports	Parameters (defaults)	Notes
`source.files`	→ files	`glob = '*'` (case-insensitive, `*` only)	Its digest includes every file's content hash.
`source.inline`	→ files	`files`	
`parse.markdown`	files → docs	`frontmatter` (set `false` to skip)	Map. Blocks: paragraph, list, fence, table, callout, quote.
`chunk.sections`	docs → chunks	`depth = 3`, `maxTokens = 450`, `minTokens = 30`, `prefixHeading`, `overlap` (tokens), `dropMermaid = true`	Map. Fences and tables stay whole up to 3 × `maxTokens`.
`chunk`	docs → chunks	`fn` λ `(doc, ctx, p) => [{ headingPath, text, blocks?, … }]`	Map. Chunk kind is the node id.
`enrich`	chunks → chunks	`name`, `fn` λ `(chunk, ctx, p) => value`, `concurrency = 3`, `budget.calls`	Map, async, per-item cache in memory and in `env.cache`. Costs calls if the lambda mentions `sample`.
`enrich.llm`	chunks → chunks	`name`, `prompt` (summary, questions, keywords), `tier = 'quick'`, `concurrency = 3`, `budget.calls`	One `sample.json` call per chunk. Deferred when `buildCalls` is `'explicit'`.
`index.bm25`	chunks → index	`fields = { title: 2, headingPath: 3, text: 1 }`, `k1 = 1.2`, `b = 0.75`, `stem = true`, `name`	Field values may be dotted paths into the chunk.

Type	Ports	Parameters (defaults)	Notes
`index.terms`	chunks → index	`keys = ['type', 'tags', 'status']`, `name`	Matches whole metadata values that appear in the query; score `1 / ln(2 + n/10)`.
`synonyms`	docs → value	`table` (object or bake)	Applied at search time to every `index.bm25` retrieve, at weight 0.5.
`bake`	docs → value	`value` (bake)	

A.6 Live components

Live components run per query. Trigger is the default; any node can override it. "Skipped" is what the node outputs when the trigger excludes it: `pass` returns its input unchanged, `null` returns nothing.

Type	Ports	Trigger	Reads	Parameters (defaults)	Skipped
`retrieve`	queries → hits	change		`index`, `k = 20`, `perQuery = true`; or `fn` λ `(s, ctx, queries, p) => hits`	input
`rewrite`	queries → queries	change		`fn` λ `(s, ctx, queries, p) => queries`	pass
`rewrite.llm`	queries → queries	submit	`q`	`n = 3`, `tier = 'quick'`	pass
`fuse.rrf`	hits* → hits	change		`k = 60`, `weights = {}` (by input id), `limit = 60`	input
`fuse.max`	hits* → hits	change		none	input
`filter`	hits → hits	change		`fn` λ `(s, ctx, hit, p) => bool`	input
`filter.where`	hits → hits	change	`filters`	`key = 'filters'`	input
`rerank`	hits → hits	change		`fn` λ `(s, ctx, hits, p) => hits`	pass
`rerank.llm`	hits → hits	submit	`q`, `k`	`top = 20`, `keep = s.k or 8`, `tier = 'quick'`	pass
`limit`	hits → hits	change		`k = s.k or 8` (number or λ)	input
`expand.neighbors`	hits → hits	change		`before = 0`, `after = 1`, `top = 5`, `maxChars = 6000`, `decay = 0.5`	input

Type	Ports	Trigger	Reads	Parameters (defaults)	Skipped
`expand.links`	hits → hits	change	`q`	`index`, `top = 5`, `max = 4`, `decay = 0.5`	input
`pack`	hits → context	change		`budget = 40000` (characters), `order = 'rank'` or `'doc'`, `maxPerDoc = 6`	input
`answer`	context → answer	submit	`q`	`instructions`, `tier = 'default'`	null
`agentic`	queries → answer	submit	`q`	`indexes`, `maxReads = 8`, `instructions`, `tier = 'default'`	null
`map`	inType → outType	change		`fn` λ `(s, ctx, input, p)`, `inType = 'hits'`, `outType = 'hits'`	input

A lambda in a non-lambda parameter (for example `limit(s => s.k)`) is evaluated per run with `(s, ctx)` and its signal reads are recorded. The static scan also adds signals read by any lambda to the node's `reads`.

`rag.define(type, def)` registers a component. `type` must look like `family` or `family.variant`. `def` takes `phase` (`'build'` or `'live'`, required), `in` (default `hits` for live, `docs` for build), `out` (default `hits` or `chunks`), `params` (defaults), `trigger`, `id`, and either `run(s, ctx, input, p)` for live or `map(item, ctx, p)` or `all(inputs, ctx, p)` for build. A build `map` that outputs chunks may return plain `{ headingPath, text }` objects; ids are assigned for it.

A.7 Lambda contexts

Lambdas are stored as source and run closed: a name that is neither an argument nor an allowed global throws `references free identifier` on first run (chapter 9). These are the arguments each kind receives.

Kind	Arguments	`ctx` members
Build slot (`chunk`, `enrich`, custom `map`)	`(item, ctx, p)`	`node`, `item`, `sample`, `sample.json`, `tokens`, `slug`, `hash`, `clip`, `log`, `files()`, `docs`, `chunks`
Bake	`(ctx)`	`docs`, `chunks`, `tokens`, `slug`

Kind	Arguments	`ctx` members
Live slot (`filter` , `rerank` , `rewrite` , `retrieve` , `map` , custom `run`)	`(s, ctx, input, p)` (`filter` gets one hit)	`node` , `sample` , `sample.json` , `signal` (abort), `signals` , `synonyms` , `tokens` , `clip` , `index(name)` , `indexNames()` , `firstIndex(kind)` , `chunk(id)` , `chunks(f)` , `doc(id)` , `docs()` , `links(docId)` , `neighbors(id, { before = 1, after = 1 })` , `log` , `note(text)` , `emit(text)`
Live parameter (for example `limit(s => s.k)`)	`(s, ctx)`	as for a live slot
Judge	`(s, { sample }, { q, answer, evidence })`	`sample` only

Hits passed to `filter` and `rerank` lambdas are expanded with `docId` , `title` , `headingPath` , `text` , `meta` , `kind` and `tokens` . A `rerank` lambda may return hit objects or bare ids; ids get score `1 / rank` .

A.8 The IR

`rag.compile` returns a plain object (Appendix C.1) with these non-enumerable methods.

Method	Returns
`ir.node(id)`	The node, or `undefined` .
`ir.downstream(id)` , `ir.upstream(id)`	Node ids reachable along edges, in IR order.
`ir.summary()`	`id:type → …` , with `!` after submit-only live nodes.
`ir.mermaid()`	A Mermaid flowchart; build nodes are parallelograms, signals circles, artifact edges dotted.
`ir.describe(id)`	The component's one-line description of its params.

Method	Returns
`ir.clone()`	A deep copy with methods.

A.9 Engine

Member	Signature	Purpose
`new rag.Engine(env)`	env: `files()`, `sample`, `cache { get, set }`, `log`, `buildCalls` (`'explicit'` defers Claude calls)	
`build`	`(ir, { calls, force, signal, onProgress })`	Builds dirty build nodes; returns a build report (C.7). `calls` overrides `buildCalls`.
`digests`	`(ir)`	`Map(nodeId → digest)` for build nodes.
`dirty`	`(ir)`	Build node ids with no memo entry for their digest.
`pending`	`(ir)`	`[{ node, items }]` for build nodes still waiting for Claude calls.
`artifacts`	`(ir)`	`{ chunks, docs, order, indexes, synonyms, links, byId }`, or `null` if anything is unbuilt.
`session`	`(ir or () => ir)`	A `Session` with its own memo.
`run`	`(ir, signals, opts)`	One-off session run.
`on`	`(fn)`	Events `node-built` and `built`; returns an unsubscribe function.
`memo`, `partial`, `items`, `bakes`	Maps	Build memo by digest, partial (deferred) outputs, per-item cache, bake values.

A.10 Session and run result

`session.run(signals, { trigger = 'submit', signal, onText, onNode, onProgress })` builds first if needed, then runs every live node in IR order. The result has these fields.

Field or method	Content
`runId`, `trigger`, `signals`	Run number, trigger, and the merged signals used.
`hits`, `context`, `answer`	The values of the IR's `outputs.hits`, `outputs.context`, `outputs.answer`.
`outputs`	Every node's output by id.
`trace`	One entry per live node (C.3).
`errors`, `ms`, `calls`	Error strings `node: message`, total time, Claude calls.
`top(n = 5, fromNodeId)`	`[{ id, score, doc, heading, reason? }]`.
`chunk(id)`	The chunk object (C.6).
`explain(id)`	As `rag.explain`. Nodes whose status was `cached` carry no ranks and are absent from the path.
`toJSON()`	`{ trigger, q, ms, calls, top, answer, errors, trace }`, with trace lines as strings.

A.11 Evals and sweeps

Member	Signature	Notes
`.case`	`(q, { doc, section, any, answerHas }, { signals, synthetic })`	Patterns: string (case-insensitive substring), RegExp, `{ $re, flags }`, or an array of these. `doc` is matched against title, file name, doc id and aliases; `section` against the heading path joined with `›`.

Member	Signature	Notes
`.metrics`	`(['hit@5', 'mrr', 'ndcg@10'])`	Also `recall@k`. The first metric defines `misses`.
`.judge`	`(name, (s, ctx, { q, answer, evidence }) => number)`	Runs only with `run(…, { judge: true })` and an answer.
`.filter`, `.concat`, `.as`, `.toJSON`		
`.run`	`(target, { trigger = 'change', k, signals, judge, signal, onCase })`	Report (C.5). Retrieval depth is at least 10.
`.compare`	`(targets, opts)`	`{ rows, delta, cases, reports }`; `delta` is the last target minus the first.
`rag.sweep(target, grid).run(evals, { metric })`	grid `{ 'node.path': [values] }`	`{ metric, rows, best, top(n) }`; rows carry the config, the summary and `rebuilt` node ids. Ties keep run order.

A.12 Workspace

Member	Signature	Returns
`new rag.Workspace(ir, { engine, name = 'main', revision = 0 })`		
`ir`, `revision`, `history`, `future`, `branches`, `name`, `readonly`	properties	
`apply`	`(ops, { expect, by = 'user', note })`	Receipt (C.4). Throws `conflict` if `expect` is not the current revision.

Member	Signature	Returns
`dryRun`	`(ops)`	`{ ok, live, dirty, changed, removed, estimate: { llmCalls } }` or `{ ok: false, error }`. `llmCalls` is dirty Claude-costing nodes × all chunks.
`rebuild`	`({ calls, force })`	`{ ok, ms, computed, reused, calls, errors }`.
`run`, `session`	`(signals, opts)`, `()`	As A.10, against the current IR.
`undo`, `redo`	`()`	`{ revision, undone, ops }`, `{ revision }`; restore stored IRs; the revision always increases.
`reset`	`(ir, note)`	Replaces the IR as one history entry.
`branch`	`(name)`	A child workspace at the same revision.
`merge`	`(branch or name)`	Replays the branch's ops; throws `conflict` if the parent moved.
`snapshot`, `restore`	`()`, `(snap)`	`{ ir, revision, history, future }`.
`diff`	`(other)`	`[{ change: added, removed, type, wiring, param or trigger, node, … }]`.
`on`	`(fn)`	Events `apply`, `build`, `undo`, `redo`, `reset`, `restore`.
`toJSON`	`()`	`{ name, revision, nodes, dirty, branches, history }`.

A.13 `gui`

`gui.app(id)` returns an `AppSpec`: `.title(t)`, `.state(obj)`, `.derive(key, (s, ctx) => v)`, `.watch(key, (s, ctx, v) => …)`, `.use(plan)`, `.view(v => …)`, `.compile()` (C.8). The other exports are `gui.mount(app, el, host)`, `gui.bake(f)`, `gui.components()`, `gui.Runtime`, `gui.AppSpec` and the helpers `md`, `chart`, `h`, `esc`, `snippet`, `fmtNum`. Every prop may be a value or a lambda `(s, ctx) => value`; a component re-renders when a state key it read changes.

Group	Method	Props read
Layout	`col(b, o)`, `row(b, o)`	`grow`, `gap` (col)
Layout	`grid(cols, b)`, `card(title, b)`, `tabs(t => t.tab(title, b))`, `when(cond, b)`, `divider()`	`cols`; `title`; `key`; `cond`
Content	`h(text, level = 2)`, `text(md)`, `note(md, tone = 'info')`, `badge(text, tone)`, `code(text, lang)`	
Content	`stat(label, value, o)`, `stats(items)`, `kv(obj)`, `json(value)`, `link(text, href)`, `list(items)`	`delta`, `unit`
Content	`table(rows, o)`	`columns`, `max`, `onRowClick`
Charts	`chart(kind, data, o)`, `line`, `bar`, `scatter`	`title`, `xLabel`, `yLabel`, `yMin`, `yMax`
Inputs	`slider(key, o)`, `number(key, o)`, `select(key, options, o)`, `toggle(key, label)`, `input(key, o)`	`label`, `min`, `max`, `step`, `placeholder`
Inputs	`button(label, (s, ctx) => …, o)`	`disabled`, `tone`
Drawing	`canvas((s, ctx, g) => …, o)`	`width`, `height`, `label`, `animate`, `onClick`, `onHover`, `onKey`
Search	`searchBox(key = 'q', o)`	`placeholder`, `submitLabel`

Group	Method	Props read
Search	`facets(key = 'filters', o)`	`fields`, `from`, `max`
Search	`results(o)`	`from`, `limit`, `item`, `onSelect`, `selectedKey`
Search	`hits(list, o)`	`onSelect`
Search	`answer(o)`	`from`, `title`, `onCite`, `selectedKey`
Search	`chunk(id, o)`	`selectedKey`
Search	`trace()`, `status(o)`, `dag(o)`	`from` (status); `all` (dag; live nodes only unless true)

The runtime's lambda `ctx` holds `set(patch)`, `get(key)`, `run(trigger = 'submit')`, `submit(patch)`, `rag` (`chunk`, `doc`, `chunks`, `docs`, `explain`, `result`, `ir`), `toast`, `log`, `copy`, `download`, and in the workbench also `ws`, `evals`, `ui` and `lib` (`{ rag, gui }`). Bound apps hold the plan's signals as state keys plus `out`, `run`, `stream` and `result`; changing a signal key schedules a `change` run after 140 ms.

A.14 `kit`

Function	Returns
`kit.reports()`	The standard corpus: `*.md`, markdown, sections (`depth 3`, `maxTokens 450`), BM25 index `text` over title 2, headingPath 3, callout 2, text 1, and terms index `meta` over type, tags, status, aliases.
`kit.goldens({ docOnly })`	The 12 golden cases (`goldens`, or `goldens-doc` with document patterns only), metrics hit@5, MRR, nDCG@10.
`kit.searchApp(plan, { id, title, facets, answer, placeholder, submitLabel, results })`	Search box, status, facets (`['type', 'tags']` unless `false`), results, answer (unless `false`) and the section viewer.

Preset scripts run with `rag`, `gui`, `log` and `kit` in scope and return `{ plan, evals, app, variants }`.

A.15 Agent scope

The agent's `eval` tool runs an async function body with `rag`, `gui`, `wb`, `ws`, `ui`, `chat`, `ref`, `log` and `console` in scope, and returns YAML truncated near 9,000 characters (chapter 28). In Read-only mode the mutating members marked ✱ throw `readonly`.

Object	Members
`wb`	`files()`, `addFile(name, text)` ✱, `removeFile(name)` ✱, `script.get()`, `script.set(text)` ✱, `script.build()` ✱, `plan`, `evals`, `variants`, `corpus`, `result`, `lastEval`, `app`, `runEvals({ trigger })`, `search(q, { signals, trigger })`, `chunk(id)`, `chunks(f)`, `doc(id)`, `docs()`, `state()`, `kit`, `presets.list()`, `presets.current()`, `presets.text(id)`, `presets.load(id)` ✱, `build()` ✱
`ws`	The workbench workspace (A.12); `apply`, `rebuild`, `undo`, `redo`, `merge`, `reset`, `restore` ✱
`ui`	`query(q, { submit, filters })`, `filters(obj)`, `signals(obj)`, `tab('dag', 'trace', 'evals', 'history', 'corpus' or 'ir')`, `select(nodeId)`, `chunk(id)`, `app(spec)`, `toast(msg)`
`chat`	`md(text)`, `note(md, tone)`, `code(text, lang)`, `json(value)`, `table(rows, { title, columns })`, `chart(kind, data, opts)`, `hits(resultOrHits, { title, k })`, `evals(report)`, `dag({ highlight })`, `ui(app)`, `download(name, data)`
`ref`	`query(q)`, `chunk(id)`, `node(id)`, `tab(name)`: `wb:` links for markdown in the agent's replies

B. Preset catalogue

The editor's picker holds 29 presets in six groups, listed here in picker order. Each row was produced by running the preset script, compiling its plan and running it once on `change` and once on `submit` against the bundled corpus with a stand-in for Claude, so the call counts are the number of Claude calls the plan makes, not real outputs. Every preset starts from `source.files` and `parse.markdown`; the components column lists what it adds. "Build / live" counts build and live nodes. Every preset ships the 12 golden cases (`judged` adds a 13th) and a search app. Chapters in the last column use the preset as a Lab or as a worked example.

B.1 Start here

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`reports`	Reports · hybrid + rerank + answer	The full pipeline: three retrievers fused, query rewrite, rerank, neighbours, packing, cited answer. Variants `lexical` and `agentic`.	`chunk.sections`, `chunk` (failure-table rows), `synonyms`, `index.bm25`, `index.terms`, `rewrite.llm`, `retrieve` × 3, `fuse.rrf`, `filter.where`, `rerank.llm`, `expand.neighbors`, `pack`, `answer`	8 / 10	3 calls on Enter (`rewrite`, `rerank`, `answer`)	2, 7, 14, 18, 22, 23
`minimal`	Minimal search	The smallest working pipeline.	`chunk.sections`, `index.bm25`, `retrieve`	4 / 1	none	1, 3, 7, 19, 21
`graph`	Graph form	The same kind of plan with every node and edge named through `rag.graph`.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`, `limit`, `pack`, `answer`	5 / 7	1 call on Enter (`answer`)	7, 10

B.2 Corpus and chunking

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`chunk-sizes`	Chunk sizes	Section chunking at 200, 450 and 900 tokens, h2-only and 40-token overlap, as variants.	`chunk.sections`, `index.bm25`, `retrieve`, `expand.neighbors`	4 / 2	none	4, 11, 18, 20
`table-rows`	Table rows as chunks	A build lambda that turns each table row into its own chunk stream.	`chunk.sections`, `chunk` (rows), `index.bm25` × 2, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`, `limit`	7 / 5	none	4, 9
`code-blocks`	Code search	`rag.define('chunk.codeBlocks')`: one chunk per fenced block, language as a facet.	`chunk.sections`, `chunk.codeBlocks`, `index.bm25` × 2, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`, `limit`	7 / 5	none	4, 8
`outline`	Outline search	Headings only, each with its section's first sentence.	`chunk` (outline lambda), `index.bm25`, `retrieve`	4 / 1	none	4
`per-doc`	Which report?	One card per document for document-level retrieval.	`chunk` (doc cards), `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`	5 / 4	none	4

B.3 Ranking

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`fields`	Field weights	BM25F field weights; variants `text-only`, `headings-x6`, `b-0.3`.	`chunk.sections`, `index.bm25`, `retrieve`	4 / 1	none	5, 19
`fusion`	Fusion	RRF over retrievers; variants `fuse-max`, `rrf-k10`, `text-weight-only`, `equal-weights`.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 3, `fuse.rrf`, `filter.where`, `limit`	6 / 6	none	14
`custom-rerank`	Rerank with a lambda	A heuristic rerank lambda with params; variants `no-rerank`, `heading-x3`.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `rerank`, `limit`	5 / 3	none	9, 16

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`define-components`	Your own components	`rag.define('diversify')` and `rag.define('boost.tags')`, chained like built-ins, with a `boostTag` signal.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `boost.tags`, `diversify`, `limit`	5 / 4	none	8, 16
`synonyms`	Query expansion	A baked synonyms table applied at search time, and an acronym-expanding rewrite lambda; variants `no-rewrite`, `no-synonyms`.	`chunk.sections`, `synonyms`, `index.bm25`, `index.terms`, `rewrite`, `retrieve`, `limit`	6 / 3	none	6, 16
`context`	Context expansion	Neighbouring sections and linked reports added before packing.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `limit`, `expand.neighbors`, `expand.links`, `pack`, `answer`	5 / 6	1 call on Enter	15

B.4 With Claude

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`llm-rerank`	Rewrite + rerank	`rewrite.llm` and `rerank.llm` with stated reasons; variant `lexical`.	`chunk.sections`, `index.bm25`, `index.terms`, `rewrite.llm`, `retrieve`, `filter.where`, `rerank.llm`	5 / 4	2 calls on Enter	16
`hyde`	HyDE	A submit-only rewrite lambda that asks Claude for a hypothetical answer and searches with it; signal `hyde` turns it off.	`chunk.sections`, `index.bm25`, `index.terms`, `rewrite`, `retrieve`, `limit`	5 / 3	1 call on Enter	16

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`doc-cards`	Doc cards	`enrich.llm` writes a summary, questions and keywords per document card; a routing retriever uses the card index.	`chunk.sections`, `chunk` (cards), `enrich.llm`, `index.bm25` × 2, `index.terms`, `retrieve` × 2, `fuse.rrf`, `limit`	8 / 4	11 calls at Build (one per card), deferred; none per query	11, 17
`agentic`	Agentic answer	One tool-using node that searches, reads and cites.	`chunk.sections`, `chunk`, `index.bm25` × 2, `index.terms`, `agentic`	7 / 1	1 sampler call on Enter (the tool loop runs inside it)	17
`two-answers`	Two answers	One packed context feeding a brief and a detailed answer node.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `limit`, `pack`, `answer` × 2	5 / 5	2 calls on Enter	15
`judged`	Judged evals	`answerHas` checks plus `grounded` and `concise` judges.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `rerank.llm`, `pack`, `answer`	5 / 4	2 calls on Enter; judges 2 more per case in judged eval runs	20

B.5 Search experiences

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`facets`	Faceted search	Frontmatter facets, `filter.where`, and a filter lambda driven by the `codeOnly` toggle.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `filter.where`, `filter`, `limit`	5 / 4	none	6, 27

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`answer-first`	Answer first	Tabs for the answer, its sources and the trace with a live DAG.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`, `rerank.llm`, `pack`, `answer`	5 / 7	2 calls on Enter	12, 26, 27
`doc-browser`	Document browser	Pick a report, read its outline, or search inside it.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`, `filter.where`, `limit`	5 / 3	none	26, 27
`compare-nodes`	Compare stages	Three result lists from three nodes of one plan.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `rerank`, `limit`	5 / 5	none	13, 26
`playground`	Ranking playground	Sliders bound to `headingBoost`, `failureBoost` and `rrfK`; only nodes that read them re-run.	`chunk.sections`, `chunk`, `index.bm25` × 2, `index.terms`, `retrieve` × 2, `fuse.rrf`, `rerank`, `limit`	7 / 5	none	12, 25, 26
`canvas-map`	Result map	A clickable canvas of hits by document and rank.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`	5 / 1	none	24, 27
`dashboard`	Corpus dashboard	Stats, charts and a table computed from the corpus.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve`	5 / 1	none	24

B.6 Evaluation

Id	Title	Demonstrates	Components	Build / live	Claude	Chapters
`synthetic-evals`	Synthetic evals	`rag.evals.generate` from a button, then scoring the generated cases.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `filter.where`	5 / 4	none per query; one call per 8 generated questions when the button is pressed	18
`sweep`	Parameter sweep	Twelve configurations scored on the goldens, with a line chart.	`chunk.sections`, `index.bm25`, `index.terms`, `retrieve` × 2, `fuse.rrf`, `limit`	5 / 4	none	20

The picker's `llm` badge comes from `presets/index.json`: `enter` for presets that call Claude on Enter, `build` for `doc-cards`, and no badge otherwise. `synthetic-evals` carries `enter` although its calls come from its Generate button rather than from Enter. The `synonyms` blurb says "index-time synonyms"; the table is built at index time but applied when searching (chapter 6.3). Chapters 28–32 use the agent panel and the page itself rather than a preset.

C. Formats

Every object in this appendix was printed by running the library in node on the bundled corpus. The examples use one small plan throughout:

```
const corpus =
rag.corpus('reports').from(rag.files('*.md')).parse(rag.markdown())
  .chunk(rag.sections({ depth: 3, maxTokens: 450 }))
  .index('text', rag.bm25({ fields: { title: 2, headingPath: 3, text: 1 } }))
  .index('meta', rag.terms({ keys: ['type', 'tags'] }));
const plan = rag.plan('small').over(corpus).signals({ q: '', k: 5 })
  .retrieve('text', { k: 20 }).retrieve('meta').fuse.rrf({ k: 60 })
  .limit(s => s.k).pack({ budget: 8000 }).answer();
```

Long chunk ids are shortened to `cv#11-why-chatgpt-still-showed-no-actions`, where `cv` stands for `project-report-coinvault-mcp-from-local-conformance-to-chatgpt-2`. The answer text comes from a stand-in for Claude.

C.1 The IR

```
{
  "kind": "rag", "version": 1, "id": "small", "corpus": "reports",
  "signals": { "q": "", "k": 5 },
  "nodes": [
    { "id": "files",   "t": "source.files",   "phase": "build", "in": [],
"p": { "glob": "*.md" }, "out": "files", "uses": [], "reads": [] },
    { "id": "q",       "t": "signal",         "phase": "signal", "in": [],
"p": {}, "out": "signal" },
    { "id": "k",       "t": "signal",         "phase": "signal", "in": [],
"p": {}, "out": "signal" },
    { "id": "parse",   "t": "parse.markdown", "phase": "build", "in":
["files"], "p": {}, "out": "docs", "uses": [], "reads": [] },
    { "id": "chunks",  "t": "chunk.sections", "phase": "build", "in":
["parse"], "p": { "depth": 3, "maxTokens": 450 }, "out": "chunks", "uses":
[], "reads": [] },
    { "id": "ix-text", "t": "index.bm25",     "phase": "build", "in":
["chunks"], "p": { "fields": { "title": 2, "headingPath": 3, "text": 1 },
"name": "text" }, "out": "index", "uses": [], "reads": [] },
    { "id": "ix-meta", "t": "index.terms",    "phase": "build", "in":
["chunks"], "p": { "keys": ["type", "tags"], "name": "meta" }, "out":
"index", "uses": [], "reads": [] },
    { "id": "text",    "t": "retrieve",       "phase": "live", "in": ["q"],
"p": { "index": "text", "k": 20 }, "out": "hits", "uses": ["ix-text"],
"reads": [] },
    { "id": "meta",    "t": "retrieve",       "phase": "live", "in": ["q"],
"p": { "index": "meta" }, "out": "hits", "uses": ["ix-meta"], "reads": [] },
    { "id": "fuse",    "t": "fuse.rrf",       "phase": "live", "in": ["text",
"meta"], "p": { "k": 60 }, "out": "hits", "uses": [], "reads": [] },
    { "id": "limit",   "t": "limit",          "phase": "live", "in":
["fuse"], "p": { "k": { "$fn": "s => s.k" } }, "out": "hits", "uses": [],
"reads": ["k"] },
    { "id": "pack",    "t": "pack",           "phase": "live", "in":
["limit"], "p": { "budget": 8000 }, "out": "context", "uses": [], "reads": []
},
    { "id": "answer",  "t": "answer",         "phase": "live", "in":
["pack"], "p": {}, "out": "answer", "uses": [], "reads": ["q"] }
```

```
  ],
  "edges": [["files","parse","data"], ["parse","chunks","data"],
["chunks","ix-text","data"], ["chunks","ix-meta","data"],
            ["q","text","signal"], ["ix-text","text","artifact"],
["q","meta","signal"], ["ix-meta","meta","artifact"],
            ["text","fuse","data"], ["meta","fuse","data"],
["fuse","limit","data"], ["k","limit","signal"],
            ["limit","pack","data"], ["pack","answer","data"],
["q","answer","signal"]],
  "outputs": { "answer": "answer", "context": "pack", "hits": "limit" },
  "pinned": [],
  "digest": "6efbebeeea640f"
}
```

Field	Meaning
`kind`, `version`	Always `"rag"` and `1`.
`id`, `corpus`	Plan id and corpus id.
`signals`	Signal names with their initial values. Each signal also appears as a node with `t: "signal"`.
`nodes`	Topologically ordered. Signal nodes are placed where the order puts them.
`edges`	`[from, to, kind]`. `data` is an input wire, `signal` a signal read or input, `artifact` a live node's use of a build output such as an index.
`outputs`	Named outputs. Unpinned ones are derived: `answer` and `context` are the last nodes of that type; `hits` is the last `rerank*` node, else the input of the context node, else the last hits node.
`pinned`	Output names set explicitly with `g.output` or the `output` op.
`digest`	Hash of node ids, types, inputs, params and triggers, plus signals and outputs.

Node field	Meaning
`id`	Starts with a letter; letters, digits, `_` and `-`.
`t`	Component type, or `signal`.
`phase`	`build`, `live` or `signal`, taken from the component.
`in`	Input node ids in port order.
`p`	Parameters. Lambdas are `{ "$fn": source }`, bakes `{ "$bake": source }`, regular expressions `{ "$re", "flags" }`.
`trigger`	Present only when set on the node; otherwise the component default applies.
`out`	Output type: `files`, `docs`, `chunks`, `index`, `value`, `queries`, `hits`, `context`, `answer`, `signal`.
`uses`	Build nodes read as artifacts, such as `ix-text`.
`reads`	Signals read by the component or by a lambda in `p`, found by static scan. A retrieve's query arrives through `in` and is not repeated here.

C.2 Operations

Ops are plain objects applied in order by `ws.apply`, `rag.applyOps` or `plan.patch`. A batch is atomic: the first failing op throws `op N (kind): message` and nothing changes. After the batch the IR is recompiled, so an op list that leaves the graph invalid throws `after ops: …`.

Op	Shape	Effect
`set`	`{ op: 'set', node, path, value }`	Sets a param; `path` is dotted (`fields.title`). Functions become lambdas.
`unset`	`{ op: 'unset', node, path }`	Removes a param.

Op	Shape	Effect
`insert`	`{ op: 'insert', node: { t, p, trigger? } or component, id?, after or before or in, only?, feeds? }`	Adds a node. `after` rewires that node's consumers (or only those in `only`) and moves outputs of the same type; `before` takes over a single-input node's input; `in` wires explicitly and `feeds` appends it to other nodes' inputs.
`remove`	`{ op: 'remove', node }`	Deletes a node and rewires consumers to its input. Refused while another node `uses` it, or when a consumer cannot be rewired.
`replace`	`{ op: 'replace', node, with: { t, p, trigger? } }` or `{ op: 'replace', node, fn, param = 'fn' }`	Swaps the component, or only a lambda.
`wire`	`{ op: 'wire', node, in: [ids] }`	Sets a node's inputs.
`signal`	`{ op: 'signal', key, value }`	Adds or changes a signal's initial value.
`trigger`	`{ op: 'trigger', node, value: 'change' or 'submit' }`	Overrides a node's trigger.
`output`	`{ op: 'output', node, name }`	Pins a named output.
`rename`	`{ op: 'rename', node, to }`	Renames a node and every reference to it.

Real refusals from the example workspace:

```
op 1 (set): no node 'nope'. Nodes: files, parse, chunks, ix-text, text, limit
after ops: 1 problem:
  fuse: input 'q' gives signal but this port takes hits
```

The second is `remove meta` on the fused plan: removing one of two retrievers rewires the query signal into `fuse`, which the checker rejects, so the edit needs a `wire` op as well.

C.3 Trace entry

One entry per live node per run, in IR order, from the `submit` run of "Why did ChatGPT show no actions after OAuth worked?":

```
{ "node": "text", "t": "retrieve", "ms": 1.4, "in": 1, "out": 20, "status":
"ran", "calls": 0,
  "top": [{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "score":
24.5179,
            "why": { "gpt": 4.97, "chatgpt": 4.97, "action": 3.97, "show":
2.89, "oauth": 2.77 } }, …] }
{ "node": "fuse", "t": "fuse.rrf", "ms": 0.2, "in": 2, "out": 27, "status":
"ran", "calls": 0,
  "top": [{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "score":
0.03202, "parts": { "text": 1, "meta": 4 } }, …] }
{ "node": "pack", "t": "pack", "ms": 0.2, "in": 5, "out": 5, "status": "ran",
"calls": 0 }
{ "node": "answer", "t": "answer", "ms": 0.3, "in": 5, "out": 5, "status":
"ran", "calls": 1 }
```

Field	Meaning
`node`, `t`	Node id and type.
`status`	`ran`; `cached` (session memo hit, same params, inputs and signal reads); `skipped` (trigger excluded the node; its input or `null` passed on); `error`.
`in`, `out`	Item counts: list length, number of evidence entries for a context or answer, else 1.
`ms`, `calls`	Time and Claude calls spent in this node.
`top`	Up to 50 hits with `id`, `score` and, when present, `why`, `parts`, `reason`. Only on hits nodes that ran.
`notes`	Lines a node wrote with `ctx.note`, such as an agentic node's searches and reads.
`error`	The message, when `status` is `error`.

C.4 Receipt, history entry and dry run

`ws.apply([{ op: 'set', node: 'fuse', path: 'k', value: 20 }, { op: 'set', node: 'ix-text', path: 'fields.title', value: 4 }], { by: 'agent', note: 'try k 20' })` returned this receipt and stored this history entry:

```
{ "revision": 1, "live": ["fuse"], "dirty": ["ix-text"], "changed": ["ix-
text"], "removed": [] }

{ "revision": 1, "by": "agent", "note": "try k 20", "at": 1792192982445,
  "ops": [{ "op": "set", "node": "fuse", "path": "k", "value": 20 },
          { "op": "set", "node": "ix-text", "path": "fields.title", "value":
4 }],
  "before": "<the previous IR>" }
```

Receipt field	Meaning
`revision`	The workspace revision after the apply.
`live`	Live nodes whose type, inputs, params or trigger changed, plus changed signals. They take effect on the next run.
`dirty`	Build nodes whose new digest has no memo entry; the next build computes them.
`changed`	Build nodes whose digest changed, whether or not a memo entry already exists.
`removed`	Node ids no longer in the IR.

A dry run classifies without applying. On a two-node plan, `ws.dryRun([{ op: 'set', node: 'chunks', path: 'maxTokens', value: 300 }, { op: 'signal', key: 'k', value: 8 }])` returned `{ "ok": true, "live": ["k"], "dirty": ["chunks", "ix-text"], "changed": ["chunks", "ix-text"], "removed": [], "estimate": { "llmCalls": 0 } }`. A stale `expect` throws `conflict: workspace 'main' is at revision 0; these ops were planned against 7. Re-read ws.ir and plan again.`

C.5 Eval report

Two cases run on `change` against the example workspace:

```
{
  "id": "one", "target": "main", "irDigest": "8139f8bf6ab55d", "trigger":
"change", "at": 1792192982545, "ms": 98.4,
  "summary": { "n": 2, "hit@5": 1, "mrr": 1, "ndcg@10": 0.909, "p50ms": 1.8,
"calls": 0 },
  "cases": [
    { "i": 1, "q": "Why did ChatGPT show no actions after OAuth worked?",
"rank": 1, "synthetic": false, "R": 2,
      "expect": { "doc": "CoinVault", "section": "/no actions|stateless/i",
"answerHas": "/stateless/i" },
      "top": [{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "score":
0.08929,
                "doc": "PROJECT REPORT - CoinVault MCP - From Local
Conformance to ChatGPT 2026-07-28",
                "heading": "11. Why ChatGPT still showed no actions" }, …],
      "ms": 1.8, "m": { "hit@5": 1, "mrr": 1, "ndcg@10": 0.8175 } },
    { "i": 2, "q": "What does the rebalance law say?", "rank": 1, "R": 1, …,
"m": { "hit@5": 1, "mrr": 1, "ndcg@10": 1 } }
  ],
  "misses": []
}
```


Field	Meaning
`target`, `irDigest`, `trigger`	What was measured: the workspace or variant name, its IR digest, and the trigger each case ran with.
`summary`	The mean of each metric over cases that have it, `n`, median case time `p50ms`, and total Claude `calls`.
`cases[].rank`	1-based position of the first relevant hit, or `null`. Hits come from `result.hits`, or from the answer's evidence when the plan has no hits output.
`cases[].R`	Number of relevant chunks in the whole corpus; the denominator of recall and of the ideal DCG. A case with `R` 0 scores 0 on every metric.
`cases[].m`	Per-case metric values, plus `answerHas` (0 or 1) and judge scores when the run produced an answer.

Field	Meaning
`cases[].top`, `answer`, `errors`	Top three hits, the answer text clipped to 600 characters, and run errors.
`misses`	Cases where the first metric is 0: `{ q, rank, got, want }`.

`report.table()` (non-enumerable) returns one row per case for display.

C.6 Hits, context, answer, chunk and document

A hit from a BM25 retrieve, the same chunk after fusion, and `result.explain` for it:

```
{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "score": 24.5179, "via":
"text",
  "why": { "gpt": 4.97, "chatgpt": 4.97, "action": 3.97, "show": 2.89,
"oauth": 2.77 },
  "q": "Why did ChatGPT show no actions after OAuth worked?" }

{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "score": 0.03202,
"parts": { "text": 1, "meta": 4 }, "via": "fuse" }

{ "chunk": "cv#11-why-chatgpt-still-showed-no-actions",
  "path": [{ "node": "text",  "rank": 1, "score": 24.5179, "terms": { "gpt":
4.97, "chatgpt": 4.97, "action": 3.97, "show": 2.89, "oauth": 2.77 } },
           { "node": "meta",  "rank": 4, "score": 1.2131, "terms": { "tags":
"chatgpt" } },
           { "node": "fuse",  "rank": 1, "score": 0.03202, "parts": { "text":
1, "meta": 4 } },
           { "node": "limit", "rank": 1, "score": 0.03202, "parts": { "text":
1, "meta": 4 } }],
  "where": "PROJECT REPORT - CoinVault MCP - From Local Conformance to
ChatGPT 2026-07-28 › 11. Why ChatGPT still showed no actions" }
```

Hit field	Meaning
`id`, `score`	Chunk id and the score at this node. Scores are comparable only within one node.
`via`	The node that produced the hit.

Hit field	Meaning
`why`	BM25: the five largest per-term contributions after stemming. Terms index: the matched key and value.
`q`	The query, of several, that gave the hit its best score.
`parts`	RRF: the hit's 1-based rank in each fused input.
`reason`	A short reason from `rerank.llm` or a rerank lambda.
`of`	For `expand.*`: the hit it was added beside.

The context from `pack` is `{ text, evidence, bytes }`. `evidence` is `[{ label: 'E1', id, docId, title, headingPath }, …]`; `text` joins `[E1] title › heading` and the chunk text with `---` separators; `bytes` counts characters. The answer is `{ text, evidence, cited, truncated }`, where `cited` lists the labels found in the text; an agentic answer adds `rounds` (`[{ tool, query or id, index?, found? }]`).

A chunk:


```
{ "id": "cv#11-why-chatgpt-still-showed-no-actions", "docId": "cv",
  "title": "PROJECT REPORT - CoinVault MCP - From Local Conformance to
ChatGPT 2026-07-28",
  "headingPath": ["11. Why ChatGPT still showed no actions"], "kind":
"section",
  "text": "At this stage, OAuth worked, Inspector worked, and protected tools
worked. ChatG…",
  "meta": { "aliases": […], "status": "active", "type": "article", "tags":
["article", "project-report", "coinvault", "mcp", "oauth", "chatgpt", …], …
},
  "callout": "", "hasCode": true, "hasTable": false, "tokens": 287, "part":
1, "parts": 1, "ord": 40 }
```


Chunk field	Meaning
`id`	`docId#slug-of-last-heading`, with the chunk kind inserted for non-section streams and `-2`, `-3` on collisions.
`kind`	`section`, or the node id of a lambda chunker.

Chunk field	Meaning
`meta`	The document's frontmatter, minus title, dates, source paths, related notes and strings over 120 characters.
`callout`, `hasCode`, `hasTable`	Derived from the chunk's blocks.
`tokens`	Estimated tokens.
`part`, `parts`	Position when a section was split by `maxTokens`.
`ord`	Position in the stream.

A document from `parse.markdown` is `{ id, name, title, meta, h1, sections, links }`. Each section is `{ level, title, path, blocks, id, text }`, with `level` 0 for text before the first heading; a block is `{ kind, text }` plus `lang` (fence), `header` and `rows` (table), or `variant` (callout). `links` lists `[[wiki link]]` targets.

C.7 Build report

`engine.build(ir)` for the example, cold:


```
{ "ok": true, "ms": 200.9, "calls": 0, "errors": [],
  "nodes": {
    "chunks":  { "id": "chunks", "t": "chunk.sections", "digest":
"ebe4a56ec01071", "reused": false, "ms": 42.9, "count": 462, "computed": 11,
"cached": 0, "calls": 0 },
    "ix-text": { "id": "ix-text", "t": "index.bm25", "digest":
"4001134f943a91", "reused": false, "ms": 123.5, "count": 462, "computed": 0,
"cached": 0, "calls": 0 },
    … },
  "passes": [
    { "pass": "resolve(sources)", "ms": 0.6, "what": "11 files", "reused":
false },
    { "pass": "parse(markdown)", "ms": 21.6, "what": "11 docs", "reused":
false },
    { "pass": "chunk", "ms": 42.9, "what": "chunks 462", "reused": false },
    { "pass": "index", "ms": 125.8, "what": "text · meta", "reused": false },
    { "pass": "lower(plan)", "ms": 0, "what": "6 live nodes · 1 λ" },
    { "pass": "check(ports)", "ms": 0, "what": "15 edges · 0 problems" } ] }
```

Node field	Meaning
`digest`	The node's content digest (chapter 11).
`reused`	True when the memo (or, without calls, the partial store) already held this digest.
`count`	Output size: items, or the index's chunk count.
`computed`, `cached`	For map components: items computed now, and items taken from the per-item cache.
`deferred`, `skipped`	Items left unprocessed because Claude calls were not allowed, or because `budget.calls` ran out. A node with either is stored as partial and listed by `engine.pending`.
`calls`, `ms`, `error`	Claude calls, time, and the error that stopped the build.

C.8 The gui IR

`app.compile()` for a three-component app:

```
gui.app('mini').title('Mini search').use(plan).state({ selected: null
}).derive('n', s => (s.out.limit || []).length)
  .view(v => v.col(c => c.searchBox('q', { placeholder: 'Ask…' })
    .results({ onSelect: (s, ctx, h) => ctx.set({ selected: h.id })
}).chunk(s => s.selected)));
```

```
{ "kind": "gui", "version": 1, "id": "mini", "title": "Mini search",
  "state": { "selected": null },
  "derived": { "n": { "$fn": "s=>(s.out.limit||[]).length" } },
  "watchers": [], "uses": { "plan": "small" },
  "root": { "id": "n4", "t": "col", "p": {}, "c": [
    { "id": "n1", "t": "searchBox", "p": { "key": "q", "placeholder": "Ask…"
} },
    { "id": "n2", "t": "results", "p": { "onSelect": { "$fn": "
(s,ctx,h)=>ctx.set({selected:h.id})" } } },
    { "id": "n3", "t": "chunk", "p": { "id": { "$fn": "s=>s.selected" } } } ]
},
  "digest": "c5bf3dea7ad230" }
```

Field	Meaning
`state`	Initial state, JSON-copied. A bound app adds the plan's signals and `out`, `run`, `stream`, `result` at mount.
`derived`	Lambdas recomputed after every state change, up to four passes until stable.
`watchers`	`{ key, fn }`: handlers called when a key changes.
`uses`	The plan id the app binds to, or `null`.
`root`	The view tree. Node ids `n1`, `n2` … are assigned in build order; containers carry children in `c`.
`p`	Props: values, or lambdas `{ $fn }`. Handler props (`onClick`, `onSelect`, `onRowClick`, `onCite`, `onSubmit`, `item`, canvas `draw`) are called with extra arguments rather than evaluated for a value.

D. Glossary

Terms as this book uses them, in alphabetical order. The chapter column gives where the term is defined; Appendix A and C give the exact API and formats.

Term	Definition	Chapter
Agent	Claude with one `eval` tool that runs JavaScript in the page against `rag`, `gui`, `wb`, `ws`, `ui`, `chat` and `ref`.	28
Agentic node	A live `agentic` node that gives Claude `search`, `read`, `outline` and `documents` tools and produces an answer with evidence labels assigned on read.	17
`answerHas`	A golden-case pattern that the answer text must match; scored 0 or 1 when the run produced an answer.	20

Term	Definition	Chapter
App (gui)	A compiled `gui` description: state, derived values, watchers, the plan it uses, and a view tree built once.	24
Artifact edge	An IR edge from a build output, such as an index, to the live node that uses it.	10
Atomic block	A fence or table that the section chunker keeps whole up to three times `maxTokens`.	4
Bake	`rag.bake(ctx => value)`: a lambda evaluated once at build time over the built docs and chunks, whose result becomes a parameter value.	9
BM25	The lexical score used by `index.bm25`: inverse document frequency times saturated term frequency with length normalization, summed over query terms; `k1 = 1.2`, `b = 0.75`.	5
BM25F field weights	Per-field multipliers (`title: 2`, `headingPath: 3`, …) applied to term counts before scoring.	5
Branch	A child workspace created at the parent's revision; merged by replaying its ops, refused if the parent has moved.	23
Build memo	The engine's map from build-node digest to output; a digest found there is reused without running the node.	11
Build phase	The phase in which source, parse, chunk, enrich, index and bake nodes run, once per digest.	11
Build slot	A lambda parameter of a build component, run per item at build time, for example `.chunk('rows', (doc, ctx, p) => …)`.	9
Callout	A `> [!kind]` block, kept as its own block kind and indexed as the chunk field `callout`.	3
Chunk	The unit that is ranked, cited and scored: a slice of one document with an id, heading path, text, metadata and token count.	4

Term	Definition	Chapter
Chunk id	`docId#slug`, stable while the document and its headings are unchanged; used by fusion, expansion, selection and `explain`.	4
Closed lambda	A lambda run without its closure: only its arguments and a fixed set of globals are visible.	9
Component	A registered node type with a phase, typed ports, a default trigger and a run, map or all function.	8
Context	The output of `pack`: evidence-labelled text under a character budget, plus the evidence list.	15
Corpus spec	The immutable build-phase description produced by `rag.corpus(…)`.	7
Cursor	The builder's record of the latest node of each port type, used to wire the next call.	7
Deferred build	A build of a Claude-costing node with calls disallowed: items pass through unprocessed, the output is stored as partial, and `engine.pending` lists the node until Build runs it.	11
Digest	A hash of a build node's type, params, input digests and, for `source.files`, file contents; it names the node by what it would compute.	10
Dirty	A build node whose current digest has no memo entry.	11
Dry run	`ws.dryRun(ops)`: classifies the effect of ops without applying them.	21
Evidence label	`E1`, `E2`, …: the label `pack` or the agentic node gives each piece of context, cited inline by the answer.	15
Facet	A count of metadata values over a result set, written back as a filter signal when selected.	6
Family method	A plan method such as `.fuse.rrf()` generated from the registry; installed on the prototype, so later definitions appear on existing plans.	8

Term	Definition	Chapter
Fragment	`rag.fragment(id, (spec, params) => spec)`: a reusable spec transformation.	7
Free identifier	A name in a lambda that is neither an argument nor an allowed global; it throws on first run.	9
Fusion	Combining several hit lists into one; `fuse.rrf` by reciprocal rank, `fuse.max` by maximum score.	14
Golden case	A question with patterns for the document, section and optionally answer text that should answer it.	18
Graph form	`rag.graph`: a plan whose nodes, inputs and outputs are all stated explicitly.	7
Heading path	The list of headings above a section, joined with `›` for display and matching.	3
Hit	`{ id, score, via, … }`: a chunk at a position in one node's ranked list.	13
IR	The compiled graph: typed nodes in topological order, edges, signals, outputs and a digest.	10
Lambda	A function stored as source text, `{ $fn }`, so a pipeline can be saved and edited as data.	9
Live phase	The phase in which query nodes run on every search.	11
Live slot	A lambda parameter of a live component, run per query with signals, context and input.	9
Metric	hit@k, MRR, nDCG@k or recall@k, computed per case from the rank of relevant chunks.	19
Op	A data description of one IR edit; ten kinds.	21
Optimistic concurrency	Planning ops against a known revision and passing it as `expect`, so a stale plan is refused.	22
Output	A named IR entry (`hits`, `context`, `answer`, …) pointing at a node; derived unless pinned.	10

Term	Definition	Chapter
Partial output	A build output stored with deferred or skipped items; reused only while calls stay disallowed.	11
Per-item cache	The cache of map-component results keyed by component, params and item hash, in memory and in the database.	11
Plan spec	The immutable live-phase description produced by `rag.plan(…)`.	7
Port	A typed input or output of a component: files, docs, chunks, index, value, queries, hits, context, answer.	8
Preset	A pipeline script in the editor's picker that returns `{ plan, evals, app, variants }`.	2
Reads	The signals a node depends on, from the component's declaration, the static scan of its lambdas, and runtime tracking.	9
Receipt	What `ws.apply` returns: the new revision and the live, dirty, changed and removed nodes.	22
Relevant chunk	A chunk whose document and section match a golden case's patterns.	18
Revision	The workspace's change counter; it increases on every apply, undo, redo, reset and restore.	22
RRF	Reciprocal rank fusion: each list adds `weight / (k + rank)`; `k = 60` by default.	14
Section	The text under one heading, with its level, path and blocks.	3
Session	A runner bound to an engine and an IR that keeps a memo between runs.	12
Session memo	Per-node record of the last run's params, input identities and signal values read; a node whose record matches is reported `cached`.	12

Term	Definition	Chapter
Signal	A named input of a query run, such as `q`, `k` or `filters`; a node in the IR and a state key in a bound app.	12
Snapshot	`{ ir, revision, history, future }` saved before each agent step so the step can be reverted.	23
Spec	An immutable builder value; every builder call returns a new spec.	7
Status	A trace entry's outcome: ran, cached, skipped or error.	12
Sweep	Scoring a grid of `set` ops on the same evals; each configuration rebuilds only what it changed.	20
Synonyms table	A build-time value mapping phrases to expansions, applied to BM25 queries at weight 0.5.	6
Synthetic case	A golden case whose question Claude wrote for a sampled section.	18
Terms index	`index.terms`: matches whole metadata values (type, tags, …) that appear in the query.	6
Trace	One entry per live node per run: status, counts, time, calls and top hits.	13
Trigger	When a live node runs: `change` on every keystroke, `submit` only on Enter.	12
Variant	A plan derived by `plan.variant(id, ops)`; the original is unchanged.	7
Workspace	The holder of the current IR, its engine, revision and history, through which edits are applied.	22